WordPress Plugin Development Cookbook

Third Edition

Explore the complete set of tools to craft powerful plugins that extend the world's most popular CMS

Yannick Lefebvre

WordPress Plugin Development Cookbook

Third Edition

Explore the complete set of tools to craft powerful plugins that extend the world's most popular CMS

Yannick Lefebvre

Packt

BIRMINGHAM—MUMBAI

WordPress Plugin Development Cookbook
Third Edition

Copyright © 2022 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Group Product Manager: Pavan Ramchandani
Publishing Product Manager: Kaustubh Manglurkar
Senior Editor: Aamir Ahmed
Content Development Editor: Feza Shaikh
Technical Editor: Shubham Sharma
Copy Editor: Safis Editing
Project Coordinator: Manthan Patel
Proofreader: Safis Editing
Indexer: Manju Arasan
Production Designer: Prashant Ghare
Marketing Coordinator: Anamika Singh

First published: July 2012
Second edition: July 2017
Third edition: March 2022
Production reference: 1170222

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham
B3 2PB, UK.

ISBN 978-1-80181-077-7
www.packt.com

To my wife, Andrée, for her everlasting love, her patience throughout the writing process, and supporting all my quirks.

To my daughters, Évelyne and Gabrielle, for always making me smile and becoming wonderful women.

To the users of my WordPress plugins, who are always pushing me to learn new things and add great new features to my creations after all these years.

– Yannick Lefebvre

Contributors

About the author

Yannick Lefebvre is a plugin developer who has created multiple plugins on the official WordPress repository. His first creation, Link Library, is used on thousands of sites worldwide. With a background in computer science, he wrote his first plugin in 2004 and quickly started sharing his creations with the community. He is actively involved in the Montreal WordPress community, has presented multiple times at WordCamp Montreal, and offers custom plugin development services. By day, Yannick works for CM Labs Simulations, a company providing software tools and simulators for vehicles and heavy equipment.

I would like to thank the WordCamp Montreal organizers for giving me the chance to speak at multiple editions of the event, for creating great videos of the presentations, and for giving me the opportunity to get involved in the community. This project would not have existed without them.

I would also like to thank Richard Archambault for his great feedback and encouragement while I wrote the first edition of this book, as well as the entire Packt Publishing team for proposing this great project to me, supporting me through the entire process, and allowing me to put together this third edition.

Finally, I would like to thank my parents for always believing in me and encouraging me in all my projects.

About the reviewers

Ali Demirci is a senior developer who has 15 years of experience in WordPress. He started working on WordPress in 2007. He is the co-founder of 2035themes. He has sold more than 6,000 WordPress themes on Themeforest. He became a freelancer as the themes business wasn't providing a stable income. He is now a full-time freelancer on the Codeable platform.

Stefan C. Densmore has been a WordPress developer since 2003 and a continuous member of the WPMUDEV development community since 2011. He has been a speaker at multiple WordCamps as a subject matter expert in WordPress Multisite development. With a formal education in human factors engineering and performance psychology, Stefan is passionate about making complex systems more user-friendly. He has a fondness for applications and plugins that enhance information sharing and knowledge management. Stefan has served as a technology broker and consultant for various information management interests and lives in the United States. He is the founder of both the United Resource Connection and Cybercasemanager enterprises, and can be found on Twitter `@StefanDensmore`.

Fred Atkinson is a telecom/IT professional with interests in web and internet technologies and website development. As an avid ham radio operator (WB4AEJ, for 47 years), he has been using WordPress for several years to build websites that facilitate the propagation of operator callsigns. He studied at Johns Hopkins University, earning a master's degree in Information and Telecommunications Systems Management in 1997. He also has an associate's degree in Enterprise Network Management from Stratford University. He is now retired and currently lives in Mesa, Arizona.

Table of Contents

Preface

1
Preparing a Local Development Environment

Technical requirements 2
Installing a local web server on your computer 2
How to do it... 3
How it works... 5
There's more... 6
Creating a local version control repository 7
How to do it... 7
How it works... 11
There's more... 12
Installing a dedicated code editor/text editor 14
Getting ready 15
How to do it... 15
How it works... 16
There's more... 17

2
Plugin Framework Basics

Technical requirements 20
Creating a plugin file and header 20
Getting ready 21
How to do it... 21
How it works... 23
See also 23
Adding output content to page headers using plugin actions 24
How to do it... 24
How it works... 26
There's more... 27
See also 28
Using WordPress path utility functions to load external files and images 28
How to do it... 28
How it works... 30
There's more... 31
See also 32

Modifying the site generator meta tag using plugin filters 32
How to do it... 32
How it works... 33
There's more... 34
See also 35

Adding text after each item's content using plugin filters 35
How to do it... 35
How it works... 38
There's more... 38
See also 39

Inserting link tracking code in the page body using plugin filters 39
Getting ready 39
How to do it... 40
How it works... 42
See also 42

Troubleshooting coding errors and printing variable content 42
How to do it... 43
How it works... 46
There's more... 47
See also 49

Creating a new simple shortcode 49
How to do it... 49
How it works... 50
See also 51

Creating a new shortcode with parameters 51
How to do it... 51
How it works... 53
See also 54

Creating a new enclosing shortcode 54
How to do it... 54
How it works... 56
See also 56

Loading a style sheet to format plugin output 56
Getting ready 56
How to do it... 56
How it works... 58
See also 58

Writing plugins using object-oriented PHP 58
Getting ready 58
How to do it... 59
How it works... 60
There's more... 60
See also 60

3

User Settings and Administration Pages

Technical requirements 62

Creating default user settings on plugin initialization 63
How to do it... 63
How it works... 64
There's more... 65
See also 65

Storing user settings using arrays 66
Getting ready 66
How to do it... 66

How it works... 68
See also 69

Removing plugin data on deletion 69
Getting ready 69
How to do it... 69
How it works... 71
See also 71

Creating an administration page menu item in the settings menu 71
Getting ready 71
How to do it... 72
How it works... 73
There's more... 74
See also 75

Creating a multi-level administration menu 75
How to do it... 75
How it works... 77
There's more... 78
See also 79

Rendering the admin page contents using HTML 79
Getting ready 79
How to do it... 80
How it works... 81
There's more... 82
See also 83

Processing and storing plugin configuration data 83
Getting ready 83
How to do it... 83
How it works... 86
See also 87

Displaying a confirmation message when options are saved 87
Getting ready 87
How to do it... 87
How it works... 88
See also 89

Adding custom help pages 89
Getting ready 89
How to do it... 89
How it works... 92
See also 92

Rendering the admin page contents using the Settings API 92
How to do it... 93
How it works... 98
There's more... 100
See also 102

Accessing user settings from action and filter hooks 102
Getting ready 102
How to do it... 102
How it works... 104
See also 104

Formatting admin pages using meta boxes 104
Getting ready 104
How to do it... 105
How it works... 109
See also 110

Splitting admin code from the main plugin file to optimize site performance 111
Getting ready 111
How to do it... 111

How it works... 112
See also 112

Storing style sheet data in user settings 113
Getting ready 113
How to do it... 113
How it works... 121
See also 122

Managing multiple sets of user settings from a single admin page 122
Getting ready 123
How to do it... 123
How it works... 130
See also 131

Creating a network-level plugin with admin pages 131
Getting ready 131
How to do it... 132
How it works... 135
See also 135

4

The Power of Custom Post Types

Technical requirements 138
Creating a custom post type 139
Getting ready 139
How to do it... 139
How it works... 143
There's more... 144
See also 146

Adding a new section to the custom post type editor 146
Getting ready 146
How to do it... 147
How it works... 150
See also 151

Displaying single custom post type items using a custom layout 151
Getting ready 151
How to do it... 151
How it works... 154
See also 155

Tailoring search output for custom post type items 155
Getting ready 155
How to do it... 155
How it works... 159
See also 160

Displaying custom post type data in shortcodes 160
Getting ready 160
How to do it... 161
How it works... 164
See also 164

Adding custom categories for custom post types 165
Getting ready 165
How to do it... 165
How it works... 168
See also 168

Adding custom fields to categories 168
Getting ready 169
How to do it... 169
How it works... 171
See also 172

Hiding the category editor from the custom post type editor 172
Getting ready 172
How to do it... 172
How it works... 175
See also 175

Displaying additional columns in the custom post list page 175
Getting ready 175
How to do it... 176
How it works... 180
See also 180

Adding filters for custom categories to the custom post list page 181
Getting ready 181
How to do it... 181
How it works... 183
See also 184

Adding Quick Edit fields for custom categories 184
Getting ready 184
How to do it... 184
How it works... 192

Updating the page title to include custom post data using plugin filters 193
Getting ready 193
How to do it... 194
How it works... 195

5

Customizing Post and Page Editors

Technical requirements 198

Capturing and displaying information using custom meta boxes 198
Getting ready 198
How to do it... 198
How it works... 201
There's more... 202

Displaying custom post data using filter functions 203
Getting ready 203
How to do it... 204
How it works... 205
See also 206

Hiding the Custom Fields section in the post editor 206
Getting ready 206
How to do it... 208
How it works... 209

Extending the post editor to attach custom files with the media uploader 210
Getting ready 210
How to do it... 210
How it works... 215
See also 215

6
Extending the Block Editor

Technical requirements 218

Setting up a block development environment and creating a simple block 219

Getting ready 219
How to do it... 219
How it works... 224
There's more... 225
See also 226

Creating an editable blockquote block 226

Getting ready 226
How to do it... 226
How it works... 230
See also 231

Creating a new WordPress REST endpoint 231

Getting ready 232
How to do it... 232
How it works... 233
See also 234

Creating a server-side rendering block that leverages an existing shortcode 234

Getting ready 234
How to do it... 234
How it works... 240
See also 241

7
Accepting User Content Submissions

Technical requirements 244

Creating a client-side content submission form 244

Getting ready 244
How to do it... 244
How it works... 248
See also 248

Saving user-submitted content in custom post types 249

Getting ready 249
How to do it... 249
How it works... 252
There's more... 253
See also 253

Sending email notifications upon new submissions 254

Getting ready 254
How to do it... 254
How it works... 257
See also 257

Implementing a CAPTCHA on user forms using an online service 257

Getting ready 258
How to do it... 258
How it works... 261
See also 262

Using a local library to implement a CAPTCHA on user forms 262
Getting ready 262
How to do it... 263
How it works... 265
See also 265

8

Customizing User Data

Technical requirements 268
Adding custom fields to the user editor 268
Getting ready 268
How to do it... 268
How it works... 270
See also 271
Processing and storing custom user data 271
Getting ready 271
How to do it... 272
How it works... 273
See also 273
Displaying new user data on the user list page 273
Getting ready 274
How to do it... 274
How it works... 278
See also 279
Using custom user data in an enclosing shortcode 279
Getting ready 279
How to do it... 279
How it works... 281
See also 281

9

Leveraging JavaScript, jQuery, and AJAX Scripts

Technical requirements 284
Safely loading jQuery onto WordPress web pages 284
Getting ready 284
How to do it... 285
How it works... 286
There's more... 287
Displaying a pop-up dialog using the built-in ThickBox plugin 287
Getting ready 287
How to do it... 287
How it works... 289
There's more... 290

Controlling pop-up dialog display using shortcodes 291
Getting ready 292
How to do it... 292
How it works... 294
See also 295

Displaying a calendar day selector using the Datepicker plugin 295
Getting ready 295
How to do it... 295
How it works... 298

Adding tooltips to admin page form fields using the TipTip plugin 298
Getting ready 299
How to do it... 299
How it works... 301
See also 301

Using AJAX to dynamically update partial page contents 301
Getting ready 302
How to do it... 302
How it works... 307
See also 309

10

Adding New Widgets to the WordPress Library

Technical requirements 312

Creating a new widget in WordPress 312
Getting ready 312
How to do it... 312
How it works... 317
See also 318

Displaying widget configuration options 318
Getting ready 318
How to do it... 318
How it works... 321
See also 322

Validating widget configuration options 322
Getting ready 322
How to do it... 322
How it works... 323
See also 324

Adding a custom dashboard widget 324
Getting ready 324
How to do it... 324
How it works... 326
See also 327

Adding a custom widget to the network dashboard 327
Getting ready 328
How to do it... 328
How it works... 330
See also 330

11

Fetching, Caching, and Regularly Updating External Site Data

Technical requirements 332

Displaying RSS items from an external feed 332

Getting ready 332
How to do it... 332
How it works... 338
There's more... 338
See also 339

Optimizing plugin performance by storing external data using transients 339

Getting ready 339
How to do it... 339
How it works... 343
See also 345

Scheduling plugin data updates using WP-Cron 345

Getting ready 345
How to do it... 345
There's more... 348
See also 348

12

Enabling Plugin Internationalization

Technical requirements 350

Changing the WordPress site language configuration 350

Getting ready 350
How to do it... 350
How it works... 351

Adapting default user settings for translation 351

Getting ready 351
How to do it... 352
How it works... 353
See also 353

Making admin page code ready for translation 353

Getting ready 354
How to do it... 354
How it works... 356
See also 357

Modifying shortcode output for translation 357

Getting ready 357
How to do it... 357
How it works... 358
See also 359

Translating text strings using Poedit 359

Getting ready 359
How to do it... 359
How it works... 361
There's more... 362
See also 362

Loading a language file in the plugin initialization 362
Getting ready 363
How to do it... 363
How it works... 365
There's more... 365
See also 368

13

Distributing Your Plugin on WordPress.org

Technical requirements 371
Creating a README file for your plugin 371
Getting ready 371
How to do it... 371
How it works... 373
There's more... 373
Applying for your plugin to be hosted on WordPress.org 374
How to do it... 374
How it works... 375
See also 375
Uploading your plugin using Subversion 375
Getting ready 375
How to do it... 376
How it works... 378
There's more... 378
See also 378
Providing plugin banner and thumbnail images 379
Getting ready 379
How to do it... 379
How it works... 381
See also 381

Index

Other Books You May Enjoy

Preface

As it approaches the twentieth anniversary of its initial release, WordPress continues to be one of the most widely used, powerful, and open **content management systems** (**CMSs**). Whether you're a site owner trying to find the right extension, a developer who wants to contribute to the community, or a website developer working to fulfill a client's needs, learning how to extend WordPress' capabilities will help you to unleash its full potential.

This book will help you become familiar with API functions to create secure plugins with easy-to-use administration interfaces. Complete with new recipes and up-to-date code samples, including new chapters on the creation of custom blocks for the Block Editor and interacting with external data sources, this third-edition WordPress plugin development book teaches you how to create plugins of varying complexity ranging from using just a few lines of code to complex extensions that provide intricate new capabilities.

Starting with WordPress' basic mechanism for creating plugins, the book covers recipes to show you how to design administration panels, enhance the post editor with custom fields, store custom data, and even create custom blocks. You'll safely incorporate dynamic elements into web pages using scripting languages, learn how to integrate data from external sources, and build new widgets that can be added to WordPress sidebars and widget areas. By the end of this book, you'll be able to create WordPress plugins to perform almost any task you can imagine.

Who this book is for

This book is for WordPress users, developers, and site integrators interested in creating new plugins to address their personal needs, fulfill client requirements, and bring new capabilities to the WordPress community. Basic knowledge of PHP and WordPress is expected.

What this book covers

Chapter 1, *Preparing a Local Development Environment*, shows plugin developers how to install and configure an efficient development environment.

Chapter 2, *Plugin Framework Basics*, explains the basic mechanics of registering user functions with WordPress to be executed at key points when web pages are displayed, forming the basis of plugin creation.

Chapter 3, *User Settings and Administration Pages*, covers the creation of administration pages that will allow users to configure the plugins you create.

Chapter 4, *The Power of Custom Post Types*, empowers developers to add whole new content management sections to the WordPress environment.

Chapter 5, *Customizing Post and Page Editors*, demonstrates how to alter the default administration post and page editing environment to add new capabilities.

Chapter 6, *Extending the Block Editor*, shows how to create new content blocks for the new Block Editor and facilitate building websites.

Chapter 7, *Accepting User Content Submissions*, allows users to submit their own content to new content sections that will be managed by your plugins.

Chapter 8, *Customizing User Data*, explains how to store additional information for users and how to modify site output based on this data.

Chapter 9, *Leveraging JavaScript, jQuery, and AJAX Scripts*, makes plugin output very dynamic by using a number of popular script libraries.

Chapter 10, *Adding New Widgets to the WordPress Library*, shows how to add new widgets that users will be able to easily drag and drop to add content to their web pages.

Chapter 11, *Fetching, Caching, and Regularly Updating External Site Data*, explains how to extract data from external sites and locally cache that data for efficient use.

Chapter 12, *Enabling Plugin Internationalization*, prepares your plugin to be translated into any language to make it easier to be used by non-English speakers.

Chapter 13, *Distributing Your Plugin on WordPress.org*, shows you how to prepare your plugin for sharing with the global WordPress community.

To get the most out of this book

All code examples in this book have been tested against WordPress 5.9 using the **Twenty Twenty-Two** theme, unless otherwise noted. As WordPress always strives for backward compatibility, they will likely work with future versions as well. All other software tools needed are covered in *Chapter 1*, *Preparing a Local Development Environment*.

Software covered in the book	Operating system requirements
WordPress 5.9	Windows, macOS, or Linux
PHP 5.6 and higher	
JavaScript, jQuery	
Local development web server	
Visual Studio Code	
Subversion	
Poedit	

If you are using the digital version of this book, we advise you to type the code yourself or access the code from the book's GitHub repository (a link is available in the next section). Doing so will help you avoid any potential errors related to the copying and pasting of code.

Download the example code files

You can download the example code files for this book from GitHub at `https://github.com/PacktPublishing/WordPress-Plugin-Development-Cookbook-Third-Edition`. If there's an update to the code, it will be updated in the GitHub repository.

We also have other code bundles from our rich catalog of books and videos available at `https://github.com/PacktPublishing/`. Check them out!

Download the color images

We also provide a PDF file that has color images of the screenshots and diagrams used in this book. You can download it here: `https://static.packt-cdn.com/downloads/9781801810777_ColorImages.pdf`

Conventions used

There are a number of text conventions used throughout this book.

`Code in text`: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. Here is an example: "The `widget` method starts by calling the standard PHP `extract` function on the first parameter received, an array named `$args`, which contains a list of styling tidbits to render the widget."

A block of code is set as follows:

```
class Book_Reviews extends WP_Widget {
    function __construct () {
        parent::__construct( 'book_reviews',
            'Book Reviews',
            array( 'description' =>
                   'Displays list of recent book reviews' ) );
    }
}
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
</tr><tr>
<td colspan="2">
<div class="g-recaptcha"
data-sitekey="[my-site-key]"></div>
</td>
```

Any command-line input or output is written as follows:

```
npm install --save-dev --save-exact @wordpress/scripts
```

Bold: Indicates a new term, an important word, or words that you see onscreen. For instance, words in menus or dialog boxes appear in **bold**. Here is an example: "**Publish** the new page and click on **View Page** to see the block on your development site."

> **Tips or Important Notes**
> Appear like this.

Sections

In this book, you will find several headings that appear frequently (*Getting ready*, *How to do it...*, *How it works...*, *There's more...*, and *See also*).

To give clear instructions on how to complete a recipe, use these sections as follows:

Getting ready

This section tells you what to expect in the recipe and describes how to set up any software or any preliminary settings required for the recipe.

How to do it...

This section contains the steps required to follow the recipe.

How it works...

This section usually consists of a detailed explanation of what happened in the previous section.

There's more...

This section consists of additional information about the recipe in order to make you more knowledgeable about the recipe.

See also

This section provides helpful links to other useful information for the recipe.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book, email us at `customercare@packtpub.com` and mention the book title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you would report this to us. Please visit `www.packtpub.com/support/errata` and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at `copyright@packt.com` with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit `authors.packtpub.com`.

Share Your Thoughts

Once you've read *WordPress Plugin Development Cookbook*, we'd love to hear your thoughts! Please click here to go straight to the Amazon review page for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

1
Preparing a Local Development Environment

Before you start writing your first WordPress plugin, it is important to have a good set of tools in place that will allow you to work locally on your computer and be more efficient in your work. While it is possible to perform some development tasks with the built-in tools that are provided with the operating system, creating a solid local development environment will help you develop plugins quickly and have full control over your server settings to be able to test different configurations.

This chapter proposes a set of free tools that can easily be installed on your computer, regardless of your preferred operating system, to facilitate the development of your future WordPress plugins. These tools include a **local web server** to speed up page access and avoid sending files constantly to a remote server, a **version control system** to keep incremental backups of your work, and a **code editor** to enhance your editing capabilities.

We will cover the following recipes in this chapter:

- Installing a local web server on your computer
- Creating a local version control repository
- Installing a dedicated code editor/text editor

Technical requirements

You can find complete code examples on GitHub under the following repository: `https://github.com/PacktPublishing/WordPress-Plugin-Development-Cookbook-Third-Edition`.

Installing a local web server on your computer

The first step to preparing a local development environment is to install a local web server on your computer. This will transform your computer into a system capable of running the core WordPress code and performing all tasks related to rendering a WordPress website locally.

Having a local web server has many benefits:

- It provides a quick response to the frequent page refreshes that are made as plugin code is written, tested, and refined, since all information is processed locally.
- It removes the need to constantly upload new plugin file versions to a remote web server to validate code changes.
- It allows development to take place when no internet connection is available (for example, when traveling on an airplane). This is only possible if your plugin code does not interact with external servers.
- It offers a worry-free programming environment where you cannot bring down a live website with a programming error or an infinite loop.

There are many free packages available online that contain all of the web server components necessary to run a WordPress installation.

How to do it...

Follow these steps to easily install the aptly named **Local** application from Flywheel:

1. Visit the **Local** website (`https://www.localwp.com/`) and download the appropriate package for your computer.

> **Note**
> Local is free and available for Windows, macOS, and Linux platforms. The screenshots and instructions in this recipe are based on **Local** version 6.1.5 running on Windows 11. The installation steps and exact dialog content may vary based on your choice of platform.

2. Launch the **Local** installer.
3. On Windows, select whether the application should be available only to the user performing the application or to all users (requires administrator privileges), and then leave all the other options at their default values.
4. Also on Windows, make sure that the **Run Local** checkbox is checked at the end of the installation process and click on **Finish** to exit the installer and start **Local**.

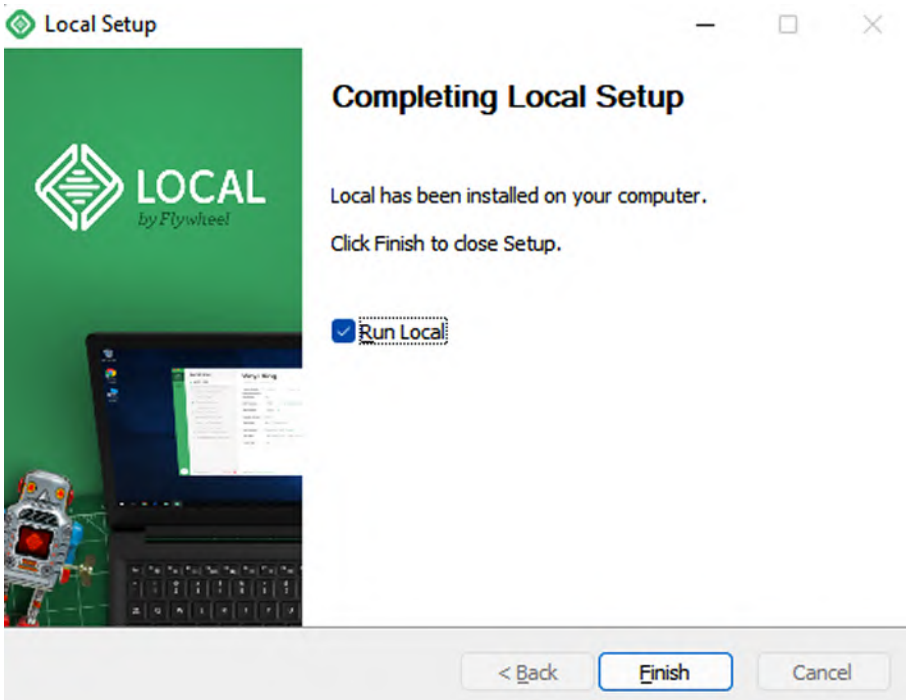


Figure 1.1 – Last page of the Local installation program on Windows

5. On macOS, open the installation package and drag the **Local** program to the **Applications** folder. Once this is done, search for `Local` and run it.
6. When **Local** first starts, you will need to accept the application's **Terms of Service** and will also be given the option to activate **Error Reporting** to automatically report problems back to the **Local** team. You can activate the error reporting feature at your discretion.
7. On the next screen, you will be prompted to create a free **Local** account. This is optional and can be bypassed by closing the account creation screen, bringing you to the main **Local** interface.

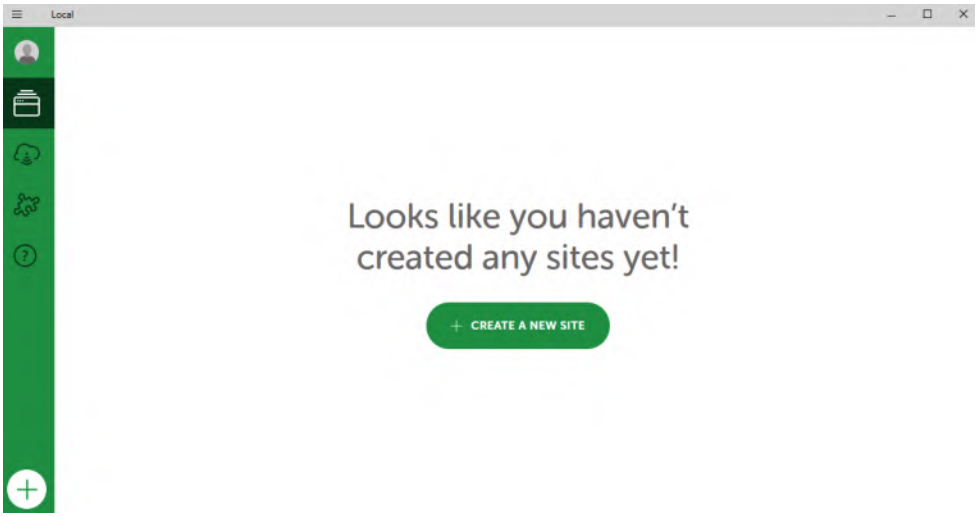


Figure 1.2 – Main Local interface

8. Click on **+ CREATE A NEW SITE** to start the site creation process.
9. Name your new site **Plugin Development Site** and click on **CONTINUE**.
10. Select the **Preferred** option to create a site running the latest versions of **PHP** and **MySQL**, along with the **nginx** web server, and then click on **CONTINUE**.
11. Specify a username, password, and email to create an account on your development WordPress site and then click on **ADD SITE**. The **Local** application will download and install the required components and start all of the required applications to have a working local WordPress installation. On Windows, you may need to allow multiple applications through the computer's firewall.

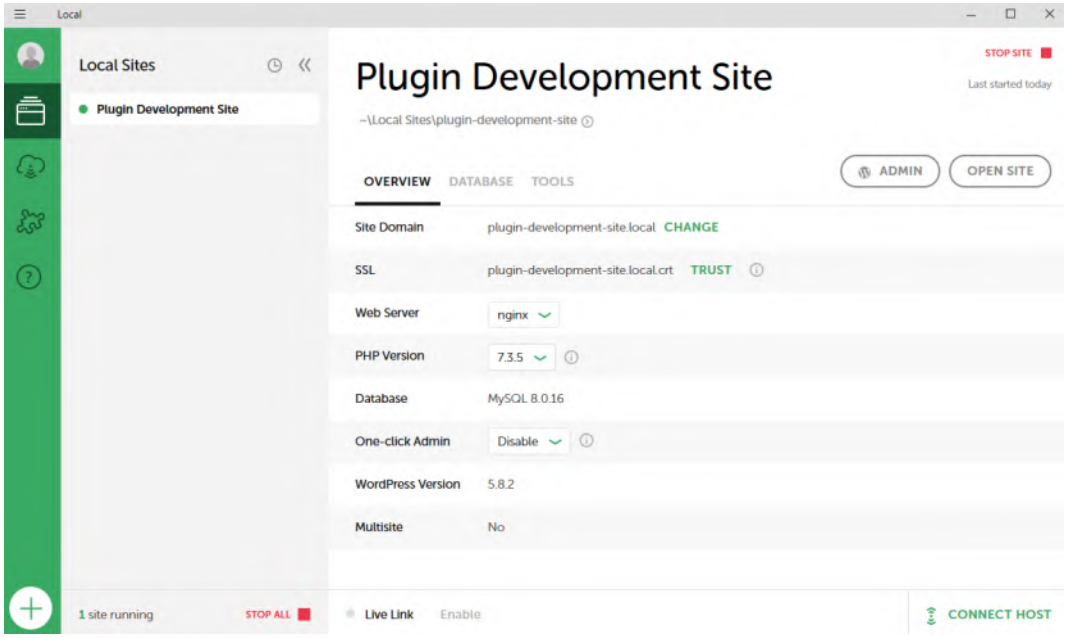


Figure 1.3 – Newly created local development site

12. Visit your local development site by clicking on the **OPEN SITE** button in the top-right corner of the **Local** interface. The default address of your site is `http://plugin-development-site.local/`.
13. Back in the **Local** interface, click on the **ADMIN** button to visit your site's login page and connect using the credentials you provided during the site creation process.
14. Make sure that the active theme is **Twenty Twenty-Two**. If not, install it and activate it.
15. Still in the **Local** interface, click on the arrow next to the site path under the site name to access the site's local files on your computer. The WordPress software files will be found under the `app\public` directory within the site directory.

How it works...

The Local application contains all of the components necessary to run a web server capable of hosting a WordPress website on your computer. These components include:

- nginx or the Apache web server
- PHP interpreter

- MySQL database server
- Adminer database management interface

Once we've created a development site with Local, the application registers a website domain that is recognized by the operating system as a request to communicate with the locally hosted development website. In this example, the domain created is `http://plugin-development-site.local`. When developing a local site that will eventually be deployed online, you can change the **Site Domain** field in the Local interface to match the domain of your target website. This allows you to work on the site locally while accessing it as if it was in its final online location.

There's more...

While Local is a full-featured local web server package and is available on the three major operating systems, there are many others available online. Most of these packages will run the required web services on the computer directly, while more advanced packages, such as **Varying Vagrant Vagrants** (**VVV**), will virtualize a complete Linux-based web server on your computer to create a more accurate replica of a final deployment environment optimized for WordPress. Here is a list of some of the most popular local web server packages:

For Windows:

- WampServer (`https://www.wampserver.com/en/`)
- EasyPHP (`https://www.easyphp.org/`)

For macOS:

- MAMP (`https://www.mamp.info/en`)

Cross-platform:

- Varying Vagrant Vagrants (`https://varyingvagrantvagrants.org`)

> **Tip**
>
> For a more complete list of web server packages, visit `https://en.wikipedia.org/wiki/List_of_AMP_packages`.

Creating a remote web development environment

If it's not possible for you to set up a local web server to develop WordPress plugins, or if you are planning to share the development tasks with one or more developers, then an alternative to setting up a local web server is to create a remote development environment.

The easiest way to create such an environment, assuming that you already have a web hosting account set up, is to create a sub-domain off your main domain. This will allow you to create a standalone test installation for WordPress that will still provide safety from the point of view of affecting a live site but will not carry the other benefits of a local installation.

Creating a local version control repository

Version control is an important part of any code development project. Version control keeps track of a project's history, provides full and organized backups, and easily rolls back changes in order to get back to a known working state. Version control is also the best and most efficient way to share code and other files when developing a project in a team environment. In addition to being a great version control system that is easy to use and configure, Subversion (often referred to as SVN) is also the technology that manages all submissions on the official WordPress plugin directory. Therefore, by setting up and using a local Subversion repository during your initial plugin development, you will immediately be ready to share your creations with the community. We'll be covering the most commonly used Subversion commands in this recipe.

How to do it...

Follow these steps to create a local Subversion version control repository:

1. On Windows, visit the **TortoiseSVN** website (`https://tortoisesvn.net/downloads.html`) to download and install the free Subversion client for your version of Windows (32-bit or 64-bit).
2. On macOS, you can download and install the Subversion binaries from **Wandisco** (`https://www.wandisco.com/source-code-management/subversion#osx`) or use **Homebrew** (`https://brew.sh/`) to easily add them to your computer.

> **Tip**
> While there are tools that offer a graphical user interface for Subversion, this recipe focuses on the creation of a local repository using Subversion's command-line tools to be more platform-agnostic.

3. Create a new folder on your hard drive that will host your local Subversion repository (for example, `c:\WPSVN`).
4. Open a **Command Prompt** or **Terminal** and navigate to the newly created directory.
5. Use the following command to create the required file structure in the target directory:

```
svnadmin create c:\WPSVN
```

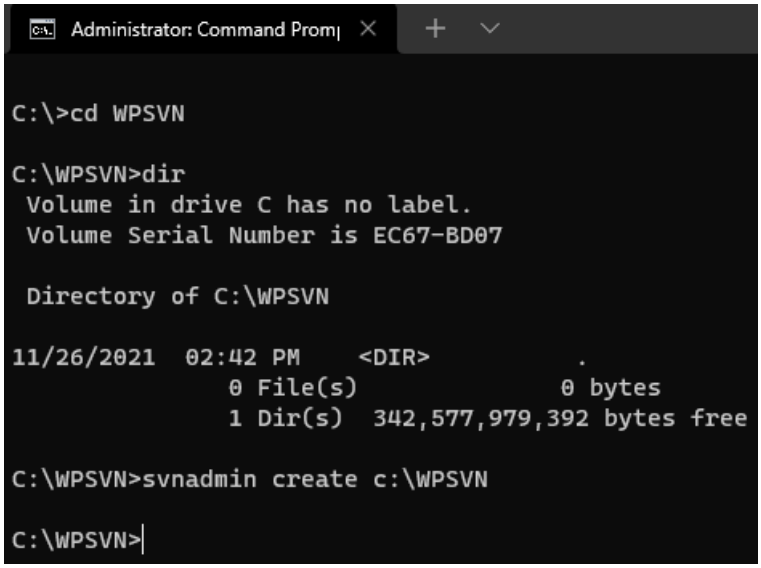


Figure 1.4 – Creating a local Subversion repository with command-line tools

6. Use the `cd` command to navigate to the `wp-content/plugins` directory of your local WordPress installation, making sure to use double quotes around the path if it contains any spaces. For example, if you are using the Local software for your development site on a Windows computer, the command would be as follows, with `ylefe` replaced with your own user directory name:

```
cd "C:\Users\ylefe\Local Sites\plugin-development-site\
app\public\wp-content\plugins"
```

7. Use the following command to import all files from the WordPress plugin folder into your local Subversion repository:

```
svn import -m "First import of complete WordPress plugin
directory to repo." file:///c:/WPSVN
```

8. Enter the following command to check out all the files from the repository and force the local files to be overwritten:

```
svn checkout --force file:///c:/WPSVN .
```

Figure 1.5 – Importing and checking out all WordPress plugin files

9. Open the `index.php` file from your WordPress plugin directory in a text editor.
10. Edit the comment on line 2 to change it from `Silence is golden` to `Silence is everything`, and then save and close the file.
11. Create a new folder in the `plugins` directory named `chapter1` using your computer's **File Explorer/Finder** or the directory creation command:

```
mkdir chapter1
```

12. Enter the following command from the **Command Prompt** or **Terminal** to add the new directory to the repository:

```
svn add chapter1
```

13. Navigate to the `chapter1` directory using your computer's **File Explorer** or **Finder** and create a new text document named `example.txt`.

14. Back in the **Command Prompt** or **Terminal**, use the following command to see the version control of the files contained in the plugin directory:

```
svn status
```

Figure 1.6 – Version control status for all files in the plugin directory

> **Note**
>
> As we can see from the output of the `svn status` command, the Subversion software correctly identified that the `index.php` file was modified (indicated by the letter **M**) and that the `chapter1` folder is to be added to the repository (indicated by the letter **A**). However, it is unsure what to do with the `example.txt` file since we did not identify it with an `svn` command (indicated by a question mark). If we were to commit our changes to the repository at this time, the `example.txt` file would not be sent with the other changes.

15. Set the `example.txt` file to be added to the repository with the following command:

```
svn add chapter1\example.txt
```

16. Commit all changes to the local repository with the following command:

```
svn commit -m "Modified index.php, created a new
directory and added a file in new directory"
```

The following figure shows the execution of the `svn commit` command and its output.

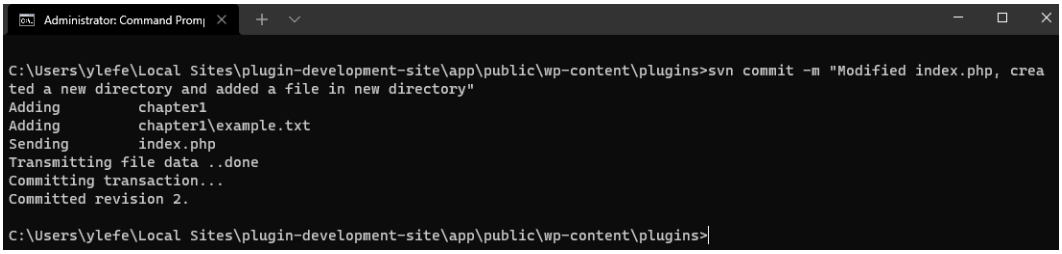


Figure 1.7 – Sending updates to the local Subversion repository

17. Delete the `chapter1\example.txt` file using your computer's **File Explorer** / **Finder** or with the appropriate command-line tool for your operating system (`del` on Windows and `rm` on macOS/Linux).
18. Back in the **Command Prompt** or **Terminal**, run the `svn status` command to see that Subversion identifies the missing file with an exclamation mark.
19. Use the following command to mark the file for deletion when the next `commit` command is executed:

```
svn delete chapter1\example.txt
```

20. Use the following command to delete the `chapter1` directory and mark it for deletion in a single step:

```
svn delete chapter1
```

How it works...

Subversion is a free, open source version control system that is designed to keep file revisions organized and backed up over the course of a project's development, as well as provide access to older versions of all files at any time. If you have ever found yourself copying a directory on your computer and giving each copy sequentially numbered names, then you've already used a primitive version control system. Subversion and other version control tools are really just a more organized and efficient method of achieving the same goal of keeping backups of code files and being able to access any old version of a file.

While the default Subversion interface is a set of command-line utilities, TortoiseSVN and many other client applications provide graphical tools to create, access, and manage local and remote repositories.

In addition to familiarizing yourself with this system for later use on `wordpress.org`, using a local Subversion repository ensures that you always have easy access to older versions of your plugin files. This can be very useful in case a code change breaks your work and you cannot figure out how to get back to a working state.

Using the `import` Subversion command copies all the selected files to the repository. In addition to storing the files themselves, Subversion identifies each file with a revision number and an optional import message. The revision number is generated by Subversion and incremented every time a group of files is added. It is especially useful when searching through a repository's history.

Performing a `checkout` operation takes copies of all files from the repository and places them in the target directory. It also creates a `.svn` sub-directory that contains information on the address of the source repository, along with original versions of all the files that were checked out. These original files are used by Subversion to determine when changes have been made to any file relative to its contents when they were last committed, checked out, or updated. It might seem a bit redundant to have an original copy of all the files in the `.svn` folder when our repository is locally hosted. However, this is the same functionality that allows Subversion to identify file changes when working on a remote repository, even when your computer is not connected to the internet.

When the `commit` operation is performed, new files are added to the repository, modified files are uploaded and stored next to their previous versions, while deleted files are tagged as no longer being part of the current project version. While some of these behaviors might seem strange, it's by preserving previous versions of files and even keeping files that are no longer part of a project that Subversion is able to let us navigate through a project's entire history or bring back past versions of files.

There's more...

Using a Subversion client with a graphical user interface can greatly facilitate some of these tasks. For example, the version control status of files can be seen overlaid on top of the file and folder icons right in Windows File Explorer when TortoiseSVN (`https://tortoisesvn.net/`) is installed.

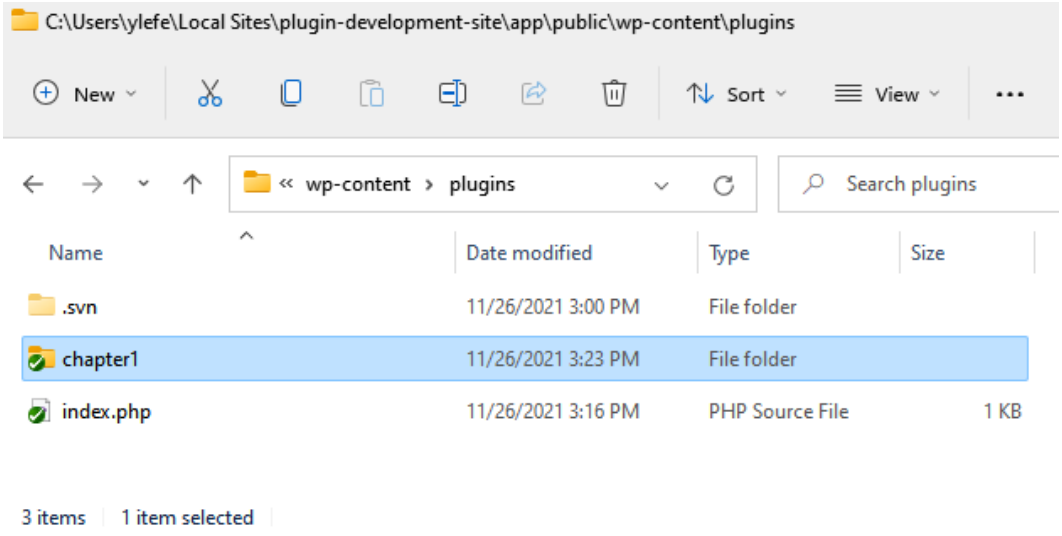


Figure 1.8 – TortoiseSVN integration in Windows File Explorer

GUI-based Subversion clients also include interactive menus for all of the commands covered in this chapter, but the actual steps to use them vary greatly from one client to the next. You should look for these graphical administration capabilities when searching for a Subversion client. For example, SmartSVN (`https://smartsvn.com`) offers free and paid versions of its client for Windows, macOS, and Linux, while Versions (`https://versionsapp.com/`) and Cornerstone (`https://cornerstone.assembla.com/`) are two popular paid applications on macOS.

Other version control systems

While Subversion is easy to learn and is the system that is used by WordPress on its official plugin repository, other version control systems, such as Git (`https://git-scm.com/`) and Mercurial (`https://mercurial-scm.org/`), are very popular in the open source development community and could also be considered to manage your plugin code. It's even possible to use a combination of version control systems with your code base. For example, you could use Subversion to commit to the official WordPress repository, but also use Git to commit the same code to a popular platform such as GitHub (`https://github.com/`), which makes it easy for your users to submit pull requests.

Viewing the differences in modified files

By using the `svn diff <filename>` command on any modified file, Subversion will bring up its built-in file difference viewer tool, highlighting the parts that are different between the last version of the file in the repository and the current version of this file. This allows users to see what changed at a glance and be sure that no code was modified inadvertently. It's also possible to display the results using a graphical diff tool, as long as the application command is in your system path, as shown in the following example:

```
svn diff index.php --diff-cmd winmerge
```

Updating files to the latest repository version

If you are the only person committing files to a repository and you are working on a single computer, then you will never need to use the `svn update` command. This function is designed to compare your local files with the repository and check whether new files or new revisions are available in the repository that are not present locally. It will then apply all the necessary changes to the local versions of these files. Remember to use the `svn update` command regularly if you are working in a team environment or are developing a project across multiple computers.

Reverting uncommitted file changes

Until a file is committed to a repository, it's possible to reverse all the changes made to it since the last checkout, update, or committal by using the `svn revert <filename>` command. This can be useful if you made changes to the code that broke its functionality and want to get back to a known good state.

Viewing file history

As multiple versions of files are committed to a repository over time, Subversion keeps track of all the versions of these files along with the messages that were associated with each commit operation. The `svn log <filename>` command allows you to see a full history of changes made to one or more files.

Installing a dedicated code editor/text editor

Most operating systems provide a built-in text editor. While it is possible to create WordPress plugins using such a simple tool, it is highly recommended to install a dedicated code editor on your computer to simplify your plugin development work.

Getting ready

Of course, not all code editors are equal. Here are some of the features that you should look for when selecting a code editing application:

- PHP syntax highlighting
- Completion of PHP function names
- Ability to search across multiple files
- Ability to highlight all instances of search keyword(s) or selected text
- Line numbering
- Ability to resize the editor text or specify a replacement font
- Possibility of opening multiple files simultaneously

The following editors contain most or all of these key features. Most are free tools, but some are paid applications.

Cross-platform:

- Visual Studio Code (`https://code.visualstudio.com`)
- Sublime Text (`https://www.sublimetext.com`)

On Windows:

- Notepad++ (`https://notepad-plus-plus.org/`)

On macOS:

- TextMate (`https://macromates.com`)

On Linux:

- Screem (`http://www.screem.org/`)

How to do it...

Follow these steps to install Visual Studio Code and learn about some of its core functions:

1. Visit the Vortex Studio Code site (`https://code.visualstudio.com`) and download the installation package for your platform of choice.
2. Run the installation program for the editor and select the default settings.
3. Launch the code editor.

4. Connect to the administration section of your WordPress development site and click on the **Plugins** menu item.
5. Check whether you have the **Hello Dolly** plugin from **Matt Mullenweg** installed. If not, go to the **Add New** section, search for it, and then click on **Install Now**.
6. Back in the code editor, open the `hello.php` file from the `plugins` or `plugins/hello-dolly` directory of your local WordPress installation. You will see that different parts of the code are displayed in different colors based on the type of code.
7. Double-click on a word to select it (for example, `Hello`). You will see any other instance of that same word highlighted across the file contents.
8. Scroll down and hover your mouse cursor over the standard `explode` PHP function to see its definition and list of arguments.

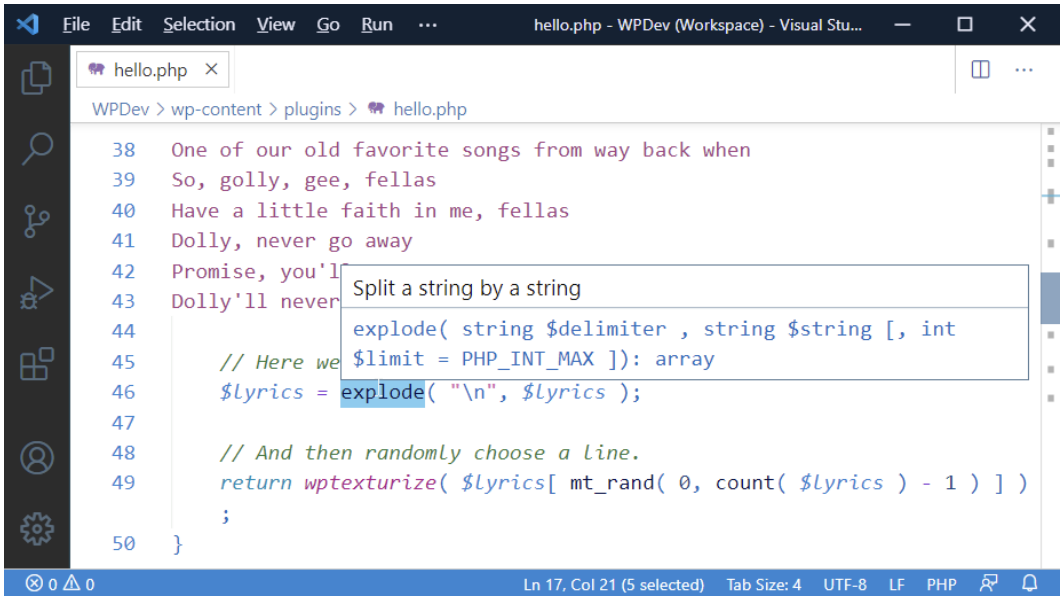


Figure 1.9 – Visual Studio Code interface with PHP function data

How it works...

Code editors have built-in parsers that enable them to identify the parts of the code that are comments, PHP language functions, text strings, and a variety of other elements. Having these elements colored on the screen makes it much easier to read through code and to see that a function's name is not spelled correctly, or to quickly identify comments.

Another functionality that is crucial when developing plugins for WordPress is the ability to see line numbers in the editor. This function comes in handy, especially when PHP code errors come up, since the filename and line of code that was being processed at the time of the error are normally displayed. In most code editors, the developer can either scroll to the specific line or enter the line number in a quick **Go To** dialog box (*Ctrl+G* in Visual Studio Code) to jump to that line right away.

There's more...

Just like WordPress, **Visual Studio Code** has a rich plugin environment that allows its users to transform it into a complete development environment. The following extensions, all freely available from the built-in **Marketplace**, are highly recommended when developing WordPress plugins:

- **PHP Extension Pack** by Felix Becker: Includes the **PHP Debug** extension to connect Visual Studio Code to an XDEBUG-enabled web server and be able to step through every line of code and set breakpoints to facilitate debugging. Also contains the **PHP IntelliSense** extension to augment the editor's autocompletion capabilities.
- **SVN** by Chris Johnson: Integrates **Subversion** version control functionality in the editor. As you modify the code, you'll be able to see modified lines in the editor margins. This extension will also identify all modified files and allow you to perform all interactions with your version control server right from the editor.
- **WordPress Hooks Intellisense** by John Billion: Adds the completion of WordPress API functions as you write your plugin code.

2
Plugin Framework Basics

From its very first beginnings, WordPress has always been designed as a very open platform. This openness has been exemplified not only through its open source licensing and distribution model but also its open plugin architecture, providing developers with the ability to deliver an even richer experience to its users.

While a basic WordPress installation provides a great amount of functionality that continues to expand from one release to the next, users often need to add one or more features to make it the perfect website management system to bring their project to life. This is where plugins come into play. They can fill this gap by augmenting or manipulating virtually any aspect of a WordPress website's display and administrative tasks.

Just like WordPress, plugins are written in the PHP programming language, which is structurally similar to more traditional languages such as C and C++. This code is stored in plain ASCII text files that are read and executed on a web server when pages are requested to be displayed. The secret ingredient that enables plugins to have such great power in WordPress is the inclusion of callback mechanisms, called **hooks**, throughout the platform's source code. These hooks come in two flavors, called **action** and **filter** hooks, which allow plugins to add content to a site and modify data before it is displayed, respectively. Whether it's rendering a site's front page, a single article, or its administration pages, WordPress has thousands of entry points where custom functions can be executed.

Beyond their ability to augment WordPress functionality, a side benefit of plugins is that most functionalities they add to a site are independent of the active theme. Therefore, users who like to change their theme frequently don't have to worry about manually adding back custom elements to their new themes when they make a switch.

This chapter explains the difference between action and filter hooks and shows how to use them to write a first set of plugins that will range in functionality, from adding information to a page header to defining new custom **shortcodes**.

In this chapter, we will cover the following topics:

- Creating a plugin file and header
- Adding output content to page headers using plugin actions
- Using WordPress path utility functions to load external files and images
- Modifying the site generator meta tag using plugin filters
- Adding text after each item's content using plugin filters
- Inserting link tracking code in the page body using plugin filters
- Troubleshooting coding errors and printing variable content
- Creating a new simple shortcode
- Creating a new shortcode with parameters
- Creating a new enclosing shortcode
- Loading a style sheet to format plugin output
- Writing plugins using object-oriented PHP

Technical requirements

You can find complete code examples on GitHub in the following repository: `https://github.com/PacktPublishing/WordPress-Plugin-Development-Cookbook-Third-Edition`.

Creating a plugin file and header

The first step of creating a plugin is to create a directory and a PHP file in the WordPress plugins directory and add the necessary information to have it recognized by the system. This first recipe shows you how to create a basic plugin file for WordPress and how to activate this new extension from the administration interface.

Getting ready

You should have access to a WordPress development environment, either on your local computer or a remote server, where you will be able to load your new plugin files (see *Chapter 1*, *Preparing a Local Development Environment*).

How to do it...

Follow these steps to create your first plugin header and activate the plugin in WordPress:

1. Navigate to the WordPress `plugins` directory (`wp-content/plugins`) of your development installation.
2. Create a new subdirectory called `ch2-plugin-header` within the plugin directory.
3. Navigate to this directory and create a new text file called `ch2-plugin-header.php`.
4. Open the new file in a code editor and add the following code:

```
<?php
/*
Plugin Name: Chapter 2 - Plugin Header
Plugin URI:
Description: Declares a plugin that will be visible in
the WordPress admin interface
Version: 1.0
Author: Yannick Lefebvre
Author URI: http://ylefebvre.ca
License: GPLv2
*/
```

> **Note**
> While the `Description` text is shown on two separate lines in the code example, it should be entered as a single line in your code file to be completely displayed in the WordPress **Plugins** list.

5. Save and close the new file.
6. Log in to the administration page of your development WordPress installation.

7. Click on **Plugins** in the left-hand navigation menu to show a list of all installed plugins. You should see your new plugin listed next to the two default ones that come pre-packaged with WordPress:

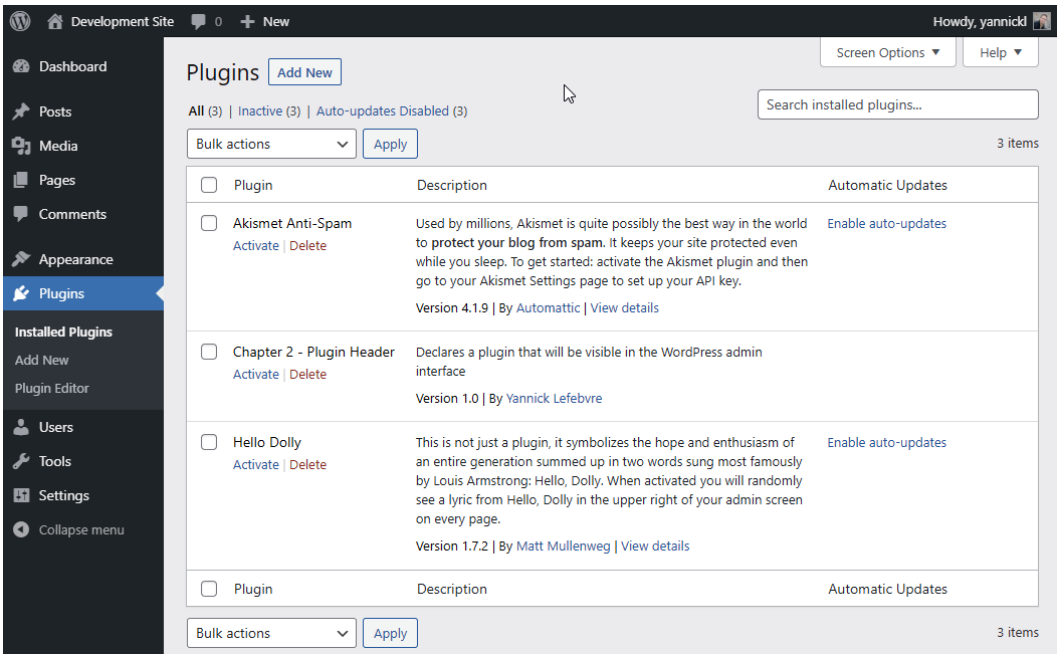


Figure 2.1 – The WordPress Plugins page showing the newly created plugin

> **Note**
>
> The list of default plugins may vary depending on how you built your development environment. You will see **Akismet** and **Hello Dolly** if you installed WordPress manually but might not see them if you are running a local development tool such as **Local**.

8. Enable the plugin by clicking on the **Activate** link under its name. You will see that the background color of your new plugin changes to indicate that it has been activated, along with a message specifying that the activation was successful.

How it works...

Plugin files can either be located directly in the `wp-content/plugins` directory or in a subdirectory under this location, with most following this second approach. When you access the installed plugins list in the administration interface, WordPress scans the entire `plugins` directory, looking for PHP files that contain comments following the format specified in this recipe. In some cases, there can be more than one PHP file containing plugin header data in any of these directories, and each of them will show up as a separate entry in the **Plugins** list.

Taking a closer look at the code that we entered in the file, the first line of the plugin file (`<?php`) is a tag that identifies the beginning of the PHP code that will be analyzed and executed by the PHP interpreter. Optionally, we could include a closing PHP tag (`?>`) at the end of the file. However, most PHP developers omit the closing tag, since having any spaces after that tag will cause warnings to be displayed by the interpreter.

> **Tip**
> To ensure compatibility with most WordPress installations, it is important to use the complete `<?php` open tag syntax in your plugin code instead of the `<?` shorthand version. Not all PHP installations are configured to support the shorthand version, and many users don't have access to change this type of configuration on their server.

The second and last lines (`/*` and `*/`) indicate that the text between these special delimiters should be considered as comments. Finally, each line within the comment contains a specific label, indicating the type of information that follows it. When this information is found, WordPress retrieves data about the plugin and adds it to the list.

When a plugin is activated, WordPress validates the file's content to be sure that it is valid PHP code. It will then execute this content every time any page is rendered on the site, whether that page is front-facing or a backend administration section. For this reason, it is preferable to activate plugins only when they are in use, to avoid site slowdowns.

Of course, at this point, our new plugin does not add or modify any functionality in our WordPress installation, since it does not contain any code, but this is still an important first step.

See also

- The *Installing a local web server on your computer* recipe in *Chapter 1*, *Preparing a Local Development Environment*

Adding output content to page headers using plugin actions

A common action performed by plugins is to add extra content to the header of visitor-facing pages generated by WordPress. This recipe shows you how to register an action hook function to add such additional content. To make this example more concrete, we will use the Google Analytics page header JavaScript code that so many people use to get good page view analytics for their site.

How to do it...

Follow these steps to learn how to register custom code to add content to a site's page headers:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch2-page-header-output`.
3. Navigate to this directory and create a new text file called `ch2-page-header-output.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 2 - Page Header Output`.
5. Add the following line of code at the end of the file to register a function that will be called when WordPress renders the page header:

```
add_action( 'wp_head', 'ch2pho_page_header_output' );
```

6. Add the following code section to provide an implementation for the `ch2pho_page_header_output` function:

```
function ch2pho_page_header_output() { ?>
    <script>
    (function(i,s,o,g,r,a,m){i['GoogleAnalyticsObject']=r;
    i[r]=i[r]||function(){
    (i[r].q=i[r].q||[]).push(arguments)},
    i[r].l=1*new Date();
    a=s.createElement(o),
    m=s.getElementsByTagName(o)[0];a.async=1;a.src=g;
    m.parentNode.insertBefore(a,m)})
```

```
        (window,document,'script',
        'https://www.google-analytics.com/analytics.js',
        'ga');

        ga('create', 'UA-0000000-0', 'auto');
        ga('send', 'pageview');
        </script>
  <?php }
```

7. Save and close the plugin file.
8. Log in to the administration page of your development WordPress installation.
9. Click on **Plugins** in the left-hand navigation menu.
10. **Activate** your new plugin.
11. Navigate to your website's front page and use your browser's **View Page Source** function to see the HTML source code for the site. Note that the exact name of this browser function will be slightly different based on which browser you are using. Search for `UA-0000000-0` to see that all of the code contained between the two curled brackets of your new function is in your website's header:

```
106  <script>
107  (function(i,s,o,g,r,a,m){i['GoogleAnalyticsObject']=r;
108  i[r]=i[r]||function(){
109  (i[r].q=i[r].q||[]).push(arguments)},
110  i[r].l=1*new Date();
111  a=s.createElement(o),
112  m=s.getElementsByTagName(o)[0];a.async=1;a.src=g;
113  m.parentNode.insertBefore(a,m)})
114  (window,document,'script',
115  'https://www.google-analytics.com/analytics.js',
116  'ga');
117
118  ga('create', 'UA-0000000-0', 'auto');
119  ga('send', 'pageview');
120  </script>
```

Figure 2.2 – Google Analytics code in the page source view

> **Note**
> If you are copying and pasting code from a digital version of this book, you will lose the original code indentation and should correct it in your code editor.

How it works...

The `add_action` function is used to associate custom plugin code to one of the two types of WordPress hooks, the action hook. As mentioned briefly in this chapter's introduction, hooks are the enabling functionality that make plugins possible in WordPress. Action hooks enable the execution of additional code at specific points when either public-facing or administration pages are prepared to be displayed. This code usually adds content to a site or changes the way a given action is performed.

In this recipe, the first line of code that we wrote registers a function named `ch2pho_page_header_output` with an action hook called `wp_head`. This action is one among more than 2,500 action hooks that are available in current versions of WordPress, and it allows any registered function to output additional content to the page header. Since all echoed content will be displayed, we can write our callback function very simply by placing `?>` and `<?php` tags around the Google Analytics code. This will tell PHP to display all the content that is within that function's body, as opposed to interpreting it.

As you may have noticed, the current code is not very flexible, since you would need to hardcode your Google Analytics account number in the plugin code for it to function properly. The creation of a configuration panel and user settings in *Chapter 3*, *User Settings and Administration Pages*, will provide a way to configure such information to make our plugins more flexible.

Now, to fully understand its syntax, let's take a closer look at the complete `add_action` function:

```
add_action( 'hook_name', 'your_function_name', [priority],
            [accepted_args] );
```

The first parameter, the hook name, indicates the name of the WordPress hook with which we want our custom function to be associated. This name must be accurately spelled; otherwise, our function will not be called and no error message will be displayed.

The second parameter is the name of the plugin function that will be called to perform an action. This function can have any name, with the only condition being that this name must be unique to avoid conflicts with function names from other plugins or from the core WordPress code. In this recipe, the function name starts with an acronym representing the name of the plugin, making it much more unique.

The `priority` parameter is optional, as indicated by the square brackets, and has a default value of `10`. It indicates the execution priority of this plugin relative to other plugin functions that hook into the same action, with a lower number indicating a higher priority. Any plugin can register one or more functions with an action hook using the `add_action` function. As it is rendering web pages, WordPress keeps a queue of all entries and calls them at the appropriate moment. It is interesting to note that the hook mechanism is also used by WordPress itself, as it regularly calls the `add_action` function in its own code to register functions to be called at the right time. If you realize that you need your function to be called before or after other plugins that are registering with the same hook, change the value of the `priority` parameter.

The last parameter of the `add_action` function, `accepted_args`, is also optional, has a default value of `1`, and should be assigned a number when set. It should only be set to a different value for some particular hooks where more than one parameter will be passed to the registered function. Some of these hooks will be covered in later recipes.

There's more...

Finding the right hooks to register plugin functions is a large part of WordPress plugin development. Fortunately, there are a number of ways to get information on existing hooks and to learn when they get called during the WordPress page generation process.

Action hooks online listings

The WordPress Codex (`https://codex.wordpress.org/`) and WordPress Code Reference (`https://developer.wordpress.org/reference/`) are documentation sites that contain a multitude of information that is useful to users and developers alike. When it comes to action hooks, the Codex contains information on the most commonly used hooks, with basic descriptions indicating how they can be used; it can be found here: `https://codex.wordpress.org/Plugin_API/Action_Reference`. However, this is not a complete listing.

There are many third-party sites that parse the WordPress source code and provide their own hook listings (for example, `http://hookr.io`). While hooks are not as eloquently documented in these types of raw listings, they do provide basic information on their names and where they are called, as WordPress generates pages for visitors and administrators. These details can often be enough to find a hook based on the functionality that you are trying to implement.

Searching for hooks in the WordPress source code

Since WordPress is open source, another way to find information about hooks is to search directly within its code. For every action hook that accepts user functions, you will see a call to the `do_action` function to execute all the registered items. As you can see, the function takes two or more arguments, with the second one(s) being optional:

```
do_action( 'tag', [$arg] );
```

For the example shown in this recipe, a search for `do_action( 'wp_head' )` reveals that it is the only function that is called when a WordPress theme makes a call to the `wp_head()` function in its header file:

```
do_action( 'wp_head' );
```

See also

- The *Creating a plugin file and header* recipe

Using WordPress path utility functions to load external files and images

On occasion, plugins need to refer to external files (for example, images, JavaScript, or jQuery script files) that are stored in the plugin directory. Since users are free to rename a plugin's folder or even install plugin files straight into the WordPress `plugins` directory, paths to any external files must be built dynamically based on the actual plugin location. Thankfully, a number of utility functions are present to simplify this task.

How to do it...

Follow these steps to create a simple plugin that will add a favicon meta tag to a website's header, pointing to an image file located in the plugin's directory:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch2-favicon`.
3. Use a web service, such as `https://favicongrabber.com/`, to retrieve a website's favicon (for example, `https://www.packtpub.com`) and store it in the `ch2-favicon` directory with its default name (`favicon.ico`).

4. Navigate to the `ch2-favicon` directory and create a new text file called `ch2-favicon.php`.
5. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 2 - Favicon`.
6. Add the following line of code to register a function that will be called when WordPress renders the page header:

```
add_action( 'wp_head', 'ch2fi_page_header_output' );
```

7. Add the following code section to provide an implementation for the `ch2fi_page_header_output` function:

```
function ch2fi_page_header_output() {
    $site_icon_url = get_site_icon_url();
    if ( !empty( $site_icon_url ) ) {
        wp_site_icon();
    } else {
        $icon = plugins_url( 'favicon.ico',
                                __FILE__ );
    ?>

    <link rel="shortcut icon"
          href="<?php echo esc_url( $icon ); ?>" />
    <?php }
}
```

8. Save and close the plugin file.
9. Log in to the administration page of your development WordPress installation.
10. Click on **Plugins** in the left-hand navigation menu.
11. **Activate** your new plugin.

12. Navigate to your website's front page and refresh it to see that the icon file that you assigned through your plugin code now appears in your browser's address bar, title bar, or navigation tab, depending on your preferred browser. The following screenshot shows how the favicon file is rendered in Microsoft Edge, Google Chrome, and Mozilla Firefox, from top to bottom:

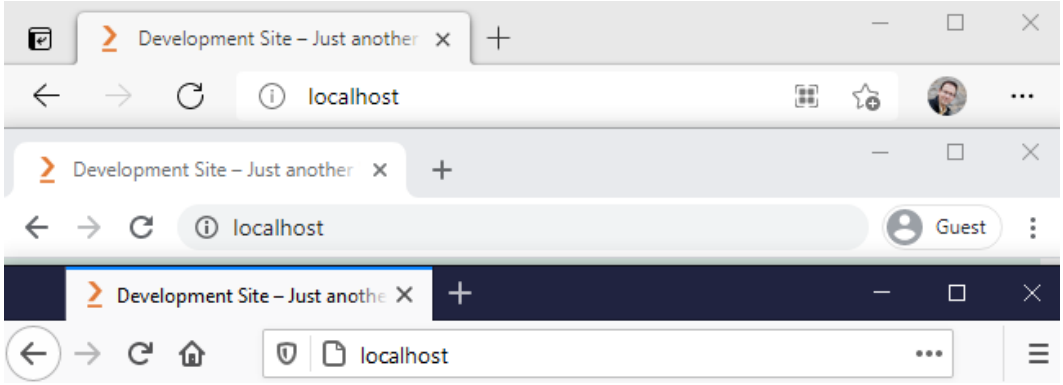


Figure 2.3 – The website favicon appearing in Microsoft Edge, Google Chrome, and Mozilla Firefox

13. Go to the **Appearance** section of your development site's **Dashboard** and **Activate** a theme other than **Twenty Twenty-Two** (for example, **Twenty Twenty-One** or **Twenty Twenty**). You may need to install one of these themes if they are not present in your development environment.
14. Select the **Customize** submenu under the **Appearance** menu.
15. Under **Site Identity**, assign a square image that is at least 512 x 512 pixels in dimension as the **Site Icon**; then, click the **Publish** button at the top of the customizer.
16. Refresh your website to see that the newly assigned site icon image is now displayed instead of the `favicon.ico` file.
17. Go back to the **Appearance** section and **Activate** the **Twenty Twenty-Two** theme.

How it works...

The `plugins_url` utility function, used in conjunction with the `__FILE__` PHP constant and the name of our favicon file, allows us to quickly get the URL of this file located in our plugin directory. Once we have it, we can print out the appropriate HTML command to notify browsers of the location of this file.

The `plugins_url` function can be called with or without parameters. In the first case, it builds a URL by appending the path or filename found in the first parameter to the location of the file specified in the second argument. In the second situation, it simply returns the location of the plugin directory:

```
plugins_url( $path, $plugin );
```

Before we display our plugin's favicon file, we also call the `get_site_icon_url` function to see whether the user has already assigned a site icon using the WordPress customizer. If that is the case, we give priority to that icon and display it using the `wp_site_icon()` function.

> **Note**
> The **Twenty Twenty-Two** theme found in WordPress 5.9 is built using a new theme creation technique and does not have a customizer section. This is why we change our site theme to demonstrate how our code takes into consideration site icons assigned in the customizer. The majority of themes available today have customizer sections, but this may change as WordPress evolves. The **Twenty Twenty-Two** theme may also add a customizer section in future releases.

There's more...

The `plugins_url` function is one of the many functions that can be used in plugins to help find the location of files in a WordPress installation. Other useful functions include the following:

- `get_theme_root()`: Returns the address of the theme installation directory
- `get_template_directory_uri()`: Retrieves the URL to the current theme's files
- `admin_url()`: Provides the address of the WordPress administrative pages
- `content_url()`: Indicates where the `wp-content` directory can be found
- `site_url()` and `home_url()`: Returns the site address
- `includes_url()`: Provides the location of WordPress include files
- `wp_upload_dir()`: Indicates the directory where user-uploaded files are stored

See also

- The *Creating a plugin file and header* recipe
- The *Adding output content to page headers using plugin actions* recipe

Modifying the site generator meta tag using plugin filters

Beyond adding functionality or content to a site, the other major task commonly performed by plugins is to augment, modify, or reduce information before it is displayed on the screen. This is done by using WordPress filter hooks, which allow plugins to register a custom function through the WordPress API to be executed when content is prepared before it is sent to the browser.

How to do it...

Follow these steps to implement your first filter callback function that modifies the contents of the generator meta tag found in a site's HTML source code:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch2-generator-filter`.
3. Navigate to this directory and create a new text file called `ch2-generator-filter.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 2 - Generator Filter`.
5. Add the following line of code to register a function that will be called when WordPress is preparing data to output the generator meta tag as part of the page header:

```
add_filter( 'the_generator', 'ch2gf_generator_filter',
            10, 2 );
```

6. Add the following code section at the end of the file to provide an implementation for the `ch2gf_generator_filter` function:

```
function ch2gf_generator_filter ( $html, $type ) {
    if ( $type == 'xhtml' ) {
        $html = preg_replace( '("WordPress.*?")',
            '"Yannick Lefebvre"', $html );
    }
```

```
        return $html;
    }
```

7. Save and close the plugin file.
8. Log in to the administration page of your development WordPress installation.
9. Click on **Plugins** in the left-hand navigation menu.
10. **Activate** your new plugin.
11. Use a web browser to visit your website and display the page source. Searching for the `generator` keyword will reveal that the content generator meta tag has been modified and now reads as follows:

```
<meta name="generator" content="Yannick Lefebvre" />
```

How it works...

The `add_filter` function is used to associate a custom plugin function to the second type of WordPress hook, the filter hook. Filter hooks give plugins the chance to augment, modify, delete, or completely replace information while WordPress is executed. To enable this, filter functions are sent data that can be modified as a function parameter. They must **return** the resulting set of data to WordPress once they have finished making the changes.

Unlike action hooks, filter functions must not output any text or HTML code, since they are executed while output is being prepared, and that would likely result in output showing up in unexpected places in the site layout. Instead, they should return the filtered data.

Taking a closer look at the parameters of the `add_filter` function, we can see that it is very similar to the `add_action` function that we saw in the last few recipes:

```
add_filter( 'hook_name', 'your_function_name', [priority],
            [accepted_args] );
```

The first parameter, the hook name, indicates the name of the WordPress hook that we want our custom function to be associated with. This name must be accurately spelled; otherwise, our function will not be called, and no error message will be displayed.

The second parameter is the name of the plugin function that will be called to filter data. This function can have any name, with the only condition being that this name must be unique to avoid conflicting with functions from other plugins or from the WordPress code.

The `priority` parameter is optional, as indicated by the square brackets, and has a default value of `10`. It indicates the execution priority of this callback relative to other callbacks that are registered by WordPress itself and by other plugins, with a lower number indicating a higher priority.

The last parameter of the function, `accepted_args`, has a default value of `1` and indicates how many parameters will be sent to your custom filter function. It should only be set to higher values when you are using filters that will send multiple parameters, as shown in this recipe with the `$html` and `$type` arguments.

There's more...

Beyond demonstrating how to change the site generator name, this plugin also showed how to use an advanced PHP function to perform the actual text replacement. We will take a closer look at this function and also explore resources to learn more about filter hooks.

The preg_replace function

The `preg_replace` function is a PHP function that can be used to perform a search and replace operation within a string based on a search pattern. We use this function rather than the simpler `str_replace` function, since we want to find and replace both the WordPress keyword and its associated version number, which changes with every version.

Filter hooks online listings and the apply_filters function

Similar to action hooks, information about commonly used filter hooks can be found at the WordPress Codex (`https://codex.wordpress.org/Plugin_API/Filter_Reference`) or on sites that provide raw function lists (for example, `http://hookr.io`).

It is also possible to learn about filter hooks by searching for occurrences of the `apply_filters` function in the WordPress code. As can be seen in the following code, this function has a variable number of arguments, with the first one being the name of the filter hook, the second representing the value that the registered function will be able to modify, and the remaining optional parameters containing additional data that may be useful in the implementation of the filter function:

```
apply_filters( $tag, $value, [$var] ... );
```

For the example shown in this recipe, a search for `apply_filters( 'the_generator'` in the WordPress code reveals that it is called within the `the_generator` template function:

```
echo apply_filters( 'the_generator',
                    get_the_generator( $type ), $type );
```

See also

- The *Creating a plugin file and header* recipe

Adding text after each item's content using plugin filters

After making a number of changes to the page header, the generator meta tag, and the site favicon, this recipe takes a more active role by adding a link to each post or page, allowing visitors to email a link to the article that they are currently viewing. This functionality is implemented using a filter hook attached to the page and post content. This allows our custom function to append custom output code to all entries that get displayed on the screen.

How to do it...

Follow these steps to create a plugin that adds an email link at the end of all posts and pages:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch2-email-page-link`.
3. Navigate to this directory and create a new text file called `ch2-email-page-link.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 2 - Email Page Link`.
5. Visit an icon download website, such as `https://iconarchive.com`, and download an email icon with a small resolution (32 x 32 pixels) in PNG format to the `ch2-email-page-link` directory, giving it the name `mailicon.png`.

6. Add the following line of code at the end of the file to register a function that will be called when WordPress is preparing data to display the content of a post or page:

```
add_filter( 'the_content',
            'ch2epl_email_page_filter' );
```

7. Add the following code section to provide an implementation for the `ch2epl_email_page_filter` function:

```
function ch2epl_email_page_filter ( $the_content ) {
    // build url to mail message icon
    $mail_icon_url = plugins_url( 'mailicon.png',
                                  __FILE__ );

    // Set value of $new_content to previous content
    $new_content = $the_content;

    // Append image with mailto link after content,
    // including the item title and permanent URL
    $new_content .= '<div class="email_link">';
    $new_content .= '<a title="Email article link"';
    $new_content .=
        'href="mailto:someone@somewhere.com?';
    $new_content .=
        'subject=Check out this interesting ';
    $new_content .= 'article entitled ';
    $new_content .= get_the_title();
    $new_content .= '&body=Hi!%0A%0AYou might ';
    $new_content .= 'enjoy this article entitled ';
    $new_content .= get_the_title() . '.%0A%0A';
    $new_content .= get_permalink();
    $new_content .= '%0A%0AEnjoy!">';

    $new_content .= '<img alt="Email icon" src="';
```

```
        $new_content .= esc_url( $mail_icon_url );
        $new_content .= '" /></a></div>';

        // Return filtered content for display on the site
        return $new_content;
    }
```

8. Save and close the plugin file.
9. Log in to the administration page of your development WordPress installation.
10. Click on **Plugins** in the left-hand navigation menu.
11. **Activate** your new plugin.
12. Visit your website to see the new mail icon at the end of each post and page. You may need to click on an article or page to see the icon:

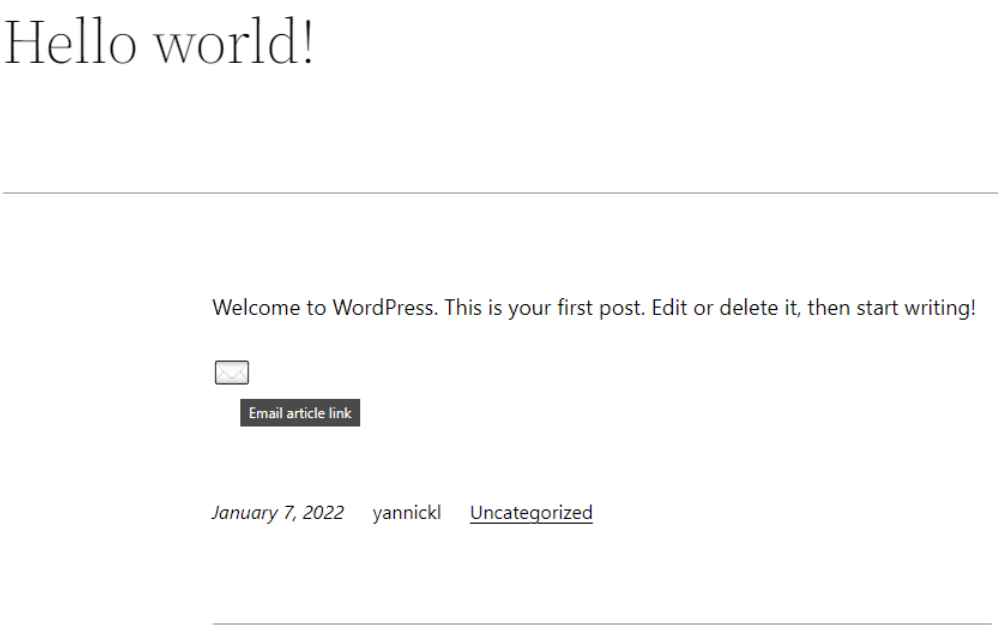


Figure 2.4 – The newly added Email article link icon

13. Click on one of the mail links:

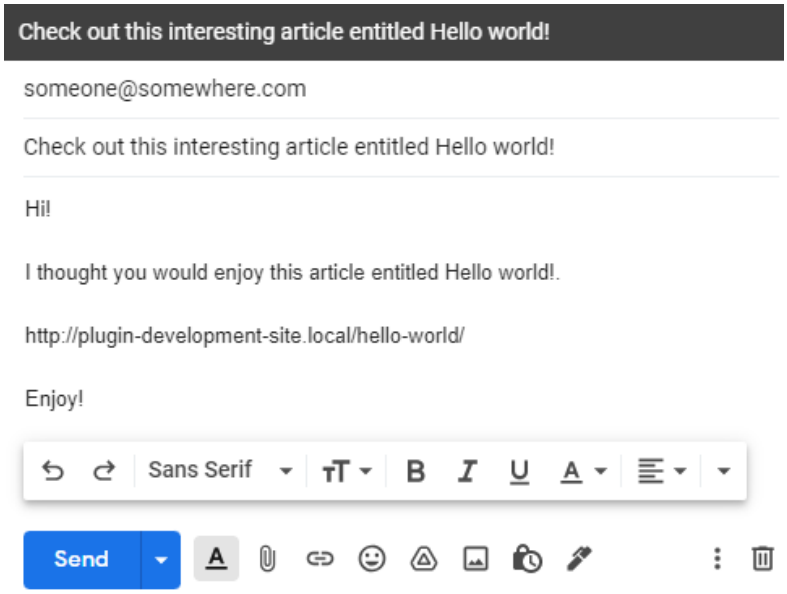


Figure 2.5 – The email generated after clicking on the link

As seen in *Figure 2.5*, your default mail client will come up with information about the item you were reading. The only information that needs to be updated is the recipient address before visitors can quickly send an email.

How it works...

Similar to the previous recipe, this plugin uses the `add_filter` function to register a custom function to be called by WordPress as it prepares an item's content to be displayed on a page. When the filter function is called, the first action that it performs is to create a URL to the email icon that was downloaded in the recipe. It then goes on to modify the original content by appending the HTML code to display a `mailto` link. The same technique can be used to create links to popular social media and link sharing sites, with simple changes to the syntax of the link. Once the new content is ready, it is returned to WordPress to be sent to any other registered filters and subsequently displayed on the site.

There's more...

This recipe also introduces a pair of useful WordPress utility functions to get access to the current item's content.

The get_the_title and get_permalink functions

While these two functions are mainly seen within theme template files, they can also be used by plugins to get easy access to the content of items that are currently being processed. More specifically, the two utility functions that are used in this recipe are as follows:

- `get_the_title()`: This function gives us quick access to the item's title.
- `get_permalink()`: This is a function that returns the item's permalink (a URL that is always associated with this post or page, even after it is no longer featured on a website's front page or blog page).

See also

- The *Creating a plugin file and header* recipe
- The *Using WordPress path utility functions to load external files and images* recipe
- The *Modifying the site generator meta tag using plugin filters* recipe

Inserting link tracking code in the page body using plugin filters

After seeing how to append text to existing content, this recipe shows you how to modify page content before it is displayed onscreen. More specifically, the following plugin expands on the Google Analytics header plugin created earlier and adds a JavaScript function to all links that are included in posts and pages to track when they are clicked by visitors.

Getting ready

You should have already followed the *Adding output content to page headers using plugin actions* recipe to have a starting point for this recipe, and the resulting plugin should be active in your development site. Alternatively, you can download the resulting code (`ch2/ch2-page-header-output/ch2-page-header-output.php`) for the book's GitHub page.

How to do it...

Follow these steps to add outbound link tracking code to your simple Google Analytics plugin:

1. Navigate to the `ch2-page-header-output` folder in the WordPress `plugins` directory of your development installation.
2. Open the `ch2-page-header-output.php` file in a code editor.
3. Add the following line after the existing code to register a function that will be called when WordPress is preparing data to display a page or post's content:

```
add_filter( 'the_content',
            'ch2lfa_link_filter_analytics' );
```

4. Add the following code section to provide an implementation for the `ch2lfa_link_filter_analytics` function:

```
function ch2lfa_link_filter_analytics ($the_content) {
    $new_content = str_replace( 'href',
        'onClick="recordOutboundLink( this );return
        false;" href',
        $the_content );
    return $new_content;
}
```

5. Add the following line of code to register a function that will be called when WordPress renders the page footer:

```
add_action( 'wp_footer',
            'ch2lfa_footer_analytics_code' );
```

6. Add the following code section to provide an implementation for the `ch2lfa_footer_analytics_code` function:

```
function ch2lfa_footer_analytics_code() { ?>
    <script type="text/javascript">
        function recordOutboundLink( link ) {
            ga( 'send', 'event', 'Outbound Links',
                'Click',
                link.href, {
                    'transport': 'beacon',
                    'hitCallback': function() {
```

```
                                    document.location = link.id;
                                }
                            } );
                        }
                    </script>
                    <?php }
```

7. Save and close the plugin file.
8. Go to the **Posts** section of the **Dashboard** and edit one of the items to add a link within the content, pointing to a location of your choice.
9. Click **Update** to save the changes to the modified post.
10. In your web browser, refresh your website to see the modified post.
11. Right-click on the new link and **Inspect** it using your browser's developer tools:

```
▼<p>
...   <a onclick="recordOutboundLink( this );return false;" href="https://www.packtpub.com">Packt
      Publishing</a> == $0
    </p>
  ▶<div class="email_link">…</div>
    ::after
  </div>
  <!-- .entry-content -->
</div>
<!-- .post-inner -->
▶<div class="section-inner">…</div>
<!-- .section-inner -->
▶<div class="comments-wrapper section-inner">…</div>
<!-- .comments-wrapper -->
</article>
<!-- .post -->
</main>
<!-- #site-content -->
▶<div class="footer-nav-widgets-wrapper header-footer-group">…</div>
<!-- .footer-nav-widgets-wrapper -->
▶<footer id="site-footer" role="contentinfo" class="header-footer-group">…</footer>
<!-- #site-footer -->
▼<script type="text/javascript">
    function recordOutboundLink( link ) {
        ga( 'send', 'event', 'Outbound Links', 'Click',
            link.href, {
                'transport': 'beacon',
                'hitCallback': function() {
                    document.location = link.href;
                }
            } );
    }

</script>
```

Figure 2.6 – Custom JavaScript code added to web links in the page output

As seen in *Figure 2.6*, the link tag now has an additional `onClick` JavaScript property with code that will be called when visitors follow it, along with the definition of the `recordOutboundLink` function in the page footer.

How it works...

The content filter function that is put in place by calling `add_filter` receives the entire content of all the posts and pages before they are rendered to the browser and is allowed to make any number of changes to this information. In this case, we are using the PHP `str_replace` function to search for any occurrence of the `href` string, which indicates a link. When the string is found, it is replaced with a call to a JavaScript function while preserving the original `href` tag.

To make this plugin complete, it also needs to provide an implementation for the JavaScript `recordOutboundLink` function. This is done by registering a custom function with the `wp_footer` action hook, which will output extra content once in the website's footer.

The resulting plugin automates many of the tasks related to tracking usage data on a website using Google Analytics.

See also

- The *Adding output content to page headers using plugin actions* recipe
- The *Adding text after each item's content using plugin filters* recipe

Troubleshooting coding errors and printing variable content

As you transcribe code segments from the pages of this book or start writing your own plugins, there is a strong chance that you will have to troubleshoot problems with your code. You might also run into trouble working with data that your plugin is meant to manipulate. This recipe shows basic techniques to identify and quickly resolve these errors while creating a plugin that will hide an item from the navigation menu for users who are not logged in to your site.

How to do it...

Follow these steps to learn how to troubleshoot problematic plugin code:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch2-nav-menu-filter`.
3. Navigate to this directory and create a new text file called `ch2-nav-menu-filter.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 2 - Nav Menu Filter`.
5. Add the following line of code to register a function that will be called when WordPress is preparing data to display the site's navigation menu:

```
add_filter( 'wp_nav_menu_objects',
            'ch2nmf_new_nav_menu_items', 10, 2 );
```

6. Add the following code section to provide an implementation for the `ch2nmf_new_nav_menu_items` function. Note that the word `functio` is mistyped on purpose at the beginning of the first line:

```
functio ch2nmf_new_nav_menu_items (
                        $sorted_menu_items, $args ) {
    print_r( $sorted_menu_items );
    return $sorted_menu_items;
}
```

> **Tip**
> If you are using a dedicated code editor, you should be able to tell that the `functio` text is not a recognized keyword, since it will not be colored as a PHP keyword and might also be underlined to indicate an error.

7. Save the plugin file and leave your code editor open.
8. Log in to the administration page of your development WordPress installation.
9. Click on **Plugins** in the left-hand navigation menu.
10. **Activate** your new plugin.

11. WordPress will display a fatal error message, indicating that the plugin could not be activated, since a parsing error was found. It also indicates the exact filename and line where the error occurred, helping to narrow down where the problem occurred. The actual format of the error message may be different based on the local development tool that you're using:

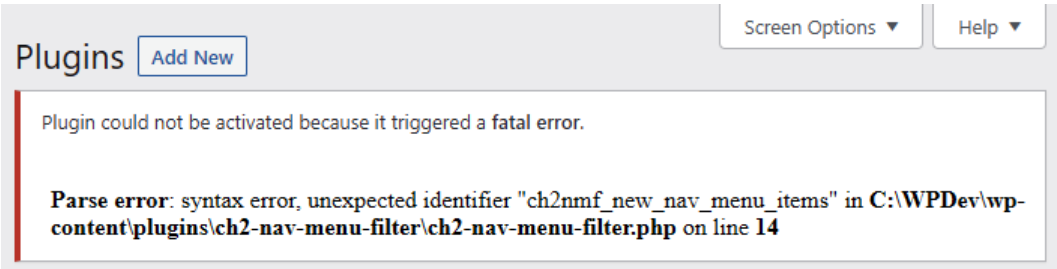


Figure 2.7 – A fatal error upon plugin activation due to a syntax error

12. Go back to your code editor, correct the spelling of the word `function`, and save the file.
13. **Activate** the plugin a second time. It should now activate correctly.
14. Back in the code editor, remove the last letter of the word `function` to re-introduce a syntax error, and save the file.
15. Visit the development site in your browser. You will now see that the entire site has been replaced with a **Parse error** message similar to the one we just saw. You may also only get a blank page or a critical error, depending on your web server configuration.
16. Go to the **Appearance** section of your development site's **Dashboard** and **Activate** a theme other than **Twenty Twenty-Two** (for example, **Twenty Twenty-One** or **Twenty Twenty**). You may need to install one of these themes if they are not present in your development environment.
17. In the WordPress **Dashboard**, navigate to the **Appearance | Menus** item.
18. If you don't have any menu defined, type `Main Menu` in the **Menu Name** field. Check the appropriate **Display location** checkboxes to assign your new menu to the theme's main navigation area. The exact name of this location will change based on the theme you're using. Click on **Create menu** to save your menu.
19. Create an additional item in your menu as a **Custom Link**, setting the **URL** to `/privatearea` and the **Link Text** to `Private Area`; then, click on **Add to Menu**:

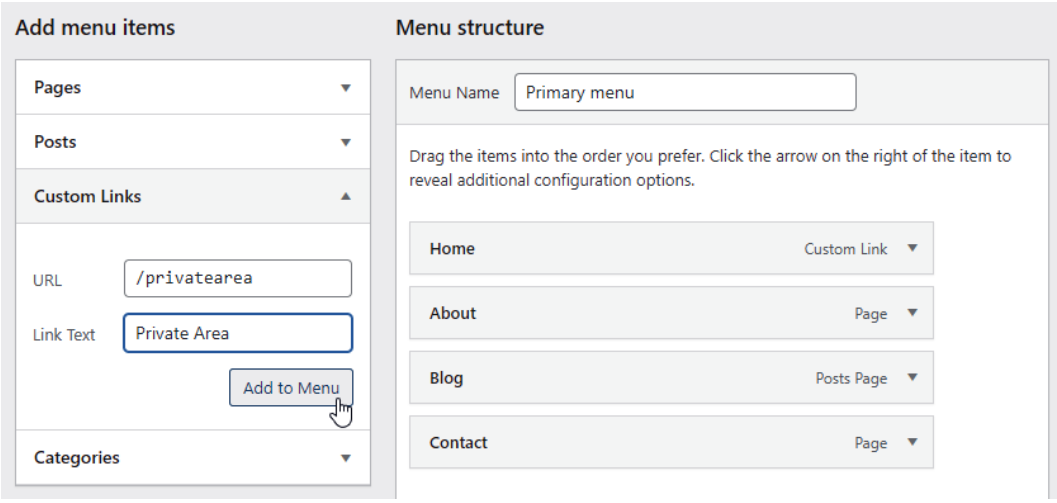


Figure 2.8 – Creating a new menu item

20. Click on the **Save Menu** button to save your changes.
21. Back in your code editor, correct the spelling error once again and refresh the page to see the website return to normal. You will also see that a lot of information is now being printed before the navigation menu. This output is generated by the `print_r` function and is meant to help us understand how the data received by our filter function is organized. Once we have a good understanding of that data, we will be able to properly make changes to this information in the next step. To view the `print_r` data in a more organized way, bring up the **View Page Source** function of your browser and search for the keyword `Array` to quickly find the menu data.
22. Back in the code editor, replace the line containing the `print_r` function call with the following code:

```
// Check if used is logged in, continue if not logged
if ( is_user_logged_in() == FALSE ) {
    // Loop through all menu items received
    // Place each item's key in $key variable
    foreach ( $sorted_menu_items as
              $key => $sorted_menu_item ) {
        // Check if menu item matches search string
        if ( 'Private Area' ==
                $sorted_menu_item->title ) {
            // Remove item from menu array if found
            // using item key
```

```
                unset( $sorted_menu_items[ $key ] );
            }
        }
    }
```

23. Refresh your website, and you will see that the large array printout has disappeared. If you are logged in as the administrator, you will also notice the **Private Area** link in your menu. Log out of your site or open an incognito session in your browser to see the page without the new menu item.
24. Go back to the **Appearance** section and **Activate** the **Twenty Twenty-Two** theme.

How it works...

As WordPress assembles a list of all available plugins to display them in the administration interface, it does not check to see whether each plugin's PHP code is valid. This check is only done when a plugin is activated. At that time, any syntax error will be caught immediately and the newly activated plugin will remain inactive, preventing a failure of the entire website.

That being said, once a plugin is activated, its code is evaluated every time WordPress renders a web page, and any subsequent code error that gets saved to the plugin file will cause the site to stop working correctly. For this reason, it is highly recommended to set up a local development environment, as discussed in *Chapter 1*, *Preparing a Local Development Environment*, to avoid affecting a live site when an inevitable error creeps up in your plugin code. To avoid potential outages on a live website, it is safer to deactivate plugins before making changes to them and then reactivate them once changes are completed. This way, all changes are validated before the plugin code is reactivated. It should be noted that, with this method, the plugin's functionality won't be available on your site while you make changes, so it is not an optimal way to modify deployed code.

Once the code is working correctly, the second part of this recipe shows us how to visualize the information received by a registered filter function. While the WordPress Codex website provides great documentation of the purpose of most available filters, it does not go into detail about the structure of the information that is sent to each filter function.

Thankfully, the PHP `print_r` function comes in very handy, since it can display the content of any variable on the screen, no matter what information is stored in the variable that it receives as an argument.

Last, but not least, the implementation of the custom filter function uses the WordPress API `is_user_logged_in()` function to see whether the person viewing the site has provided login credentials. If the return value of the function is `true`, our code goes on to parse all the menu items and remove the **Private Area** menu item.

> **Note**
>
> This is the second time in this chapter that we ask you to activate a theme other than the default WordPress 5.9 **Twenty Twenty-Two** theme. With WordPress 5.9, WordPress is introducing a new theme-building paradigm where users will be able to modify their site layout using a graphical tool. Some action and filter hooks don't work with this new theme structure in this first version. That being said, all code samples in this book work with the large majority of themes that are available today and are running on most WordPress sites.

There's more...

In addition to the debugging techniques used in this recipe, WordPress offers a number of built-in tools to facilitate plugin troubleshooting.

Built-in WordPress debugging features

While the `wp-config.php` file, located at the top of the WordPress file structure, is primarily used to store basic site configuration data, it can also be used to trigger a number of debugging features. The first of these is debug mode, which will display all PHP errors, warnings, and notices at the top of site pages. For example, having this option active will show any undefined variables that you try to access in your code along with any deprecated WordPress function. To activate this tool, change the second parameter of the line defining the `WP_DEBUG` constant from `false` to `true` in `wp_config.php`:

```
define( 'WP_DEBUG', true );
```

If the `WP_DEBUG` line does not appear in your development site's `wp-config.php` file, you can add it before the line that reads the following:

```
/* That's all, stop editing! Happy publishing. */
```

To prevent debug messages from affecting the site's layout, you can download a useful plugin called **Debug Bar** (`https://wordpress.org/plugins/debug-bar/`) to collect messages and display them in the admin bar, as shown here:

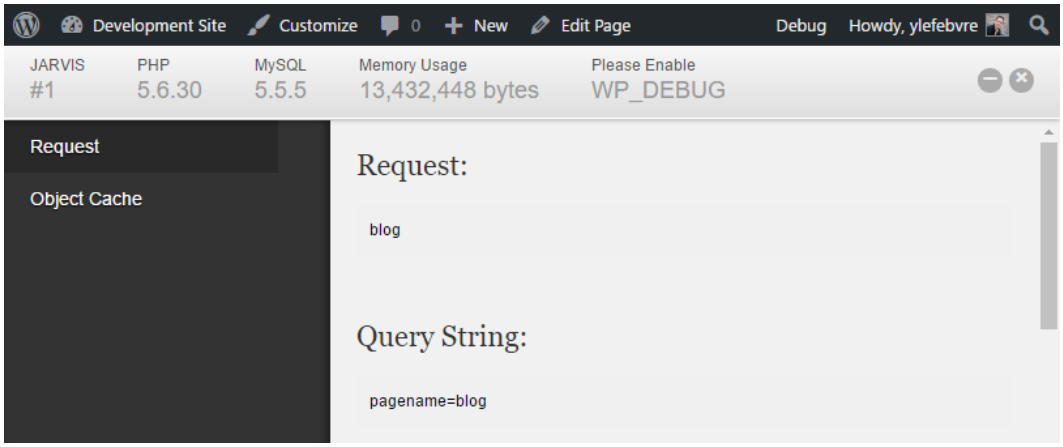


Figure 2.9 – The Debug Bar plugin

Other debugging features that can be activated from the `wp-config.php` file are as follows:

- `WP_DEBUG_LOG`: Stores all debug messages in a file named `debug.log` in the site's `wp-content` directory for later analysis
- `WP_DEBUG_DISPLAY`: Indicates whether error messages should be displayed on the site
- `SAVEQUERIES`: Stores database queries in a variable that can be displayed in the page footer (see `https://wordpress.org/support/article/editing-wp-config-php/#save-queries-for-analysis` for more information)

Better debugging with Xdebug

While all of the debugging techniques mentioned in this chapter are valid, they do require manual intervention to insert and remove code, especially in the case of the `print_r` function called to view data received by the filter function. If you forget to remove a manual debugging statement, your users will end up with debug data on their site if you distribute your work.

As you become more invested in plugin development, you should consider setting up a development server that supports Xdebug. One of the benefits of this debugging tool is that coding errors will be displayed with more details about where they were found and the code that was executed leading up to the error. More importantly, Xdebug allows developers to connect a code editor such as Visual Studio Code to the web server to be able to step through each individual line of code as they are executed. It is also possible to set breakpoints to stop execution in specific places and see data in all variables every step of the way.

See also

- The *Modifying the site generator meta tag using plugin filters* recipe

Creating a new simple shortcode

Shortcodes are a very popular tool in WordPress that allow users to easily add content generated by plugins or themes to any page or post without needing to be familiar with PHP code and editing theme template files. As they are very simple to create, shortcodes can also be used to easily automate the display of content that repeatedly needs to be inserted on your site.

How to do it...

Follow these steps to create a new custom shortcode that will accelerate the task of linking to a specific Twitter page in any post or page:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch2-twitter-shortcode`.
3. Navigate to this directory and create a new text file called `ch2-twitter-shortcode.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 2 - Twitter Shortcode`.
5. Add the following line of code to declare a new shortcode, simply using the two `tl` characters, and specify the name of the function that should be called when the code is encountered in posts or pages:

```
add_shortcode( 'tl', 'ch2ts_twitter_link_shortcode' );
```

6. Add the following code section to provide an implementation for the `ch2ts_twitter_link_shortcode` function:

```
function ch2ts_twitter_link_shortcode( $atts ) {
    $output =
        '<a href="https://twitter.com/ylefebvre">';
    $output .= 'Twitter Feed</a>';
    return $output;
}
```

7. Save and close the plugin file.
8. Log in to the administration page of your development WordPress installation.
9. Click on **Plugins** in the left-hand navigation menu.
10. **Activate** your new plugin.
11. Edit an existing post on your site and use the `[tl]` shortcode in the code editor:

Figure 2.10 – Inserting the [tl] shortcode in the new page contents

12. Save and view the post to see that the shortcode was replaced by a link to a Twitter page attached to the words `Twitter Feed`.

How it works...

Shortcodes have similarities with both action hooks and filter hooks, since their associated custom function is called when it is time to perform a task, just like an action hook, but they must return their output through a return value, just like a filter hook. In terms of external data, the function associated with a shortcode will receive data in the case of some types of shortcodes, while it will only produce output in other cases.

When used in the content of a post or page, any shortcode surrounded by a pair of square brackets is identified by the WordPress engine, which then searches for functions registered for that specific code. If found, the associated function is called, and the expected result is used to replace the original shortcode text in the item's content.
Just like filter functions, shortcode functions must not output any text directly, since it will likely appear in an unexpected place in the page layout, as WordPress calls all shortcode-processing functions before displaying the body of an item.

For simple shortcodes, such as the one found in this recipe, the plugin functions associated with them must return information, but they do not receive any additional data through function parameters. That being said, they can rely on utility functions, such as `get_the_ID`, `get_the_title`, and other WordPress utility functions, to get more information on the item that contains them and be able to produce tailored output. Other types of shortcodes seen in later recipes will have more context and configuration options. It is also possible for shortcodes to access stored options data, which will be covered in *Chapter 3*, *User Settings and Administration Pages*.

See also

- The *Creating a plugin file and header* recipe

Creating a new shortcode with parameters

Simple shortcodes already provide a lot of potential to output complex content to a page by entering a few characters in the post or page editors. That being said, they become even more useful when they are coupled with parameters that will be passed to their associated processing function. Using this technique, it becomes very easy to create a shortcode that accelerates the insertion of external content in WordPress posts or pages by only needing to specify the shortcode and the unique identifier of the source element to be displayed.

How to do it...

Follow these steps to create a shortcode that will be used to quickly add Twitter feeds to posts or pages:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch2-twitter-embed`.
3. Navigate to this directory and create a new text file called `ch2-twitter-embed.php`.

4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 2 - Twitter Embed`.
5. Add the following line of code to declare a new shortcode and specify the name of the function that should be called when the shortcode is found in posts or pages:

```
add_shortcode( 'twitterfeed',
                'ch2te_twitter_embed_shortcode' );
```

6. Add the following code section to provide an implementation for the `ch2te_twitter_embed_shortcode` function:

```
function ch2te_twitter_embed_shortcode( $atts ) {
    extract( shortcode_atts( array(
        'user_name' => 'ylefebvre'
    ), $atts ) );

    if ( empty( $user_name ) ) {
        $user_name = 'ylefebvre';
    } else {
        $user_name =
            sanitize_text_field( $user_name );
    }

    $output = '<p><a class="twitter-timeline" href="';
    $output .=
        esc_url( 'https://twitter.com/'.$user_name );
    $output .=
        '">Tweets by ' . esc_html( $user_name );
    $output .= '</a></p><script async ';
    $output .=
        'src="//platform.twitter.com/widgets.js"';
    $output .= 'charset="utf-8"></script>';

    return $output;
}
```

7. Save and close the plugin file.
8. Log in to the administration page of your development WordPress installation.

9. Click on **Plugins** in the left-hand navigation menu.
10. **Activate** your new plugin.
11. Create a new page and use the `[twitterfeed user_name='WordPress']` shortcode in the page editor, where `WordPress` is the Twitter username of the feed to display:

Figure 2.11 – Inserting the Twitter feed shortcode with its user_name parameter

12. **Publish** and view the page to see that the shortcode has been replaced by an embedded Twitter feed on your site. The Twitter feed may be quite large, based on the theme you are using on your development site.
13. **Edit** the page and remove the `user_name` parameter and its associated value, only leaving the core `[twitterfeed]` shortcode in the post; then **Update** to save changes.
14. Refresh the page to see that the feed is still being displayed but now shows tweets from a default account specified by the code.

How it works...

When shortcodes are used with parameters, these extra pieces of data are sent to the associated processing function in the `$atts` parameter variable. By using a combination of the standard PHP `extract` and WordPress-specific `shortcode_atts` functions, our plugin is able to parse the data it receives and create an array of identifiers and values that are subsequently transformed into PHP variables. These variables are used in the rest of our shortcode implementation function. In this specific example, we expect a single variable to be used, called `user_name`, which will be stored in a PHP variable called `$user_name`. If the user enters the shortcode without any parameter, a default value of `ylefebvre` will be assigned to the `$user_name` variable to ensure that the plugin still works. Since we are going to accept user input in this code, we also verify that the user did not provide an empty string. We also use the `sanitize_text_field` function to make sure that there is no hazardous code in what the user entered, along with the `esc_html` and `esc_url` functions to be absolutely sure to remove any potentially harmful HTML characters before we output the destination link and its accompanying text to the browser.

Once we have access to the Twitter username, we can put together the required HTML code that will embed a Twitter feed in our page and display the selected user's tweets.

While this example only has one argument, it is possible to define multiple parameters for a shortcode. In such a situation, parameters can be provided in any order by the user.

See also

- The *Creating a new simple shortcode* recipe

Creating a new enclosing shortcode

A different type of shortcode that exists in WordPress is one that encloses content in posts and pages. Using a syntax similar to HTML tags, enclosing shortcodes can be used to identify parts of an item's content that need to be treated in a special way. For example, it is possible to use this type of shortcode to style a part of the post.

As an example of how to create enclosing shortcodes, this recipe shows you how to create a set of tags that will identify part of a post or page that should only be shown to visitors that are logged in to a site. In this way, the shortcode acts similarly to a filter hook, with the added bonus that you do not need to parse for instances of these tags in your code, as would normally be done in a filter callback.

How to do it...

Follow these steps to create a new enclosing shortcode to identify private parts of website content:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch2-private-item-text`.
3. Navigate to this directory and create a new text file called `ch2-private-item-text.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 2 - Private Item Text`.
5. Add the following line of code to declare a new shortcode and specify the name of the function that should be called when the shortcode is found in posts or pages:

```
add_shortcode( 'private',
               'ch2pit_private_shortcode' );
```

6. Add the following code section to provide an implementation for the `ch2pit_private_shortcode` function:

```
function ch2pit_private_shortcode( $atts,
                                   $content = null ) {
    if ( is_user_logged_in() ) {
        return '<div class="private">' . $content .
            '</div>';
    } else {
        $output = '<div class="register">';
        $output .= 'You need to become a member to ';
        $output .= 'access this content.</div>';
        return $output;
    }
}
```

7. Save and close the plugin file.
8. Log in to the administration page of your development WordPress installation.
9. Click on **Plugins** in the left-hand navigation menu.
10. **Activate** your new plugin.
11. Create a new post and wrap some of the content with the `[private]` and `[/private]` tags:

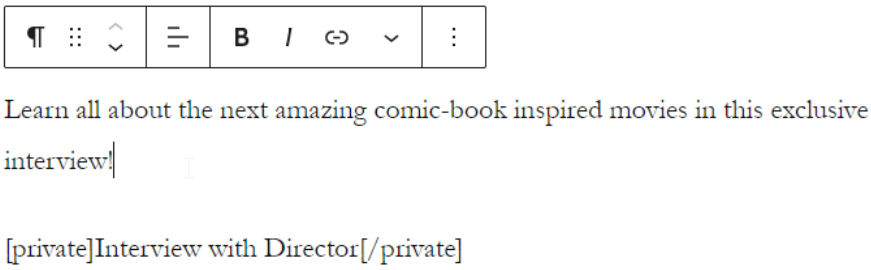


Figure 2.12 – Using the private enclosing shortcode

12. Save and view the post to see that the text is visible while you are logged in to your site.
13. Open an incognito browser view and visit the page to see that the enclosed text has been replaced by a general message.

How it works...

Similar to a filter function, enclosing shortcodes receive a copy of the text that has been wrapped with new tags through the `$content` parameter. It is then possible to return this text with additional HTML code, or completely replace it with new content. In this specific case, we used the `is_user_logged_in` WordPress function to determine whether the current visitor is logged in to the site. Based on the result of that query, the code determines whether the original content should be displayed with some additional `div` tags, or whether the visitor should see a message encouraging them to join the website to see this content.

See also

- The *Creating a new simple shortcode* recipe

Loading a style sheet to format plugin output

When a plugin adds custom content or inserts styling tags to a post or page's existing content, as was done in the previous recipe that showed how to create an enclosing shortcode, it often needs to load a custom style sheet to style these new elements. This recipe shows how to add a style sheet in the WordPress style queue to format the private output created in the previous recipe. This queue is processed when the page header is rendered, listing all the style sheets that need to be loaded to display the site correctly.

Getting ready

You should have already followed the *Creating a new enclosing shortcode* recipe to have a starting point for this recipe, and the resulting plugin should still be active in your development site. Alternatively, you can download the resulting code (`ch2/ch2-private-item-text/ch2-private-item-text.php`) of that recipe from the book's GitHub page.

How to do it...

Follow these steps to add insert custom **Cascading Style Sheet** (**CSS**) code in your page output:

1. Navigate to the `ch2-private-item-text` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch2-private-item-text.php` file in a code editor.

3. Add the following line after the existing code to register a function that will be called at the beginning of the WordPress page display process:

```
add_action( 'wp_enqueue_scripts',
            'ch2pit_queue_stylesheet' );
```

4. Add the following code section to provide an implementation for the `ch2pit_queue_stylesheet` function:

```
function ch2pit_queue_stylesheet() {
    wp_enqueue_style( 'privateshortcodestyle',
        plugins_url( 'stylesheet.css', __FILE__ ) );
}
```

5. Save and close the plugin file.
6. Create a new text file in the `ch2-private-item-text` directory called `stylesheet.css` and open it in a code editor.
7. Add the following content to the file:

```
.private {
    color: #6E6A6B;
}

.register {
    background-color: #ff4d4d;
    color: #fff;
    padding-left: 10px;
}
```

8. Save and close the text file.
9. Navigate to your website, making sure you are logged in, and refresh the page containing the private text content. You should notice that the private text is now displayed in gray.
10. Open the site in an incognito browser view to see that the registration message styling has changed to be displayed in white with a red background.

How it works...

While it would have been possible to write straight HTML code to load the CSS file by registering a function with the `wp_head` action hook, as we have done previously, WordPress has utility functions designed to help avoid loading duplicate style sheets or scripts on a site. In this specific example, `wp_enqueue_script` is used to place the plugin's style sheet file in a queue that will be processed when the plugin header is rendered, with the associated name `privateshortcodestyle`. Once WordPress has processed all the plugins and boiled down all the style sheet requests to single instances, it will output the necessary HTML code to load all of them.

The content of the `stylesheet.css` file is standard CSS code that specifies that any text that is assigned the `private` class should be displayed in gray, while the text displayed to non-registered users should be displayed in white on a red background.

See also

- The *Creating a new enclosing shortcode* recipe

Writing plugins using object-oriented PHP

So far, all the plugin examples that have been covered in this chapter have been written using the procedural PHP programming style. In this style, all functions are declared directly in the main body of the plugin and the hook registration functions have direct access to these functions.

WordPress plugins can also be written using an object-oriented PHP approach. This recipe shows how the code from the previous recipe can be restructured to be written in object-oriented PHP.

Getting ready

You should have already followed the *Loading a style sheet to format plugin output* recipe to have a starting point for this recipe. Alternatively, you can download the resulting code (`ch2/ch2-private-item-text/ch2-private-item-text-v2.php`) for that recipe from the book's GitHub page.

How to do it...

Follow these steps to transform an existing plugin's code into object-oriented PHP:

1. Log in to the administration page of your WordPress installation.
2. Click on **Plugins** in the left-hand navigation menu.
3. Check whether the **Chapter 2 - Private Item Text** plugin is currently active and **Deactivate** it if it is.
4. Copy the entire contents of the `ch2-private-item-text` directory and rename the copy `ch2-oo-private-item-text`.
5. Navigate to the newly renamed folder and rename the main PHP code file `ch2-oo-private-item-text.php`.
6. Open the newly renamed plugin file in a code editor.
7. Update the plugin header to change the name of the plugin to `Chapter 2 - Object-Oriented - Private Item Text`.
8. Right after the plugin header, add the following text to declare a new class for the plugin and specify a constructor method for this class:

```
class CH2_OO_Private_Item_Text {
    function __construct() {
    }
}
$my_ch2_oo_private_item_text =
    new CH2_OO_Private_Item_Text();
```

9. Move the calls to the `add_shortcode` and `add_action` functions to be placed inside of the class constructor method (`__construct`).
10. Modify the second argument of the `add_shortcode` and `add_action` functions as follows:

```
add_shortcode( 'private', array( $this,
    'ch2pit_private_shortcode' ) );
add_action( 'wp_enqueue_scripts', array( $this,
    'ch2pit_queue_stylesheet' ) );
```

11. Move the complete `ch2pit_private_shortcode` and `ch2pit_queue_stylesheet` functions inside of the class body (after the `__construct` method and before the class closing bracket).

12. Save and close the modified file.
13. Log in to the administration page of your development WordPress installation.
14. Click on **Plugins** in the left-hand navigation menu.
15. **Activate** the new plugin.
16. Visit your site to see that the private item content functionality is still in place and works as it did before.

How it works...

The code changes that we applied to the plugin first declare a class for all of our plugin's functionality and also contain a constructor method for that class. The `__construct` method is called once, as soon as the class is instantiated by the last line in the plugin's code, and can be used to associate custom functions with all action hooks, filter hooks, and shortcodes.

The main benefit of using an object-oriented approach is that you don't have to be as careful when naming your hook callbacks and all other functions, since these names are local to the class and can be the same as function names declared in any other classes or in procedural PHP code.

There's more...

If you enjoy object-oriented plugin development and create a lot of plugins, you might benefit from using a boilerplate generator.

WordPress plugin boilerplate generator

By visiting the WordPress plugin boilerplate generator (`https://wppb.me/`), you can easily create code that needs to be written each time you create a plugin. After entering basic data about your plugin, you will receive a download with the core structure for your new plugin. This template contains a number of object-oriented concepts that are best suited to developers who are well versed in object-oriented programming.

See also

- The *Creating a new enclosing shortcode* recipe

3
User Settings and Administration Pages

As we saw in *Chapter 2*, *Plugin Framework Basics*, it is very easy for a plugin to register custom functions with action and filter hooks to change or augment the way WordPress renders web pages. That being said, some of the examples covered in *Chapter 2*, *Plugin Framework Basics,* have limitations when it comes to dealing with custom user information, such as the inability to easily specify a Google Analytics account number.

To make plugins easy to use for a wide audience, it is usually important to create one or more administration pages where users will be able to provide details that are specific to their installation, enter information on external accounts, and customize some of the aspects of the plugin's functionality. As an example, the **Akismet** plugin, provided in default WordPress installations, offers a configuration page that can be found under the **Settings | Akismet** configuration menu. Thankfully, WordPress has a rich set of functions that allows plugin developers to easily put together configuration pages that will seamlessly blend with the rest of the administrative panels.

This chapter covers how to use the WordPress options-related **Application Programming Interface (API)** functions to store and access user options in the site database. We then go on to explore the different techniques that are available to create custom dialogs that provide users with complete control over the configuration of the plugins that you create.

This chapter is focused on setting up pages that will enable users to configure plugin settings. It covers the following recipes:

- Creating default user settings on plugin initialization
- Storing user settings using arrays
- Removing plugin data on deletion
- Creating an administration page menu item in the **Settings** menu
- Creating a multi-level administration menu
- Rendering the admin page contents using HTML
- Processing and storing plugin configuration data
- Displaying a confirmation message when options are saved
- Adding custom help pages
- Rendering the admin page contents using the Settings API
- Accessing user settings from action and filter hooks
- Formatting admin pages using meta boxes
- Splitting admin code from the main plugin file to optimize site performance
- Storing style sheet data in user settings
- Managing multiple sets of user settings from a single admin page
- Creating a network-level plugin with admin pages

Technical requirements

You can find complete code examples on GitHub in the following repository: `https://github.com/PacktPublishing/WordPress-Plugin-Development-Cookbook-Third-Edition`.

Creating default user settings on plugin initialization

A typical first step of most user-configurable plugins is to create a default set of values for all options when the plugin is activated. These default options will subsequently be used to populate the plugin's configuration page when it is visited by the site administrator.

How to do it...

Follow these steps to register a function that is called when a plugin is activated and learn how to store options data in the site database:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch3-individual-options`.
3. Navigate to this directory and create a new text file called `ch3-individual-options.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 3 - Individual Options`.
5. Add the following line of code to register a function that will be executed when the plugin is activated, after its initial installation, or following an upgrade:

```
register_activation_hook( __FILE__,
                          'ch3io_set_default_options' );
```

6. Add the following code section to provide an implementation for the `ch3io_set_default_options` function:

```
function ch3io_set_default_options() {
    if ( false ===
            get_option( 'ch3io_ga_account_name' ) ) {
        add_option( 'ch3io_ga_account_name',
                    'UA-0000000-0' );
    }
}
```

7. Save and close the plugin file.
8. Go to the **Plugins** section of the administration interface.

9. Execute the activation function that was just added by clicking on the **Activate** option of the new `Chapter 3 - Individual Options` plugin.
10. In your web server's MySQL database administration tool (**phpMyAdmin**, **Adminer**, or other), select your WordPress database, click on the **SQL** or **SQL Command** tab, replace the default query with the following statement, and then execute the code:

```
select * from wp_options where option_name =
'ch3io_ga_account_name'
```

```
select * from wp_options where option_name =
'ch3io_ga_account_name'
```

option_id	option_name	option_value	autoload
155	ch3io_ga_account_name	UA-0000000-0	yes

1 row (0.000 s) Edit, Explain, Export

Figure 3.1 – Newly created plugin option in the wp_options table

As seen in *Figure 3.1*, the query returns a single row with the default value assigned to the new option. The **option_id** field value will likely be different in your query results.

How it works...

The `register_activation_hook` function is used to indicate to WordPress the name of a function that should be called when it activates the plugin. Unlike other hooks, this function requires the complete path and filename of the main plugin code file to be sent as its first argument, along with the name of the associated function. To do this easily, we are leveraging the PHP `__FILE__` constant, which provides the complete path and filename of the plugin.

When the callback function is called, we can use the Options API to create, update, or delete settings in the options table of the site's MySQL database. In this specific example, we are using the `add_option` function to easily create an option called `ch3io_ga_account_name` with a default value of `UA-0000000-0`.

> **Tip**
> Just like function names, you should be careful when naming plugin options to avoid conflicts with other plugins. A good practice is to add a unique prefix to the beginning of each variable name.

Before making a call to create the new option, the activation function checks whether the option is present in the WordPress options table by using the `get_option` function. If the return value is `false`, indicating that the option was not found, a new default option can be created. Any other result would show that the plugin has been activated on the site previously and that options may have been changed from their default values. It is important to keep in mind when writing this code that plugins get deactivated and reactivated each time they are updated using the WordPress update tool, resulting in a call to their activation function. It is also possible that a user might have deactivated a plugin temporarily to debug site issues and brought it back at a later time, also resulting in the activation function being called.

Finally, it should be noted that it is possible to call the `add_option` function multiple times if more than one option is needed to implement a plugin's desired functionality. That being said, it is not necessary to verify the presence of all options, as checking for a single one would indicate that they were all previously set.

There's more...

Beyond the creation of default values for a plugin, the activation hook can also be used to perform more advanced tasks, such as doing data initialization, as will be seen in later chapters. In contrast, the similar deactivation function hook does not have any real use within the creation of most plugins.

Deactivation function

Similar to the activation function that we used in this recipe, WordPress provides a way to register a deactivation function (using `register_deactivation_hook`). While it may be tempting to use this function to remove options created by the plugin, it is not possible to know why the deactivation function was called. The three situations that could trigger this call are a plugin upgrade, a temporary deactivation to debug a site problem, or just before the plugin gets deleted. Since it is best to keep user options in the first two situations, any cleanup and data removal code should be placed in a plugin's uninstallation file instead, as described in a later recipe in this chapter.

See also

- The *Removing plugin data on deletion* recipe

Storing user settings using arrays

While the previous recipe worked quite well in creating entries in the site's options table for each individual plugin option, another way to manage user settings is to store them as arrays in the database.

This recipe creates the same option as the previous one, but also adds a second option indicating whether outgoing links should be tracked. It uses an array instead of individual options to store these settings. It also incorporates an upgrade strategy to deal with the creation of additional options as a plugin evolves over time.

Getting ready

You should have already followed the recipe entitled *Inserting link tracking code in the page body using plugin filters* in *Chapter 2*, *Plugin Framework Basics*, to have a starting point for this recipe. Alternatively, you can get the resulting code (`ch2/ch2-page-header-output/ch2-page-header-output-v2.php`) from the book's GitHub page and rename the file `ch2-page-header-output.php`.

How to do it...

Follow these steps to learn how to store arrays of options in the site options table:

1. Navigate to the `ch2-page-header-output` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch2-page-header-output.php` file in a code editor.
3. Add the following line of code at the end of the file to register a function to be called when the plugin is activated:

```
register_activation_hook( __FILE__,
    'ch2pho_set_default_options_array' );
```

4. Add the following code section to provide an implementation for the `ch2pho_set_default_options_array` function:

```
function ch2pho_set_default_options_array() {
    ch2pho_get_options();
}
```

5. Add the following code to provide an implementation for the `ch2pho_get_options` function:

```
function ch2pho_get_options() {
    $options = get_option( 'ch2pho_options',
                           array() );

    $new_options['ga_account_name'] = 'UA-0000000-0';
    $new_options['track_outgoing_links'] = false;

    $merged_options = wp_parse_args( $options,
                                     $new_options );

    $compare_options =
        array_diff_key( $new_options, $options );
    if ( empty( $options ) ||
            !empty( $compare_options ) ) {
        update_option( 'ch2pho_options',
                       $merged_options );
    }
    return $merged_options;
}
```

6. Save and close the plugin file.
7. Go to the **Plugins** section of the administration interface.
8. Click on the **Deactivate** link for the `Chapter 2 - Page Header Output` plugin if it was still activated, followed by a click on the **Activate** link to execute the activation function that was just added.
9. Using your web server's MySQL database administration tool, query the `wp_options` table of your WordPress installation for an option with the name `ch2pho_options`:

```
select * from wp_options where option_name =
'ch2pho_options'
```

select * from wp_options where option_name = 'ch2pho_options'

option_id	option_name	option_value	autoload
159	ch2pho_options	a:2:{s:15:"ga_account_name";s:12:"UA-0000000-0";s:20:"track_outgoing_links";b:0;}	yes

1 row (0.000 s) Edit, Explain, Export

Figure 3.2 – A serialized array of plugin options in the wp_options table

As seen in *Figure 3.2*, the query returns a single row with a serialized set of data representing all of the fields in the array.

How it works...

The `add_option`, `get_option`, and `update_option` functions accept values as single variables or arrays of data. When given an array, they transform the information received to a serialized array that gets stored in the site database.

The main advantage of using arrays over multiple options is that all of the information can be retrieved with a single function call, optimizing access to the MySQL database and simplifying your plugin's code. This is especially important when your plugin options need to be queried every time a site page needs to be rendered. Of course, this advantage is only true if you need to use most plugin options at the same time. Otherwise, your code will be managing large amounts of data for no reason.

Another benefit of using arrays instead of individual options is that the names of each option can be much shorter and simpler, since you only need to worry about avoiding naming conflicts at the top option name level, as opposed to each key in the array. Finally, having all the options stored in a single array makes the bulk removal of these options much easier than if they were all stored separately, as we will see in the next recipe.

The bulk of this recipe's code defines a utility function called `ch2pho_get_options`, which is used to make sure that we always get good values when retrieving options, even when our plugin runs for the first time or new options are introduced by an upgrade. As part of this utility function, we use the `wp_parse_args` function to quickly compare an existing set of options retrieved by the `get_option` function with the current set of default plugin options, specified by the `$new_options` array. An array of existing values merged with any missing array elements is returned by the `wp_parse_args` function. The last part of our option-retrieval function checks whether the previous option array was empty or whether any new keys were added to the new array, using the PHP `array_diff_key` function. In either of these cases, it will save the updated options array back to the site database.

It should finally be noted that while the `ch2pho_get_options` function returns an array of all plugin options, we are not storing its return value since we are not using it in this specific recipe, but will use it in later recipes in this chapter.

See also

- The *Adding output content to page headers using plugin actions* recipe in *Chapter 2*, *Plugin Framework Basics*
- The *Inserting link tracking code in the page body using plugin filters* recipe in *Chapter 2*, *Plugin Framework Basics*
- The *Removing plugin data on deletion* recipe

Removing plugin data on deletion

As with any piece of software, it is quite possible that users might decide to remove a plugin from their WordPress installation if they no longer require the functionality that it provides, or they have found an alternative that they prefer.

When this happens, the plugin author must decide whether all of the configuration data stored in the site's database should be left in place, making it easier to re-install the plugin down the road, or to remove all of this information, leaving a clean database behind.

Getting ready

You should have already followed the *Storing user settings using arrays* recipe to have options data ready for deletion, and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch3/ch2-page-header-output/ch2-page-header-output-v3.php`) from the book's GitHub page. You should rename the `ch2-page-header-output-v3.php` file as `ch2-page-header-output.php` and have activated the plugin once before starting this recipe.

How to do it...

Follow these steps to create a de-installation function that will remove options data from a site's database:

1. Navigate to the `ch2-page-header-output` folder of the WordPress `plugins` directory of your development installation.
2. Create a new file called `uninstall.php`.
3. Open the new file in a text editor and add the following code to it:

```
<?php
// Check that code was called from WordPress with
// uninstallation constant declared
```

```
if ( !defined( 'WP_UNINSTALL_PLUGIN' ) ) {
    exit;
}

// Check if options exist and delete them if present
if ( false != get_option( 'ch2pho_options' ) ) {
    delete_option( 'ch2pho_options' );
}
```

4. Save and close the plugin file.
5. Navigate to the administration page of your development WordPress installation and click on **Plugins** on the left-hand navigation menu.
6. **Deactivate** the `Chapter 2 - Page Header Output` plugin.
7. Make a copy of your plugin code and uninstallation files to avoid losing them upon deletion of the plugin in the following steps. The copy should be moved outside of the `plugins` folder to avoid WordPress seeing two copies of the plugin.
8. Click on the **Delete** link under the `Chapter 2 - Page Header Output` plugin.
9. Click on the **OK** button to delete all the plugin files.

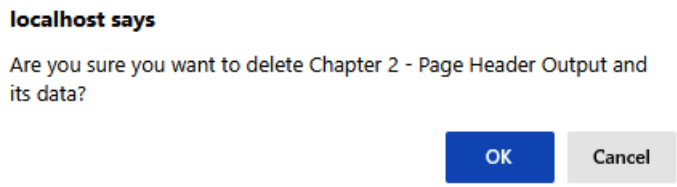


Figure 3.3 – Plugin deletion confirmation dialog

> **Note**
> Depending on the configuration of your development site, you may be prompted to provide FTP credentials before WordPress is able to delete the plugin files.

10. Using your web server's MySQL database administration tool, query the `wp_options` table of your WordPress installation for an option with the name `ch2pho_options` to see that the option has been deleted:

```
select * from wp_options where option_name =
'ch2pho_options'
```

How it works...

When a plugin is inactive and a site administrator clicks on its deletion link, WordPress checks for the presence of a file called `uninstall.php` in the plugin directory. If the file exists and the user clicks the **OK** button to delete, WordPress proceeds with the deletion of all the plugin files and executes the content of the `uninstall.php` file. This file should contain PHP code that deletes all the plugin options and any other content created by the plugin. Once executed, the uninstall script will be deleted along with the rest of the plugin files.

Looking at the content of the uninstall script, the first few lines of code check for the presence of a constant that WordPress should have set before calling the script. If it is not present, the script will abort immediately for security purposes. This ensures that an external visitor knowing that a certain plugin is installed won't be able to try to run the `uninstall.php` script. Once the intent has been verified, the rest of the code checks for the existence of the `ch2pho_options` array that was created in the previous recipe and deletes it. If you created more than one option to store your configuration data, you will need to delete each option with individual calls to the `delete_option` function.

See also

- The *Storing user settings using arrays* recipe

Creating an administration page menu item in the settings menu

After defining default values for plugin configuration options, the next step to creating a plugin configuration page is to build a place where users will be able to view and change these values. By using the WordPress API, we can create new items in the administration menu that will later allow us to create custom plugin configuration pages.

Getting ready

You should have already followed the *Storing user settings using arrays* recipe to have options data available to manage. Alternatively, you can get the resulting code (`ch3/ch2-page-header-output/ch2-page-header-output-v3.php`) from the book's GitHub page. You should rename the file `ch2-page-header-output-v3.php` as `ch2-page-header-output.php` before starting this recipe.

How to do it...

Follow these steps to create a new menu item that will appear under the **Settings** subsection of the administration menu:

1. Navigate to the root directory of your WordPress development installation.
2. Open the `wp-config.php` file in a text editor.
3. Set the `WP_DEBUG` variable to be `true`. If there is no `WP_DEBUG` line in your `wp-config.php` file, create it just above the comment that reads: `/* That's all, stop editing! Happy publishing. */`:

```
define('WP_DEBUG', true);
```

4. Save the configuration file.
5. Navigate to the `ch2-page-header-output` folder of the WordPress plugin directory of your development installation.
6. Open the `ch2-page-header-output.php` file in a text editor.
7. Add the following line of code at the end of the file to register a function to be called when WordPress is building the administration pages menu:

```
add_action( 'admin_menu', 'ch2pho_settings_menu' );
```

8. Add the following code section to provide an implementation for the `ch2pho_settings_menu` function:

```
function ch2pho_settings_menu() {
    add_options_page(
        'My Google Analytics Configuration',
        'My Google Analytics', 'manage_options',
        'ch2pho-my-google-analytics',
        'ch2pho_config_page' );
}
```

9. Save and close the plugin file.
10. Navigate to the administration page of your development WordPress installation.
11. **Activate** the `Chapter 2 - Page Header Output` plugin if it is still deactivated after following the previous recipe.
12. Click on the **Settings** section on the left-hand navigation menu to expand it. You will see a new menu item called **My Google Analytics** in the tree, created from the code that was just added to the plugin.

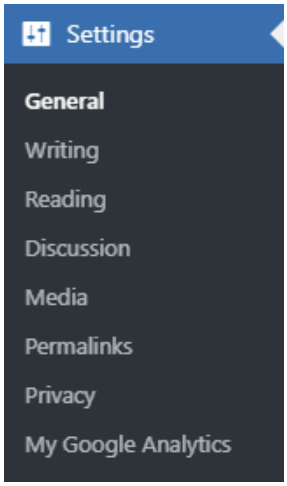


Figure 3.4 – New menu item in the Settings menu

13. Click on the **My Google Analytics** menu item. You will see an error message displayed since WordPress cannot find the function intended to populate the configuration page. This error will go away once you perform the recipe entitled *Rendering the admin page contents using HTML* later in this chapter.
14. Back in the `wp-config.php` file, return the `WP_DEBUG` variable to its default value of `false`:

```
define('WP_DEBUG', false);
```

15. Save and close the configuration file.

> **Note**
> Depending on the web server used and its configuration, you might only see a blank page instead of an error message after performing *step 13* if you don't set the `WP_DEBUG` variable to `true`.

How it works...

The first line of code of this recipe registers a function to be called when WordPress is building the administration menu. When it is executed, the custom function that we created makes a call to the `add_options_page` function to add an item to the **Settings** menu. This function has a number of parameters that we will look at, as follows:

```
add_options_page( $page_title, $menu_title, $capability,
                  $menu_slug, $function );
```

The first two parameters are text strings that will be visible to site administrators, with the first one appearing in the browser title bar or tab title, and the second being the text of the submenu item that will appear under the **Settings** menu.

The third parameter is a bit more complicated and refers to the **user capability** required to be able to see and access this menu item. When creating users in a WordPress installation, each user is assigned one of the five default user roles (**Subscriber**, **Administrator**, **Editor**, **Author**, or **Contributor**). Each of these roles is mapped to a number of **permissions** that determine the actions that users with this role can perform. For a full list of roles and their associated capabilities, please refer to the WordPress Codex page on the topic (`https://wordpress.org/support/article/roles-and-capabilities/`). In this example, we specified the user capability `manage_options`, which is assigned to users who have administrative rights on the site and to super admins when working in a **network** WordPress installation.

The fourth menu item, `menu_slug`, is a text string that will be used internally by WordPress to identify the menu item. This string should be unique to avoid conflicts with other plugins.

> **Tip**
>
> The `menu_slug` parameter should be all lowercase to ensure that more advanced functionalities, such as WordPress meta boxes, work correctly if you decide to use them.

The last parameter specifies the name of the function to be called to display the contents of the configuration page when the submenu item is clicked.

The **Settings** menu is a perfect location for plugins that only require a single configuration page, as you may have seen when installing other plugins, while more complex plugins that require multiple configuration sections should use the technique shown in the next recipe to have a multi-level menu.

There's more...

While new items will always be located under the default **Settings** menu items created by WordPress (**General**, **Writing**, **Reading**, and so on), plugin developers do have some control over the location of their plugin in the list.

Setting hook priority to determine the menu order

As mentioned in the previous chapter, when action hooks were first introduced, the `add_action` function's third parameter is used to indicate the priority of a registered callback over other functions registered for the same hook (in this case, the `admin_menu` hook). To ensure that the newly created menu item is as high as possible in the menu, the `priority` argument of the registered function can be set to a value of `1`:

```
add_action( 'admin_menu', 'ch2pho_settings_menu', 1 );
```

> **Note**
> It should be noted that other plugins can also set their callback to this priority. In such cases, alphabetical priority and activation sequence are other factors in determining which menu item will be displayed first after **Privacy**.

See also

- The *Storing user settings using arrays* recipe

Creating a multi-level administration menu

When plugins grow in complexity, their configuration options often grow in numbers, giving users a high level of flexibility in choosing how the plugin behaves on their site. While it is possible to display all the plugin options on a single lengthy configuration page, creating a new top-level menu item with multiple sections can help organize parameters in logical groupings that will allow users to find what they are looking for more quickly. It also has the side benefit of making your plugin appear more prominently in the WordPress administration menu.

How to do it...

Follow these steps to create a new top-level menu item in the administration menu with an accompanying submenu item:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch3-multi-level-menu`.
3. Navigate to this directory and create a new text file called `ch3-multi-level-menu.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 3 - Multi-level menu`.

5. Add the following line of code to register a function that will be called when WordPress is preparing data to display the site's administration menu:

```
add_action( 'admin_menu', 'ch3mlm_admin_menu' );
```

6. Add the following code section to provide an implementation for the `ch3mlm_admin_menu` function:

```
function ch3mlm_admin_menu() {
    // Create top-level menu item
    add_menu_page(
        'My Complex Plugin Configuration Page',
        'My Complex Plugin', 'manage_options',
        'ch3mlm-main-menu', 'ch3mlm_my_complex_main',
        'dashicons-menu-alt3' );

    // Create a sub-menu under the top-level menu
    add_submenu_page( 'ch3mlm-main-menu',
        'My Complex Menu Sub-Config Page',
        'Sub-Config Page',
        'manage_options', 'ch3mlm-sub-menu',
        'ch3mlm_my_complex_submenu' );
}
```

7. Save and close the plugin file.
8. Navigate to the **Plugins** section of your site's administration area.
9. **Activate** your new plugin.
10. You will now see a new menu item in the administration menu.

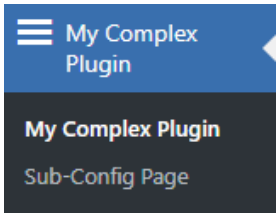


Figure 3.5 – New multi-level administration menu

11. Expand the top-level new menu item to see the submenu items.

> **Note**
> As with the previous recipe, clicking on the menu items will display an error or a blank page since we have not implemented a function to generate actual content for these menu items. We will not be implementing pages for these two particular menu items, but you can use a technique similar to what is shown in the *Rendering the admin page contents using HTML recipe* to create your own `ch3mlm_my_complex_main` and `ch3mlm_my_complex_submenu` functions.

How it works...

The `add_menu_page` function is very similar to the `add_options_page` function seen in the previous recipe, with its first five parameters being identical:

```
add_menu_page( $page_title, $menu_title, $capability,
               $menu_slug, $function, $icon_url, $position );
```

The last two items are specific to this function, with the first allowing us to display an icon in the menu next to our new top-level item, and the second specifying where the new menu should be positioned within the administration menu.

> **Note**
> While it might seem interesting to use the position argument of the `add_menu_page` function to specify an exact position for a new menu item, doing so is risky, since only one menu item will be displayed if two plugins create entries with the same position value. If the position parameter is not specified, WordPress will organize the new menu sections appropriately.

Once the first top-level menu item has been created, the `add_submenu_page` function can be used to attach an additional submenu item. Its parameters can be seen in the following snippet and are virtually identical to the `add_options_page` function, except for the first parameter, which should be the unique string identifier of the top-level menu item to which the submenu should be attached:

```
add_submenu_page( $parent_slug, $page_title, $menu_title,
                  $capability, $menu_slug, $function );
```

While it is possible to use this technique to create top-level menu items for plugins with a single configuration page, these simpler extensions typically just create a single entry under the **Settings** or **Tools** menus, as shown in the previous recipe.

There's more...

Let's take a closer look at the string used to specify the icon when we called the `add_menu_page` function.

Three possible sources for menu icons

The `icon_url` field of the `add_menu_page` function can accept three different types of values, with the first one listed being the method used in this recipe:

- The name of an item from the built-in WordPress **Dashicons** font, which contains a few hundred icons (`https://developer.wordpress.org/resource/dashicons/`). Dashicons are scalable and will automatically match the visual look of the WordPress interface. However, there is a high chance that other plugins might have selected the same icon you choose.
- A base64-encoded **Scalable Vector Graphic** (**SVG**) icon that will be colored to match the rest of the WordPress admin interface. It should be noted that you will need to add a `fill="black"` attribute to all paths in your SVG file so that the icon is properly colored when displayed. The following example draws the Packt logo as the menu icon, with its distinctive orange color when the menu is selected:

```
$menu_icon = '<?xml version="1.0" encoding="utf-8"?>
<svg version="1.1" id="layer" xmlns="http://www.
w3.org/2000/svg" xmlns:xlink="http://www.w3.org/1999/
xlink" x="0px" y="0px" viewBox="0 0 100 130">
<style type="text/css">.st0{fill:#EC670F;}</style>
<g id="g27" transform="translate(-531.85618,-266.66067)">
<polygon class="st0" fill="black" points="620.6,329.1
612.1,340.5 576.5,385.6 546.9,385.6 592.1,329 547.3,278.8
576.7,278.8 612,319.3" id="polygon25" />
</g>
</svg>';

add_menu_page( 'My Complex Plugin Configuration Page',
    'My Complex Plugin', 'manage_options',
    'ch3mlm-main-menu', 'ch3mlm_my_complex_main',
    '' .
        base64_encode( $menu_icon )
);
```

This code generates the following output:

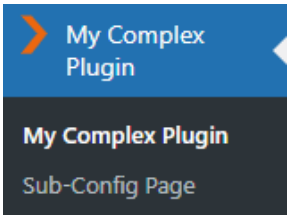


Figure 3.6 – Menu item with a SVG logo

- The URL of an image file located in your plugin directory. While this option might seem like the easiest method for assigning an icon, the color and size of the icon will likely not match the rest of the WordPress interface with this method.

See also

- The *Creating an administration page menu item in the Settings menu* recipe

Rendering the admin page contents using HTML

Once a custom menu item has been created, WordPress will call the function associated with it when it gets visited. The assigned function's main purpose is to render a configuration page containing a form with all the options available to the user and to send the captured data back to WordPress for processing.

Two main methods can be used to render plugin configuration pages: straight HTML and the **Settings API**. This recipe explores the use of HTML to create a configuration panel, while a later recipe will show how to use the Settings API to prepare the page output.

Getting ready

You should have already followed the *Creating an administration page menu item in the Settings menu* recipe. The resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch3/ch2-page-header-output/ch2-page-header-output-v4.php`) from the book's GitHub page. You should rename the `ch2-page-header-output-v4.php` file as `ch2-page-header-output.php` before starting this recipe.

How to do it...

Follow these steps to create an HTML-based configuration page for your plugin:

1. Navigate to the `ch2-page-header-output` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch2-page-header-output.php` file in a text editor.
3. Add the following lines of code at the end of the file to implement the rendering code for the plugin options page:

```
function ch2pho_config_page() {
    // Retrieve plugin options from database
    $options = ch2pho_get_options();
    ?>

    <div id="ch2pho-general" class="wrap">
    <h2>My Google Analytics</h2><br />

    <form method="post" action="admin-post.php">
    <input type="hidden" name="action"
           value="save_ch2pho_options" />

    <!-- Adding hidden security referrer field -->
    <?php wp_nonce_field( 'ch2pho' ); ?>
    Account Name: <input type="text"
        name="ga_account_name"
        value="<?php echo esc_html(
            $options['ga_account_name'] ); ?>"/>
        <br />
    Track Outgoing Links: <input type="checkbox"
           name="track_outgoing_links"
           <?php checked(
             $options['track_outgoing_links'] ); ?>/>
           <br /><br />
    <input type="submit" value="Submit"
           class="button-primary"/>
```

```
        </form>
        </div>
    <?php }
```

4. Save and close the plugin file.
5. Click on the **Settings** section on the administration pages.
6. Click on the **My Google Analytics** menu item.

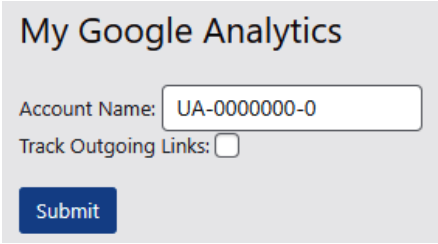


Figure 3.7 – HTML-based plugin configuration page

As seen in *Figure 3.7*, the plugin's configuration page is displayed based on our plugin's HTML code.

How it works...

Any output generated within the configuration page implementation function will be sent to the browser, enclosed within the WordPress administration interface layout. In this recipe's code, we start by using the `ch2pho_get_options` function that we defined earlier in this chapter to retrieve all options for the plugin, conveniently organized in an array that we can store in a single variable.

We then use a closing PHP bracket (`?>`) to be able to write direct HTML code for the rest of the function's body, sending this content directly to the browser. The HTML code takes care of creating a standard form, rendering a text field to display and accept new values for the Google Analytics account number, and a checkbox for the user to specify whether or not outgoing links should be tracked. Finally, the HTML code adds a **Submit** button to allow users to submit any changes made to the plugin's configuration.

Taking a closer look at the code, it also contains small snippets of PHP code that display the current configuration values when the options page is displayed and uses the `checked` WordPress utility function to output the correct HTML code when the passed argument evaluates as `true`.

The biggest advantage of using straight HTML to render a plugin's configuration page is that it allows for the creation of intricate layouts to present all of the options to the end user. This is in sharp contrast to using the Settings API, as we will see in a later recipe. HTML is also easier to understand for many web designers as opposed to working with intricate functions.

> **Note**
> It should be noted that any changes submitted from this form in its current state won't be saved since we have not implemented the code necessary to process the submitted data and store it back in the options database table. Submitting the form will also result in WordPress displaying a blank page. This will be addressed in the next recipe.

There's more...

As soon as user submission processing comes into play, it is important to think about security, to be sure to avoid the most common application security risks (`https://owasp.org/www-project-top-ten/`). The form that was created in this recipe is no exception.

wp_nonce_field

The `wp_nonce_field` function that was used in this recipe is part of a security measure to ensure that the data being sent for submission comes from the WordPress administration pages and not an external source. By adding this function call, a hidden text field is added to the plugin configuration form with information that will be checked when the post data is received.

While it is optional, the first argument of the function is a unique identifier that should always be set to ensure better security. If it is not set, default values will be used, facilitating security breaches. The function also has a number of other optional parameters, as follows:

```
wp_nonce_field( [$action], [$name], [$referer], [$echo] );
```

The other three arguments are used to specify a name for the **nonce**, which would need to be matched on the receiving end, a Boolean variable to indicate whether the `referer` field should be set for validation, and another Boolean parameter to determine whether the hidden form field should be displayed immediately or returned.

See also

- The *Creating an administration page menu item in the Settings menu* recipe
- The *Storing user settings using arrays* recipe

Processing and storing plugin configuration data

With the configuration page in place, plugin users will be able to modify configuration options and submit them to be stored in the WordPress database. The missing link at this time is the creation of a data processing function that will receive the data posted by the user and store it in the site's `options` table.

Getting ready

You should have already followed the *Rendering the admin page contents using HTML* recipe and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch3/ch2-page-header-output/ch2-page-header-output-v5.php`) from the book's GitHub page. You should rename the `ch2-page-header-output-v5.php` file as `ch2-page-header-output.php` before starting this recipe.

How to do it...

Follow these steps to implement a data processing function to validate the plugin's options data and store it in an options array:

1. Navigate to the `ch2-page-header-output` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch2-page-header-output.php` file in a text editor.
3. Add the following line of code at the end of the file to register a function to be called when WordPress first identifies that the requested page is an administration page:

```
add_action( 'admin_init', 'ch2pho_admin_init' );
```

4. Add the following code section to provide an implementation for the `ch2pho_admin_init` function:

```
function ch2pho_admin_init() {
    add_action( 'admin_post_save_ch2pho_options',
                'process_ch2pho_options' );
}
```

5. Add the following code section to define the `process_ch2pho_options` function that was declared in the previous step:

```
function process_ch2pho_options() {

}
```

6. Add the following block of code within the `process_ch2pho_options` function to validate user capabilities, check for the referrer code, and retrieve the plugin options:

```
// Check that user has proper security level
if ( !current_user_can( 'manage_options' ) ) {
    wp_die( 'Not allowed' );
}

// Check if nonce field configuration form is present
check_admin_referer( 'ch2pho' );

// Retrieve original plugin options array
$options = ch2pho_get_options();
```

7. Add this second block of code to the body of the `process_ch2pho_options` function, after the code from the previous step, to process and validate all posted data:

```
// Cycle through all text form fields and store their
// values in the options array
foreach ( array( 'ga_account_name' )
          as $option_name ) {
    if ( isset( $_POST[$option_name] ) ) {
        $options[$option_name] =
```

```
                sanitize_text_field(
                    $_POST[$option_name] );
        }
    }

    // Cycle through all check box form fields and set the
    // options array to true or false values
    foreach ( array( 'track_outgoing_links' )
              as $option_name ) {
        if ( isset( $_POST[$option_name] ) ) {
            $options[$option_name] = true;
        } else {
            $options[$option_name] = false;
        }
    }
```

8. Add this last block of code to the implementation of the `process_ch2pho_options` function to store the resulting options array to the options table and redirect the user to the plugin configuration page:

```
    // Store updated options array to database
    update_option( 'ch2pho_options', $options );

    // Redirect the page to the configuration form
    wp_redirect( add_query_arg(
                  'page',
                  'ch2pho-my-google-analytics',
                  admin_url( 'options-general.php' ) ) );
    exit;
```

9. Save and close the plugin file.
10. Click on the **Settings** section of the administration menu.
11. Click on the **My Google Analytics** menu item to display the configuration page.
12. Change the value of one of the fields and click on the **Submit** button.
13. When the page refreshes, you will see that the values displayed reflect the values submitted.

How it works...

This recipe is the first to introduce an action hook that has a variable name. Instead of writing a specific action hook name when calling `add_action`, this hook name starts with the words `admin_post_` and is followed by the name of an action that it expects to match with a hidden form field. In this case, the action name is `save_ch2pho_options`. Going back to the previous recipe, you can see that this text is the same as the one that was placed in the hidden form field called `action`:

```
<input type="hidden" name="action"
       value="save_ch2pho_options" />
```

When the configuration page form is submitted, it sends all of its data to the `admin-post.php` script, which checks for an `action` field and then calls the associated function, if present.

Once the processing function is executed, the calls to `current_user_can` and `check_admin_referer` are security measures where we check to see whether the user who is currently logged in has administrative rights and whether the nonce field that was part of the form is present. An error in the permission check will result in a specific error message, letting the user know that he does not have the right to perform this action, while the nonce check will display a vague error message to throw off potential hackers (*Figure 3.8*):

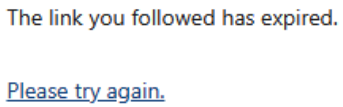


Figure 3.8 – Security message displayed when the nonce check fails

The rest of the function focuses on retrieving the current set of plugin options using the `ch2pho_get_options` function, processing and sanitizing the posted fields, and storing the updated values back in the site database. While using `foreach` loops as shown here might seem to be overly complicated when it comes to storing two simple data fields, this approach can easily scale up to support large amounts of configuration fields. Similar loops can be created to sanitize URLs, HTML code, or any other type of user input.

The final step is a call to the `wp_redirect` function to send the browser back to the plugin options page after all the data has been stored. It is important to always call the `exit` PHP function following a call to `wp_redirect`, as shown in this recipe.

See also

- The *Rendering the admin page contents using HTML* recipe

Displaying a confirmation message when options are saved

An important usability aspect of any user interface is to display an acknowledgment message when users have completed a task successfully. As you may have noticed in the previous recipe, WordPress does not provide any built-in user feedback after configuration data has been saved to the options table.

Getting ready

You should have already followed the *Processing and storing plugin configuration data* recipe and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch3/ch2-page-header-output/ch2-page-header-output-v6.php`) from the book's GitHub page. You should rename the `ch2-page-header-output-v6.php` file to `ch2-page-header-output.php` before starting this recipe.

How to do it...

Follow these steps to display an acknowledgment message on the configuration page after the user has updated the plugin's configuration options:

1. Navigate to the `ch2-page-header-output` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch2-page-header-output.php` file in a text editor.
3. Modify the call to `wp_redirect` at the end of the `process_ch2pho_options` function, as follows, with the modifications shown in bold:

```
wp_redirect( add_query_arg(
    array( 'page' => 'ch2pho-my-google-analytics',
           'message' => '1' ),
    admin_url( 'options-general.php' ) ) );
```

4. Add the following code (in bold) after the configuration page title within the `ch2pho_config_page` function:

```
<h2>My Google Analytics</h2><br />

<?php if ( isset( $_GET['message'] ) &&
        $_GET['message'] == '1' ) { ?>
    <div id='message' class='updated fade'>
    <p><strong>Settings Saved</strong></p></div>
<?php } ?>
```

5. Save and close the plugin file.
6. Click on the **Settings** section of the administration menu.
7. Click on the **My Google Analytics** menu item.
8. Change the value of one of the fields and click on the **Submit** button:

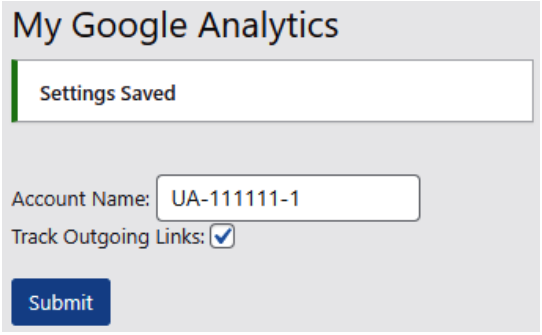


Figure 3.9 – Save confirmation message

As shown in *Figure 3.9*, the newly created message indicates that the settings have been saved.

How it works...

When a redirection call is made, user-submitted fields and PHP variables do not carry forward to the target page. Therefore, we need to use another method, query arguments, to determine that a confirmation message should be displayed.

The first part of the recipe modifies the existing call to `wp_redirect` slightly to add a new query variable called `message`, set to a value of `1`.

Once it receives this variable, the code responsible for rendering the options page can display a message, following the standard WordPress styling.

The same mechanism could be used to display different messages based on the outcome of the options storage. For example, if some fields need to receive data formatted a certain way, the `process_ch2pho_options` function could set the message value differently depending on the success or failure of the data processing operation.

See also

- The *Processing and storing plugin configuration data* recipe

Adding custom help pages

As descriptive as field labels can be, a good plugin always needs to be accompanied by a set of documentation to allow users to quickly understand how to activate the plugin and perform the right steps to get the expected results. While a README file is often what developers first think of producing, users almost never read an external file or instructions on the official WordPress plugin page. They just install the plugin and try to figure it out by themselves.

To address this concern, WordPress introduced the ability to create elaborate multi-section help pages right in the plugin's administration pages to enable users to quickly get answers to their questions.

Getting ready

You should have already followed the *Displaying a confirmation message when options are saved* recipe, and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch3/ch2-page-header-output/ch2-page-header-output-v7.php`) from the downloaded code bundle. You should rename the `ch2-page-header-output-v7.php` file as `ch2-page-header-output.php` before starting this recipe.

How to do it...

Follow these steps to add a help section to your plugin configuration page, containing multiple tabs of information:

1. Navigate to the `ch2-page-header-output` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch2-page-header-output.php` file in a text editor.
3. Find the `ch2pho_settings_menu` function in the existing code.

4. Modify the body of the function code to store the return value of the `add_options_page` function call to a variable:

```
$options_page =
    add_options_page(
        'My Google Analytics Configuration',
        'My Google Analytics',
        'manage_options',
        'ch2pho-my-google-analytics',
        'ch2pho_config_page' );
```

5. Add the following block of code at the end of the `ch2pho_settings_menu` function body to register an action that will be called when the plugin's options page is loaded:

```
if ( !empty( $options_page ) ) {
    add_action( 'load-' . $options_page,
                'ch2pho_help_tabs' );
}
```

6. Add the following code at the end of the plugin file to implement the newly declared `ch2pho_help_tabs` function:

```
function ch2pho_help_tabs() {
    $screen = get_current_screen();
    $screen->add_help_tab( array(
        'id' => 'ch2pho-plugin-help-instructions',
        'title' => 'Instructions',
        'callback' =>
            'ch2pho_plugin_help_instructions',
    ) );

    $screen->add_help_tab( array(
        'id' => 'ch2pho-plugin-help-faq',
        'title' => 'FAQ',
        'callback' => 'ch2pho_plugin_help_faq',
    ) );

    $screen->set_help_sidebar(
```

```
            '<p>This is the sidebar content</p>' );
    }
```

7. Add the following code section at the end of the plugin file to provide an implementation for the `ch2pho_plugin_help_instructions` function:

```
    function ch2pho_plugin_help_instructions() { ?>
        <p>These are instructions explaining how to use
        this plugin.</p>
    <?php }
```

8. Add the following code section at the end of the plugin file to provide an implementation for the `ch2pho_plugin_help_faq` function:

```
    function ch2pho_plugin_help_faq() { ?>
        <p>These are the most frequently asked questions
        on the use of this plugin.</p>
    <?php }
```

9. Save and close the plugin file.
10. Click on the **Settings** section of the administration menu.
11. Click on the **My Google Analytics** menu to display the plugin configuration page. You will now see a **Help** tab appear in the top-right corner of the page.
12. Click on the **Help** tab.

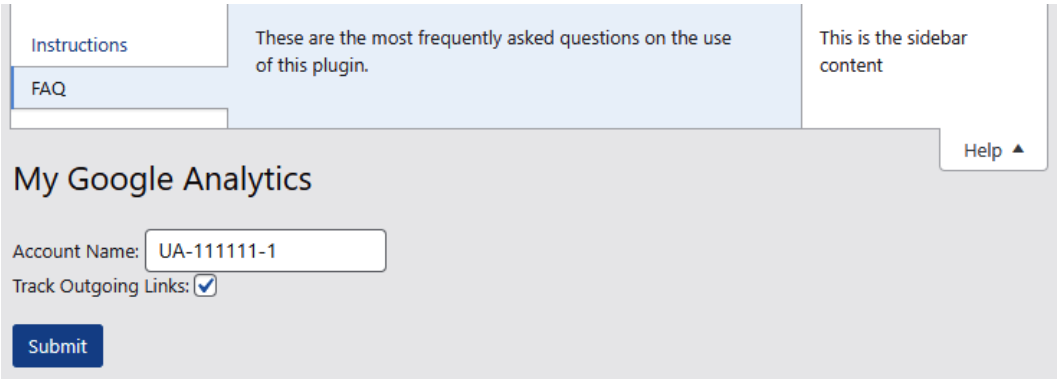


Figure 3.10 – Custom help pages

As seen in *Figure 3.10*, the help content that has been added to the plugin appears in the **Help** tab.

How it works...

As first discussed in the *Processing and storing plugin configuration data* recipe, some WordPress action hooks have names that contain a variable element. This allows plugin developers to have code executed when a specific page is rendered, or when data from a specific form is submitted. In this example, the `load-<pagename>` hook is used to register a function that gets executed when a specific administration page is accessed by the user.

Once the callback occurs, the function's code retrieves a reference to the WordPress `screen` object, which contains data about the screen that is currently displayed, along with a number of utility functions to manipulate and add content to the page. The code from the recipe then proceeds to register functions to render the content of two sections in the **Help** tab using the `add_help_tab` function.

The `add_help_tab` function is a little different from the functions that we have seen before, expecting a single array of options as its parameter. These options indicate a unique identifier (`id`) for the menu section, a label to be displayed on each tab (`title`), and the name of the function that will render the tab contents (`callback`). It is also possible to replace the callback argument with a parameter called `content`, which would directly contain the HTML code intended to be displayed in the **Help** tab. With this information, WordPress is able to integrate the HTML code provided when rendering the configuration page interface. This includes all of the necessary wrapper code to make the **Help** tab open and close, as well as allow the user to switch between the different sections.

The other function used in this recipe, `set_help_sidebar`, is even simpler than `add_help_tab`, with a single argument indicating the HTML content to be displayed on the right-hand side of the **Help** section.

See also

- The *Rendering the admin page contents using HTML* recipe

Rendering the admin page contents using the Settings API

In addition to creating administration pages through HTML code, WordPress also offers a set of functions referred to as the Settings API that can be used to automate the creation of configuration pages. While the work required to put this rendering technique in place is overly complex for plugins that only have a handful of options, it is definitely useful if you are dealing with tens or hundreds of configuration fields, simplifying the task of writing out HTML code for every single item to calling a single function for each of them. It also provides some automation around the processing and storing of plugin configuration data.

This recipe explains how to specify the contents of a configuration page using the Settings API and how to provide rendering functions for the most commonly used types of form fields used in configuration pages. It uses the same set of configuration options as the other recipes in this chapter to show how the two techniques compare.

How to do it...

Follow these steps to learn how to build a plugin configuration page using the Settings API:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch3-settings-api`.
3. Navigate to this directory and create a new text file called `ch3-settings-api.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 3 - Settings API`.
5. Add the following line of code to register a function that will be called when WordPress activates the plugin:

```
register_activation_hook(
    __FILE__, 'ch3sapi_set_default_options' );
```

6. Add the following code section to provide an implementation for the `ch3sapi_set_default_options` function to set default plugin options:

```
function ch3sapi_set_default_options() {
    ch3sapi_get_options();
}
```

7. Add the following code to provide an implementation for the `ch3sapi_get_options` function:

```
function ch3sapi_get_options() {
    $options = get_option( 'ch3sapi_options',
                           array() );

    $new_options['ga_account_name'] = 'UA-0000000-0';
    $new_options['track_outgoing_links'] = false;

    $merged_options = wp_parse_args( $options,
                                     $new_options );
```

```
    $compare_options = array_diff_key( $new_options,
                                       $options );
    if ( empty( $options ) ||
            !empty( $compare_options ) ) {
        update_option( 'ch3sapi_options',
                       $merged_options );
    }
    return $merged_options;
}
```

8. Add the following registration function to associate a callback with the `admin_init` action hook:

```
add_action( 'admin_init', 'ch3sapi_admin_init' );
```

9. Add an implementation for the `ch3sapi_admin_init` function, creating the settings group for the plugin and defining its contents:

```
function ch3sapi_admin_init() {
    // Register a setting group with a validation
    // function so that post data handling is done
    // automatically for us
    register_setting( 'ch3sapi_settings',
        'ch3sapi_options',
        'ch3sapi_validate_options' );

    // Add a new settings section within the group
    add_settings_section( 'ch3sapi_main_section',
        'Main Settings',
        'ch3sapi_main_setting_section_callback',
        'ch3sapi_settings_section' );

    // Add each field with its name and function to
    // use for our new settings, put in new section
    add_settings_field( 'ga_account_name',
        'Account Name',
        'ch3sapi_display_text_field',
```

```
            'ch3sapi_settings_section',
            'ch3sapi_main_section',
            array( 'name' => 'ga_account_name' ) );
        add_settings_field( 'track_outgoing_links',
            'Track Outgoing Links',
            'ch3sapi_display_check_box',
            'ch3sapi_settings_section',
            'ch3sapi_main_section',
            array( 'name' => 'track_outgoing_links' ) );
    }
```

10. Define a body for the `ch3sapi_validate_options` function, which was declared when registering the settings in the previous section:

```
    function ch3sapi_validate_options( $input ) {
        foreach ( array( 'ga_account_name' )
                  as $option_name ) {
            if ( isset( $input[$option_name] ) ) {
                $input[$option_name] =
                    sanitize_text_field(
                        $input[$option_name] );
            }
        }

        foreach ( array( 'track_outgoing_links' )
                  as $option_name ) {
            if ( isset( $input[$option_name] ) ) {
                $input[$option_name] = true;
            } else {
                $input[$option_name] = false;
            }
        }
        return $input;
    }
```

11. Define a body for the `ch3sapi_main_setting_section_callback` function, declared when the settings section was created:

```
function ch3sapi_main_setting_section_callback() { ?>
    <p>This is the main configuration section.</p>
<?php }
```

12. Provide an implementation for the `ch3sapi_display_text_field` function, declared when a text field was added to the settings section:

```
function ch3sapi_display_text_field( $data = array() ) {
    extract( $data );
    $options = ch3sapi_get_options();
    ?>

    <input type="text"
        name="ch3sapi_options[<?php echo
                                esc_html($name); ?>]"
        value="<?php echo
            esc_html( $options[$name] ); ?>"/>
    <br />
<?php }
```

13. Define the `ch3sapi_display_check_box` function, declared when a checkbox was added to the settings section:

```
function ch3sapi_display_check_box($data = array()) {
    extract ( $data );
    $options = ch3sapi_get_options();
    ?>

    <input type="checkbox"
        name="ch3sapi_options[<?php echo
                                esc_html($name); ?>]"
        <?php checked( $options[$name] ); ?>/>
<?php }
```

14. Add the following line of code to register a function that will be called when WordPress is preparing data to display the site's administration menu:

```
add_action( 'admin_menu', 'ch3sapi_settings_menu' );
```

15. Provide code for the implementation of the `ch3sapi_settings_menu` function:

```
function ch3sapi_settings_menu() {
    add_options_page(
        'My Google Analytics Configuration',
        'My Google Analytics - Settings API',
        'manage_options',
        'ch3sapi-my-google-analytics',
        'ch3sapi_config_page' );
}
```

16. Add a definition for the `ch3sapi_config_page` function, defined when the new options page was declared:

```
function ch3sapi_config_page() { ?>
    <div id="ch3sapi-general" class="wrap">
    <h2>My Google Analytics - Settings API</h2>

    <form name="ch3sapi_options_form_settings_api"
          method="post" action="options.php">
    <?php settings_fields( 'ch3sapi_settings' ); ?>
    <?php do_settings_sections(
        'ch3sapi_settings_section'); ?>
    <input type="submit" value="Submit"
           class="button-primary" />
    </form></div>
<?php }
```

17. Save and close the plugin file.
18. Navigate to the **Plugins** menu of the administration area.
19. **Activate** your new plugin.

20. Navigate to the **Settings** menu and click on the **My Google Analytics – Settings API** menu item to see the configuration page for this plugin.

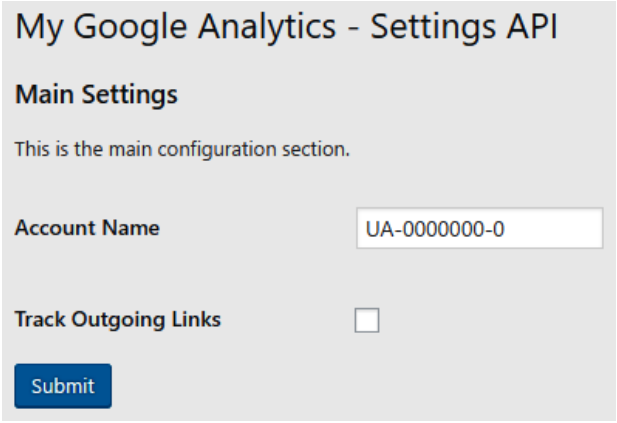


Figure 3.11 – Settings page built using the Settings API

21. Make a change to the options and click on **Submit** to see that the changes are automatically handled by WordPress without having written express code to save options.

How it works...

The Settings API is an intricate series of callbacks that allow plugin developers to streamline the creation of administration pages and to automatically store user options.

This self-contained plugin recipe starts with the creation of a new set of default options, to avoid inadvertently deleting options from previous recipes.

The code continues with registering a function to be called whenever admin pages are prepared for display using the `admin_init` action hook. Upon getting called, the callback function takes care of registering a new setting group, a setting section belonging to this group, and two fields that will display the desired options within the section. As can be seen throughout this code, additional functions are registered to validate the user-submitted data, to display custom text at the beginning of the section, and to display the two different types of fields required to capture and display user input.

Taking a closer look at each of the functions that were just used, the first function has three parameters, which are as follows:

```
register_setting( $option_group, $option_name,
                  $sanitize_callback );
```

Within these parameters, the first option is a unique identifier for the settings group, the second is the name of the options array that will be used to store configuration data in the site database, while the third is the name of a callback function that will receive user input for validation.

Moving on to the second function used in this example, `add_settings_section`, the four parameters that it requires respectively indicate a unique identifier for the section, the title string that will be displayed when the section is rendered, a callback function that will be used to display a description for the section, and finally, a page identifier that will be used to render all similar functions later within the plugin code:

```
add_settings_section( $id, $title, $callback, $page );
```

The third function of the Settings API used in this recipe, `add_settings_field`, is called multiple times to define the fields that make up each section:

```
add_settings_field( $id, $title, $callback, $page,
                    $section, [$args] );
```

Similar to the other functions, the first parameter is a unique identifier for the field, the second parameter is a label that will be displayed next to the field, and the third parameter is a callback function that will be executed to output the necessary HTML code to display the field. The next three parameters indicate the page that the field belongs to, the section that it is contained in, and an optional array of additional data to be sent to the callback function. As can be seen in the rest of this recipe, we are leveraging this optional additional data argument to send data to the field processing function to make them more generic.

When the configuration page is visited, the top-level form is created using regular HTML code, setting the action to `options.php`. This script is responsible for automating the processing of user data. The rest of the form is quite simple since it gets generated by the `settings_fields` and `do_settings_sections` functions.

When these two functions are called, the setting group created earlier is rendered, followed by calls to the functions designed to draw all the sections that it contains and all the registered fields within these sections.

While the Settings API provides full control over the layout of the form fields themselves, its use dictates the general layout of the configuration page, creating a two-column table that contains the labels for each field in the first column and the code produced by the plugin's callback functions in the second one. As the functions for each type of field are called, they receive the array data that was associated with each of them and use it to retrieve the current field values and to specify the name of each field to be stored back upon user input.

The last piece of the puzzle is the validation function that was registered when the setting group was first created. The purpose of this function is to allow plugin developers to perform data type or content validation as user data is submitted through the form, similar to the way we validate user-submitted data in the *Processing and storing plugin configuration data* recipe.

There's more...

While this recipe shows how to create rendering functions for two types of data fields, you may require other types of options for your plugin. The following are code examples that show how to handle most typical data types used in plugin options.

Rendering a drop-down list settings field

Before adding the `select` list itself, we would first add a line to initialize the `select_list` option in the `ch3sapi_get_options` function, if not found in the options retrieved from the database: `$new_options['select_list'] = 'First';`.

The next step to rendering a drop-down list is to provide a list of all possible options, along with the option name, in the optional field data array. Here is an example of a call to the `add_settings_field` function call with such a list:

```
add_settings_field( 'select_list', 'Select List',
    'ch3sapi_select_list', 'ch3sapi_settings_section',
    'ch3sapi_main_section',
    array( 'name' => 'select_list',
    'choices' => array( 'First', 'Second', 'Third' ) ) );
```

With this information, we can provide an implementation for the `ch3sapi_select_list` function, which will be able to render an HTML `select` element using the `choices` array to populate it:

```
function ch3sapi_select_list( $data = array() ) {
    extract ( $data );
    $options = ch3sapi_get_options();
    ?>
    <select
        name="ch3sapi_options[<?php
                echo esc_html($name); ?>]">
    <?php foreach( $choices as $item ) { ?>
        <option value="<?php echo esc_html( $item ); ?>"
```

```
        <?php selected( $options[$name] == $item ); ?>>
        <?php echo esc_html( $item ); ?></option>;
    <?php } ?>
    </select>
<?php }
```

Rendering a text area settings field

Another common field type used in configuration pages is a multi-line text area. Once again, we would need to update the `ch3sapi_get_options` function to specify a default value for this new option if not found. Once this is done, the `add_settings_field` function is identical to the `text` and `checkbox` examples shown in the recipe:

```
add_settings_field( 'ga_text_area', 'Big text field',
        'ch3sapi_display_text_area',
        'ch3sapi_settings_section',
        'ch3sapi_main_section',
        array( 'name' => 'ga_text_area' ) );
```

Meanwhile, the implementation of the `ch3sapi_display_text_area` function would look as follows:

```
function ch3sapi_display_text_area( $data = array() ) {
    extract ( $data );
    $options = ch3sapi_get_options();
    ?>
    <textarea type="text"
        name="ch3sapi_options[<?php echo
                                esc_html( $name ); ?>]"
        rows="5" cols="30">
<?php echo esc_html ( $options[$name] ); ?></textarea>
<?php }
```

Additional validation

As additional fields and new types of fields are added, you should update the `ch3sapi_validate_options` function to process your new fields.

See also

- The *Rendering the admin page contents using HTML* recipe
- The *Processing and storing plugin configuration data* recipe

Accessing user settings from action and filter hooks

After creating a default set of values for our plugin's configuration and creating an interface to allow users to modify and update those values, we are now ready to start using these options when pages are rendered using our additional plugin functionality. Going back to the Google Analytics example created in *Chapter 2*, *Plugin Framework Basics*, this recipe shows how to access the plugin options data using a familiar function to make the existing code much more flexible.

Getting ready

You should have already followed the *Adding custom help pages* recipe, and the resulting plugin should be active on your development site. Alternatively, you can get the resulting code (`ch3/ch2-page-header-output/ch2-page-header-output-v8.php`) from the book's GitHub page. You should rename the `ch2-page-header-output-v8.php` file as `ch2-page-header-output.php` before starting this recipe.

How to do it...

Follow these steps to learn how to access plugin options in filter and action hooks:

1. Navigate to the `ch2-page-header-output` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch2-page-header-output.php` file in a text editor.
3. Modify the implementation of the `ch2pho_page_header_output` function to retrieve the plugin options array and embed the account number in the page header code. The new sections are identified in bold and the first block of code should be inserted between the function opening parenthesis (`{`) and the original PHP closing statement (`?>`):

```
function ch2pho_page_header_output() {
    $options = ch2pho_get_options();
    ?>
    <script>
```

```
    (function(i,s,o,g,r,a,m){i['GoogleAnalyticsObject']=r;
    i[r]=i[r]||function(){
    (i[r].q=i[r].q||[]).push(arguments)},
    i[r].l=1*new Date();
    a=s.createElement(o),
    m=s.getElementsByTagName(o)[0];a.async=1;a.src=g;
    m.parentNode.insertBefore(a,m)})
    (window,document,'script',
    'https://www.google-analytics.com/analytics.js',
    'ga');
    ga( 'create',
    '<?php echo esc_html(
        $options['ga_account_name']); ?>',
    'auto' );
    ga('send', 'pageview');
    </script>
<?php }
```

4. Find the `add_filter` function call related to the `the_content` filter in the plugin code. Add the code identified in bold in the following snippet to check whether the option to track outgoing links has been selected before registering an action hook to filter all post and page content:

```
$options = ch2pho_get_options();
if ( true == $options['track_outgoing_links'] ) {
    add_filter( 'the_content',
                'ch2lfa_link_filter_analytics' );
}
```

5. Use the same check to determine whether page footer code should be added to provide the JavaScript necessary for outgoing link tracking to occur with the changes made identified in bold:

```
if ( true == $options['track_outgoing_links'] ) {
    add_action( 'wp_footer',
                'ch2lfa_footer_analytics_code' );
}
```

6. Save and close the plugin file.
7. Visit the site and look at the page source to see that the string `UA-0000000-0` has been replaced by the last value saved in the plugin's configuration page. You can also set the link tracking code to be inserted or not in the page's code by changing the **Track Outgoing Links** option.

How it works...

As we saw earlier in this chapter, when creating administrative pages, our custom `ch2pho_get_options` function can query the site's database and return the plugin configuration data that it contains. This data can be in the form of a single variable or an array of information. In this case, the plugin options array is retrieved and used to inject values in the page header, footer, and content using action and filter hooks. As always, we use escaping functions to make sure all the data is clean before displaying it.

See also

- The *Storing user settings using arrays* recipe

Formatting admin pages using meta boxes

As a plugin's administration page becomes longer and more complex, it becomes important to divide its content into multiple sections. While standard HTML headers or `fieldset` tags could be used for this task, they lack the usefulness and nice visual appearance of meta boxes. Meta boxes are the containers that show up in many WordPress content editors, as well as on the main administration Dashboard page.

Beyond visually organizing content, meta boxes are very powerful, since they allow site administrators to collapse configuration sections that they don't use, reorder sections based on their needs, and even hide elements that they don't want to see.

Getting ready

You should have already followed the *Accessing user settings from action and filter hooks* recipe. Alternatively, you can get the resulting code (`ch3/ch2-page-header-output/ch2-page-header-output-v9.php`) from the book's GitHub page. You should rename the `ch2-page-header-output-v9.php` file as `ch2-page-header-output.php` before starting the recipe.

How to do it...

Follow these steps to convert the HTML-based configuration page that was created earlier in this chapter to use the built-in meta box system:

1. Browse to the **Plugins** section of the administration section of your site and **Deactivate** the `Chapter 2 - Page Header Output` plugin.
2. Navigate to the `ch2-page-header-output` folder of the WordPress `plugins` directory of your development installation.
3. Copy the `ch2-page-header-output.php` file to `ch2-page-header-output-metaboxes.php`.
4. Open the `ch2-page-header-output-metaboxes.php` file in a text editor.
5. Change the plugin name in the header from `Chapter 2 - Page Header Output` to `Chapter 2 - Page Header Output Meta Boxes`.
6. Right under the top plugin header comment, add a line of code to declare a global variable to hold the identifier for the options page:

```
global $options_page;
```

7. Find the `ch2pho_settings_menu` function in the existing code.
8. Add a line at the top of the function body to point to the global `$options_page` variable:

```
global $options_page;
```

9. Find the `ch2pho_help_tabs` function within the plugin code.
10. Add the following block of code at the end of the function body to create meta boxes to be drawn on the screen and specify functions to be called to populate these boxes:

```
global $options_page;
add_meta_box( 'ch2pho_general_meta_box',
              'General Settings',
              'ch2pho_plugin_meta_box', $options_page,
              'normal', 'core' );
add_meta_box( 'ch2pho_second_meta_box',
              'Second Settings Section',
              'ch2pho_second_meta_box', $options_page,
              'normal', 'core' );
```

11. Add a line of code at the end of the plugin code file to register a function to be called when administration page styles are placed in a queue:

```
add_action( 'admin_enqueue_scripts',
            'ch2pho_load_admin_scripts' );
```

12. Insert the following code segment to provide an implementation for the `ch2pho_load_admin_scripts` function:

```
function ch2pho_load_admin_scripts() {
    global $current_screen;
    global $options_page;

    if ( $current_screen->id == $options_page ) {
        wp_enqueue_script( 'common' );
        wp_enqueue_script( 'wp-lists' );
        wp_enqueue_script( 'postbox' );
    }
}
```

13. Create a new function to implement the `ch2pho_plugin_meta_box` function that was declared a few steps back. Notice that the body of the function is a direct copy and paste of the previous form code that was used to render the **Account Name** and **Track Outgoing Links** fields:

```
function ch2pho_plugin_meta_box( $options ) { ?>
    Account Name: <input type="text"
        name="ga_account_name"
        value="<?php echo
            esc_html( $options['ga_account_name'] );
            ?>"/><br />
    Track Outgoing Links <input type="checkbox"
        name="track_outgoing_links"
        <?php checked(
            $options['track_outgoing_links'] ); ?>/>
        <br />
<?php }
```

14. Add the following code to provide an implementation for the `ch2pho_second_meta_box` function to display a second meta box. This second box will not have any real content. It will only be used to illustrate some of the meta box functionality:

```
function ch2pho_second_meta_box( $options ) { ?>
    <p>This is the content of the second metabox.</p>
<?php }
```

15. Find the `ch2pho_config_page` function and add the following line of code after the call to `ch2pho_get_options` to get access to the global `$options_page` variable:

```
$options = ch2pho_get_options();
global $options_page;
```

16. Add two additional calls to `wp_nonce_field` after the original one:

```
<!-- Adding security through hidden referrer field -->
<?php wp_nonce_field( 'ch2pho' ); ?>

<!-- Security fields for meta box save processing -->
<?php wp_nonce_field( 'closedpostboxes',
                      'closedpostboxesnonce', false ); ?>
<?php wp_nonce_field( 'meta-box-order',
                      'meta-box-order-nonce', false ); ?>
```

17. Still in the `ch2pho_config_page` function, replace the code displaying the `Account Name` text field, the `Track Outgoing Links` checkbox, and the `Submit` button, with the following code. Notice that the `form` and `div` closing tags are retained while all the new code is shown in bold:

```
<div id="poststuff" class="metabox-holder">
    <div id="post-body">
        <div id="post-body-content">
        <?php do_meta_boxes( $options_page, 'normal',
                             $options ); ?>
        <input type="submit" value="Submit"
               class="button-primary"/>
        </div>
    </div>
    <br class="clear"/>
```

```
</div>
</form>
</div>
<script type="text/javascript">
//<![CDATA[
jQuery( document ).ready( function( $ ) {
    // close postboxes that should be closed
    $( '.if-js-closed' ).removeClass(
        'if-js-closed' ).addClass( 'closed' );

    // postboxes setup
    postboxes.add_postbox_toggles(
        '<?php echo $options_page; ?>' );
});
//]]>
</script>
```

18. Save and close the plugin file.
19. **Activate** your new plugin.
20. Click on the **Settings** section on the left-hand navigation menu to expand it.
21. Click on the **My Google Analytics** menu in the tree to display the redesigned administration page.

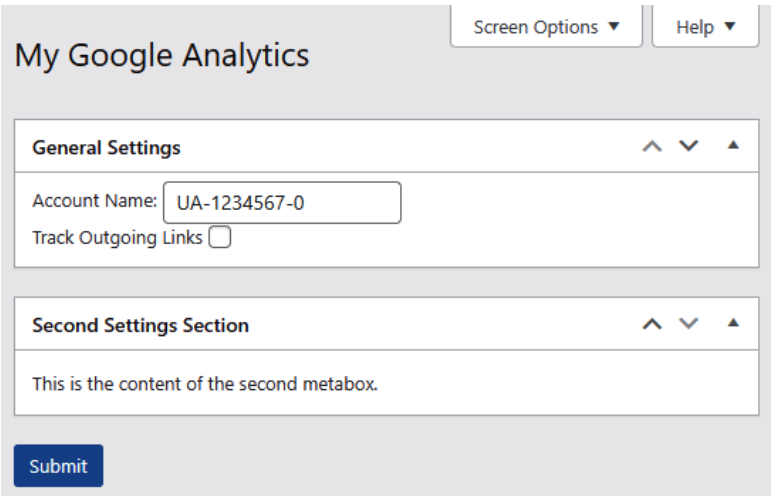


Figure 3.12 – Plugin admin section with meta boxes

22. Drag and drop one of the meta boxes to re-order them or use the up and down arrows on the top-right of each meta box.
23. Click on one of the meta box titles to collapse that section.
24. Click on the **Screen Options** menu in the top-right corner to open a menu to control the visibility of all the meta boxes.
25. Move to another section of the administration menu and come back to the **My Google Analytics** section to see that all changes made to the layout of the configuration page have been retained.

How it works...

The setup of the meta box functionality is done in the `load-<pagename>` callback function by calling the `add_meta_box` function multiple times based on the desired number of boxes to be displayed on the screen.

The function takes a number of arguments, as shown:

```
add_meta_box( $id, $title, $callback, [$page], [$context],
              [$priority], [$callback_args] );
```

Going over the parameters in this function, the first is a unique identifier for the meta box, while the second is the string that will be displayed as the title of the box itself. It is also the name that will show up in the **Screen Options** configuration tab. The third parameter is the name of the function to be called to render the contents of the meta box. The fourth argument identifies the page where the meta boxes will be rendered. In this case, we use the value of the `$options_page` global variable for this parameter, to be sure that it will be assigned the correct page identifier. **Global PHP variables** are powerful tools that can help us share data between functions in a plugin. By using the `global` keyword in front of the name of a variable, a website's PHP interpreter will know that it has to access a common memory space to store and access information.

The fifth parameter, the context, will be assigned values such as `normal`, `side`, or `advanced` to determine where it will appear.

The sixth argument indicates the priority of the registered meta box within the section it belongs to, relative to the other meta boxes. If all the boxes have the same priority, the order in which the calls to the `add_meta_box` function were made will determine their original drawing order. Of course, as was seen in this recipe, this order can be overridden by the user through a simple drag-and-drop operation. The final parameter is optional and can be used to send information to the function that will render the meta box contents.

While it is actually possible to call `add_meta_box` from other action hook callbacks, only meta boxes registered during a `load-<pagename>` callback will show up in the **Screen Options** list to allow the user to control their visibility. You may want to create meta boxes outside of this action hook on purpose, to be sure that important boxes are always shown and to provide a standard user experience for all users.

In addition to the calls to `add_meta_box`, we must make multiple calls to `wp_enqueue_script` in the page load function to request that three JavaScript scripts be loaded when our configuration page is rendered. These scripts provide the drag-and-drop, minimize, and hiding functionalities that were demonstrated at the end of the recipe, with just a few initialization calls needed to be done from our code through JavaScript functions.

Once the meta boxes have been created, the bulk of the work is done within the options page rendering function. As we can see in the modified code, the first thing that is done is to create new `nonce` fields. These unique numbers will be generated as hidden data on the page and will be used for authentication to save layout changes within the configuration page. Next, we create a number of `div` sections with specific `id` names that contain a nested call to the `do_meta_boxes` function. These `div` tags are used to ensure that the meta boxes are styled using the WordPress administration pages style sheet.

Once called, the `do_meta_boxes` function takes care of drawing all of the meta boxes that were created for the given page (specified in the first argument) and the given section (second argument). It also passes along any data specified in the third function argument to the functions associated with each box.

The remaining change to the page rendering function is a block of JavaScript code that takes care of closing down any meta box section that was closed by the user during a previous visit to the page. It also assigns jQuery callbacks to the meta boxes so that any user interaction with them is saved to the site database by sending **AJAX** requests to the web server.

Last, but not least, the meta box rendering functions are responsible for rendering the content inside each meta box. They can do this by outputting straight HTML. By passing along the complete options array to these functions, the code that is contained within them can be exactly the same as before to render the various options fields.

See also

- The *Rendering the admin page contents using HTML* recipe

Splitting admin code from the main plugin file to optimize site performance

As mentioned in *Chapter 2*, *Plugin Framework Basics*, the entire content of the main code file of a WordPress plugin gets evaluated every time any page is rendered on the site, whether it's a visitor-facing page or a backend administration page. This means that large amounts of PHP code can potentially be parsed on every iteration, wasting processing cycles on the site's server, even though some of this code will never be active when regular visitors are browsing the site.

A prime example of this waste is all of the code samples that we have been building in this chapter. While this code is extremely useful for site administrators, there is no sense in having the web server parse and validate that code when regular pages are displayed. For this reason, it is better to isolate this code in a separate file that will only be loaded and parsed when someone is visiting the site's dashboard.

Getting ready

You should have already followed the *Creating a multi-level administration menu* recipe to have a starting point for this recipe, and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch3/ch3-multi-level-menu/ch3-multi-level-menu.php`) from the book's GitHub page.

How to do it...

Follow these steps to learn how to isolate less frequently required code to a separate file and only load it when a user is visiting the site administration section:

1. Navigate to the `ch3-multi-level-menu` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch3-multi-level-menu.php` file in a text editor.
3. Create a new PHP code file called `ch3-multi-level-menu-admin-functions.php` in the same directory and open it in a text editor.
4. Move the calls to the `add_action` function and the definition of the `ch3hmi_hide_menu_item` function to the new file, preceded by a standard PHP open tag, and check for the definition of a few constants at the top:

```
<?php
if ( !defined( 'ch3mlm' ) || !defined( 'ABSPATH' ) ) {
    exit;
}
```

5. Back in the main plugin code file (`ch3-multi-level-menu.php`), add code that will define a constant and check whether the current page being rendered is an administration page. If so, proceed to load the administration functions code file:

```
define( 'ch3mlm', 1 );
if ( is_admin() ) {
    require_once plugin_dir_path( __FILE__ ) .
        'ch3-multi-level-menu-admin-functions.php';
}
```

6. Save and close the plugin file.
7. While the plugin will continue to work exactly as it did before, the action hook registration code will only be processed when an administration page is displayed.

How it works...

The `is_admin` function is used to quickly tell whether the page currently being rendered is an administration page. If it is, our plugin code uses the standard PHP `require_once` function to load, validate, and execute the contents of a separate file. In this case, the file is a second PHP file located in the plugin's directory. To be flexible with regard to the location of the plugin files, we build a path to the file containing the administration functions using the WordPress `plugin_dir_path` function.

While the benefit of placing so little code in a separate file is minimal, this technique has a larger impact on performance when dealing with larger administration panels. In addition to not having to register an action hook on every page load, the PHP interpreter does not have to make sure that the syntax for the entire contents of that second file is valid when rendering front-facing pages. The same technique could be used to isolate code that transforms shortcodes into content if a plugin declares many of them but users typically only use a few at a time.

See also

- The *Creating a multi-level administration menu* recipe

Storing style sheet data in user settings

While most common plugin options are typically presented to users as simple textboxes, checkboxes, or drop-down lists, there are instances where more text needs to be stored for user settings. A good example of this is plugin-specific style sheets, which can allow users to change the visual appearance of plugin output. While loading a separate style sheet file worked well in the *Loading a style sheet to format plugin output* recipe in *Chapter 2*, *Plugin Framework Basics*, this approach did not give users any liberty in changing these styling rules to work better with their site design, since any changes that users make to the stylesheet will get overwritten when the plugin is updated using the WordPress plugin upgrade process.

One solution to this problem is to store style sheet data with the rest of the configuration options in the site database. This way, the information will remain intact when upgrades are performed. This recipe shows how to change the plugin created in the previous chapter to initialize the plugin options using an external file. It also shows how to create an administration panel to allow users to modify or reset the style sheet and how to use the new data to output the style information to the page header. Many of the lessons learned in this chapter will be put to use to create the final result.

Getting ready

You should have already followed the *Loading a style sheet to format plugin output* recipe in the previous chapter to have a starting point for this recipe. Alternatively, you can get the resulting code (`ch2/ch2-private-item-text/ch2-private-item-text-v2.php`) from the book's GitHub page and rename the file to `ch2-private-item-text.php` before starting the recipe.

How to do it...

Follow these steps to build a style sheet editor for the **Chapter 2 - Private Item Text** plugin created earlier.

1. Navigate to the `ch2-private-item-text` folder of the WordPress `plugins` directory of your development installation.
2. Download the `class.csstidy.php`, `class.csstidy_optimise.php`, `class.csstidy_print.php`, and `data.inc.php` files from the **CSSTidy** project (`https://github.com/Cerdic/CSSTidy`) and save them to a new `csstidy` sub-folder under your plugin directory. You can view each code file individually and click on **Raw** mode before saving it or download a zip file of the entire project and pick out the files needed.

3. Open the `ch2-private-item-text.php` file in a text editor.
4. Add the following block of code just under the plugin header to define and implement the `ch2pit_validate_css` utility function:

```
function ch2pit_validate_css( $css ) {
    require_once plugin_dir_path( __FILE__ ) .
        '/csstidy/class.csstidy.php';
    $csstidy = new csstidy();
    $csstidy->set_cfg( 'optimise_shorthands', 2 );
    $csstidy->set_cfg( 'template', 'low' );
    $csstidy->set_cfg( 'discard_invalid_properties',
                       true );
    $csstidy->set_cfg( 'remove_last_;', false );

    $csstidy->parse( $css );
    return $csstidy->print->plain();
}
```

5. Add the following lines of code at the bottom of the file to declare a function that will initialize plugin options and register it to be called upon plugin activation:

```
register_activation_hook( __FILE__,
                          'ch2pit_get_options' );

function ch2pit_get_options() {
    $options = get_option( 'ch2pit_options',
                           array() );

    $stylesheet_location =
        plugin_dir_path( __FILE__ ) . 'stylesheet.css';
    $new_options['stylesheet'] =
        ch2pit_validate_css(
            file_get_contents( $stylesheet_location ) );

    $merged_options = wp_parse_args( $options,
                                     $new_options );
    $compare_options =
        array_diff_key( $new_options, $options );
```

```
    if ( empty( $options ) ||
            !empty( $compare_options ) ) {
        update_option( 'ch2pit_options',
                       $merged_options );
    }
    return $merged_options;
}
```

6. Add the following code segment to register a function to be called when the menu is built to add an additional item under the **Settings** menu and implement that function:

```
add_action( 'admin_menu', 'ch2pit_settings_menu' );

function ch2pit_settings_menu() {
    add_options_page(
        'Private Item Text Configuration',
        'Private Item Text', 'manage_options',
        'ch2pit-private-item-text',
        'ch2pit_config_page' );
}
```

7. Add the following code block to declare a function to be called to queue scripts when rendering administration pages and implement that function:

```
add_action( 'admin_enqueue_scripts',
            'add_page_scripts_enqueue_script' );

function add_page_scripts_enqueue_script( $hook ) {
    if( 'settings_page_ch2pit-private-item-text' ===
            $hook ) {
        $cm_settings['codeEditor'] =
            wp_enqueue_code_editor(
                array( 'type' => 'text/css' ) );
        wp_localize_script( 'jquery', 'cm_settings',
                            $cm_settings );
        wp_enqueue_script( 'wp-theme-plugin-editor' );
        wp_enqueue_style( 'wp-codemirror' );
```

```
    }
}
```

8. Insert the following code to declare the `ch2pit_config_page` function:

```
function ch2pit_config_page() {
    $options = ch2pit_get_options(); ?>

<?php }
```

9. Insert the following block of code within the `ch2pit_config_page` function to retrieve options, display the configuration page title, and display confirmation messages as needed:

```
<div id="ch2pit-general" class="wrap">
<h2>Private Item Text</h2>

<?php if ( isset( $_GET['message'] ) &&
        $_GET['message'] == '1' ) { ?>
    <div id='message' class='updated fade'><p>
    <strong>Settings Saved</strong></p></div>
<?php } elseif ( isset( $_GET['message'] )
        && $_GET['message'] == '2' ) { ?>
    <div id='message' class='updated fade'><p>
    <strong>Stylesheet reverted</strong></p></div>
<?php } ?>
```

10. Insert the next block of code in the `ch2pit_config_page` function, after the previous code, to display a form containing a stylesheet editor and close the `div` section that was previously started.

```
<form name="ch2pit_options_form" method="post"
      action="admin-post.php">
<input type="hidden" name="action"
       value="save_ch2pit_options" />
<?php wp_nonce_field( 'ch2pit' ); ?>

Stylesheet<br />
<textarea name="stylesheet" id="fancy-textarea"
    rows="10" cols="40"
```

```
    style="font-family:Consolas,Monaco,monospace">
<?php echo esc_html ($options['stylesheet'] ); ?>
</textarea><br />

<input type="submit" value="Submit"
       class="button-primary" />
<input type="submit" value="Reset" name="resetstyle"
       class="button-primary" />
</form>
</div>
```

> **Note**
> The call to the `echo` function that prints the value of the style sheet option in the `textarea` (shown in bold) should start at the beginning of a new line to avoid having extra spaces at the beginning of the style sheet editor. Also, make sure you do not lose the hyphen in `font-family` if you are copying the code from a digital copy of this book.

11. Complete the body of the `ch2pit_config_page` function by adding the following block of JavaScript code to enhance the stylesheet editor from a basic `textarea` to an editor that supports syntax highlighting:

```
<script type="text/javascript">
    jQuery(document).ready(function() {
        wp.codeEditor.initialize(
            jQuery('#fancy-textarea'), cm_settings );
    });
</script>
```

12. Add the following block of code at the end of the file to register a function to be called when user options are saved, and to declare the option processing function:

```
add_action( 'admin_init', 'ch2pit_admin_init' );

function ch2pit_admin_init() {
    add_action( 'admin_post_save_ch2pit_options',
                'process_ch2pit_options' );
}
```

```
function process_ch2pit_options() {

}
```

13. Add the following block of code to the `process_ch2pit_options` function body to validate user permissions, check for the presence of an admin referer, and retrieve the plugin options from the database:

```
// Check that user has proper security level
if ( !current_user_can( 'manage_options' ) ) {
    wp_die( 'Not allowed' );
}

// Check if nonce field is present
check_admin_referer( 'ch2pit' );

// Retrieve original plugin options array
$options = ch2pit_get_options();
```

14. Continue the `process_ch2pit_options` function body with the following block of code to reset the stylesheet with its original value found in the `stylesheet.css` file if a reset command was issued, or store the user-submitted style sheet if present:

```
if ( isset( $_POST['resetstyle'] ) ) {
    $stylesheet_location =
        plugin_dir_path( __FILE__ ) . 'stylesheet.css';
    $options['stylesheet'] =
        ch2pit_validate_css(
          file_get_contents( $stylesheet_location ) );
    $message = 2;
} elseif ( !empty( $_POST ) ) {
    foreach ( array( 'stylesheet' )
              as $option_name ) {
        if ( isset( $_POST[$option_name] ) ) {
            $options[$option_name] =
                ch2pit_validate_css(
                    $_POST[$option_name] );
        }
```

```
        }
        $message = 1;
    }
```

15. Complete the `process_ch2pit_options` function body by inserting this final block of code that saves the plugin options and redirects the user back to the plugin configuration page:

```
    update_option( 'ch2pit_options', $options );

    wp_redirect( add_query_arg(
        array(
            'page' => 'ch2pit-private-item-text',
            'message' => $message
        ),
        admin_url( 'options-general.php' )
    ) );
    exit;
```

16. Locate and delete the existing call to the `add_action` function, which associated the `ch2pit_queue_stylesheet` function with the `wp_enqueue_scripts` action hook, along with the `ch2pit_queue_stylesheet` function itself.
17. Add the following code at the end of the file to display the user-modifiable style sheet code as part of the page header:

```
add_action( 'wp_head', 'ch2pit_page_header_output' );

function ch2pit_page_header_output() { ?>
    <style type='text/css'>
    <?php
        $options = ch2pit_get_options();
        echo esc_html( $options['stylesheet'] );
    ?>
    </style>
<?php }
```

18. Save and close the plugin file.
19. **Deactivate** and then **Activate** the **Chapter 2 - Private Item Text** plugin from the administration interface.

20. Navigate to the **Settings** menu and select the **Private Item Text** submenu item to see the newly created configuration panel. It allows users to submit changes to the style sheet or reset it to its initial state, as shown in the following screenshot:

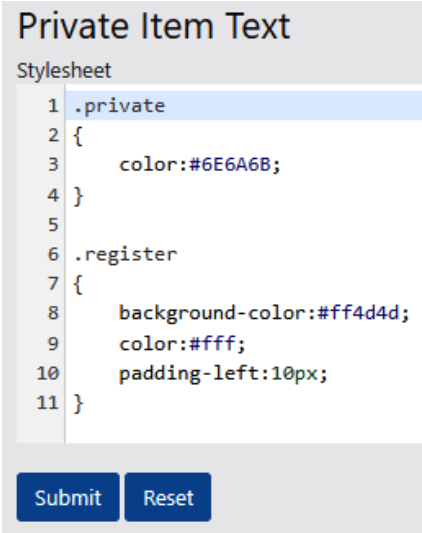


Figure 3.13 – Editor for custom stylesheet content

21. Try to add new properties to some of the existing CSS rules to see how the enhanced editor provides autocompletion services as well as indicates any incorrect or incomplete syntax; for example, adding a border property to the `.private` class:

```
border: 1px black #000;
```

22. Visit the website and look at the page source to see that the style sheet data entered in the configuration page shows up in the HTML header:

```
<style type='text/css'>
.private
{
    color: #6E6A6B;
}
.register
{
    background-color: #ff4d4d;
    color: #fff;
    padding-left: 10px;
```

```
}
</style>
```

How it works...

Re-using many of the elements covered in this chapter, this recipe creates a simple, yet effective, configuration interface to allow users to make changes to the color that is used to highlight private text in posts, instead of this color being hardcoded in a plugin file.

That being said, this recipe does introduce a few new concepts. The first is the initialization of the plugin options by reading data from a file instead of having all of that information stored in the PHP code. This technique is useful when dealing with an option that has a lot of content, such as a style sheet.

The next element of interest lies within the data processing function, where the code checks to see which button was pressed between the one to reset the style sheet and the one to submit user changes to be stored in the site database. Based on the result, the processing code will either read back the initial style sheet from the file or use the user-posted data to update the configuration data.

We've also created a utility function using a third-party PHP library to clean and validate user-submitted style sheet data as well as the style sheet that is read from a local plugin file. This is necessary since WordPress does not have any built-in data sanitization functions that are specific to validating CSS code.

> **Note**
>
> If you plan to distribute your plugin in the official WordPress plugin repository, you need to be careful when incorporating code from other developers in your plugin and always check the type of license that is associated with that code. In this specific example, the CSSTidy library is licensed under the GPL 2 license, which is the same as WordPress, so we can include this code with no issues. The same could be said of any GPL-compatible licenses, but you'll get in trouble if you include code that is not compatible with GPL. You can visit `https://www.gnu.org/licenses/license-list.en.html` for a comprehensive list of licenses that are compatible, or not, with GPL.

The last new element that we used in this recipe is the built-in WordPress **CodeEditor** extension that enhances our style sheet editor from a simple `textarea` to a clean code editor with line numbers, syntax completion, and code highlighting. If you dig around, you'll find that WordPress incorporates a large number of tools that are accessible to plugin developers.

Beyond these new concepts, the other main change we made is to the code that outputs stylesheet code to the page header. In this new version, we echo the content of the style sheet that is stored in the options table directly to the browser instead of reading it from a file and queuing it for display.

See also

- The *Creating a new enclosing shortcode* recipe in *Chapter 2*, *Plugin Framework Basics*
- The *Loading a style sheet to format plugin output* recipe in *Chapter 2*, *Plugin Framework Basics*

Managing multiple sets of user settings from a single admin page

Throughout this chapter, you have learned how to create configuration pages to manage single sets of configuration options. In some cases, only being able to specify a single set of options will not be enough. For example, looking back at the Twitter Embed shortcode plugin that was created in the previous chapter, we might want to add some options to specify the number of tweets to display and the width of the Twitter feed. It would also be interesting to be able to create multiple configurations and select the one to use when placing the shortcode in a page or post, as shown in this shortcode example: `[twitterfeed user_name="WordPress" option_id="2"]`.

While the first thought that might cross your mind to configure such a plugin is to create a multi-level menu item with submenus to store a number of different settings, this method would produce a very awkward interface for users to navigate. A better way is to use a single configuration page and to give the user a way to select between multiple sets of options to be modified.

In this recipe, you will learn how to enhance the previously created Twitter feed shortcode plugin to be able to control the embedded feed width and number of tweets to display from the plugin configuration panel and give users the ability to configure multiple display sizes.

Getting ready

You should have already followed the *Creating a new shortcode with parameters* recipe in the previous chapter to have a starting point for this recipe. The resulting plugin should also be activated on your development site. Alternatively, you can get the resulting code (`ch2/ch2-twitter-embed/ch2-twitter-embed.php`) from the book's GitHub page.

How to do it...

Follow these steps to learn how to create a multi-setting plugin configuration page:

1. Navigate to the `ch2-twitter-embed` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch2-twitter-embed.php` file in a text editor.
3. Add the following lines of code at the end of the file to assign an activation callback to initialize plugin options when it is installed or upgraded and implement that function:

```
register_activation_hook( __FILE__,
    'ch2te_set_default_options_array' );

function ch2te_set_default_options_array() {
    ch2te_get_options();
}
```

4. Add the following block of code to implement the `ch2te_get_options` function, referenced in the previous block of code:

```
function ch2te_get_options( $id = 1 ) {
    $options = get_option( 'ch2te_options_' . $id,
                           array() );

    $new_options['setting_name'] = 'Default';
    $new_options['width'] = 560;
    $new_options['number_of_tweets'] = 3;

    $merged_options = wp_parse_args( $options,
                                     $new_options );
    $compare_options =
```

```
            array_diff_key( $new_options, $options );
        if ( empty( $options ) ||
                !empty( $compare_options ) ) {
            update_option( 'ch2te_options_' . $id,
                                $merged_options );
        }
        return $merged_options;
    }
```

5. Insert the following code segment to register a function to be called when the administration menu is assembled. When this happens, the callback function adds an item to the **Settings** menu and specifies the function to be called to render the configuration page:

```
add_action( 'admin_menu', 'ch2te_settings_menu' );

function ch2te_settings_menu() {
    add_options_page( 'Twitter Embed Configuration',
                      'Twitter Embed',
                      'manage_options',
                      'ch2te-twitter-embed',
                      'ch2te_config_page' );
}
```

6. Add the following code to provide a partial implementation for the configuration page rendering function:

```
function ch2te_config_page() {
    if ( isset( $_GET['option_id'] ) ) {
        $option_id = intval( $_GET['option_id'] );
    } elseif ( isset( $_POST['option_id'] ) ) {
        $option_id = intval( $_POST['option_id'] );
    } else {
        $option_id = 1;
    }

    $options = ch2te_get_options( $option_id ); ?>

<?php }
```

7. Insert the following code within the `ch2te_config_page` function body, after the call to `ch2te_get_options` and the closing php statement (`?>`), and before the closing `<?php } ` statement, to display the configuration page title along with an update message when options are saved:

```
<div id="ch2te-general" class="wrap">
<h2>Twitter Embed</h2>

<!-- Display message when settings are saved -->
<?php if ( isset( $_GET['message'] ) &&
        $_GET['message'] == '1' ) { ?>
    <div id='message' class='updated fade'>
    <p><strong>Settings Saved</strong></p></div>
<?php } ?>
```

8. Insert the next block of code, still in the `ch2te_get_options` function body, after the `<?php } ?>` line of code and before the closing `<?php } ` line of code, to display a series of tabs that allow the user to switch between multiple configurations:

```
<!-- Option selector -->
<div id="icon-themes" class="icon32"><br></div>
<h2 class="nav-tab-wrapper">
<?php for ( $counter = 1;$counter <= 5;$counter++ ) {
    $temp_options = ch2te_get_options( $counter );
    $class = ( $counter == $option_id ) ?
              ' nav-tab-active' : ''; ?>

<a class="nav-tab<?php echo esc_html( $class ); ?>"
   href="<?php echo
   add_query_arg(
       array( 'page' => 'ch2te-twitter-embed',
              'option_id' =>intval( $counter ) ),
              admin_url( 'options-general.php' ) );
   ?>"><?php echo intval( $counter );
if ( $temp_options !== false ) {
   echo ' (' .  esc_html(
        $temp_options['setting_name'] ) . ')';
} else {
```

```
    echo '(Empty)';
} ?>
</a>
<?php } ?>
</h2><br />
```

9. Add this final block of code to the body of the `ch2te_get_options` function body, after the `</h2></br>` line of code and before the closing `<?php }` statement, to display the actual configuration fields:

```
<!-- Main options form -->
<form name="ch2te_options_form" method="post"
      action="admin-post.php">
<input type="hidden" name="action"
       value="save_ch2te_options" />
<input type="hidden" name="option_id"
       value="<?php echo intval( $option_id ); ?>" />
<?php wp_nonce_field( 'ch2te' ); ?>
<table>
<tr><td>Setting name</td>
    <td><input type="text" name="setting_name"
           value="<?php echo
           esc_html( $options['setting_name'] ); ?>"/>
    </td>
</tr>
<tr><td>Feed width</td>
    <td><input type="text" name="width"
               value="<?php
               echo intval( $options['width'] );
    ?>"/></td>
</tr>
<tr><td>Number of Tweets to display</td>
<td><input type="text" name="number_of_tweets"
           value="<?php echo
           intval( $options['number_of_tweets'] ); ?>"
           />
</td></tr>
```

```
</table><br />
<input type="submit" value="Submit"
       class="button-primary" />
</form>
</div>
```

10. Add the following block of code at the end of the plugin file to register a function that will process user options when submitted to the site:

```
add_action( 'admin_init', 'ch2te_admin_init' );

function ch2te_admin_init() {
    add_action( 'admin_post_save_ch2te_options',
                'process_ch2te_options' );
}
```

11. Add the following code to implement the `process_ch2te_options` function, declared in the previous block of code:

```
function process_ch2te_options() {
    if ( !current_user_can( 'manage_options' ) ) {
        wp_die( 'Not allowed' );
    }

    check_admin_referer( 'ch2te' );

    if ( isset( $_POST['option_id'] ) ) {
        $option_id = intval( $_POST['option_id'] );
    } else {
        $option_id = 1;
    }

    $options = ch2te_get_options( $option_id );

    foreach ( array( 'setting_name' )
              as $param_name ) {
        if ( isset( $_POST[$param_name] ) ) {
            $options[$param_name] =
```

```
                    sanitize_text_field(
                        $_POST[$param_name] );
            }
        }

        foreach( array( 'width',
                        'number_of_tweets' )
                as $param_name ) {
            if ( isset( $_POST[$param_name] ) ) {
                $options[$param_name] =
                    intval( $_POST[$param_name] );
            }
        }

    }
```

12. Insert the following block of code in `process_ch2te_options`, in the function body after the code from the previous step, to actually save the updated options to the site options table and redirect the user to the plugin configuration page:

```
    $options_name = 'ch2te_options_' . $option_id;
    update_option( $options_name, $options );

    $clean_address = add_query_arg( array( 'message' => 1,
                     'option_id' => $option_id,
                     'page' => 'ch2te-twitter-embed' ),
                     admin_url( 'options-general.php' ) );
    wp_redirect( $clean_address );
    exit;
```

13. Find the `ch2te_twitter_embed_shortcode` function and modify it as follows to accept the new `option_id` parameter and load the plugin options to produce the desired output. The changes are identified in bold:

```
    function ch2te_twitter_embed_shortcode( $atts ) {
        extract( shortcode_atts( array(
                     'user_name' => 'ylefebvre',
                     'option_id' => '1'
```

```
    ), $atts ) );

    if ( intval( $option_id ) < 1 ||
            intval( $option_id ) > 5 ) {
        $option_id = 1;
    }
    $options =
        ch2te_get_options( intval( $option_id ) );

    if ( empty( $user_name ) ) {
        $user_name = 'ylefebvre';
    } else {
        $user_name =
            sanitize_text_field( $user_name );
    }

    if ( !empty( $user_name ) {
        $output =
            '<a class="twitter-timeline" href="';
        $output .=
            esc_url( 'https://twitter.com/' .
                     $user_name );
        $output .= '" data-width="';
        $output .= intval( $options['width'] );
        $output .= '"data-tweet-limit="';
        $output .=
            intval( $options['number_of_tweets'] );
        $output .= '">Tweets by ' .
                       esc_html( $user_name );
        $output .= '</a><script async ';
        $output .=
            'src="//platform.twitter.com/widgets.js"';
        $output .= ' charset="utf-8"></script>';
    } else {
        $output = '';
    }
```

```
        return $output;
    }
```

14. Save and close the plugin file.
15. **Deactivate** and then **Activate** the **Chapter 2 - Twitter Embed** plugin from the administration interface to execute its activation function and create default settings.
16. Navigate to the **Settings** menu and select the **Twitter Embed** submenu item to see the newly created configuration panel with the first set of options being displayed and more sets of options accessible through the tabs shown at the top of the page.
17. Edit and **Submit** the options for the first configuration tab.

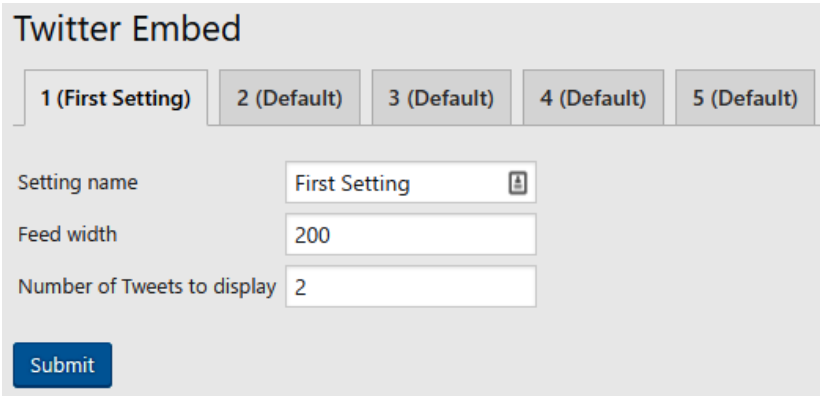


Figure 3.14 – Multi-tab interface for plugin configuration options

18. To select the set of options to be used, add the `option_id` parameter to the shortcode used to display a Twitter feed, as follows:

```
[twitterfeed user_name="WordPress" option_id="1"]
```

How it works...

This recipe shows how we can leverage options arrays to create multiple sets of options simply by creating the name of the options array on the fly. Instead of having a specific option name in the first parameter of the `get_option` function call, we create a string with an option ID. This ID is sent through as a URL parameter on the configuration page and as a hidden text field when processing the form data.

The rest of the code is similar to the other examples that we have seen in this chapter, but with the extra twist of creating a slick tab interface, since the way to access the array elements remains the same.

See also

- The *Rendering the admin page contents using HTML* recipe

Creating a network-level plugin with admin pages

A very powerful feature of WordPress is its ability to run multiple websites from a single installation of the platform. Each site can have its own content and its own visual identity, while administrators can manage all sites from one dashboard. When you run WordPress as a multisite network, it adds an additional section to the dashboard, for network-level management. All of the plugins we have created so far in this chapter present configuration panels at the site level, allowing for the custom configuration of options on a per-site basis. While this will be the desired behavior for most plugins, it might be preferable for some configuration elements to be set in a single place by a network-level administrator and applied to all the sites in the network.

In this recipe, you will learn how to modify our Google Analytics plugin to be configured at the network level when running on a multi-site installation and at the site level on regular WordPress configurations. This would allow a network site administrator to set Google Analytics credentials once for all sites that will be running off the network.

Getting ready

You should have already followed the *Accessing user settings from action and filter hooks* recipe, and the resulting plugin should be active on your development site. Alternatively, you can get the resulting code (`ch3/ch2-page-header-output/ch2-page-header-output-v9.php`) from the book's GitHub page. You should rename the `ch2-page-header-output-v9.php` file as `ch2-page-header-output.php` before starting this recipe.

You should also have access to a WordPress installation configured in Network mode. You can read more about the steps required to configure WordPress to run as a multisite at `https://wordpress.org/support/article/create-a-network/`. If you're using Local as your development web server, you can simply create a new test site and set the **multisite** option in the **Advanced Options** section when creating your admin user account.

How to do it...

Follow these steps to convert a plugin we created previously to display its configuration page at the network level or single-site level depending on the type of WordPress installation:

1. Navigate to the `ch2-page-header-output` folder of the WordPress plugin directory of your development installation.
2. Open the `ch2-page-header-output.php` file in a text editor.
3. Modify the implementation of the `ch2pho_get_options` function to replace both option functions with network-level versions. The code section shows the two lines that need to be changed with the modifications in bold:

```
$options = get_site_option( 'ch2pho_options',
                            array() );

update_site_option( 'ch2pho_options',
                    $merged_options );
```

4. Locate the `add_action( 'admin_menu'` function call that adds a callback to populate the admin menu and add code to check whether the installation is a single site or multisite before registering callbacks. The following code shows new elements in bold:

```
if ( is_multisite() ) {
    add_action( 'network_admin_menu',
                'ch2pho_settings_menu' );
} else {
    add_action( 'admin_menu',
                'ch2pho_settings_menu' );
}
```

5. Modify the `ch2pho_settings_menu` function to add new items to different menus based on whether the site is a regular site or a multisite, with the new sections identified in bold:

```
function ch2pho_settings_menu() {
    if ( is_multisite() ) {
        $options_page = add_submenu_page(
            'settings.php',
            'My Google Analytics Configuration',
```

```
                'My Google Analytics',
                'manage_options',
                'ch2pho-my-google-analytics',
                'ch2pho_config_page' );
        } else {
            $options_page =
                add_submenu_page( 'options-general.php',
                    'My Google Analytics Configuration',
                    'My Google Analytics',
                    'manage_options',
                    'ch2pho-my-google-analytics',
                    'ch2pho_config_page' );
        }
        if ( !empty( $options_page ) ) {
            add_action( 'load-' . $options_page,
                        'ch2pho_help_tabs' );
        }
    }
```

6. In the `process_ch2pho_options` function, replace the call to `update_option` with a call to `update_site_option`:

```
update_site_option( 'ch2pho_options', $options );
```

7. Still in the `process_ch2pho_options` function, make the following changes around the `wp_redirect` function call, with new elements shown in bold:

```
if ( is_multisite() ) {
    $redirect_page = '/network/settings.php';
} else {
    $redirect_page = 'options-general.php';
}
wp_redirect( add_query_arg(
    array( 'page' => 'ch2pho-my-google-analytics',
           'message' => '1' ),
    admin_url( $redirect_page ) ) );
```

8. Modify the `ch2pho_config_page` function to modify the form action so that it finds `admin-post.php` in the multisite installations, with the new code shown in bold:

```
<form method="post"
      action="<?php echo
          admin_url( 'admin-post.php' ); ?>">
```

9. Save and close the plugin file.
10. In a network installation of WordPress, visit the **Plugins** section of **Network Admin**.
11. **Network Activate** the **Chapter 2 - Page Header Output** plugin. You will see that a new menu item labeled **My Google Analytics** has appeared in the **Settings** menu.

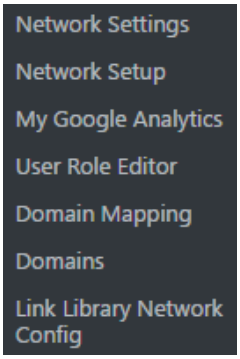


Figure 3.15 – Network-level plugin configuration menu

12. Visit any site in the network and inspect the page source.
13. Search for the user account that you configured on the network-level administration page to see that the Google Analytics code is still added to the page header. The plugin also works correctly when installed in single-site installations.

How it works...

When creating a network-level plugin, the `is_multisite` function becomes very useful to determine whether or not a site is configured as a multisite. In the recipe, we used it to register callbacks to be called when building different menus (the regular single-site administration menu and the network management menu). We also used the `is_multisite` function when registering the actual menu item to determine where users should be redirected after saving options.

The other useful functions in these modifications are `get_site_option` and `update_site_option`. When called in a multisite configuration, these functions will retrieve network-level options from the database. Alternatively, when running on a regular single site, they will default to access site-level options. By using these functions in our updated code, we make all the interactions with site options compatible with single and multisite installations.

The last element of this recipe relates to how we activated this plugin at the network level. This means that it will be active on all sites created in the network. More specifically, in the case of our plugin, this means that all the sites will output Google Analytics code in their header, with the associated account being specified once on the network configuration.

See also

- The *Accessing user settings from action and filter hooks* recipe

4
The Power of Custom Post Types

Building on its history of openness and ease of use, WordPress reached new heights in customization when it introduced custom post types.

Custom post types are a new category of items that are created by using the WordPress API and that appear in the WordPress administration interface as completely new sections, next to the default **Posts** and **Pages** sections. These custom items can be used to store any type of information, including events, bug reports, recipes, movie reviews, and many more.

When using custom post types to implement this kind of functionality, developers can take advantage of WordPress' internal content editing capabilities, including its powerful text editor and user-friendly media uploader. Custom post types also simplify data management for developers, since all of the information related to these new entries is stored in the site database using the existing table structures. Finally, custom post types can leverage the established theme and template system to display the information that site administrators store in these new content types.

If you have ever taken a peek at the MySQL database behind a WordPress site, you know that posts, pages, attachments, revisions, and navigation menu items share the same tables. In essence, all of these data elements are custom post types, with some of them using the standard text editor, while others, such as the navigation menus, have a custom management interface. Each of these types of items also has a different mechanism to be displayed on a site.

Using custom post types opens up endless possibilities to tailor the functionality of a WordPress installation and provide a custom solution to end users without needing to invest a large amount of time re-inventing the wheel. This chapter covers all facets of creating custom post types through the creation of a book review system. This includes the creation of new types of elements, displaying the newly stored information on the website, and customizing the environment to create an editor with unique capabilities.

This chapter covers one of the most powerful features of WordPress, custom post types, through the following recipes:

- Creating a custom post type
- Adding a new section to the custom post type editor
- Displaying single custom post type items using a custom layout
- Tailoring search output for custom post type items
- Displaying custom post type data in shortcodes
- Adding custom categories for custom post types
- Adding custom fields to categories
- Hiding the category editor from the custom post type editor
- Displaying additional columns in the custom post list page
- Adding filters for custom categories to the custom post list page
- Adding Quick Edit fields for custom categories
- Updating the page title to include custom post data using plugin filters

Technical requirements

You can find complete code examples on GitHub in the following repository: `https://github.com/PacktPublishing/WordPress-Plugin-Development-Cookbook-Third-Edition`.

Creating a custom post type

The initial creation of a custom post type is extremely easy. It only requires a single function to be called from an action hook callback. Once in place, a lot of functionality immediately becomes available to editors, administrators, and site visitors.

Getting ready

You should have access to a WordPress development environment, either on your local computer or a remote server, where you will be able to load your new plugin files.

How to do it...

Follow these steps to create a new custom post type that will be used to store book reviews:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch4-book-reviews`.
3. Navigate to this directory and create a new text file called `ch4-book-reviews.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 4 - Book Reviews`.
5. Add the following line of code to register a function that will be executed during the initialization phase every time WordPress generates a page:

```
add_action( 'init', 'ch4_br_create_book_post_type' );
```

6. Add the following code block to provide an implementation for the `ch4_br_create_book_post_type` function:

```
function ch4_br_create_book_post_type() {
    register_post_type( 'book_reviews',
        array(
            'labels' => array(
                'name' => 'Book Reviews',
                'singular_name' => 'Book Review',
                'add_new' => 'Add New',
                'add_new_item' =>
                    'Add New Book Review',
                'edit' => 'Edit',
```

```
                    'edit_item' => 'Edit Book Review',
                    'new_item' => 'New Book Review',
                    'view' => 'View',
                    'view_item' => 'View Book Review',
                    'search_items' =>
                        'Search Book Reviews',
                    'not_found' =>
                        'No Book Reviews found',
                    'not_found_in_trash' =>
                        'No Book Reviews found in Trash',
                    'parent' => 'Parent Book Review',
                ),
                'public' => true,
                'menu_position' => 20,
                'supports' =>
                    array( 'title', 'editor', 'comments',
                           'thumbnail', 'custom-fields' ),
                'taxonomies' => array( '' ),
                'menu_icon' => 'dashicons-book-alt',
                'has_archive' => false,
                'exclude_from_search' => true,
            )
        );
    }
```

7. Save and close the plugin file.
8. Navigate to the **Plugins** management page and **Activate** the **Chapter 4 – Book Reviews** plugin.
9. Click on the newly available **Book Reviews** menu item, located under the **Pages** section, to see the **Book Reviews** creation and management interface.

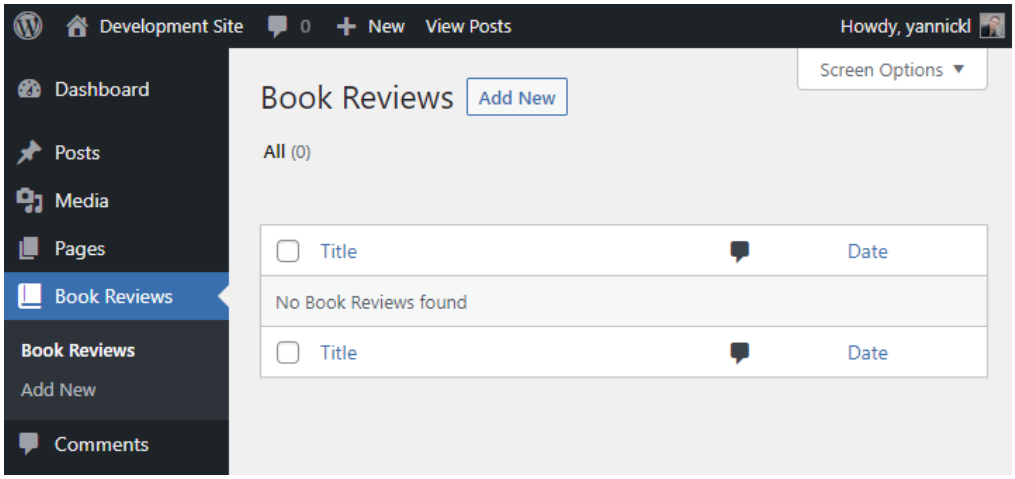


Figure 4.1 – The new Book Reviews custom post type

10. Click on the **Add New** button, next to the section title, to display the **Book Review** editor featuring the complete WordPress text editor, the **Custom Fields** editor, **Discussion** control, publishing controls, and the **Featured Image** section.
11. Fill in the new entry by specifying the **Book Review** title (for example, `WordPress Plugin Development Cookbook`).
12. Enter text in the main content editor to provide a short review for the book.
13. Scroll to the **Custom Fields** section and type `book_author` as the **Name** of the field and `Yannick Lefebvre` for **Value**. Click **Add Custom Field** to save the values and create a second field.

> **Note**
> If some custom fields already exist in your WordPress installation (from other plugin data entries), you will need to click on **Enter new** before being able to set **Name** to `book_author`.

14. Set the **Name** of the second field to `book_rating` and **Value** to `5`. Click on **Add Custom Field** to save this second entry.
15. Find and download a book cover image from a website such as Google Images (`https://images.google.com`), Packt (`https://www.packtpub.com`), or the book's GitHub page (`ch4/ch4-book-reviews/WPPluginDevCookbook.png`).

16. Click on the **Set featured image** link located in the right-hand sidebar of the editing interface.
17. Click on **Upload Files** to pick the image that you downloaded and have WordPress upload it to the `wp-content/uploads` folder of your site.
18. Once the file is uploaded and WordPress displays information about it, look at the bottom of the media upload dialog and click on the **Set featured image** link to associate it with the book review that you are creating.

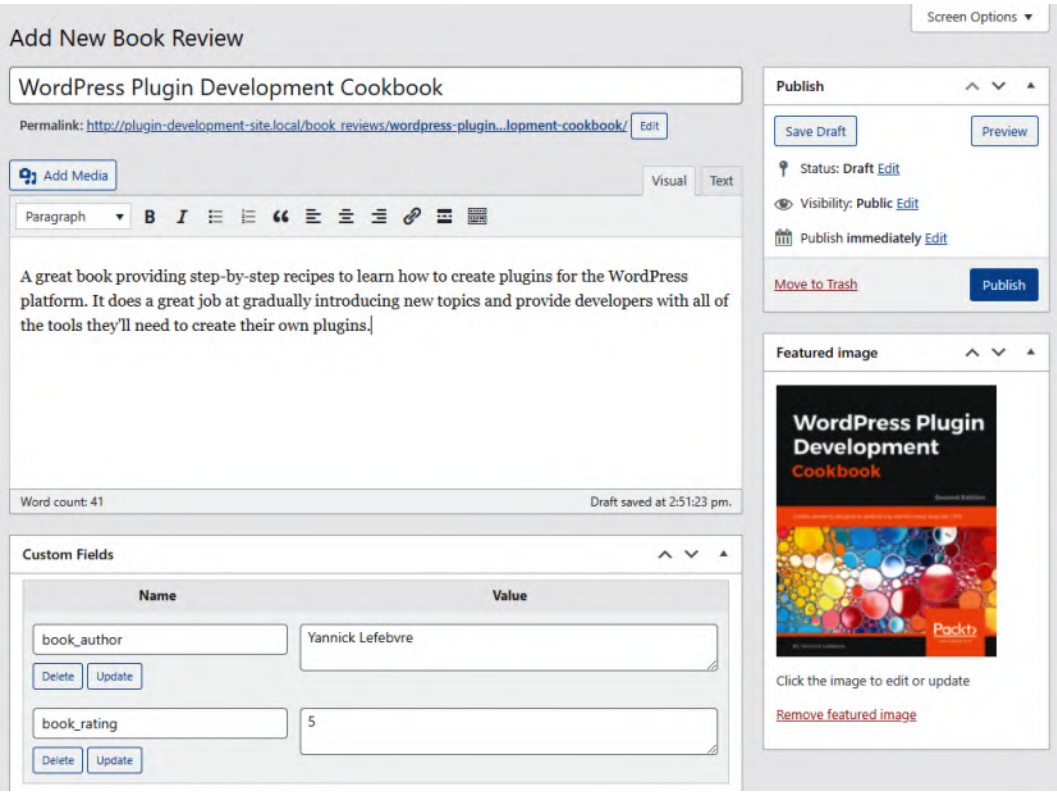


Figure 4.2 – Book Review editor

19. Click on the **Publish** button to save this first book review.
20. Visit the **Permalinks** section of the **Settings** menu and click on the **Save Changes** button (without making any changes to your **Permalinks** settings).
21. Click on the **Book Reviews** dashboard menu link to see your review and click on the **View** link next to it to see the newly created content in your web browser.

How it works...

By making a call to the `register_post_type` function, the entire WordPress environment becomes aware of the existence of this new post type. This awareness includes the creation of a dedicated section to create and edit posts of this type and the ability to process web page requests for book reviews.

As mentioned at the beginning of this recipe, the function is quite simple to use and only requires two arguments:

```
register_post_type( $post_type, $args );
```

The first argument is a text string that indicates the name of the post type. Please note when choosing this name that it will be used as the default value for the permalinks of all items that use the new type and that it should be unique enough to avoid potential conflicts with other plugins.

The second argument is an array of properties that specify the characteristics of the new post type and determine how this type will be edited.

In this specific example, the first element of the properties array is actually another array that contains a number of labels. These labels indicate the text strings that should be displayed when managing items created under the new post type. For example, if we look at *Figure 4.1*, the message **No Book Reviews found** came directly from the definition of the `not_found` label in this array.

The second argument, named `public`, determines whether the post type's administration interface should be shown to manage it and whether visitors should be able to view single items. Next is the `menu_position` member of the configuration array, indicating the desired position of the new element in the administration menu. In this example, a value of `20` indicates that it should be displayed after the **Pages** menu item. Visit the WordPress Code Reference site (`https://developer.wordpress.org/reference/functions/register_post_type/#menu_position`) for a full list of potential values for this parameter and their associated positions.

The `supports` parameter is another array that indicates which parts of the content editor should be displayed for items that use the custom post type. In this case, we left out some sections, such as `author`, `excerpt`, `trackbacks`, `revisions`, `page-attributes`, and `post-formats`, as they were not desirable for our book review system.

The next few parameters in the configuration array indicate that we do not want to define custom taxonomies at this time and specify the name of a Dashicons font element that should be displayed next to the post type's name in the administration menu. Similar to what we saw in the *Creating a multi-level administration menu* recipe from *Chapter 3*, *User Settings and Administration Pages*, we can also use an SVG vector image or an image file for this parameter. Finally, the last two arguments determine whether WordPress should present an archive listing page for the new type when users visit the `/book_reviews` page on the site and whether or not book reviews should be excluded from search results.

There are actually many other parameters that can be included in the configuration array to get more precise control over some aspects of the new custom post type. Please visit the WordPress Code Reference site (`https://developer.wordpress.org/reference/functions/register_post_type/`) to learn more about them.

There's more...

While the editor for our book reviews currently looks more like the classic WordPress editor, it's possible to use the new modern **Block Editor** for our new custom post type. It is also possible to improve the permalinks for our book review system instead of using the default internal post type name.

Using the WordPress Block Editor for custom post types

Recently introduced to WordPress, the block editor (also known by its code name **Gutenberg**) drastically changed the experience of creating content in WordPress. By adding a single variable in the array of properties sent to the `register_post_type` function, we can tell WordPress to use this new editor for our custom post type:

```
'show_in_rest' => true,
```

While the property name `show_in_rest` might not seem related to the block editor, it is actually linked to the new editor's reliance on the **REST** API. Once the additional parameter is added to the array sent to the `register_post_type` function, editing or creating new book reviews displays the block editor:

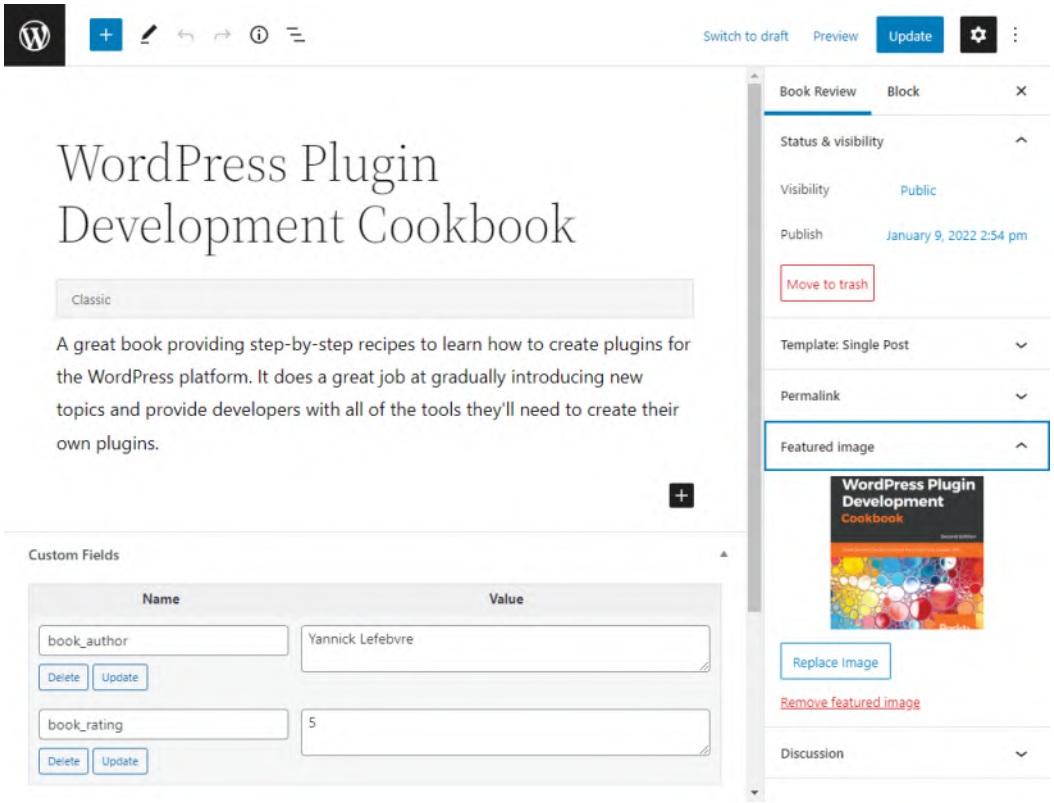


Figure 4.3 – A new book review created with the WordPress Block Editor

That being said, a lot of plugin developers still prefer to use the *Classic* editor for their custom post types and don't set this property. This older editor tends to be easier to use and is also better suited to adding custom fields to assign additional data to our new items. In this cookbook, all recipes have been tested with both editor modes, with most screenshots representing the Classic Editor.

> **Note**
>
> The **Custom Fields** editor does not appear by default when using the block editor even if it was specified in the `supports` parameter when we called `register_post_type`. Users must go to the drop-down menu in the top-right corner of the editor (shown as three vertical dots, next to the **Publish** button and the gear icon), select the **Preferences** menu, go to the **Panels** tab, enable **Custom fields,** and click the **Enable & Reload** button to display that section.

Changing the custom post type permalinks slug

An optional member of the custom post type configuration is the rewrite parameter. It can be defined as follows:

```
'rewrite' => array( 'slug' => 'awesome-book-reviews' ),
```

Although this may seem very simple, the permalinks will only change over after you go to the **Permalinks** section of the administration interface and click on the **Save Changes** button, as we did in the recipe. Alternatively, it is possible to call functions related to the WordPress **rewrite** module to programmatically request for the permalinks configuration to be rebuilt. As this is not something that should be done every time WordPress displays a page, but would be too early to do when a plugin gets initialized or upgraded, a good place to call these functions would be within the plugin options storage and update function. You might even decide to give administrators the ability to specify their own slug in a plugin configuration page. The code to reset the permalinks rules is as follows:

```
global $wp_rewrite;
$wp_rewrite->flush_rules();
```

See also

- The *Creating a multi-level administration menu* recipe in *Chapter 3*, *User Settings and Administration Pages*

Adding a new section to the custom post type editor

While the custom post editor that has been put in place so far is functional, it is not the friendliest of user interfaces, especially with the custom fields section, where users need to type or select the names of each field as they create new items. A cleaner approach is to create a custom interface using the meta box mechanism that we saw in the previous chapter to display all the data associated with our book reviews.

Getting ready

You should have already followed the *Creating a custom post type* recipe to have a starting point for this recipe, and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch4/ch4-book-reviews/ch4-book-reviews-v1.php`) from the book's GitHub page and rename the file `ch4-book-reviews.php`.

How to do it...

Follow these steps to create a meta box that will be associated with a custom post type and learn how to save the information that is entered in that new interface:

1. Navigate to the `ch4-book-reviews` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch4-book-reviews.php` file in a code editor.
3. Add the following line of code after the existing functions to register a function to be called when the administration interface is visited:

```
add_action( 'admin_init', 'ch4_br_admin_init' );
```

4. Add the following code section to provide an implementation for the `ch4_br_admin_init` function and register a meta box to be associated with the `book_reviews` post type:

```
function ch4_br_admin_init() {
    add_meta_box( 'ch4_br_review_details_meta_box',
        'Book Review Details',
        'ch4_br_display_review_details_mb',
        'book_reviews', 'normal', 'high' );
}
```

5. Insert this function to implement the `ch4_br_display_review_details_mb` function and render the meta box contents:

```
function ch4_br_display_review_details_mb(
                                        $book_review ) {
    $book_author = get_post_meta( $book_review->ID,
        'book_author', true );
    $book_rating = get_post_meta( $book_review->ID,
        'book_rating', true );
?>
    <table>
        <tr>
            <td>Book Author</td>
            <td><input type="text" style="width:100%"
                name="book_review_author_name"
                value="<?php echo
```

```
                        esc_html( $book_author ); ?>" />
                </td>
            </tr>
            <tr>
                <td style="width: 150px">Book Rating</td>
                <td>
                    <select style="width: 130px"
                            name="book_review_rating">
                    <option value="">
                        Select rating</option>
  <?php for ( $rating = 5;$rating >= 1;$rating -- ) { ?>
        <option value="<?php echo intval( $rating ); ?>"
        <?php echo selected( $rating, $book_rating ); ?>>
        <?php echo intval( $rating ); ?> stars
  <?php } ?>
                    </select>
                </td>
            </tr>
        </table>
  <?php }
```

6. Add the following code segment to register a function that will be called when posts are saved to the database:

```
add_action( 'save_post',
            'ch4_br_add_book_review_fields',
            10, 2 );
```

7. Add an implementation for the `ch4_br_add_book_review_fields` function defined in the previous `add_action` call:

```
function ch4_br_add_book_review_fields(
        $book_review_id, $book_review ) {
    if ( 'book_reviews' != $book_review->post_type ) {
        return;
    }

    if ( isset($_POST['book_review_author_name']) ) {
```

```
            update_post_meta( $book_review_id,
                'book_author',
                sanitize_text_field(
                    $_POST['book_review_author_name'] ) );
        }
        if ( isset( $_POST['book_review_rating'] ) &&
                !empty( $_POST['book_review_rating'] ) ) {
            update_post_meta( $book_review_id,
                'book_rating',
                intval( $_POST['book_review_rating'] ) );
        }
    }
```

8. Find the `ch4_br_create_book_post_type` function, where the new book type was originally created, and remove the `custom-fields` element from the `supports` array:

```
    'supports' => array( 'title', 'editor', 'comments',
                         'thumbnail' ),
```

9. Save and close the plugin file.
10. Back on your development site's dashboard, edit the book review created previously.

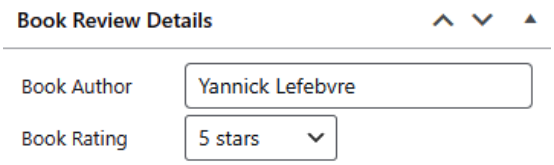


Figure 4.4 – New section for specifying the book review details

As we can see, a new **Book Review Details** meta box is now available, containing a text field to specify the author's name and a drop-down list for the rating.

How it works...

This recipe uses the WordPress built-in meta box system to create a clean interface that will allow users to manage fields specific to custom post types without having to use the cumbersome default **Custom Fields** editor. As we saw in *Chapter 3*, *User Settings and Administration Pages*, custom meta boxes can be created using the `add_meta_box` function. In addition to declaring the meta box and associating it with the custom post type, `add_meta_box` defines a callback that is responsible for rendering the contents of the box.

The next step of the recipe implements the function that renders the meta box content. As we can see, this box receives a variable that contains information about the book review that is being displayed in the post editor. Using this object, our code retrieves the current book review post ID and uses it to query the site database for the author name and rating associated with the entry. Once the custom field data has been retrieved from the database, it can be used to render both fields on screen. When new book reviews are first being created, both calls to `get_post_meta` will return an empty string, resulting in the display of an empty text field and the first entry in the drop-down list.

Taking a closer look at the `get_post_meta` function, we can see that it has three parameters:

```
get_post_meta( $post_id, $field_name, $single );
```

The first parameter is the post ID, which is used to identify the post with which the custom information is associated. The second argument is the custom field name, which should be unique and should match the name specified when the fields were created in the previous recipe. The third and final argument indicates whether the return value should be a single value or an array of values. If set to `false`, it will produce an array containing a single element even if the custom field only contains a single value. In most cases, it should be set to `true` to receive a single value that can be accessed directly.

The final steps in this recipe take care of registering a function that will be called when posts of all types are saved or deleted from the administration interface. Since it will deal with all types of data, the saving callback should first check the type of the post data received. If it's a book review, the code proceeds to check whether the received data is valid and stores the information in the post metadata table. In this recipe, the parameters for the `update_post_meta` function are similar to the `get_post_meta` function, except for the third argument, which is used to specify the data to be stored.

One last detail that should be mentioned about this recipe is the use of the fourth parameter of the `add_action` function when associating a callback with the `save_post` action hook. This argument indicates that two arguments will be received by the registered callback. If this argument is not set, the callback function will not receive that second piece of data.

See also

- The *Creating a custom post type* recipe
- The *Formatting admin pages using meta boxes* recipe in *Chapter 3*, *User Settings and Administration Pages*

Displaying single custom post type items using a custom layout

When displaying an entry created in our new custom post type, the default layout offered by our current site theme may not always be able to pleasantly display the information it contains. In most cases, you will be able to see the main post content, but not any of the custom fields data that is associated with the post.

Getting ready

You should have already followed the *Adding a new section to the custom post type editor* recipe to have a starting point for this recipe and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch4/ch4-book-reviews/ch4-book-reviews-v2.php`) from the book's GitHub page and rename the file `ch4-book-reviews.php`.

How to do it...

Follow these steps to create a custom layout to display all the elements that we stored in the book review created in the previous recipe:

1. Navigate to the `ch4-book-reviews` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch4-book-reviews.php` file in a code editor.

3. Add the following line of code after the existing code to register a function to be called when WordPress is deciding which theme template to use to render content:

```
add_filter( 'template_include',
            'ch4_br_template_include', 1 );
```

4. Add the following code section to provide an implementation for the `ch4_br_template_include` function:

```
function ch4_br_template_include( $template_path ) {
    if ( is_single() &&
            'book_reviews' == get_post_type() ) {
        // checks if the file exists in theme first,
        // otherwise install content filter
        if ( $theme_file = locate_template( array
                ( 'single-book_reviews.php' ) ) ) {
            return $theme_file;
        } else {
            add_filter( 'the_content',
                'ch4_br_display_single_book_review',
                20 );
        }
    }
    return $template_path;
}
```

5. Add the following code section to implement the `ch4_br_display_single_book_review` function to display book reviews, including their custom fields:

```
function ch4_br_display_single_book_review( $content ) {
    if ( empty( get_the_ID() ) ) {
        return $content;
    }

    // Display featured image
    $content = '<div style="float: right; ';
    $content .= 'margin: 10px">';
    $content .= get_the_post_thumbnail( get_the_ID(),
                                        'medium' );
```

```
        $content .= '</div>';
        $content .= '<div class="entry-content">';

        // Display Author Name
        $content .= '<strong>Author: </strong>';
        $content .= esc_html( get_post_meta( get_the_ID(),
                                 'book_author', true ) );
        $content .= '<br />';

        // Display yellow stars based on rating -->
        $content .= '<strong>Rating: </strong>';
        $nb_stars = intval( get_post_meta( get_the_ID(),
                              'book_rating', true ) );
        $content .=
            str_repeat( '<img style="margin: 0" src="'
            . plugins_url( 'star-icon.png', __FILE__ )
            . '" />', $nb_stars );
        $content .=
            str_repeat( '<img style="margin: 0" src="' .
            plugins_url( 'star-icon-grey.png', __FILE__ )
            . '" />', 5 - $nb_stars );

        // Display book review contents
        $content .= '<br /><br />';
        $content .= get_the_content( get_the_ID() );
        $content .= '</div>';
        return $content;
    }
```

6. Save and close the plugin file.
7. Find and download a PNG format pixel star icon measuring 32 x 32 pixels from a site such as IconArchive (`https://www.iconarchive.com`) and save it as `star-icon.png` in the `ch4-book-reviews` directory.
8. Create a grayscale version of the star icon using any graphic processing tool (for example, the free multi-platform **XnViewMP** tool, found at `https://www.xnview.com/en/`) and save it as `star-icon-grey.png`.

9. Go to the **Book Reviews** management page and click on the **View** link under the existing entry created in the previous recipe.

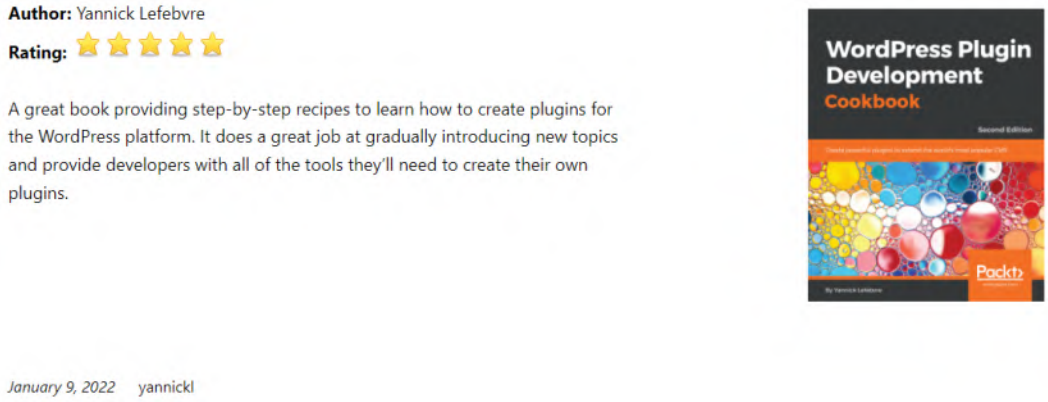


Figure 4.5 – Custom layout for the new post type

As we can see, our custom template function is used to display the review.

How it works...

When rendering any web page, the default WordPress functionality is to search the current theme directory for an applicable template suitable for the content at hand. In the case of a single custom post type item, such as our book reviews, it first looks for a single item template named `single-<post-type-name>.php`, where the latter part is the actual post type name. If it does not find this file, it defaults to the general single item template. In the first recipe of this chapter, the template that was used to show the book review was the default single item template, simply named `single.php`.

To add better support for our new post type, this recipe associates a function with the `template_include` filter hook to change that behavior. More specifically, we use the `locate_template` function to check whether the user provided a template for the `book_reviews` post type in the theme directory. If no template is found, we register a filter to overwrite the page contents with our own layout. This gives users the flexibility to use our predefined layout or to provide their own, but avoids using the default single item template that is not aware of our custom fields.

The rest of the recipe implements our fallback filter function for book review content. This code makes use of many WordPress template functions, such as `get_the_ID()` and `get_the_content()`, as well as the `get_post_meta` function, to display various elements of the current item, including the book author and its rating, as well as the main post content and the featured image.

To help users build their own theme template for your custom post type, you should provide code snippets in your plugin documentation, showing how to retrieve your custom post type's custom fields.

See also

- The *Creating a custom post type* recipe

Tailoring search output for custom post type items

When we first declared our custom post type, we specified that its items should be excluded from search results to keep things simple. However, users are most likely to want custom items created from our plugins to appear in their site search results.

Getting ready

You should have already followed the *Displaying single custom post type items using a custom layout* recipe to have a starting point for this recipe, and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch4/ch4-bookreviews/ch4-book-reviews-v3.php`) from the book's GitHub page and rename the file `ch4-book-reviews.php`.

How to do it...

Follow these steps to learn how to include our custom post types in search results and how to customize the way they appear:

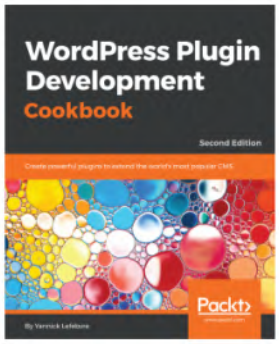
1. Navigate to the `ch4-book-reviews` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch4-book-reviews.php` file in a code editor.
3. Search for the `exclude_from_search` array member in the call to `register_post_type` and change its value from `true` to `false`:

```
  'exclude_from_search' => false
```

4. Save the plugin file.

5. Visit your development site and run a search for a keyword found in one of your book review titles to see how custom post type items are displayed by default, as dictated by your site theme.

WordPress Plugin Development Cookbook



A great book providing step-by-step recipes to learn how to create plugins for the WordPress platform. It does a great job at gradually introducing new topics and provide developers with all of the tools they'll need to create their own plugins.

Figure 4.6 – Default search results for a custom post type item

> **Note**
> The **Twenty Twenty-Two** theme does not offer a search box by default. However, you can use the search box available at the far right of the admin bar at the top of the screen. Other themes often have a search box for visitors.

6. Add the following lines of code at the bottom of the plugin file to associate the callback with two excerpt-related filters and the main content filter:

```
add_filter( 'get_the_excerpt',
            'ch4_br_search_display' );
add_filter( 'the_excerpt', 'ch4_br_search_display' );
add_filter( 'the_content', 'ch4_br_search_display' );
```

7. Add the following block of code to provide an implementation for the `ch4_br_search_display` function:

```
function ch4_br_search_display( $content ) {
    if ( !is_search() &&
            'book_reviews' != get_post_type() ) {
```

```
        return $content;
    }

    $content =
        '<div style="float: right; margin: 10px">'
        . get_the_post_thumbnail( get_the_ID(),
                                  'medium' )
        . '</div><div class="entry-content">'
        . '<strong>Author: </strong>'
        . esc_html( get_post_meta( get_the_ID(),
                            'book_author', true ) )
        . '<br /><strong>Rating: </strong>';

    $nb_stars = intval( get_post_meta( get_the_ID(),
                        'book_rating', true ) );
    $content .=
        str_repeat( '<img style="margin: 0" src="' .
        plugins_url( 'star-icon.png', __FILE__ )
        . '" />', $nb_stars );
    $content .=
        str_repeat( '<img style="margin: 0" src="' .
        plugins_url( 'star-icon-grey.png', __FILE__ )
        . '" />', 5 - $nb_stars );

    $content .= '<br /><br />';
    $content .= wp_trim_words(
        get_the_content( get_the_ID() ), 20 );
    $content .= '</div>';
    return $content;
}
```

8. Add the following line of code to register a callback function with the `the_title` filter hook:

```
add_filter( 'the_title', 'ch4_br_review_title',
            10, 2 );
```

9. Insert the following code to provide an implementation for the `ch4_br_review_title` function:

```
function ch4_br_review_title( $title, $id = null ) {
    if ( !is_admin() && is_search()
            && !empty( $id ) ) {
        $post = get_post( $id );
        if ( !empty( $post ) && $post->post_type
                == 'book_reviews' ) {
            return 'Book review: ' . $title;
        }
    }
    return $title;
}
```

10. Save the plugin file.
11. Refresh the search results page on your development site.

Figure 4.7 – Customized search content display

As we can see, our filter functions have changed the way that search results are displayed. Since the **Twenty Twenty-Two** theme already displays the featured image in the search results, we end up with duplicate images shown. Many other themes don't display featured images in search results. We could add a plugin option to specify whether or not our tailored search output should include an image to alleviate this issue. This is covered in the *Accessing user settings from action and filter hooks* recipe in *Chapter 3*, *User Settings and Administration Pages*. Another tactic would be to assign the image to our book review, using a field other than the featured image. This is covered in the *Extending the post editor to attach custom files with the media uploader* recipe in *Chapter 5*, *Customizing Post and Page Editors*. This way, our image would appear in our custom search output but not as a featured image.

How it works...

The `get_the_excerpt`, `the_excerpt`, and `the_content` filter hooks allow plugins to modify content to be displayed when WordPress and theme files call the functions that share the same names. Since we want to produce the same content no matter which one of these three functions is called, we associate the same callback function with both.

> **Note**
>
> It should be noted that while this code will work with most WordPress themes, it will not necessarily work for all of them since there are no hard rules on how to create templates for search result pages. A common good practice is to use the `get_the_excerpt` or `the_excerpt` functions to retrieve or display the post content, but that does not prevent theme developers from accessing the post contents using a different technique. We associated our callback function with the `the_content` filter hook to help with this, but there may still be other edge cases where our custom output won't be displayed.

When called, our callback function checks whether the site is currently rendering search results (`is_search()`) and whether the post type of the current item being processed for display is `book_reviews`. If both of these conditions are met, we use a template similar to the one used in the previous recipe to display the item's review text, along with its author and star rating. The main difference found in this template is the use of the `wp_trim_words` function to ensure that no more than a specified number of words of the post content are displayed in search results, in addition to putting an ellipsis where the sentence cuts off. This function has three parameters that respectively indicate the text to be trimmed, the maximum number of words to trim it to, and the text to be displayed if the text gets trimmed. The last two parameters are optional and have default values of `55` and an ellipsis (...):

```
wp_trim_words( $text, $num_words, $more )
```

The recipe also registers a callback function for the `the_title` filter hook, to add some text before the title of any book review items that are displayed in search results. Since this function is called from a number of different places as pages are rendered, including when navigation menus are built, our callback code checks to see whether we are currently rendering search results and uses the ID parameter it receives to check whether a post with the `book_reviews` type is being displayed before altering the title.

See also

- The *Displaying single custom post type items using a custom layout* recipe
- The *Accessing user settings from action and filter hooks* recipe in *Chapter 3*, *User Settings and Administration Pages*
- The *Extending the post editor to attach custom files with the Media Uploader* recipe in *Chapter 5*, *Customizing Post and Page Editors*

Displaying custom post type data in shortcodes

To help visitors navigate through the items added using our new custom post type, we will need to allow users to display a list of all the book reviews on their site, along with navigation elements to be able to handle large numbers of items. While WordPress offers a built-in mechanism to list post items in the form of archive pages, it is not easy for a plugin to be able to modify the layout of these pages in a consistent way across all possible user themes. Users also often want the ability to place custom content on any of their pages and not have to mess with theme template files. This recipe shows how to create a shortcode that will display one or more posts in any place selected by the user, including a page, a post, or even on the site's front page.

Getting ready

You should have already followed the *Tailoring search output for custom post type items* recipe to have a starting point for this recipe, and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch4/ch4-book-reviews/ch4-book-reviews-v4.php`) from the book's GitHub page and rename the file `ch4-book-reviews.php`.

How to do it...

Follow these steps to create a shortcode that will retrieve and display five book reviews at a time with accompanying navigation links.

1. Navigate to the `ch4-book-reviews` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch4-book-reviews.php` file in a code editor.
3. Add the following line of code after the existing functions to register a function that declares the new shortcode:

```
add_shortcode( 'book-review-list',
               'ch4_br_book_review_list' );
```

4. Add the following code section to provide a partial implementation for the `ch4_br_book_review_list` function:

```
function ch4_br_book_review_list() {
    $query_params = array( 'post_type' =>
                               'book_reviews',
                           'post_status' => 'publish',
                           'posts_per_page' => 5 );

    $book_review_query = new WP_Query;
    $book_review_query->query( $query_params );

    // Check if any posts were returned by the query
    if ( $book_review_query->have_posts() ) {

    }
    wp_reset_postdata();
    return $output;
}
```

5. Insert the following code between the two parentheses of the `if` statement that checks whether posts are present (`if ( $book_review_query->have_posts() )`):

```
// Display posts in table layout
$output = '<table>';
$output .= '<tr><th style="text-align:left;">';
$output .= '<strong>Title</strong></th>';
$output .= '<th style="text-align:left">';
$output .= '<strong>Author</strong></th></tr>';

// Cycle through all items retrieved
while ( $book_review_query->have_posts() ) {
    $book_review_query->the_post();
    $output .= '<tr><td style="padding-right: 20px">';
    $output .= '<a href="' . get_permalink();
    $output .= '">' . get_the_title( get_the_ID() );
    $output .= '</a></td><td>';
    $output .= esc_html( get_post_meta( get_the_ID(),
                         'book_author', true ) );
    $output .= '</td></tr>';
}
$output .= '</table>';

// Display page navigation links
if ( $book_review_query->max_num_pages > 1 ) {
    $output .= '<nav id="nav-below">';
    $output .= '<div class="nav-previous">';
    $output .= get_next_posts_link (
        '<span class="meta-nav">&larr;</span>' .
        ' Older reviews',
        $book_review_query->max_num_pages );
    $output .= '</div><div class="nav-next">';
    $output .= get_previous_posts_link(
        'Newer reviews ' .
        '<span class="meta-nav">&rarr;</span>',
        $book_review_query->max_num_pages );
```

```
        $output .= '</div></nav>';
    }
```

6. Save the plugin file.
7. Create a new page and insert the shortcode `[book-review-list]`.
8. **Publish** and **View** the page to see that a list of book reviews will be displayed in place of the shortcode. If you do not see the navigation controls under the list, create additional reviews to have more than five.

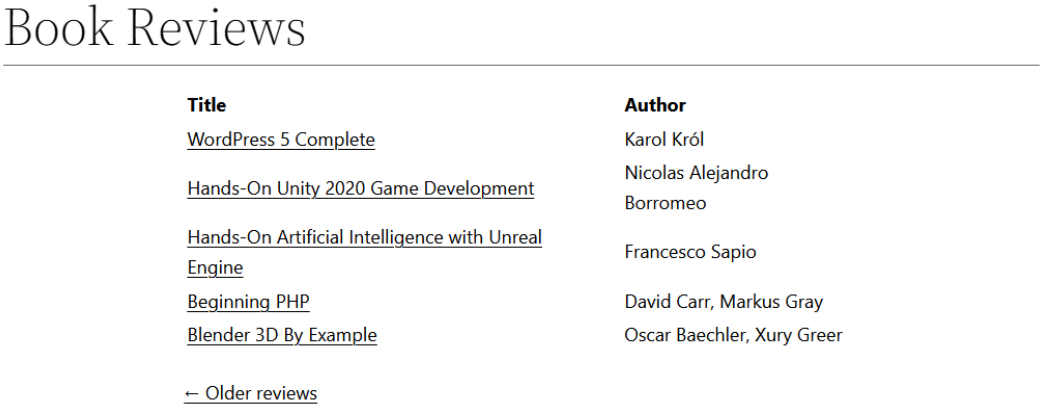


Figure 4.8 – Book review list generated by shortcode

9. Click on the **Older reviews** navigation link. You will see that the URL in the browser address bar changes to reflect the new page to be displayed, but that the list of entries still shows the same first five items as before.
10. Back in the `ch4-book-reviews.php` file, add the following highlighted code near the top of the `ch4_br_book_review_list` function body, right after the line initializing the value of the `$query_params` variable:

```
    // Preparation of query to retrieve 5 book reviews
    $query_params = array( 'post_type' => 'book_reviews',
                           'post_status' => 'publish',
                           'posts_per_page' => 5 );

    // Retrieve page query variable, if present
    $page_num = ( get_query_var( 'paged' ) ?
                      get_query_var( 'paged' ) : 1 );

    // If page number is higher than 1, add to query array
```

```
if ( $page_num != 1 ) {
    $query_params['paged'] = $page_num;
}
```

11. Save and close the plugin file.
12. Refresh the page that includes our new shortcode and use the navigation links to see that the list of items now changes properly based on the selected page.

How it works...

As we saw in *Chapter 2*, *Plugin Framework Basics*, shortcodes are text elements that can be inserted into any page and post and that are replaced with content generated by the plugin when they are found. The registered callback function must prepare the output and send it back as a return value at the end of its execution.

The first part of the `ch4_br_book_review_list` function takes care of preparing an array of configuration elements to be passed to a new instance of the `WP_Query` class. This class allows developers to easily extract information from the site database's post table. In this example, the parameters that are being set in the query are the internal post type name (`post_type`), the status of the items that we want to display (`post_status`), and the number of items that should be retrieved at a time (`posts_per_page`).

Once the configuration array is in place, we create a global variable called `book_review_query` and assign to it a new instance of a `WP_Query` object. We then call the object's `query` method, passing it the configuration array that was just assembled. If posts are found by the object, as indicated by the `have_posts()` method, we output HTML code to create a table and use a `while` loop to cycle through all the items found and display their title and author using code similar to the previous two recipes.

As part of this recipe, we also saw that if the query object's `max_num_pages` property is higher than `1`, we can use the `get_next_posts_link` and `get_previous_posts_link` functions to add navigation controls under the table of entries. We were then able to use the `get_query_var` function to see whether a page number was requested and add that number to our query parameters.

See also

- The *Creating a new simple shortcode* recipe in *Chapter 2*, *Plugin Framework Basics*

Adding custom categories for custom post types

To keep items organized on a site, administrators often use the built-in WordPress categories and terms to identify similar items. Looking back at the book review system that we have been putting in place so far in this chapter, a type of categorization that would be helpful to add is a book type (for example, science fiction, documentary, fiction, and poetry).

Getting ready

You should have already followed the *Displaying custom post type data in shortcodes* recipe to have a starting point for this recipe, and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch4/ch4-book-reviews/ch4-book-reviews-v5.php`) from the book's GitHub page and rename the file `ch4-book-reviews.php`.

How to do it...

Follow these steps to create a new category (known as a **taxonomy** in the WordPress backend) and associate it with the book review custom post type.

1. Navigate to the `ch4-book-reviews` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch4-book-reviews.php` file in a code editor.
3. Find the `ch4_br_create_book_post_type` function and add the following code to the function body after the existing call to `register_post_type` (which ends with a parenthesis and semi-colon `);`) to create the new taxonomy:

```
register_taxonomy(
    'book_reviews_book_type',
    'book_reviews',
    array(
        'labels' => array(
            'name' => 'Book Type',
            'add_new_item' => 'Add New Book Type',
            'new_item_name' => 'New Book Type Name'
        ),
        'show_ui' => true,
        'show_tagcloud' => false,
```

```
            'hierarchical' => true,
        )
    );
```

> **Note**
> If you are using the block editor for your custom post type, you will need to add the following line after the `hierarchical` parameter in the data array to see the **Book Type** section appear in the editor when editing book reviews: `'show_in_rest' => true`.

4. Save and close the plugin file.
5. Open a previously created book review to see the newly added **Book Type** section on the right-hand side of the post editor.
6. Click on the **+ Add New Book Type** link to create a new item and assign it as the current item's type.

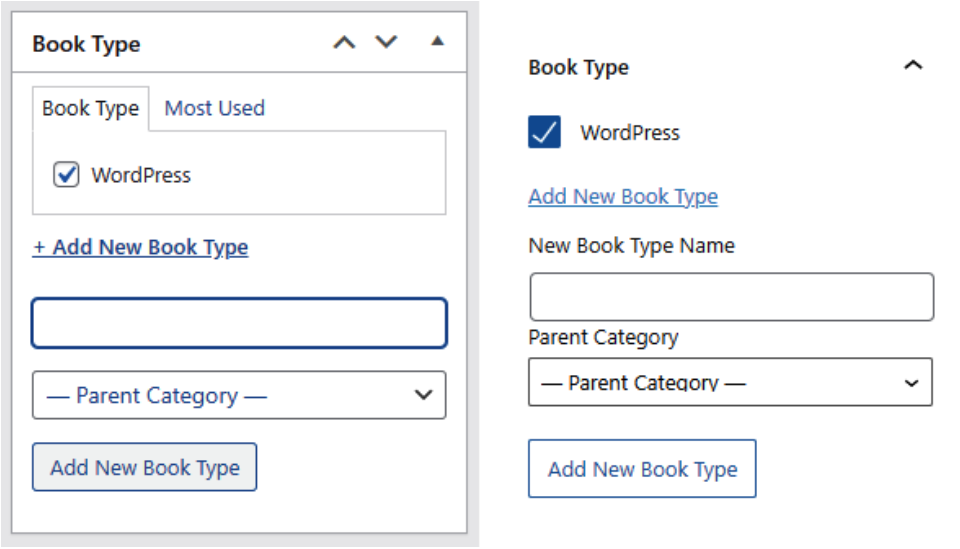


Figure 4.9 – Custom category assignment in Classic Editor (left) and Block Editor (right)

7. Click on the **Update** button in the top-right section of the post editor to save the review.
8. Look at the left-hand administration menu to see that a new menu item was added to manage book types, leading to an editor similar to the post and page category editor.

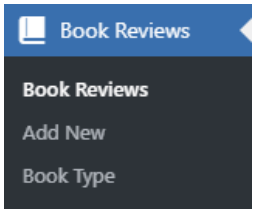


Figure 4.10 – Book Type sub-menu added to the custom post type menu

9. Back in the plugin file, add the following code to the `ch4_br_display_single_book_review` function after the section displaying the rating to display the book type and before displaying the content, with the new code shown in bold:

```
$content .=
    str_repeat( '<img style="margin: 0" src="' .
        plugins_url( 'star-icon-grey.png', __FILE__ )
        . '" />', 5 - $nb_stars );

$book_types = wp_get_post_terms( get_the_ID(),
                    'book_reviews_book_type' );

$content .= '<br /><strong>Type: </strong>';

if ( $book_types ) {
    $type_array = array();
    foreach ( $book_types as $book_type ) {
        $type_array[] = $book_type->name;
    }
    $content .=
        esc_html( implode( ',', $type_array ) );
} else {
    $content .= 'None Assigned';
}

// Display book review contents
$content .= '<br /><br />';
```

10. Save and close the template file.

Visit a book review page to see the book type displayed under the rating.

How it works...

The `register_taxonomy` function is used to create a new type of category in WordPress and associate it with a post type. It has three parameters:

```
register_taxonomy( $taxonomy_name, $post_type, $options );
```

The first argument is a unique identifier for the taxonomy. The second parameter is the post type that it should be associated with, which should match the type declared with the `register_post_type` function. The third argument is an array of parameters that determine how the new taxonomy will behave.

In this example, we have set a few taxonomy options, including a first item called `labels` that contains an array of text strings that will be used in the interface when referring to the new taxonomy. We also specified a second element, `show_ui`, which controls the display of the taxonomy meta box in the post editor and the presence of a link to access the taxonomy editor in the administration menu. Next is an option called `show_tagcloud`, which we set to `false` to avoid displaying a tag cloud of all taxonomy values. Finally, the last item in the `options` array is called `hierarchical`. When set to `true`, taxonomy items will be able to have parent/child relationships and will be accessible as a list of checkboxes in the post editor. If set to `false`, all taxonomies are organized as a flat list and can be selected using an interface similar to the tag window in the post and page editor.

There are many more options available for the `register_taxonomy` function, as can be seen by visiting the WordPress Code Reference site (`https://developer.wordpress.org/reference/functions/register_taxonomy/`), but the ones found here are the essential ones to define a basic taxonomy.

We also made use of the standard PHP `implode` function in the code we added when displaying single book reviews to combine all potential categories assigned to a book review into a single comma-separated string to display.

See also

- The *Creating a custom post type* recipe

Adding custom fields to categories

In addition to specifying names for categories, it may be useful to attach additional information to custom taxonomies created in a WordPress plugin. For example, we may want to assign custom colors to categories that will be used when they are displayed, or we might want to identify the categories of content that are only accessible to paying members on a website.

Getting ready

You should have already followed the *Adding custom categories for custom post types* recipe to have a starting point for this recipe, and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch4/ch4-book-reviews/ch4-book-reviews-v6.php`) from the book's GitHub page and rename the file `ch4-book-reviews.php`.

How to do it...

1. Follow these steps to display additional fields in the taxonomy editor and learn how to store the additional data in the site's database:
2. Navigate to the `ch4-book-reviews` folder of the WordPress `plugins` directory of your development installation.
3. Open the `ch4-book-reviews.php` file in a code editor.
4. Add the following lines of code after the existing functions to assign a function to two action hooks that will be called when users create or edit taxonomy items:

```
add_action( 'book_reviews_book_type_edit_form_fields',
            'ch4_br_book_type_new_fields', 10, 2 );
add_action( 'book_reviews_book_type_add_form_fields',
            'ch4_br_book_type_new_fields', 10, 2 );
```

5. Add the following code section to provide an implementation for the `ch4_br_book_type_new_fields` function:

```
function ch4_br_book_type_new_fields( $tag ) {
    $mode = 'new';
    if ( is_object( $tag ) ) {
        $mode = 'edit';
        $cat_color = get_term_meta( $tag->term_id,
                         'book_type_color', true );
    }

    $cat_color =
        empty( $cat_color ) ? '#' : $cat_color;

    if ( 'edit' == $mode ) {
        echo '<tr class="form-field">';
```

```
            echo '<th scope="row" valign="top">';
        } elseif ( 'new' == $mode ) {
            echo '<div class="form-field">';
        } ?>

        <label for="book_type_color">Color</label>
        <?php if ( 'edit' == $mode ) {
            echo '</th><td>';
        } ?>

        <input type="text" id="book_type_color"
               name="book_type_color"
               value="<?php echo $cat_color; ?>" />
        <p class="description">Color associated with book
        type (e.g. #199C27 or #CCC)</p>

        <?php if ( 'edit' == $mode ) {
            echo '</td></tr>';
        } elseif ( 'new' == $mode ) {
            echo '</div>';
        }
    }
```

6. Add the following lines of code at the end of the file to assign a function that will be called when users create or update taxonomy items:

```
add_action( 'edited_book_reviews_book_type',
            'ch4_br_save_book_type_new_fields',
            10, 2 );
add_action( 'created_book_reviews_book_type',
            'ch4_br_save_book_type_new_fields',
            10, 2 );
```

7. Add the following code section to provide an implementation for the `ch4_br_save_book_type_new_fields` function:

```
function ch4_br_save_book_type_new_fields( $term_id,
                                           $tt_id ) {
```

```
        if ( !$term_id ||
                !isset( $_POST['book_type_color'] ) ) {
            return;
        }

        if ( '#' == $_POST['book_type_color']
                || preg_match( '/#([a-f0-9]{3}){1,2}\b/i',
                $_POST['book_type_color'] ) ) {
            update_term_meta( $term_id, 'book_type_color',
                sanitize_text_field(
                    $_POST['book_type_color'] ) );
        }
    }
```

8. Save and close the plugin file.
9. Edit one of the **Book Type** entries created in the previous recipe to see the newly added **Color** field. Enter a valid CSS color code and **Update** the entry to see the data saved.

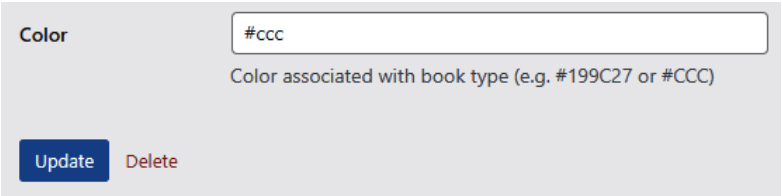


Figure 4.11 – Custom field added to the category editor

Create a new book type to see that the **Color** field is also available when creating new entries.

How it works...

This recipe uses many variable name action hooks to register functions to be called when users create or modify book types in the taxonomy editor. The first two calls to `add_action` refer to the `<taxonomy>_edit_forms_fields` and `<taxonomy>_add_form_fields` hooks. While you might expect to see two different functions associated with each of these action hooks, we actually register the same function in both cases, since rendering an additional field is similar in both cases. That being said, part of the function we register checks to see whether it receives a valid object as a parameter to know how it should render the new field so that it fits on the page irrespective of whether the user is creating a new category or editing an existing one.

We use a similar technique with the `edited_<taxonomy>` and `created_<taxonomy>` action hooks, which are respectively called when you first save a new taxonomy and update an existing one. In this case, the code does not have any significant differences depending on the action, since we only need to validate and save the incoming value for the new field. It should be noted that the `preg_match` function is used to check whether the received text starts with a `#` character, followed by three or six characters that range from `A` to `F` or `0` to `9`, to be valid hexadecimal color values.

See also

- The *Adding custom categories for custom post types* recipe

Hiding the category editor from the custom post type editor

As we saw in the previous recipe, when we associate a new taxonomy with the book reviews custom post type, the `show_ui` (Classic Editor) and `show_in_rest` (Block Editor) options control the visibility of the taxonomy assignment meta box and the admin menu link to the taxonomy editor. In some cases, it is desirable to give users access to the full taxonomy editor, but only let editors choose from a controlled drop-down list when they create new entries in the custom post type editor.

This recipe shows how to hide the taxonomy interface from the post editor and how to update the custom post type meta box created in the previous recipe to assign a type to new book reviews and save this information in the site's database.

Getting ready

You should have already followed the *Adding custom fields to categories* recipe to have a starting point for this recipe, and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch4/ch4-book-reviews/ch4-book-reviews-v7.php`) from the book's GitHub page and rename the file `ch4-book-reviews.php`.

How to do it...

Follow these steps to learn how to replace the custom post type category editor with a drop-down list:

1. Navigate to the `ch4-book-reviews` folder of the WordPress `plugins` directory of your development installation.

2. Open the `ch4-book-reviews.php` file in a code editor.
3. Find the call to the `register_taxonomy` function within the `ch4_br_create_book_post_type` function and add a new member to the configuration array named `meta_box_cb` with a value set to `false`, as shown highlighted in bold here:

```
'show_tagcloud' => false,
'meta_box_cb' => false,
'hierarchical' => true,
```

> **Note**
> If you are using the block editor for your custom post type, you should also set the `show_in_rest` option to `false` or remove it from the configuration array.

4. Save the plugin and edit a book review to see that the **Book Type** taxonomy box is no longer displayed while the **Book Type** menu item is still available under the **Book Reviews** menu.
5. Locate the `ch4_br_display_review_details_mb` function in the code and add the following code within the existing table-rendering code to add a new row containing a drop-down selection box for the book type:

```
<tr>
    <td>Book Type</td>
    <td>
    <?php
    // Retrieve array of types assigned to post
    $assigned_types =
        wp_get_post_terms( $book_review->ID,
                           'book_reviews_book_type' );
    // Retrieve array of all book types in system
    $book_types =
        get_terms( 'book_reviews_book_type',
                   array( 'orderby' => 'name',
                          'hide_empty' => 0 ) );

    if ( $book_types ) {
        echo '<select name="book_review_book_type"';
```

```
            echo ' style="width: 400px">';
            echo '<option value="">Select type</option>';

            foreach ( $book_types as $book_type ) {
                echo '<option value="';
                echo intval( $book_type->term_id ) . '" ';
                if ( !empty( $assigned_types ) ) {
                    selected( $assigned_types[0]->term_id,
                              $book_type->term_id );
                }
                echo '>' . esc_html( $book_type->name );
                echo '</option>';
            }
            echo '</select>';
        } ?>
        </td>
    </tr>
```

6. Find the `ch4_br_add_book_review_fields` function and add the following code segment within the `if` statement that checks whether the post type is `book_reviews` to save the selected book type in the site's database upon submission of the post:

```
if ( isset( $_POST['book_review_book_type'] ) ) {
    wp_set_post_terms( $book_review->ID,
        intval( $_POST['book_review_book_type'] ),
        'book_reviews_book_type' );
}
```

7. Save and close the plugin file.
8. Open a previously created book review.

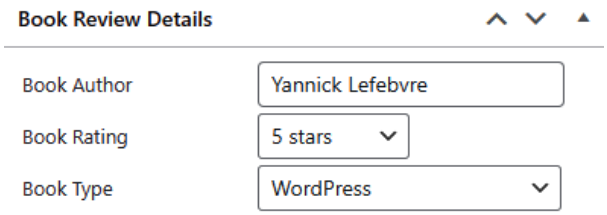


Figure 4.12 – Expanded Book Review Details section

As we can see, a new drop-down list has been added to the **Book Review Details** meta box, allowing users to select a single book type for their reviews.

How it works...

This recipe uses one of the many parameters of the `register_taxonomy` function to remove the ability for users to create new book types as they create or edit book reviews. It then goes on to create a drop-down list to be able to assign a single book type to reviews.

In this process, the recipe makes use of three functions related to storing and retrieving taxonomy entries related to posts. The first, `wp_get_post_terms`, retrieves an array of terms associated with a post based on its ID and the name of the taxonomy. The second, `wp_set_post_terms`, assigns a term to a post based on its ID and the taxonomy name. Finally, `get_terms` retrieves an array of all the terms in the taxonomy, ordered based on the query string found in the second argument.

See also

- The *Adding custom fields to categories* recipe

Displaying additional columns in the custom post list page

After customizing the post editor to give content creators a tailored environment to create and edit custom post type entries, this recipe turns its efforts toward the **Book Reviews** management page, where all the entries for this type are listed. By default, custom post type listings are quite simple and only show the title, publication date, and the number of comments for each item. To make it easier to identify and sort data in this management page, WordPress offers a number of customization capabilities, starting with the ability to change the columns that are displayed.

Getting ready

You should have already followed the *Hiding the category editor from the custom post type editor* recipe to have a starting point for this recipe, and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch4/ch4-book-reviews/ch4-book-reviews-v8.php`) from the book's GitHub page and rename the file to `ch4-book-reviews.php`.

How to do it...

Follow these steps to learn how to add and remove columns in the post management page, as well as enable sorting in these new columns:

1. Navigate to the `ch4-book-reviews` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch4-book-reviews.php` file in a code editor.
3. Add the following line of code at the end of the plugin file to register a function to be called when the book reviews listings page is being prepared:

```
add_filter( 'manage_edit-book_reviews_columns',
            'ch4_br_add_columns' );
```

4. Add the following code section to provide an implementation for the `ch4_br_add_columns` function:

```
function ch4_br_add_columns( $columns ) {
    $new_columns = array();
    $new_columns['book_reviews_author'] = 'Author';
    $new_columns['book_reviews_rating'] = 'Rating';
    $new_columns['book_reviews_type'] = 'Type';

    unset( $columns['comments'] );
    $columns = array_slice( $columns, 0, 2 ) +
        $new_columns + array_slice( $columns, 2 );

    return $columns;
}
```

5. Add the following line of code to assign a function to be called when the column's data is being retrieved for each row in the post listing:

```
add_action( 'manage_posts_custom_column',
            'ch4_br_populate_columns' );
```

6. Insert the following code segment at the end of the file to provide an implementation for the `ch4_br_populate_columns` function:

```
function ch4_br_populate_columns( $column ) {
    if ( 'book_reviews' != get_post_type() ) {
        return;
    }

    if ( 'book_reviews_author' == $column ) {
        echo esc_html( get_post_meta( get_the_ID(),
                                 'book_author', true ) );
    } elseif ( 'book_reviews_rating' == $column ) {
        $rating = intval( get_post_meta(
                        get_the_ID(), 'book_rating',
                        true ) );
        if ( $rating > 0 ) {
            echo $rating . ' stars';
        } else {
            echo 'None Assigned';
        }
    } elseif ( 'book_reviews_type' == $column ) {
        $book_types =
            wp_get_post_terms( get_the_ID(),
                'book_reviews_book_type' );
        if ( $book_types ) {
            $cat_color = get_term_meta(
                $book_types[0]->term_id,
                'book_type_color', true );

            if ( !empty( $cat_color ) &&
                    '#' != $cat_color ) {
                echo '<span style="background-color: ';
                echo esc_html( $cat_color ) . '; ';
                echo 'color: #fff; padding: 6px;">';
                echo esc_html( $book_types[0]->name );
                echo '</span>';
            } else {
```

```
                    echo esc_html( $book_types[0]->name );
                }
            } else {
                echo 'None Assigned';
            }
        }
    }
```

7. Save the plugin file and navigate to the **Book Reviews** listing page to see that the list of columns has been altered to read as **Title**, **Author**, **Rating**, **Type**, and **Date**. You will also see that data stored in the post custom fields is now displayed for each item in the list. Finally, will see that only the **Title** and **Date** columns can be sorted at this time.
8. Back in the code editor, add the following code at the end of the plugin file to register a function to be called when WordPress identifies columns that will be sortable for the book reviews custom post type:

```
add_filter(
    'manage_edit-book_reviews_sortable_columns',
    'ch4_br_author_column_sortable' );
```

9. Append the following code to provide an implementation for the `ch4_br_author_column_sortable` function:

```
function ch4_br_author_column_sortable( $columns ) {
    $columns['book_reviews_author'] =
        'book_reviews_author';
    $columns['book_reviews_rating'] =
        'book_reviews_rating';
    return $columns;
}
```

10. Add the following block of code to register a function that will be called when data is requested to display post lists:

```
add_filter( 'request', 'ch4_br_column_ordering' );
```

11. Insert the following code segment to implement the `ch4_br_column_ordering` function:

```
function ch4_br_column_ordering( $vars ) {
    if ( !is_admin() ) {
        return $vars;
    }

    if ( isset( $vars['orderby'] ) &&
            'book_reviews_author' ==
                $vars['orderby'] ) {
        $vars = array_merge( $vars, array(
                    'meta_key' => 'book_author',
                    'orderby' => 'meta_value' ) );
    } elseif ( isset( $vars['orderby'] ) &&
            'book_reviews_rating' ==
                $vars['orderby'] ) {
         $vars = array_merge( $vars, array(
                   'meta_key' => 'book_rating',
                   'orderby' => 'meta_value_num' ) );
    }
    return $vars;
}
```

12. Save and close the plugin file.
13. Refresh the **Book Reviews** listing.

Figure 4.13 – Custom columns in the Book Reviews list

As we can see, the **Author** and **Rating** column headers are now links that can be clicked to sort these columns.

How it works...

Customizing the post listings page requires an intricate mix of action and filter hooks to achieve our final goal. The first function we registered is associated with the variable filter name `manage_edit-<post_type>_columns`, where `<post_type>` is replaced with the internal post type name. When the registered function is called, it receives the default column list that will be shown while listing **Book Reviews** entries as an argument. Using this data, it proceeds to create an array of three new columns for `author`, `rating`, and `type`, followed by the removal of the `comments` column from the original array. Finally, the code inserts the three new columns between the remaining original columns using a combination of the `array_slice` PHP function and the `+` operator. Once finished, it returns the modified array.

The second part of the recipe registers the function that will be responsible for populating the new columns. Since this function gets called when any custom post type column is rendered, the code first checks whether the function is being called for book reviews and then checks which column is currently requested before echoing the requested data to the browser. The function makes calls to `get_the_ID()` to get the index of the currently displayed row and to be able to find its associated data using `get_post_meta` and `wp_get_post_terms`.

At this point in the recipe, the new columns are visible on the book reviews management page and data is displayed for each of them. The purpose of the rest of the recipe is to make the `author` and `rating` columns sortable. This is done by first registering a function with the variable filter name `manage_edit-<post_type>_sortable_columns`, where `<post_type>` is replaced with the post type name. When the function is executed, it adds two items to the array of columns that can be sorted. This takes care of making the column header links that can be clicked for sorting associated with the appropriate URLs.

The last function that is registered is associated with the `request` filter and takes care of adding elements to the query array based on the variables that came through in the query URL.

The final result allows administrators to easily reorder book reviews based on these two columns, as well as to see information about each entry at a glance.

See also

- The *Hiding the category editor from the custom post type editor* recipe

Adding filters for custom categories to the custom post list page

A second customization method for the custom post listings is to create a drop-down box that will allow administrators to only display items that belong to a single category at a time. This can help significantly reduce the number of entries that are shown to quickly find the desired content.

Getting ready

You should have already followed the *Displaying additional columns in the custom post list page* recipe to have a starting point for this recipe and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch4/ch4-book-reviews/ch4-book-reviews-v9.php`) from the book's GitHub page and rename the file `ch4-book-reviews.php`.

How to do it...

Follow these steps to add a filter mechanism based on the book review type to the listings page:

1. Navigate to the `ch4-book-reviews` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch4-book-reviews.php` file in a code editor.
3. Add the following line of code at the end of the file to register a function to be called when WordPress is preparing the filter drop-down boxes for the post listings:

```
add_action( 'restrict_manage_posts',
            'ch4_br_book_type_filter_list' );
```

4. Add the following code section to provide an implementation for the `ch4_br_book_type_filter_list` function:

```
function ch4_br_book_type_filter_list() {
    $screen = get_current_screen();
    if ( 'book_reviews' != $screen->post_type ) {
        return;
    }

    global $wp_query;
```

```
    wp_dropdown_categories( array(
        'show_option_all' => 'Show All Book Types',
        'taxonomy' => 'book_reviews_book_type',
        'name' => 'book_reviews_book_type',
        'orderby' => 'name',
        'selected' =>
    ( isset( $wp_query->query['book_reviews_book_type'] )
    ? $wp_query->query['book_reviews_book_type'] : '' ),
        'hierarchical' => false,
        'depth' => 3,
        'show_count' => false,
        'hide_empty' => true,
    ) );
}
```

5. Insert the following line of code to register a function that will be called when the post display query is being prepared:

```
add_filter( 'parse_query',
            'ch4_br_perform_book_type_filtering' );
```

6. Implement the `ch4_br_perform_book_type_filtering` function with the following code segment:

```
function ch4_br_perform_book_type_filtering($query) {
    $qv = &$query->query_vars;
    if ( isset( $qv['book_reviews_book_type'] ) &&
            !empty( $qv['book_reviews_book_type'] ) &&
            is_numeric(
               $qv['book_reviews_book_type'] ) ) {
        $term = get_term_by( 'id',
                     $qv['book_reviews_book_type'],
                     'book_reviews_book_type' );
        $qv['book_reviews_book_type'] = $term->slug;
    }
}
```

7. Save and close the plugin file.
8. Visit the **Book Reviews** listings.

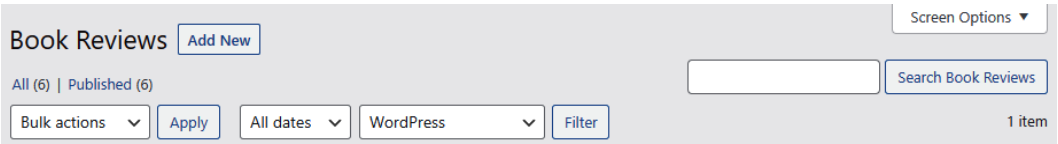


Figure 4.14 – Custom filter on the book reviews listing page

As we can see, a new drop-down list has appeared next to the existing date filter to select what book types are to be displayed.

How it works...

This recipe starts by registering an action callback that will be executed when WordPress renders the various filter controls that are available for each post type listing. When the function is called, it retrieves a global variable to know the post type that is currently being displayed and determine whether it should show the book type filter list. It also accesses the global `wp_query` variable to see whether a book type filter is already in place and sets the correct drop-down list entry to be selected if needed.

The callback then proceeds to use the `wp_dropdown_categories` function to display a list of all of the taxonomy items registered for book types. This utility function expects to receive an array of parameters that determine which taxonomy list to display, the name of the drop-down list field name, and the label to be displayed for the option to show all the types. This array should also contain a few parameters to determine the order in which the items should be displayed, specify the item to set as selected, indicate the maximum depth to show for hierarchical taxonomies, and determine whether or not items count and empty items be shown.

Once the new book type selection list is in place, selecting an entry and clicking on the **Filter** button triggers a refresh of the page and leads to the execution of the second registered callback. The filter callback function receives the current WordPress post query object and starts by first getting a pointer to the query variables that are stored inside the query object. With this in hand, it moves on to verify that a book type is part of the query variables and that it is numeric. If the result is positive, it replaces the numeric value with the textual name for the selected book type so that the query can take place.

Once all of this code is executed, users are able to quickly filter which book types should be displayed on the **Book Reviews** management page. Users are also still able to use the column sorting mechanism implemented in the previous recipe.

See also

- The *Displaying additional columns in the custom post list page* recipe

Adding Quick Edit fields for custom categories

A great feature of WordPress is the ability for site editors to quickly make changes to any post in the admin section by clicking on the **Quick Edit** link associated with any of the items shown. While our custom post type taxonomy appears in the **Quick Edit** section, it is not a drop-down list of choices, as we had in the book review editor to ensure that only a single type is selected. It would also be useful for the author and rating fields to appear in the **Quick Edit** section to be able to quickly make changes to these fields.

This recipe shows how to add custom fields when quickly editing book reviews. As you perform the following steps, you will see that some of the code that we put in place is not as cleanly written as code from previous recipes, since the WordPress Quick Edit customization infrastructure is not as well-formed as other areas of the platform and involves a mix of PHP and JavaScript.

Getting ready

You should have already followed the *Adding filters for custom categories to the custom post list page* recipe to have a starting point for this recipe, and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch4/ch4-book-reviews/ch4-book-reviews-v10.php`) from the book's GitHub page and rename the file `ch4-book-reviews.php`.

How to do it...

Follow these steps to learn how to add custom fields to the **Quick Edit** tool:

1. Navigate to the `ch4-book-reviews` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch4-book-reviews.php` file in a code editor.

3. Find the call to the `register_taxonomy` function within the `ch4_br_create_book_post_type` function and add a new member to the configuration array named `show_in_quick_edit` with a value set to `false`, as highlighted in bold here:

```
'show_tagcloud' => false,
'meta_box_cb' => false,
'show_in_quick_edit' => false,
'hierarchical' => true,
```

4. Add the following line of code at the end of the file to register a function to be called when WordPress is preparing to render the contents of the **Quick Edit** section:

```
add_action( 'quick_edit_custom_box',
            'ch4_br_display_custom_quickedit_link',
            10, 2 );
```

5. Add the following code section to provide a declaration for the `ch4_br_display_custom_quickedit_link` function:

```
function ch4_br_display_custom_quickedit_link(
                        $column_name, $post_type ) {

}
```

6. Insert the following block of code to implement the body of the `ch4_br_display_custom_quickedit_link` function:

```
if ( 'book_reviews' != $post_type ) {
    return;
}

switch ( $column_name ) {
    case 'book_reviews_author': ?>
        <fieldset class="inline-edit-col-right"><div>
        <label>
            <span class="title">Author</span>
        </label>
        <input type="text" name="book_reviews_author"
               value=""></div>
```

```
        <?php break;
        case 'book_reviews_rating': ?>
            <div>
                <label>
                    <span class="title">Rating</span>
                </label>
                <select name="book_reviews_rating">
                <option value="">None Assigned</option>
                <?php
                for ($rating = 5;$rating >= 1;$rating--) {
                ?>
                    <option value="<?php
                        echo $rating; ?>">
                    <?php echo $rating; ?> stars
                <?php } ?></select></div>
        <?php break;
        case 'book_reviews_type': ?>
            <div><label>
                <span class="title">Type</span></label>
            <?php $terms = get_terms(
                 array( 'taxonomy' =>
                            'book_reviews_book_type',
                        'hide_empty' => false ) ); ?>
            <select name="book_reviews_type">
            <option value="">None Assigned</option>
            <?php foreach ($terms as $index => $term) {
                echo '<option ';
                echo 'value="' . $term->term_id . '"';
                selected( 0, $index );
                echo '>' . $term->name. '</option>';
            } ?></select></div></fieldset>
        <?php break;
    }
```

7. Add the following line of code to register a function to be called when WordPress is rendering the footer of the administration pages:

```
add_action( 'admin_footer', 'ch4_br_quick_edit_js' );
```

8. Implement the `ch4_br_quick_edit_js` function with the following code snippet:

```
function ch4_br_quick_edit_js() {
    global $current_screen;
    if ( ( 'edit-book_reviews' !=
              $current_screen->id ) ||
         ( 'book_reviews' !=
              $current_screen->post_type ) ) {
        return;
    } ?>

<?php }
```

9. Continue the implementation of the `ch4_br_quick_edit_js` function with the following code inserted just before the PHP tag and closing function bracket (`<?php }`):

```
<script type="text/javascript">
function set_inline_book_reviews( reviewArray ) {
    inlineEditPost.revert();

    var BookAuthor =
        document.getElementsByName(
            'book_reviews_author')[0];
    BookAuthor.value = reviewArray[0];

    var Rating =
        document.getElementsByName(
        'book_reviews_rating')[0];
    for (i = 0; i < Rating.options.length; i++) {
        if ( Rating.options[i].value ==
                reviewArray[1] ) {
            Rating.options[i].setAttribute(
```

```
                    'selected', 'selected' );
            } else {
                Rating.options[i].removeAttribute(
                    'selected' );
            }
        }

        var BookType =
            document.getElementsByName(
                'book_reviews_type')[0];
        for (i = 0; i < BookType.options.length; i++) {
            if (BookType.options[i].value ==
                    reviewArray[2]) {
                BookType.options[i].setAttribute(
                    'selected', 'selected' );
            } else {
                BookType.options[i].removeAttribute(
                    'selected' );
            }
        }
    }
    </script>
```

10. Add the following code to register a function to replace the original **Quick Edit** code that is generated for each post on the book reviews page:

```
    add_filter( 'post_row_actions',
                'ch4_br_quick_edit_link',
                10, 2 );
```

11. Add the following block of code to provide an implementation for the `ch4_br_quick_edit_link` function:

```
    function ch4_br_quick_edit_link( $act, $post ) {
        global $current_screen;
        $post_id = '';

        if ( ( isset( $current_screen ) &&
```

```
        $current_screen->id != 'edit-book_reviews' &&
        $current_screen->post_type != 'book_reviews' )
        || ( isset( $_POST['screen'] ) &&
        $_POST['screen'] != 'edit-book_reviews' ) ) {
        return $act;
    }

    if ( !empty( $post->ID ) ) {
        $post_id = $post->ID;
    } elseif ( isset( $_POST['post_ID'] ) ) {
        $post_id = intval( $_POST['post_ID'] );
    }

    if ( !empty( $post_id ) ) {
        $book_author =
            esc_html( get_post_meta( $post_id,
                                     'book_author', true ) );
        $book_rating =
            esc_html( get_post_meta( $post_id,
                                     'book_rating', true ) );
        $book_reviews_types =
            wp_get_post_terms( $post_id,
                'book_reviews_book_type',
                array( 'fields' => 'all' ) );
        if ( empty( $book_reviews_types ) ) {
            $book_review_type = 0;
        } else {
            $book_review_type =
            $book_reviews_types[0]->term_id;
        }

        $idx = 'inline hide-if-no-js';
        $act[$idx] = '<a href="#" class="editinline" '
          . " onclick=\"var reviewArray = "
          . "new Array('"
          . "{$book_author}', '{$book_rating}', "
```

```
            . "'{$book_review_type}');"
            . "set_inline_book_reviews(reviewArray)\">"
            . __( 'Quick Edit' )
            . '</a>';
        }
        return $act;
    }
```

12. Add the following function call to register a function that will be executed when post data is updated from the **Quick Edit** section:

```
add_action( 'save_post',
            'ch4_br_save_quick_edit_data',
            10, 2 );
```

13. Provide an implementation for the `ch4_br_save_quick_edit_data` function with the following block of code:

```
function ch4_br_save_quick_edit_data( $ID = false,
                                      $post = false ) {
    $post_item = get_post( $ID );
    if ( ( isset( $_POST['post_type'] ) &&
           'book_reviews' != $_POST['post_type'] ) ||
           !isset( $_POST['action'] ) ||
           'inline-save' != $_POST['action'] ) {
        return;
    }

    if ( isset( $_POST['book_reviews_author'] ) ) {
        update_post_meta( $ID, 'book_author',
            sanitize_text_field(
                $_POST['book_reviews_author'] ) );
    } else {
        update_post_meta( $ID, 'book_author', '' );
    }

    if ( isset( $_POST['book_reviews_rating'] ) ) {
        update_post_meta( $ID, 'book_rating',
```

```
                intval( $_POST['book_reviews_rating'] ) );
        } else {
            update_post_meta( $ID, 'book_rating', '' );
        }

    }
```

14. Complete the implementation of the `ch4_br_save_quick_edit_data` function body by inserting the following code before its closing parenthesis:

```
    if ( isset( $_POST['book_reviews_type'] ) &&
            !empty( $_POST['book_reviews_type'] ) ) {
        $term = term_exists(
            intval( $_POST['book_reviews_type'] ),
            'book_reviews_book_type' );
        if ( !empty( $term ) ) {
            wp_set_object_terms( $ID,
                intval( $_POST['book_reviews_type'] ),
                'book_reviews_book_type' );
        }
    } else {
        $assigned_types = wp_get_post_terms( $ID,
                                 'book_reviews_book_type' );
        foreach( $assigned_types as $assigned_type ) {
            wp_remove_object_terms( $ID,
                $assigned_type->term_id,
                'book_reviews_book_type' );
        }
    }
```

15. Save and close the plugin file.

16. Visit the **Book Reviews** listings page and click on **Quick Edit** to see the newly added **Author**, **Rating**, and **Type** fields.

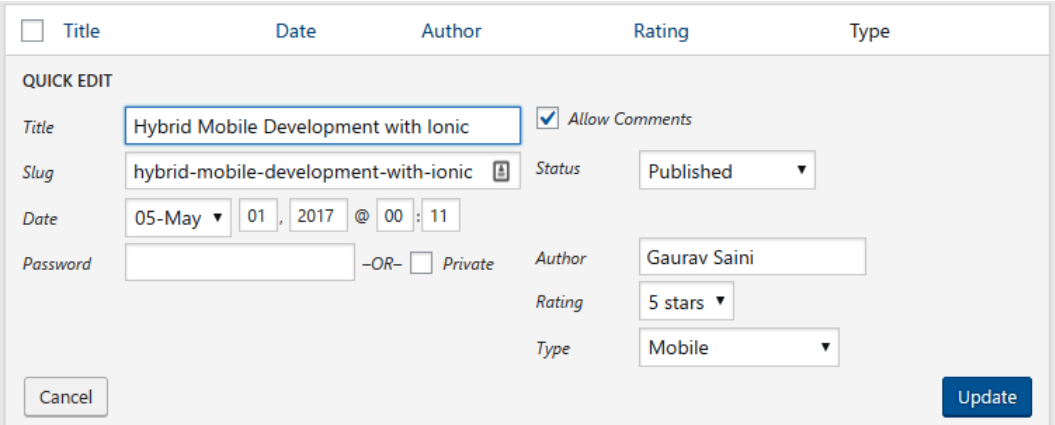


Figure 4.15 – Customized Quick Edit dialog

17. Change the values in these fields and save them using the **Update** button. You will see that the values get updated in the **Book Reviews** list, accordingly.

How it works...

While the **Quick Edit** box is a great tool that WordPress users appreciate, customizing it is a bit tricky in current versions of WordPress. While we could easily add data to each individual item in the post editor, the **Quick Edit** section requires more work to add custom fields. The root of the issue stems from the fact that there is really only a single instance of the Quick Edit section that gets rendered as part of the edit page. That section is hidden at first, then appears, and moves to the correct position when a user clicks on the **Quick Edit** link. With this in mind, we cannot just assign proper values to the custom fields in the **Quick Edit** box. Instead, we need to store hidden information as each item gets listed to be able to populate the **Quick Edit** fields appropriately based on which item is edited.

The first section of our recipe code starts off by rendering a text input field and two drop-down lists within the **Quick Edit** section. You will notice that we do not set the value of the text field or set options to be selected in the select lists. Another interesting point with this callback is that it sends us the same list of columns that we used in the *Displaying additional columns in the custom post list page* recipe. This means that if we wanted to display fields other than the ones we have added to the post table, we would have to place these field names in the list of custom columns. We would then have to use some tricks to hide them from the table and only make them accessible in the **Quick Edit** section.

Once these extra fields are in place, we add code to the footer of the book review listing admin page to create a JavaScript function that will be called when it is time to update our new custom fields in the **Quick Edit** section. The function receives an array of data, locates the custom fields in the page using the `document.getElementsByName` function, and then updates their values based on the incoming data array.

The next block of code we added creates a new **Quick Edit** link to replace the original one for each book review item. The new link not only enables users to display the **Quick Edit** section but also embeds values for each item within the `onclick` JavaScript code. The updated link also includes a call to the function that we added to the footer so that a new set of values is assigned to each custom field when the user decides to quickly edit a book review.

Finally, we store data from our custom fields when the user clicks on the **Update** button. Interestingly, we do this by registering a second callback for the `save_post` action. This means that both of our functions will be called when posts are saved. However, this second saving function checks for a number of conditions to be true before actually saving values, and the names of the fields that it saves are different from the save function we put in place for the post editor.

Updating the page title to include custom post data using plugin filters

The last customization touch that can be put in place to support our book reviews custom post type is to add custom information about the posts in the title bar when displaying them. For example, we could add the author's name next to the book title.

Getting ready

You should have already followed the *Adding Quick Edit fields for custom categories* recipe to have a starting point for this recipe, and the resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch4/ch4-book-reviews/ch4-book-reviews-v11.php`) from the book's GitHub page and rename the file `ch4-book-reviews.php`.

How to do it...

Follow these steps to learn how to use the `document_title_parts` filter to alter the post title for book reviews:

1. Navigate to the `ch4-book-reviews` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch4-book-reviews.php` file in a code editor.
3. Add the following line of code after the existing functions to register a function to be called when WordPress prepares the text to be displayed in the browser's title bar:

```
add_filter( 'document_title_parts',
            'ch4_br_format_book_review_title' );
```

4. Add the following code section to provide an implementation for the `ch4_br_format_book_review_title` function:

```
function ch4_br_format_book_review_title( $the_title ) {
    if ( 'book_reviews' == get_post_type() &&
            is_single() ) {
        $book_author = esc_html( get_post_meta(
            get_the_ID(), 'book_author', true ) );
        if ( !empty( $book_author ) ) {
            $the_title['title'] .= ' by ';
            $the_title['title'] .= $book_author;
        }
    }
    return $the_title;
}
```

5. Save and close the plugin file.
6. Visit a book review page and hover over the browser tab to see the complete, modified page title.

Figure 4.16 – Customized page title

As we can see, the book's author is now displayed after the book title.

How it works...

The `document_title_parts` filter allows plugins to modify or completely replace the page title contents. In this case, the code of the function that we associated with the filter hook receives the title that WordPress intends to display as an array. It then proceeds to check whether the item that is being prepared for display is a book review and whether or not it is a single item. While the first condition is something obvious to check for, the `is_single` verification is done to make sure that the code does not try to add a book author to the book reviews archive listing page, if present.

5
Customizing Post and Page Editors

In the last few chapters, you learned how to create configuration panels for your plugins and how to set up custom post types. With this knowledge in hand, we will now see how to customize the post and page editors using meta boxes.

Meta boxes are a very useful tool in the creation of WordPress plugins. They were first used to organize large administration panels into manageable sections in *Chapter 3*, *User Settings and Administration Pages*, and then continued to be a key element in the creation of tailored interfaces to edit custom post types in *Chapter 4*, *The Power of Custom Post Types*.

This chapter explores how meta boxes can be used to augment the default post and page editors' capabilities. While WordPress posts and pages already offer a lot of functionalities in a default installation, custom data entry fields go a long way in crafting a user experience that is much smoother than using the **Custom Fields** editor. These extra fields can be used to store anything. For example, they can be used to specify alternative language links for blog entries or to assign a pop-up dialog to specific articles on a site.

In this chapter, you will learn how to customize the core post and page editors through the following recipes:

- Capturing and displaying information using custom meta boxes
- Displaying custom post data using filter functions
- Hiding the Custom Fields section in the post editor
- Extending the post editor to attach custom files with the media uploader

Technical requirements

You can find complete code examples on GitHub in the following repository: `https://github.com/PacktPublishing/WordPress-Plugin-Development-Cookbook-Third-Edition`.

Capturing and displaying information using custom meta boxes

While the WordPress block editor did away with traditional meta boxes as its main interface building block when it was introduced in WordPress 5.0, plugin developers can still use them to easily create new sections when editing posts, pages, or any other post type that is configured to use the block editor, or on sites that are running the Classic Editor plugin.

Getting ready

You should have access to a WordPress development environment, either on your local computer or a remote server, where you will be able to load your new plugin files.

How to do it...

Follow these steps to add a custom meta box that will be used to display and capture information about the name and web address of the source materials used when writing a new post or page entry:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch5-post-source-link`.
3. Navigate to the directory and create a text file called `ch5-post-source-link.php`.

4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 5 - Post Source Link`.
5. Add the following line of code to register a function that will be executed when WordPress is preparing a list of meta boxes for all the administration areas:

```
add_action( 'add_meta_boxes',
            'ch5_psl_register_meta_box' );
```

6. Add the following code segment to provide an implementation for the `ch5_psl_register_meta_box` function:

```
function ch5_psl_register_meta_box() {
    add_meta_box( 'ch5_psl_source_meta_box',
                  'Post/Page Source',
                  'ch5_psl_source_meta_box',
                  array( 'post', 'page' ), 'normal' );
}
```

7. Insert this code to provide an implementation for the `ch5_psl_source_meta_box` function, responsible for rendering the meta box contents:

```
function ch5_psl_source_meta_box( $post ) {
    // Retrieve source name and address for post
    $source_name =
        esc_html( get_post_meta( $post->ID,
                      'source_name', true ) );
    $source_address =
        esc_html( get_post_meta( $post->ID,
                      'source_address', true ) );
    ?>
    <table>
        <tr>
            <td style="width: 100px">Source Name</td>
            <td><input type="text" size="40"
                       name="source_name"
                       value="<?php echo
                                $source_name; ?>" />
            </td>
        </tr>
```

```
        <tr>
            <td>Source Address</td>
            <td><input type="text" size="40"
                       name="source_address"
                       value="<?php echo
                             $source_address; ?>" />
            </td>
        </tr>
    </table>
<?php }
```

8. Insert the following block of code to register a function that will be called when any type of post is saved:

```
add_action( 'save_post', 'ch5_psl_save_source_data',
            10, 2 );
```

9. Append the following code section to provide an implementation for the `ch5_psl_save_source_data` function:

```
function ch5_psl_save_source_data( $post_id = false,
                                   $post = false ) {
    if ( in_array( $post->post_type,
            array( 'page','post' ) ) ) {
        if ( isset( $_POST['source_name'] ) ) {
            update_post_meta( $post_id,
               'source_name',
                sanitize_text_field(
                    $_POST['source_name'] ) );
        }
        if ( isset( $_POST['source_address'] ) ) {
            update_post_meta( $post_id,
                'source_address', esc_url_raw(
                    $_POST['source_address'] ) );
        }
    }
}
```

10. Save and close the plugin file.
11. Navigate to the **Plugins** management page and **Activate** the **Chapter 5 - Post Source Link** plugin.
12. Go to the **Posts** section of the administration section and click on one of the entries to open the post editor:

Figure 5.1 – The new post/page source meta box in the post editor

As seen in *Figure 5.1*, a new meta box is shown, containing the **Source Name** and **Source Address** fields. Any valid information entered in either of these fields will be saved along with the rest of the editor data.

How it works...

As we've seen in the last two chapters, plugin administration pages and custom post type editor pages can both benefit from using meta boxes to create their user experiences. Now, while the main **Posts** and **Pages** editors have moved to a new content creation paradigm in WordPress 5.0 with the block editor, these two editors have remained compatible with meta boxes. It should also be noted that while the block editor is the default editor for posts and pages, millions of WordPress users run a plugin called **Classic Editor** (`https://wordpress.org/plugins/classic-editor/`) to keep editing content as they used to before WordPress 5.0. All recipes found in this chapter are compatible with the block editor and the Classic Editor.

At the top of this recipe, we use the `add_meta_box` function to associate a new meta box with the page and post editors. As WordPress stores posts and pages in the same database table, it will automatically make sure that all data related to both item types will have unique IDs.

The other function that we have seen before is `get_post_meta`, which is used to retrieve custom metadata associated with post entries.

The content of the meta box itself is displayed using standard HTML. As this box will be part of a larger editor, there is no need to worry about declaring a form in this box.

Once the new dialog section is created, the next task in our recipe is to put code in place to store data from the additional fields upon submission through the use of the `save_post` action. Functions associated with this hook are called when posts of any type are saved. When executed, the associated function receives two parameters from WordPress that contain the ID of the post being saved and a copy of all the data that has been processed to be saved so far. The callback also has access to all the server post data received from the user and uses the `sanitize_text_field` and `esc_url_raw` functions to make sure that no malicious content was received.

As indicated in the previous chapter, it is important to set the fourth parameter of the `add_action` call – that is, `accepted_args` – for actions and filters that receive more than one argument. If it is not specified, these additional arguments will not be available to the receiving hook function.

Working with the assumption that the meta box was only added to the post and page editors, the code contained in the `ch5_psl_save_source_data` function first checks to see whether the post type is set as a post or page. If it is one of these two types, it moves on to check for the presence of post data for the source name and address fields. If found, `update_post_meta` is called once for each field to store the new information in the site's database associated with the posts that it belongs to. Making a call to the `update_post_meta` function actually updates the post custom field data if it exists or creates it in the case of new post or page entries.

There's more...

While this recipe specifically adds a new section to the post and page editors, it may be desirable to make these new fields available to all post types found in a WordPress installation, including custom ones.

Adding a new meta box to all post types (including custom ones)

This recipe specifies an array of post types to associate its new meta box with the post and page editors. This concept does not extend to registering a custom section with all post types found in a WordPress installation, since custom types created by other plugins are not known by our code. Thankfully, there is an easy way to get an array of all post types found in a WordPress site to associate our new meta box with all the content editors, using a quick `foreach` loop.

The following code shows how the `ch5_psl_register_meta_box` function can be rewritten to associate the new box with all post types:

```
function ch5_psl_register_meta_box() {
    $post_types = get_post_types( array(), 'objects' );
    foreach ( $post_types as $post_type ) {
        add_meta_box( 'ch5_psl_post_source_meta_box',
                      'Post/Page Source',
                      'ch5_psl_source_meta_box',
                      $post_type->name, 'normal' );
    }
}
```

As we can see, the `get_post_types` function provides us with a list of all registered post types. We can then use a simple `foreach` loop to iterate through all items received and add our meta box to each of them. If you are still running the book reviews plugin from *Chapter 4*, *The Power of Custom Post Types*, you'll be able to see the **Post/Page Source** meta box when editing reviews.

Displaying custom post data using filter functions

Once we have captured additional data in the post editor, the next logical step is to add code to display it when visitors view posts and pages. In the case of our source link data, the most logical place to show this link is after each post's content, which can easily be accomplished by assigning a function to the `the_content` filter.

Getting ready

You should have already followed the *Capturing and displaying information using custom meta boxes* recipe to have a starting point for this recipe. The resulting plugin should still be active in your development site. Alternatively, you can get the resulting code (`ch5/ch5-post-source-link/ch5-post-source-link-v1.php`) from the book's GitHub page and rename the file `ch5-post-source-link.php`.

How to do it...

Follow these steps to create a filter function to display the source data associated with a post or page item as a clean link:

1. Navigate to the `ch5-post-source-link` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch5-post-source-link.php` file in a code editor.
3. Add the following line of code at the end of the file to register a filter function to be called when the post and page content is prepared for display:

```
add_filter( 'the_content',
            'ch5_psl_display_source_link' );
```

4. Add the following code section to provide an implementation for the `ch5_psl_display_source_link` function:

```
function ch5_psl_display_source_link ( $content ) {
    $post_id = get_the_ID();
    if ( empty( $post_id ) ) {
        return $content;
    }

    if ( in_array( get_post_type( get_the_ID() ),
                   array( 'page','post' ) ) ) {
        // Retrieve source name and address for post
        $source_name =
            get_post_meta( $post_id, 'source_name',
                           true );
        $source_address =
            get_post_meta( $post_id, 'source_address',
                           true );

        // Output information to browser
        if ( !empty( $source_name ) &&
                !empty( $source_address ) ) {
            $content .= '<div class="source_link">';
            $content .= 'Source: <a href="';
            $content .= esc_url( $source_address );
```

```
                $content .= '">';
                $content .= esc_html( $source_name );
                $content .= '</a></div>';
            }
            return $content;
        }
    }
```

5. Save and close the plugin file.
6. Add source data to one of your site's posts and click to **View** its associated page:

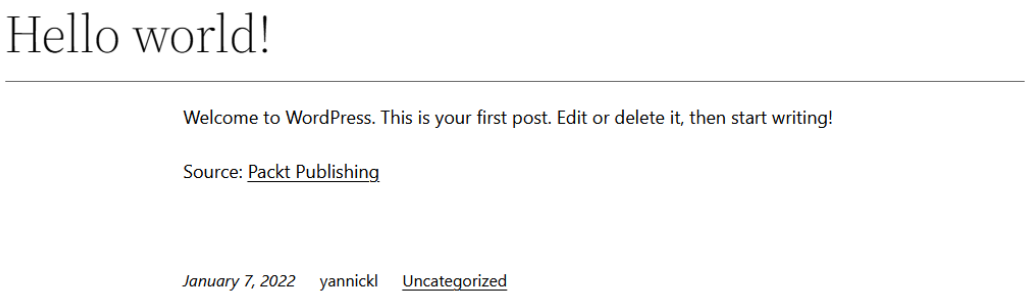


Figure 5.2 – Displaying the post source link

As seen in *Figure 5.2*, a new link is displayed after the post content with the information provided in the **Post/Page Source** meta box.

How it works...

Similar to what was done in *Chapter 2*, *Plugin Framework Basics*, in the *Adding text after each item's content using plugin filters* recipe, this recipe registers a callback with the `the_content` filter hook. This hook allows us to add an additional link after each post or page where the source name and source address fields have been filled in. We use the `get_post_type` function in conjunction with the standard PHP `in_array` function in our callback code to first check whether the item being displayed is a post or page. We then use the `get_post_meta` function to retrieve the post's source information and display a source link if both fields contain data.

We also use the `esc_url` and `esc_html` functions as a precaution to be sure that the data stored in these two post meta fields is clean before displaying it.

See also

- The *Capturing and displaying information using custom meta boxes* recipe

Hiding the Custom Fields section in the post editor

After having full control over which meta boxes are shown when creating custom post type editor controls and putting together plugin configuration pages, things are a little different when it comes to altering the basic post and page editors. More specifically, instead of choosing which meta boxes to display, the editor sections created by WordPress need to be removed to tailor the user experience.

Getting ready

You should have access to a WordPress development environment, either on your local computer or a remote server, where you will be able to load your new plugin files.

If you're using the default block editor to edit your posts and pages, follow these steps to display the **Custom Fields** section in the post editor:

1. **Edit** a post.
2. Go to the drop-down menu at the top-right corner of the editor (shown as three vertical dots, next to the **Publish** button and the gear icon).
3. Select the **Preferences** menu.
4. Go to the **Panels** tab.
5. Enable **Custom fields**.

6. Click the **Enable & Reload** button:

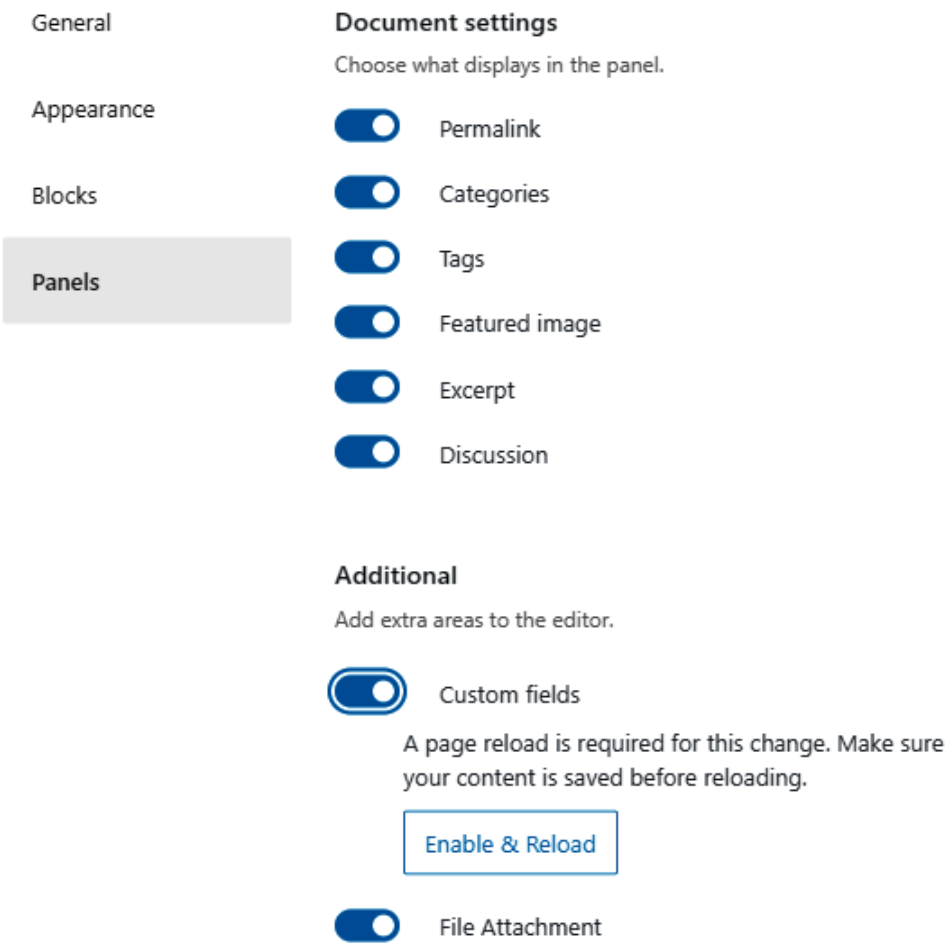


Figure 5.3 – Displaying custom fields in the block editor

You should now see the **Custom Fields** section in the editor:

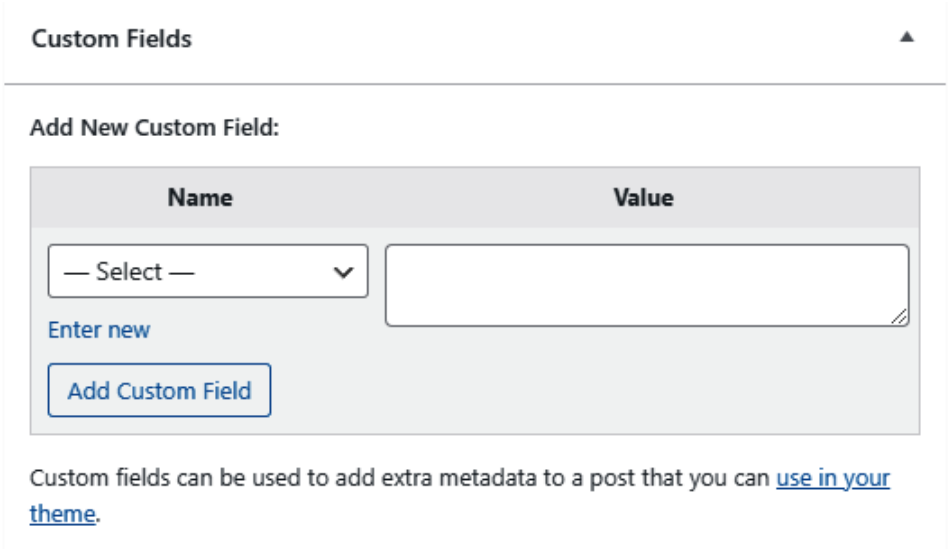


Figure 5.4 –The Custom Fields editor

How to do it...

Follow these steps to learn how to remove the **Custom Fields** meta boxes from the post and page editors:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch5-hide-custom-fields`.
3. Navigate to this directory and create a new text file called `ch5-hide-custom-fields.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 5 - Hide Custom Fields`.
5. Add the following line of code to register a function that will be executed when WordPress is preparing a list of meta boxes for all the administration areas:

```
add_action( 'add_meta_boxes',
            'ch5_hcf_remove_custom_fields_metabox' );
```

6. Add the following code section to provide an implementation for the `ch5_hcf_remove_custom_fields_metabox` function:

```
function ch5_hcf_remove_custom_fields_metabox() {
    remove_meta_box( 'postcustom',
                     array( 'post', 'page' ),
                     'normal' );
}
```

7. Save and close the plugin file.
8. Navigate to the **Plugins** management page and **Activate** the **Chapter 5 - Hide Custom Fields** plugin.
9. Go to the **Posts** section of the administration section and click on one of the entries to open the post editor. You will see that the **Custom Fields** section is no longer visible in the editor. It is also no longer possible to open this section from the editor **Preferences** section, while users of the Classic Editor plugin will no longer see **Custom Fields** listed in the **Screen Options** section.

How it works...

This short recipe contains only a few lines of code that register a function to be called when WordPress is preparing meta boxes for all the administration sections, followed by the implementation of this function. The function itself calls the `remove_meta_box` function to remove the **Custom Fields** section from the post and page editors. This function requires three parameters:

```
remove_meta_box( $id, $page, $context );
```

The first argument is the meta box identifier that was used when it was first created. While you may not know where the creation code for a given meta box is located within the WordPress source code, a quick look at the box's `div id` in the page source from a browser reveals its name. In this case, the ID is `postcustom`. The other two arguments indicate the name of the editor from which the meta box is to be removed and the context of the meta box (`normal`, `advanced`, or `side`).

Once the plugin is activated, the designated box disappears from the interface immediately, both in the default block editor or if the Classic Editor plugin is installed.

Extending the post editor to attach custom files with the media uploader

WordPress offers a very complete media upload tool to easily attach **featured images** to posts or pages. With a few simple function calls, plugin developers can create an instance of the WordPress media uploader in their custom interfaces, allowing users to easily upload and assign images.

Getting ready

You should have access to a WordPress development environment, either on your local computer or a remote server, where you will be able to load your new plugin files.

How to do it...

Follow these steps to learn how to leverage the WordPress media uploader to allow users to attach files to posts or pages and display them when these items are viewed:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch5-custom-file-attachment`.
3. Navigate to this directory and create a new text file called `ch5-custom-file-attachment.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 5 - Custom File Attachment`.
5. Insert the following line of code to register a function to be called when WordPress is preparing the meta boxes for all the administration sections:

```
add_action( 'add_meta_boxes',
            'ch5_cfa_register_meta_box' );
```

6. Add the following block of code to implement the `ch5_cfa_register_meta_box` function:

```
function ch5_cfa_register_meta_box() {
    add_meta_box( 'ch5_cfa_attach_file',
                  'File Attachment',
                  'ch5_cfa_attach_meta_box',
                  array( 'post', 'page' ), 'normal' );
}
```

7. Implement the function responsible for rendering the meta box contents, `ch5_cfa_attach_meta_box`, with the following code:

```
function ch5_cfa_attach_meta_box( $post ) {
    $attach_url = get_post_meta( get_the_ID(),
                        'attach_url', true );
    ?>
<table>
    <tr>
        <td style="width: 150px">Attachment URL</td>
        <td><input type="text" id="attach_url"
            class="components-text-control__input"
            name="attach_url" size="80"
            value="<?php echo $attach_url; ?>"
            /></td>
        <td><input type="button"
            id="attach_file_button"
            class="components-button is-secondary"
            value="Assign using Media Uploader"></td>
    </tr>
</table>

<?php }
```

8. Add this second block of code to the `ch5_cfa_attach_meta_box` function body, after the closing `</table>` tag and just before the closing `<?php }` statement:

```
<script type="text/javascript">
jQuery( document ).ready( function () {
    var file_frame;
    jQuery( '#attach_file_button' ).on( 'click',
                                function ( event ) {
        event.preventDefault();

        // If media frame already exists, reopen
        if ( file_frame ) {
            file_frame.open();
```

```
            return;
        }

        // Create the media frame.
        file_frame =
          wp.media.frames.file_frame = wp.media( {
              title : 'Select post file attachment',
              button : { text: 'Assign attachment' },
              multiple: false
        } );

        // When an image is selected, run a callback.
        file_frame.on( 'select', function () {
            attachment = file_frame.state().get(
                'selection' ).first().toJSON();
            jQuery( '#attach_url' ).val(
                                    attachment.url );
        });

        file_frame.open();
    });
});
</script>
```

9. Add the following line of code, which calls the `add_action` function to register a callback that will be executed when post data is processed to be saved:

```
add_action( 'save_post', 'ch5_cfa_save_attachment',
            10, 2 );
```

10. Implement `ch5_cfa_save_attachment` with the following code:

```
function ch5_cfa_save_attachment( $post_id = false,
                                  $post = false ) {
    if ( in_array( $post->post_type,
                   array( 'page', 'post' ) ) ) {
        if ( isset( $_POST['attach_url'] ) ) {
            update_post_meta( $post_id, 'attach_url',
```

```
                    esc_url_raw( $_POST['attach_url'] ) );
            }
        }
    }
```

11. Add the following line of code to register a filter function to be called when content is prepared for display:

```
    add_filter( 'the_content',
                'ch5_cfa_display_pdf_link' );
```

12. Insert the following code to provide an implementation for the `ch5_cfa_display_pdf_link` function:

```
    function ch5_cfa_display_pdf_link( $content ) {
        $post_id = get_the_ID();
        if ( empty( $post_id ) ||
                !in_array( get_post_type( $post_id ),
                           array( 'page', 'post' ) ) ) {
            return $content;
        }

        $attach_url =
            get_post_meta( $post_id, 'attach_url', true );
        if ( !empty( $attach_url ) ) {
            $content .= '<div class="file_attachment">';
            $content .= '<a target="_blank" href="';
            $content .= esc_url( $attach_url );
            $content .= '">';
            $content .= 'Download additional info';
            $content .= '</a></div>';
        }
        return $content;
    }
```

13. Add the following line of code to register a filter function that will be called when WordPress determines the list of file types that can be uploaded:

```
add_filter( 'upload_mimes', 'ch5_cfa_mime_types' );
```

14. Insert the following code to provide an implementation for the `ch5_cfa_mime_types` function:

```
function ch5_cfa_mime_types( $mime_types ) {
    $mime_types['svg'] = 'image/svg+xml';
    return $mime_types;
}
```

15. Save and close the plugin file.
16. Navigate to the **Plugins** management page and **Activate** the **Chapter 5 - Custom File Attachment** plugin.
17. Edit any post on your site to see the new **File Attachment** meta box:

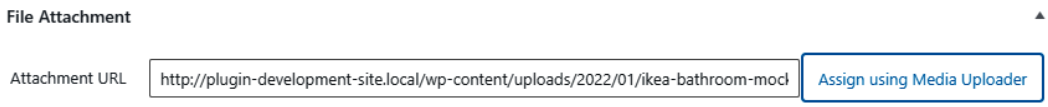


Figure 5.5 – The custom file attachment dialog

18. Enter a URL for the attachment or click on **Assign using Media Uploader** to open the WordPress media uploader tool. Select an existing media file or upload a new one.
19. Click on **Update** to save the file attachment.
20. View the edited post on the site:

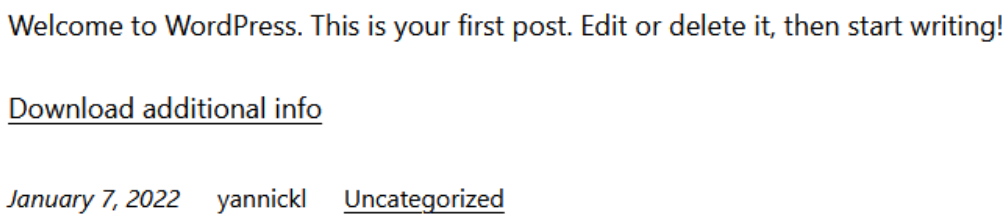


Figure 5.6 – The post download attachment

As we can see, the post is now followed by a link to download the attached file.

How it works...

The first part of the recipe registers a meta box, as we have seen in many recipes so far. This meta box displays a new editor section with a field to specify an attachment URL and a button used to launch the built-in WordPress media uploader. This tool is extremely familiar to WordPress users, since it's used to attach images to posts and pages. In addition to its familiarity, the Uploader provides us with a ton of functionality, such as allowing users to select from existing items or uploading new ones, validating to only allow safe file types, and easily providing the URL of the selected item back to our plugin code.

The media uploader is actually opened through a block of JavaScript and jQuery code that is also embedded in the meta box output. More specifically, when the button we created is clicked by the user, we first check to see whether the upload tool already exists and simply open it if that is the case. If it does not exist, we call the `wp.media` function to create the dialog with some parameters. We also attach code to be called when the user makes a selection in the upload tool, which will retrieve the URL of the selected item and copy it into the **Attachment URL** field.

Moving on to the function responsible for processing the post data, the recipe's code first checks whether the post being saved is a page or a post. If it matches one of these two types, we proceed to store the specified URL in the post metadata table.

This recipe also implements a pair of filter functions. The first is a filter for `the_content` to display a link to the attached file attachment under posts and pages. The second filter is associated with the `upload_mimes` hook to allow SVG files to be attached in addition to the default list of supported file types. While there are more than 90 common files types that can be uploaded in WordPress by default (`https://developer.wordpress.org/reference/functions/get_allowed_mime_types/#comment-3870`), SVG files are not part of that list, and it's quite possible that you may want your users to be able to attach files from other file types. For a full list of media types, consult the **Internet Assigned Numbers Authority** (**IANA**) media types listing (`https://www.iana.org/assignments/media-types/media-types.xhtml`).

See also

- The *Capturing and displaying information using custom meta boxes* recipe

6
Extending the Block Editor

As mentioned a few times thus far, a large evolution in WordPress functionality came in version 5.0 with the introduction of the **Block Editor**, also well known by its development codename, **Gutenberg**. While the original page editor was quite functional and had been supporting content creators for many years, it did not provide full **What You See Is What You Get** (**WYSIWYG**) functionality. More specifically, users building a complex website were often required to use shortcodes that are only seen once a page is saved and viewed on the site, or even had to type in HTML code. Of course, many third-party plugins (**Visual Composer**, **WPBakery**, **Elementor**, **Divi**, and so on) saw this as an opportunity to provide more advanced page building interfaces, eventually leading to the WordPress development team creating their own solution with the block editor.

The block editor introduced a new content creation paradigm with an array of building blocks that can be added and immediately seen on the page and post editors. The block editor also introduced a new programming paradigm for plugin developers, with the majority of its code being written in JavaScript instead of PHP. Furthermore, new blocks can either be written in traditional JavaScript or using **ESNext**, a new version of JavaScript that is not yet supported by browsers, along with **JSX**, a JavaScript module created by the React project.

While the same results can be achieved with both techniques, creating new blocks using ESNext and JSX is much easier, since developers can write simple code that will be converted to browser-compatible JavaScript through a build process. The resulting code is also better optimized to support all browsers with no additional work on the developer's part, and all compiled code is automatically combined and **minimized** in a single script file, allowing browsers to load it rapidly.

We'll develop three new blocks in this chapter using ESNext and JSX in the following recipes:

- Setting up a block development environment and creating a simple block
- Creating an editable blockquote block
- Creating a new WordPress REST endpoint
- Creating a server-side rendering block that leverages an existing shortcode

Curious readers can take a look at a set of sample block codes provided by the WordPress development team on GitHub (`https://github.com/wordpress/gutenberg-examples/`) to see how code differs between standard JavaScript and ESNext/JSX samples.

When developing plugins, supporting the Block Editor can help widen your audience, since many users have started using the new editor and are always on the lookout for plugins that support blocks. That being said, only supporting blocks to display your plugin's content or functionality can be very limiting, since users running the **Classic Editor** plugin, a different page builder, or the **ClassicPress** (`https://www.classicpress.net/`) WordPress fork won't be able to use your creations.

Technical requirements

You can find complete code examples on GitHub in the following repository: `https://github.com/PacktPublishing/WordPress-Plugin-Development-Cookbook-Third-Edition`.

You should also make sure that you are not running the Classic Editor WordPress plugin on your development site, since that plugin's only purpose is to suppress the new block editor, which we'll be extending.

Setting up a block development environment and creating a simple block

As mentioned in the introduction, code to create custom blocks written using ESNext and JSX must be compiled before being able to run in standard browsers. This recipe shows how to download a copy of **Node.js**, the tool that will allow us to build our code, along with a set of necessary support modules to create a reusable block development environment.

Getting ready

You should have access to a WordPress development environment, either on your local computer or a remote server, where you will be able to load your new plugin files.

How to do it...

Follow these steps to create your first simple block:

1. Download and install the current **long-term support** (**LTS**) version of Node.js (`https://nodejs.org/en/`). At the time of writing, the version is 16.13.2. The libraries that are currently used for WordPress block development are not currently compatible with Node.js version 17. On the Windows platform, make sure that the option to add the application to the system's path is selected as we go through the installation process.
2. If you are running a code editor with an integrated terminal such as Visual Studio Code, restart it so that it can be aware of Node.js.
3. Navigate to the WordPress `plugins` directory of your development installation.
4. Create a new directory called `ch6-block-template`.
5. Open **Command Prompt** or **Terminal** and navigate to the `plugins/ch6-block-template` directory.

> **Note**
>
> The actual location of the plugin directory will vary depending on your operating system and the local development web server selected. If you normally write code directly on a remote development web server, you will need to work on this chapter's code locally, compile it, and then upload the compiled version of the plugin code to see the recipe's results.

6. Run the **`npm init`** command from Command Prompt to start the interactive Node package manager initialization sequence.
7. Press the *Enter* key to accept default values for all of the fields except for **description**, which you can set to `Block template`; **entry point**, which should be set to `build/index.js`; and **license**, which should be set to `GPL-2.0-only`. A file called `package.json` will be created in the plugin directory once all the questions have been answered:

```
Administrator: Command Prom

 Directory of C:\WPDev\wp-content\plugins\ch6-block-template

07/04/2021  09:31 AM    <DIR>          .
07/04/2021  09:31 AM    <DIR>          ..
               0 File(s)              0 bytes
               2 Dir(s)  408,622,084,096 bytes free

C:\WPDev\wp-content\plugins\ch6-block-template>npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (ch6-block-template)
version: (1.0.0)
description: Block template
entry point: (index.js) build/index.js
test command:
git repository:
keywords:
author: ylefebvre
license: (ISC) GPL-2.0-only
About to write to C:\WPDev\wp-content\plugins\ch6-block-template\package.json:
```

Figure 6.1 – The Node package manager initialization

8. Still in Command Prompt, run the following command to download and install all the required packages to be able to compile the ESNext/JSX block creation code:

```
npm install --save-dev --save-exact @wordpress/scripts
```

This command takes less than a minute to run. When the command has finished running, we will see that a new directory called `node-modules` was created in our plugin directory, along with a new file called `package-lock.json`.

> **Note**
>
> As the `npm install` command runs, it displays various messages and warnings about the packages that are being downloaded. While some of these messages are deprecation messages and vulnerabilities, the maintainers of the `@wordpress/scripts` package are responsible for the selection of the scripts that it contains.

9. Edit the `package.json` file (created in the `ch6-block-template` directory by the `npm init` command) using a text editor and find the `scripts` section. Add a comma to the existing line in this section, followed by a new configuration line, both shown in bold as follows:

```
"scripts": {
    "test": "echo \"Error: no test specified\" && exit
             1",
    "build": "wp-scripts build"
},
```

10. Create a new directory in the `ch6-block-template` folder called `src` and create a new file named `index.js` under it.
11. Insert the following code in the newly created `src\index.js` file:

```
import { registerBlockType } from '@wordpress/blocks';

registerBlockType( 'ch6bt/twitter-feed', {
    title: 'Twitter Feed',
    icon: 'twitter',
    category: 'design',
    edit: () =>
          <p><a href="https://twitter.com/
          ylefebvre">Twitter Feed</a></p>,
    save: () =>
          <p><a href="https://twitter.com/
          ylefebvre">Twitter Feed</a></p>,
} );
```

12. Back in Command Prompt or Terminal, with the current directory still set to the `ch6-block-template` folder, run the following command to compile the block's JavaScript code:

```
npm run build
```

> **Note**
> It should be noted that if any changes are made to the contents of `src/index.js` after running the `npm run build` command, it will need to be executed once again to compile the latest changes.

13. Create a new file called `ch6-block-template.php` in the `ch6-block-template` directory.
14. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 6 – Block template`.
15. Add the following code to register a function to be called when the WordPress initialization function runs:

```
add_action( 'init', 'ch6bt_register_block' );
```

16. Insert the following code section at the end of the file to provide an implementation for the `ch6bt_register_block` function:

```
function ch6bt_register_block() {
    if ( !function_exists( 'register_block_type' ) ) {
        return;
    }

    $asset_file =
        include( plugin_dir_path( __FILE__ ) .
                                 'build/index.asset.php' );

    wp_register_script(
        'ch6bt-twitter-feed',
        plugins_url( 'build/index.js', __FILE__ ),
        $asset_file['dependencies'],
        $asset_file['version']
    );
    register_block_type( 'ch6bt/twitter-feed',
```

```
            array( 'editor_script' =>
                        'ch6bt-twitter-feed', ) );
  }
```

17. Save and close the plugin file.
18. Navigate to the **Plugins** management page and **Activate** the **Chapter 6 – Block Template** plugin.
19. Go to the **Pages** section of the WordPress administration interface and select the **Add New** sub-menu to create a new page.
20. Click on the + sign that appears on the right side of the first content section:

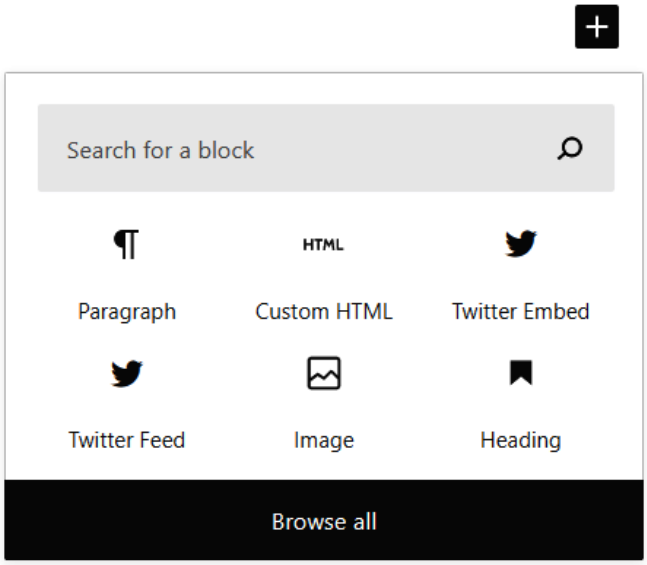


Figure 6.2 – A new Twitter Feed block

21. Click on the new **Twitter Feed** block to create an instance of it and see a link appear in the editor. If your block is not in the shortlist of blocks shown, search for its name in the search box.
22. **Publish** the new page and click on **View Page** to see the block on your development site.

> **Note**
> If you are running your code on a remote server and not in a local development environment, you should be creating the `ch6-block-template` folder in your server's `plugins` directory and then uploading the `ch6-block-template.php` in this folder, along with the entire `build` directory. You do not need to upload any of the `.json` files, the `node_modules` folder, or the `src` folder, as they are not required to run your new block.

How it works...

Node.js is a powerful tool that allows developers to run JavaScript code outside the browser from Command Prompt and is the cornerstone behind the creation of ESNext/JSX WordPress blocks. Once installed, we're able to run the necessary commands to download all scripts and modules required to compile our code and perform the actual compilation.

The `npm init` command's purpose is to create a file called `package.json`, defining our project's configuration. While many lines in this file are purely informational, the important lines are the ones that define where the compiled code should be saved (`build/index.js`) and the command to run to compile the code (`wp-scripts build`).

Our subsequent call to `npm install --save-dev --save-exact @wordpress/scripts` fetches all components that make up the `@wordpress/scripts` Node.js bundle, along with the `wp-scripts` executable that we referenced in our build command. While Node.js only needs to be installed once on your computer, steps 5 to 8 will need to be repeated for any new plugin you create that creates custom blocks. Alternatively, you can copy and modify the `package.json` file, along with a straight copy of the `node-modules` directory, to a new folder to save yourself time when starting a new project.

Once we have our tools in place, the next section of this recipe creates one of the simplest blocks possible, which only displays a short block of HTML code. This is done in `src/index.js` by first importing the `RegisterBlockType` function from the block development libraries, and then calling it with a number of parameters. These parameters respectively define the name of the new block, the name of the Dashicons font element to use as an icon for the block, the block editor category that the block should be assigned to, and finally, the code used when rendering the block in the editor (`edit`) and on visitor-facing web pages (`save`). As you can see, we use the same code in both cases in this recipe, and that code is essentially the same that we used in the *Creating a new simple shortcode* recipe in *Chapter 2*, *Plugin Framework Basics*.

Once all code has been placed in the `src/index.js` file, running the `npm run build` command calls the `wp-scripts` tool to compile our code and places the resulting output in `build/index.js`, overwriting any previous version of the file in the process.

The last few steps of this recipe are closer to the plugin development work that we did in previous chapters. More specifically, we write PHP code to register a function to be called when WordPress performs its initialization routine, and then provide an implementation for this callback function. Our callback first checks for the existence of the `register_block_type` WordPress function and exits immediately if the function does not exist. Its absence either indicates a version of WordPress older than 5.0, or a ClassicPress installation, where the block editor does not exist. In either case, trying to execute further code will only result in a fatal error on the site. If the function does exist, our code loads a small PHP file that was created during the compilation process called `index.asset.php` and uses variables found in this file in the following call to `wp_register_script`. Once the script has been included, our function calls the `register_block_type` function. In this function, the first parameter should match the block name defined when calling the `RegisterBlockType` function in our JavaScript code, and the second argument is an array of configuration parameters. For this first example, the array contains a single item specifying the script to be loaded to create the custom block.

Now, a block such as this specific one is perhaps not quite as useful as the shortcode that we created before, since it cannot be placed in the middle of a sentence to display a link to a Twitter feed. That being said, it shows us how to write and compile a very basic example of a block and can still be useful to create reusable HTML blocks for custom projects.

There's more...

With all of the files that were created in this recipe, let's take a look at the ones that should be placed under version control and distributed with your plugin.

Version control for custom blocks

If placing your plugin source files under version control, the `package-lock.json` file and `node_modules` directory do not need to be stored in your code repository. They can easily be recreated by running the `npm install` and `npm run build` commands.

Files required for plugin distribution

If you are planning on distributing your plugin, the only files that need to be included are the `ch6-block-template.php` file and the entire `build` directory. All of the other files (the `.json` files, the `src` folder, and the `node_modules` folder) are not used when running the code. If you plan to distribute your work as open source software, you should also include the `package.json` file and all files in the `src` directory. You should never include the `node_modules` directory, as it is quite large and can be recreated following the steps in this recipe.

See also

- The *Creating a new simple shortcode* recipe in *Chapter 2*, *Plugin Framework Basics*

Creating an editable blockquote block

After creating a simple static HTML custom block in the *Setting up a block development environment and creating a simple block* recipe, this recipe shows you how to create a custom block to display blockquotes where users will be able to provide their own content.

Getting ready

You should have already followed the *Setting up a block development environment and creating a simple block* recipe to have a working Node.js installation and be able to copy previously created files to your new plugin directory. Alternatively, you can perform steps 1 to 8 of the *Setting up a block development environment and creating a simple block* recipe when asked to copy files to the new folder you'll be creating.

How to do it...

Follow these steps to create a blockquote block with user-editable content:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch6-blockquote-block`.
3. Copy the `package.json` file and the `node_modules` directory from the `ch6-block-template` directory to your new folder.
4. Edit the `package.json` file in a text editor and modify the `name` field to become `ch6-blockquote-block`, change `description` to `Blockquote example for block editor`, and then save the file.

5. Create a new directory in the `ch6-blockquote-block` folder called `src` and create a new file named `index.js` under it.
6. Insert the following code in the newly created `src\index.js` file and save it:

```
import { registerBlockType } from '@wordpress/blocks';
import { RichText } from '@wordpress/block-editor';

registerBlockType( 'ch6bb-blockquote-block/blockquote', {
    title: 'Chapter 6 - Blockquote',
    icon: <svg height=»1792» viewBox=»0 0 1792
1792» width=»1792» xmlns=»http://www.w3.org/2000/
svg»><path d=»M832 320v704q0 104-40.5 198.5t-109.5
163.5-163.5 109.5-198.5 40.5h-64q-26 0-45-19t-19-45v-
128q0-26 19-45t45-19h64q106 0 181-75t75-181v-32q0-
40-28-68t-68-28h-224q-80 0-136-56t-56-136v-384q0-80
56-136t136-56h384q80 0 136 56t56 136zm896 0v704q0
104-40.5 198.5t-109.5 163.5-163.5 109.5-198.5 40.5h-
64q-26 0-45-19t-19-45v-128q0-26 19-45t45-19h64q106
0 181-75t75-181v-32q0-40-28-68t-68-28h-224q-80
0-136-56t-56-136v-384q0-80 56-136t136-56h384q80 0 136
56t56 136z»/></svg>,
    category: 'layout',
    attributes: {
        content: {
            type: 'array',
                source: 'children',
                selector: 'p',
        },
    },
    edit: ( props ) => {
        const { attributes: { content },
            setAttributes, className } = props;
        const onChangeContent = ( newContent ) => {
            setAttributes( { content: newContent } );
        };
        return (
            <RichText tagName="p"
                      className={ className }
                      onChange={ onChangeContent }
```

```
                        value={ content } />
            );
        },
        save: ( props ) => {
            return <RichText.Content tagName="p"
                value={ props.attributes.content } />;
        },
    } );
```

7. Open **Command Prompt** or **Terminal** and change the working directory to the `ch6-blockquote-block` folder.
8. Run the following command to compile the block's JavaScript code:

```
npm run build
```

9. Create a new file called `ch6-blockquote-block.php` in the `ch6-blockquote-block` directory.
10. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 6 – Blockquote Block`.
11. Add the following to register a function to be called when the WordPress initialization function runs:

```
add_action( 'init', 'ch6bb_register_block' );
```

12. Append the following code section to provide an implementation for the `ch6bt_register_block` function:

```
function ch6bb_register_block() {
    if ( !function_exists( 'register_block_type' ) ) {
        return;
    }

    $asset_file =
        include( plugin_dir_path( __FILE__ ) .
                            'build/index.asset.php');

    wp_register_script(
        'ch6bb-blockquote-block',
        plugins_url( 'build/index.js', __FILE__ ),
```

```
        $asset_file['dependencies'],
        $asset_file['version']
    );
    wp_register_style(
        'ch6bb-blockquote-style',
        plugins_url( 'css/style.css', __FILE__ ),
        array( ),
        filemtime( plugin_dir_path( __FILE__ ) .
                   '/css/style.css' )
    );
    register_block_type(
        'ch6bb-blockquote-block/blockquote',
        array(
            'style' => 'ch6bb-blockquote-style',
            'editor_script' =>
                'ch6bb-blockquote-block',
    ) );
}
```

13. Save and close the plugin file.
14. Create a new directory in the `ch6-blockquote-block` folder called `css` and create a new file named `style.css` under it.
15. Add the following CSS styling code to the newly created file and save it:

```
.wp-block-ch6bb-blockquote-block-blockquote {
    background: #f9f9f9;
    border-left: 10px solid #ccc;
    padding: 0.5em 10px !important;
    quotes: "\201C""\201D""\2018""\2019";
}
.wp-block-ch6bb-blockquote-block-blockquote:before {
    color: #ccc;
    content: open-quote;
    font-size: 4em;
    line-height: 0.1em;
    margin-right: 0.25em;
    vertical-align: -0.4em;
```

```
}
.wp-block-ch6bb-blockquote-block-blockquote p {
    display: inline;
}
```

16. Navigate to the **Plugins** management page and **Activate** the **Chapter 6 – Blockquote Block** plugin.
17. Go to the **Pages** section of the WordPress administration interface and select the **Add New** sub-menu to create a new page.
18. Click on the + sign that appears on the right side of the first content section.
19. Click on the new **Chapter 6 - Blockquote Block** to create an instance of it in the editor. If your block is not in the shortlist of blocks shown, search for its name in the search box.
20. Type some text in the new block to test out your new block:

> Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec nibh mauris, porta a tristique id, eleifend eu arcu. Sed at fermentum neque. Suspendisse mattis leo nulla, id malesuada nunc tempor non.

Figure 6.3 – The new blockquote block instance with dummy text

21. **Publish** the new page and click on **View Page** to see the block on your website.

How it works...

This second block creation recipe adds a few concepts to the simple static block that we created in the *Setting up a block development environment and creating a simple block* recipe. If we first look at `src/index.js`, we can see that we are importing a second element from the WordPress block development libraries, namely the `RichText` component.

Our main block code then proceeds to call the `RegisterBlockType` function but with a few differences from our first example. It first uses a block of SVG code to define the `icon` parameter and thus make our block stand out from other default blocks in the WordPress editor with a custom icon. It then proceeds to define `attributes` for the block. In this case, there is a single one called `content`, defined as an array capable of holding one or more HTML paragraphs.

The `edit` section of the `RegisterBlockType` function is also different from what we've seen before. It receives an argument called `props`. The `props` argument contains the block's data, which is assigned to the `attributes` structure that was previously created. The `edit` section also creates a function called `onChangeContent` that will be called when the user edits the blockquote's content. Finally, the `edit` section renders an element of `RichText` type and assigns a callback function to its `onChange` property. It also initializes its value to the text received from the block's data and assigns any stored CSS class name to it.

The `save` section of the `RegisterBlockType` function is much simpler but still uses the `RichText` element to render onscreen text based on the `content` attribute saved in the block's data.

The main plugin PHP file is almost exactly the same as the one created in this chapter's first recipe. More specifically, our code registers an `init` action hook callback. This callback proceeds to verify whether the `register_block_type` function exists. If it does, it follows up with calls to register the block's main JavaScript compiled file, along with creating a reference to a style sheet file. Finally, it calls the `register_block_type` function with an additional array member in its second argument to load the registered CSS style sheet.

The content of the CSS style sheet file that we created is used both in the block editor and in visitor-facing pages to style our new blockquote block.

See also

- The *Setting up a block development environment and creating a simple block* recipe

Creating a new WordPress REST endpoint

The **REST API** is an interface that allows application developers to exchange data with the core of a WordPress site. This API can be used for anything, from developing a mobile application that displays WordPress pages or custom post types to creating a complete replacement interface for content creation.

The new WordPress block editor was built using the REST API to interact with the rest of WordPress. As such, plugins occasionally need to extend the REST API to be able to implement new **endpoints**. This allows custom block JavaScript code to get access to data that is not part of the default WordPress REST API.

Getting ready

You should have already followed the *Managing multiple sets of user settings from a single admin page* recipe in *Chapter 3*, *User Settings and Administration Pages*, to have a starting point for this recipe. The resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch3/ch2-twitter-embed/ch2-twitter-embed-v2.php`) from the book's GitHub page and rename the file as `ch2-twitter-embed.php`.

How to do it...

Follow these steps to create a new endpoint used to retrieve a list of plugin options created in a previous recipe.

1. Navigate to the `ch2-twitter-embed` folder of the WordPress plugins directory of your development installation.
2. Open the `ch2-twitter-embed.php` file in a text editor.
3. Add the following line of code at the end of the file to assign a callback function to be called when WordPress initializes its REST API components:

```
add_action( 'rest_api_init', 'ch2te_rest_api_init' );
```

4. Continue the plugin's code file by inserting the following code to provide an implementation for the `ch2te_rest_api_init` function:

```
function ch2te_rest_api_init() {
    register_rest_route( 'twitter-embed/v1',
        '/optionlist',
        array(
            'methods' => 'GET',
            'callback' => 'ch2te_rest_option_list',
            'permission_callback' => '__return_true'
    ) );
}
```

5. Add the following block of code to the plugin file to provide an implementation for the `ch2te_rest_option_list` function that was mentioned in the previous step:

```
function ch2te_rest_option_list(
        WP_REST_Request $request ) {
```

```
        $option_array = array();
        for ( $counter = 1; $counter <= 5; $counter++ ) {
            $temp_options = ch2te_get_options( $counter );
            $option_array[$counter] =
            $temp_options['setting_name'];
        }
        $response = new WP_REST_Response( $option_array );
        return $response;
    }
```

6. Save and close the plugin file.
7. Enter the following address in your browser: `http://plugin-development-site.local/wp-json/twitter-embed/v1/optionlist`, replacing `http://plugin-development-site.local` with your development site's address. You should see the following output appear in your browser:

```
{"1":"First Setting","2":"Default",
 "3":"Default","4":"Default","5":"Default"}
```

As you can see, visiting this address returns a complete list of configuration names, as defined in our plugin's administration page. Note that the first part of the address will vary depending on the web server that you're using to test your development work.

How it works...

Our new code first takes care of registering a function to be called when WordPress initializes the REST API system. This function contains a single call to the `register_rest_route` function, where the first two parameters are used to specify the URL of the new REST API endpoint. They are followed by an array of additional parameters that specify the `methods` that will be used to query this data (**GET** and/or **POST**), the `callback` function that should be called when the new endpoint is queried, and finally, an optional `permission_callback` function that could be called to validate whether requests to this endpoint have proper permissions.

The remainder of this recipe's code implements the function that gets called when the endpoint is accessed. By using a simple `for` loop, this function's code retrieves the names of all configurations for the plugin and puts them in a new array. This is placed in a `WP_REST_Response` object and sent out to answer the request.

See also

- The *Managing multiple sets of user settings from a single admin page* recipe in *Chapter 3*, *User Settings and Administration Pages*

Creating a server-side rendering block that leverages an existing shortcode

As mentioned at the start of this chapter, block creation uses a different programming pipeline by leaning more heavily on JavaScript. This means that providing a simple block to complement an existing simple shortcode, as we've done in this chapter's first recipe, results in code duplication in our plugin. It's also not always easy for developers who are more accustomed to PHP and HTML to create complex blocks using pure JavaScript code. To alleviate these issues, WordPress offers a method by which plugins can use a server-side PHP function to render a block's content. This allows developers to offer plugins with shortcodes and blocks that share the same code base.

Getting ready

You should have already followed the *Creating a new WordPress REST endpoint* recipe to have a starting point for this recipe. The resulting plugin should still be active on your development site. Alternatively, you can get the resulting code (`ch6/ch2-twitter-embed/ch2-twitter-embed-v3.php`) from the book's GitHub page and rename the file as `ch2-twitter-embed.php`.

You should also have already followed the *Setting up a block development environment and creating a simple block* recipe to have a working Node.js installation and be able to copy previously created files to your new plugin directory. Alternatively, you can perform steps 1 to 8 of the *Setting up a block development environment and creating a simple block* recipe when asked to copy files to the new folder you'll be creating.

How to do it...

Follow these steps to use server-side rendering to render an embedded Twitter Feed block using previously created shortcode code:

1. Navigate to the `ch2-twitter-embed` folder of the WordPress `plugins` directory of your development installation.
2. Copy the `package.json` file and the `node_modules` directory from the `ch6-block-template` directory to the `ch2-twitter-embed` directory.

3. Edit the `package.json` file in a text editor and modify the `name` field to become `ch2te-twitter-embed`, change `description` to `Twitter Embed Block`, and then save the file.
4. Create a new directory in the `ch2-twitter-embed` folder called `src` and create a new file named `index.js` under it.
5. Insert the following code in the `src/index.js` file and save it:

```
import edit from './edit';
import { registerBlockType } from '@wordpress/blocks';

registerBlockType(
    'ch2twe-twitter-embed/twitter-embed', {
        title: 'Twitter Embed',
        icon: 'twitter',
        category: 'widgets',
        attributes: {
            user_name: {
                type: 'string',
                default: 'WordPress',
            },
            option_id: {
                type: 'string',
                default: '1',
            },
        },
        edit: edit,
        save() { return null; },
    }
);
```

6. Create a second file in the `src` sub-directory called `edit.js`.
7. Insert the following code to start creating the `src/edit.js` file:

```
import ServerSideRender from
    '@wordpress/server-side-render';
import { SelectControl, PanelBody, TextControl } from
    '@wordpress/components';
```

```
import { InspectorControls } from
    '@wordpress/block-editor';

const optionIdOptions = [];

wp.apiFetch(
    {path: "/twitter-embed/v1/optionlist"} ).then(
    posts => {
        jQuery.each( posts, function( key, val ) {
            optionIdOptions.push(
                {label: 'Option #' + key
                    + ': ' + val, value: key} );
    } );
} ).catch()
```

8. Add the following block of code to the `src/edit.js` file, after the code from the previous step:

```
const edit = props => {
    const { attributes: { user_name, option_id },
            className,
            setAttributes } = props;

    const setOptionID = option_id => {
        props.setAttributes( { option_id } );
    };

    const setUserName = user_name => {
        props.setAttributes( { user_name } );
    }
    const inspectorControls = (
        <InspectorControls key="inspector">
        <PanelBody>
        <TextControl
            label = "Twitter Account User Name"
            value = { user_name }
            onChange = { setUserName } />
```

```
            <SelectControl
                label="Options" value={ option_id }
                options= { optionIdOptions }
                onChange = { setOptionID } />
            </PanelBody>
            </InspectorControls> );
        return [
            <div className={ props.className }
                 key="returneddata">
                <div className="ch2te-block-warning">Twitter
                 Embed Warning: List of tweets for selected
                 user will only appear when viewing page</
                 div>
                <ServerSideRender
              block="ch2twe-twitter-embed/twitter-embed"
                     attributes = { props.attributes } />
                { inspectorControls }
            </div>
        ];
    };

    export default edit;
```

9. Save and close the `src/edit.js` file.
10. Open **Command Prompt** or **Terminal** and change the working directory to the `ch2-twitter-embed` folder.
11. Run the following command to compile the block's JavaScript code:

```
npm run build
```

12. Create a new directory in the `ch2-twitter-embed` folder called `css` and create a new file named `editor.css` within it.
13. Insert the following code in the newly created `editor.css` file:

```
.ch2te-block-warning {
    font-size: 14px !important;
    padding: 8px !important;
    border: solid 1px red !important;
```

```
    margin-bottom: 20px;
}
```

14. Edit the `ch2-twitter-embed.php` file and add the following line of code at the end of the file to declare a function to be called when WordPress initializes:

```
add_action( 'init', 'ch2te_register_block' );
```

15. Add the following block of code to implement the `ch2te_register_block` function:

```
function ch2te_register_block() {
    if ( !function_exists( 'register_block_type' ) ) {
        return;
    }

    $asset_file =
        include( plugin_dir_path( __FILE__ ) .
                                 'build/index.asset.php');
    wp_register_script(
        'ch2twe-twitter-embed',
        plugins_url( 'build/index.js', __FILE__ ),
        $asset_file['dependencies'],
        $asset_file['version']
    );
    wp_register_style(
        'ch2twe-editor-style',
        plugins_url( 'css/editor.css', __FILE__ ),
        array( ),
        filemtime( plugin_dir_path( __FILE__ ) .
                   '/css/editor.css' )
    );
    register_block_type(
        'ch2twe-twitter-embed/twitter-embed',
        array(
            'editor_script' => 'ch2twe-twitter-embed',
            'editor_style' => 'ch2twe-editor-style',
            'render_callback' =>
```

```
                    'ch2te_twitter_embed_shortcode',
                'attributes' => array(
                    'user_name'    => array(
                        'type'  => 'string',
                        'default'   => 'WordPress',
                    ),
                    'option_id'    => array(
                    'type'  => 'string',
                    'default'   => '1',
                ),
            )
        ) );
    }
```

16. Save and close the `ch2-twitter-embed.php` file.
17. Go to the **Pages** section of the WordPress administration interface and select the **Add New** sub-menu to create a new page.
18. Click on the + sign that appears on the right side of the first content section.
19. Click on the new **Twitter Embed** block to create an instance of it in the editor. If your block is not in the shortlist of blocks shown, search for its name in the search box:

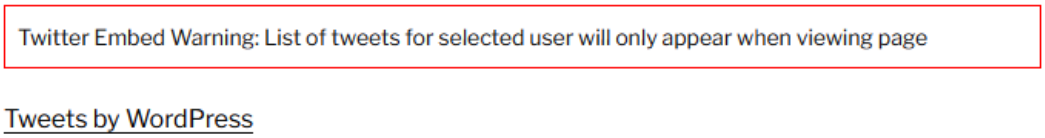


Figure 6.4 – The Twitter Embed block in the editor

20. Change the value of the **Twitter Account User Name** block configuration field to a different valid account to see the block output update.
21. Select a different set of **Options** from the drop-down list.
22. **Publish** the new page and click on **View Page** to see your new block in action.

As we can see, only the title of our embedded tweets section is visible, along with a warning when we insert our new block on a page, but the block behaves normally when viewed on the site.

How it works...

The first thing to note about this recipe is that while we've broken out our block JavaScript code into two separate files, both are combined together automatically in a single `index.js` file in the `build` directory when we compile our code. This allows developers to organize their development code in smaller files while maintaining good performance when running their custom blocks.

If we first look within `src/index.js`, we can see that we are calling the `registerBlockType` function as before but, this time, declaring two attributes for our block, `user_name` and `option_id`. We can also see that the `save` parameter of the function does not return anything, since block rendering is actually handled by a PHP function in our main plugin file. Finally, the `edit` parameter is just assigned a value of `edit`, a variable imported at the top of the script.

Moving on to the `src/edit.js` file, our code first imports a number of new components: `ServerSideRender`, `SelectControl`, `PanelBody`, `TextControl`, and `InspectorControls`. These will be used to build the configuration panel for our new block and to implement server-side rendering. These lines are followed by a section of code that uses the REST endpoint that we created in the previous recipe and stores the list of options it receives in an array called `optionIdOptions`.

The `edit` variable itself starts with the reception of the block's data and storing it locally. It then registers two callbacks to be called when the Twitter username value is changed or when a new option is selected from the block's configuration drop-down list.

In its next section, our code defines the block's configuration panel using the various elements that were imported at the top of the file using a syntax similar to HTML. We also use the `optionIdOptions` array to initialize the `SelectControl` component and show a drop-down list of options.

The last part of the `edit` section contains a user warning written in standard HTML, along with an instance of the `ServerSideRender` component and the configuration panel defined previously. When the block is added to the editor, the warning text shown in *Figure 6.4* appears, along with a small block of text.

The reason we are not seeing actual tweets is that the block editor is a dynamic environment where content is loaded as soon as blocks are created. When using server-side rendering, this does not always allow dynamic content triggers to run properly. In this case, our plugin output partly renders on screen, but the JavaScript code it loads from the Twitter website does not get triggered as it normally would. Thankfully, we are able to define different output when our block is rendered in the editor and on web pages. This allows us to display a warning to content creators to explain the situation.

We could rework our custom block code to display the tweets in the block editor by using ESNext and JSX, but that would defeat the purpose of using server-side PHP rendering and working off a single code base. Direct HTML output would render without a hitch using server-side rendering, but it would not teach us this lesson.

Moving on to the plugin's main PHP file, `ch2-twitter-embed.php`, its code is very similar to what we used for our blockquote block, with the addition of two items in the array of parameters sent to the `register_block_type` function. These items are the `render_callback`, which is the PHP function to be called to display the block's contents, and an array of `attributes` that should exactly match what was defined in `src/index.js`. While we used `attributes` before with our blockquote block, our PHP code did not need to be aware of them until now, since all rendering in the previous recipe was done within the JavaScript code.

It's interesting to note that the `render_callback` function that is assigned in our call to `register_block_type` is identical to the code we had previously defined to render our shortcode content. By making our attributes match the names of our previous shortcode arguments, we have complete code reusability between our existing shortcode implementation and our new block.

See also

- The *Creating a new simple shortcode* recipe in *Chapter 2*, *Plugin Framework Basics*
- The *Managing multiple sets of user settings from a single admin page* recipe in *Chapter 3*, *User Settings and Administration Pages*
- The *Creating an editable blockquote block* recipe
- The *Creating a new WordPress REST endpoint* recipe

7
Accepting User Content Submissions

Giving users the ability to contribute content to a website is always a good way to engage the community and keep content fresh on a website. That being said, WordPress does not provide any built-in mechanism for site visitors to submit content to site administrators. This is even more obvious when you consider that adding a simple contact form to a site requires the installation of a plugin.

Going back to the book review system that was put in place in *Chapter 4*, *The Power of Custom Post Types*, this chapter will show you how to create a user-facing form to allow visitors to submit their own book reviews to a website. We'll also see how to save this information in the site database, send out email notifications, and implement spam protection through the following recipes:

- Creating a client-side content submission form
- Saving user-submitted content in custom post types
- Sending email notifications upon new submissions

- Implementing a CAPTCHA on user forms using an online service
- Using a local library to implement a CAPTCHA on user forms

Technical requirements

You can find complete code examples on GitHub in the following repository: `https://github.com/PacktPublishing/WordPress-Plugin-Development-Cookbook-Third-Edition`.

Creating a client-side content submission form

The first step toward giving visitors the ability to contribute to a website is to present a form that they can use to submit new content. This recipe shows how to create a shortcode that can easily be inserted on any WordPress page to render a simple form.

Getting ready

You should be running the final version of the **Chapter 4 - Book Reviews** plugin created in *Chapter 4*, *The Power of Custom Post Types*. Alternatively, you can download the final resulting code (`ch4/ch4-book-reviews/ch4-book-reviews-v12.php`) from the book's GitHub page.

How to do it...

Follow these steps to create a new shortcode that displays a simple form:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch7-book-review-user-submission` and open it.
3. Create a text file called `ch7-book-review-user-submission.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 7 - Book Review User Submission`.
5. Add the following line of code to declare a new shortcode and its associated function:

```
add_shortcode( 'submit-book-review',
               'ch7_brus_book_review_form' );
```

6. Add the following code segment to provide an implementation for the `ch7_brus_book_review_form` function:

```
function ch7_brus_book_review_form() {
    if ( !is_user_logged_in() ) {
        echo '<p>You need to be a website member to ';
        echo 'submit book reviews.</p>';
        return;
    }
    ob_start();
    ?>

    <?php $content = ob_get_contents();
    ob_end_clean();
    return $content;
}
```

7. Insert the following block of code in the `ch7_brus_book_review_form` function body, between the closing PHP tag (`?>`) and the opening PHP tag (`<?php`):

```
<form method="post" id="add_book_review" action="">
<?php wp_nonce_field( 'add_review_form',
                      'br_user_form' ); ?>
<table style="width:100%"><tr>
    <td style="width:20%">Book Title</td>
    <td><input type="text" name="book_title"
               style="width:100%" /></td>
</tr><tr>
    <td>Book Author</td>
    <td><input type="text" name="book_author"
               style="width:100%" /></td>
</tr><tr>
    <td>Review</td>
    <td><textarea name="book_review_text"
                  style="width:100%"
                  rows="4"></textarea></td>
```

```
</tr><tr>
    <td>Rating</td>
    <td><select name="book_review_rating"
                style="width:200px">
    <?php for ( $r = 5; $r >= 1; $r-- ) { ?>
        <option value="<?php echo $r; ?>">
        <?php echo $r; ?> stars
    <?php } ?>
    </select></td>
</tr><tr>
    <td>Book Type</td>
    <td><?php
        $book_types = get_terms(
            'book_reviews_book_type',
            array( 'orderby' => 'name',
                   'hide_empty' => 0 ) );

        if ( !is_wp_error( $book_types )
                && !empty( $book_types ) ) { ?>
            <select name="book_review_book_type"
                    style="width:200px">
            <?php foreach ( $book_types as $type ) {
                echo '<option value="';
                echo $type->term_id;
                echo '">' . $type->name . '</option>';
            } ?>
            </select>
        <?php } ?></td>
</tr></table>
<input type="submit" name="submit"
       value="Submit Review" />
</form>
```

8. Insert the following inline stylesheet code in the `ch7_brus_book_review_form` function body, after the form closing tag (`</form>`) and the opening PHP tag (`<?php`):

```
<style>
    #add_book_review table {
        border-collapse: collapse;
    }
    #add_book_review tr {
        border-bottom: 1px solid #eee !important;
    }
    #add_book_review td {
        padding: 6px;
    }
</style>
```

9. Save and close the plugin file.
10. **Activate** the **Chapter 7 - Book Review User Submission** plugin.
11. Create a new page and insert the newly created `[submit-book-review]` shortcode in the item's content.
12. **Publish** and view the page to see the form.

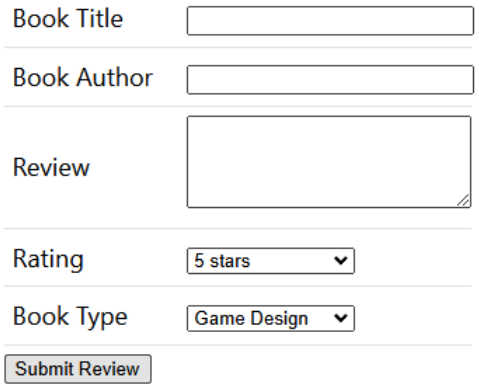


Figure 7.1 – Visitor-facing book review submission form

If you submit the form now, the review will not actually be saved since we have not implemented a processing function to parse and save the submitted data.

How it works...

As seen in the previous chapters, shortcodes are special blocks of text that can be easily added to any post or page to be replaced by content when they are found in pages visited by users. This recipe uses the `add_shortcode` function to declare a new shortcode, `[submit-book-review]`. The shortcode gets replaced with a review submission form when placed on a post or page. Of course, you could also try making a block out of it, as seen in the *Creating a server-side rendering block that leverages an existing shortcode* recipe in *Chapter 6*, *Extending the Block Editor*.

The form itself is created using standard HTML and displays a number of text fields. It also uses a bit of PHP code to dynamically build the list of ratings and book types defined in the system. The form includes a call to the `wp_nonce_field` function, which was previously seen when creating plugin configuration panels. This function adds hidden fields that will be used as a security measure in the associated data processing function. Finally, the code checks to see whether the user visiting the page is logged in to the website. In the case of a negative result, it displays a short message instead of the submission form.

When submitted, the `action` associated with the form sends visitor-provided content to the page where the book review form is displayed. This new content will be intercepted and processed in the code that will be added in the next recipe.

Finally, the `ob_start`, `ob_get_contents` and `ob_end_clean` functions are standard PHP functions that allow us to capture any output that would normally be sent to the browser, as well as have access to this output buffer before clearing it out. By using these functions, we are able to simplify the way we implement our shortcode function instead of having to append all of the form's HTML code to a content variable.

> **Note**
>
> We could have added this code to the plugin file that we created across all recipes in *Chapter 4*, *The Power of Custom Post Types*. Placing it in a separate file helps us start with a clean slate and also shows us that one plugin can augment another plugin's functionality. If you search for `Contact Form 7` or `Elementor` on the official WordPress plugin repository, you'll see that many developers have found ways to extend these popular plugins.

See also

- The *Creating a new simple shortcode recipe* in *Chapter 2*, *Plugin Framework Basics*

Saving user-submitted content in custom post types

When visitors click on the **Submit Review** button on the form created in the previous recipe, the target of the form is set to be the page that is hosting the submission form. Since this page is not capable of handling form data, we must implement an action hook that intercepts this post data and sends it to a processing function that we will define before returning visitors to the submission form page.

Getting ready

You should be running the final version of the **Chapter 4 - Book Reviews** plugin created in *Chapter 4*, *The Power of Custom Post Types*, and should have already followed the *Creating a client-side content submission form* recipe. Alternatively, you can get the files from the book's GitHub page (`ch4/ch4-book-reviews/ch4-book-reviews-v12.php` and `ch7/ch7-book-review-user-submission/ch7-book-review-user-submission-v1.php`) and rename `ch7-book-review-user-submission-v1.php` to `ch7-book-review-user-submission.php`.

How to do it...

Follow these steps to implement a function responsible for processing user input for the review submission form:

1. Navigate to the `ch7-book-review-user-submission` directory of the WordPress `plugins` folder of your development installation.
2. Open the `ch7-book-review-user-submission.php` file in a text editor.
3. Add the following line of code to register a function that will intercept user-submitted book reviews:

```
add_action( 'template_redirect',
            'ch7_brus_match_new_book_reviews' );
```

4. Add the following block of code to implement the `ch7_brus_match_new_book_reviews` function:

```
function ch7_brus_match_new_book_reviews( $template ) {
    if ( !empty(
            $_POST['ch7_brus_user_book_review'] ) ) {
        ch7_brus_process_user_book_reviews();
```

```
    } else {
        return $template;
    }
}
```

5. Insert the following code to provide an implementation for the `ch7_brus_process_user_book_reviews` function:

```
function ch7_brus_process_user_book_reviews() {
    // Check all required fields present and non-empty
    if ( !wp_verify_nonce( $_POST['br_user_form'],
                           'add_review_form' ) ||
          empty( $_POST['book_title'] ) ||
          empty( $_POST['book_author'] ) ||
          empty( $_POST['book_review_text'] ) ||
          empty( $_POST['book_review_book_type'] ) ||
          empty( $_POST['book_review_rating'] ) ) {
        $abort_message = 'Some fields were left ';
        $abort_message .= 'empty. Please go back ';
        $abort_message .= 'and complete form.';
        wp_die( $abort_message );
        exit;
    }

    $new_br_data = array( 'post_status' => 'publish',
        'post_title' => sanitize_text_field(
                            $_POST['book_title'] ),
        'post_type' => 'book_reviews',
        'post_content' => sanitize_text_field(
                      $_POST['book_review_text'] ) );

    // Store ID from post insertion
    $new_book_review_id =
        wp_insert_post( $new_br_data );

    add_post_meta( $new_book_review_id, 'book_author',
        sanitize_text_field(
```

```
                $_POST['book_author'] ) );
        add_post_meta( $new_book_review_id, 'book_rating',
            (int) $_POST['book_review_rating'] );

        $type = sanitize_text_field(
                    $_POST['book_review_book_type'] );
        if ( term_exists( $type,
                'book_reviews_book_type' ) ) {
            wp_set_post_terms( $new_book_review_id,
                               $type,
                               'book_reviews_book_type' );
        }

        $redirect_address =
            ( empty( $_POST['_wp_http_referer'] ) ?
            site_url() : $_POST['_wp_http_referer'] );
        wp_redirect( add_query_arg( 'add_review_message',
                        '1', $redirect_address ) );
        exit;
    }
```

6. In the original `ch7_brus_book_review_form` function, add the following code shown in bold between the `wp_nonce_field` function call and the opening `table` tag:

```
<?php wp_nonce_field( 'add_review_form',
                      'br_user_form' ); ?>

<?php if ( isset( $_GET['add_review_message'] )
    && $_GET['add_review_message'] == 1 ) { ?>
<div style="margin: 8px;border: 1px solid #ddd;
    background-color: #ff0;">
Thank for your submission!</div>
<?php } ?>
<!-- Variable to indicate user-submitted items -->
<input type="hidden" name="ch7_brus_user_book_review"
```

```
          value="1" />

  <table style="width:100%"><tr>
```

7. Save and close the plugin file.
8. Go back to the page containing the book review submission form and refresh the page to have the latest version of the form code.
9. Complete all fields and click on **Submit Review** to send all the fields to the newly created processing function. After processing the new content, the script will return to the form and display a confirmation message.
10. Visit the **Book Reviews** section of your site's administration pages to see your newly submitted entry.

How it works...

The purpose of this recipe is to process the book review data submitted by users with the form created in the previous recipe. In the first few steps, we assign a function to the `template_redirect` action hook to allow us to capture this data. This hook function is executed very early in the WordPress processing sequence. If a specific `POST` variable is found, we call our processing function that is defined in the rest of the recipe. Otherwise, we let WordPress process the page request normally.

The first thing that is done in our processing function is to check whether the proper hidden data field is found as part of the post data, using the `wp_verify_nonce` function. If it is not present, it indicates that someone may be trying to send data to the website without having visited the frontend form. In this case, our code displays an error message using the `wp_die` function. We also check whether all the form fields are non-empty, displaying the same error message if any field does not contain a value.

Once we are sure that all input is present and the posted data is valid, the next steps place all received data in an array. This data includes the book's title and review text, along with a post status and the `book_reviews` post type name. The resulting array is sent to the `wp_insert_post` function to create a book review and store the information within. As we can see, `wp_insert_post` only requires a single parameter, which receives the array that we just created. While we only define four elements in that data array, many more are available, as can be seen by consulting the WordPress Code Reference (`https://developer.wordpress.org/reference/functions/wp_insert_post/`).

Now, calling `wp_insert_post` only takes care of storing some key data elements that belong in the post data. We must follow up this code with calls to `update_post_meta` and `wp_set_post_terms` to store the remaining book review information in the website database.

Once all the information is stored, we use a combination of the `wp_redirect` and `add_query_arg` functions to send the user back to the page where they submitted their book review. Our code also makes sure that only one instance of the `add_review_message` variable is in the target address.

Finally, this recipe makes a small modification to the book review form to add a confirmation message shown to visitors when information is accepted by the plugin.

There's more...

In a world of spam bots, who are tasked with creating bogus content on any website, setting new book reviews to be immediately visible on the website might not be the wisest idea.

Moderating user-submitted content

Instead of setting a status of `publish` for new post entries, we can use a value of `draft` to make the new entry visible only in the backend administration area. To give plugin users more flexibility, you could also give administrators a way to choose what method they prefer in a plugin configuration page.

See also

- The *Creating a client-side content submission form* recipe
- The *Processing and storing plugin configuration data* recipe in *Chapter 3*, *User Settings and Administration Pages*
- The *Adding a new section to the custom post type editor* recipe in *Chapter 4*, *The Power of Custom Post Types*

Sending email notifications upon new submissions

Just like WordPress sends out email notifications to the administrator when new comments are posted, it can be very useful to send out emails when visitors post new book reviews. This allows website managers to review new content as it comes in and decide whether they approve it to be published online.

Getting ready

You should be running the final version of the Book Reviews plugin created in *Chapter 4, The Power of Custom Post Types*, and should have already followed the *Saving user-submitted data in custom post types* recipe (including changing the post status to `draft` as indicated in the *There's more...* section). Alternatively, you can get the resulting files from the book's GitHub page (`ch4/ch4-book-reviews/ch4-book-reviews-v12.php` and `ch7/ch7-book-review-user-submission/ch7-book-review-user-submission-v2.php`) and rename `ch7-book-review-user-submission-v2.php` to `ch7-book-review-user-submission.php`. Finally, you should have access to a WordPress installation on a hosted web server, as emails are usually not sent when running it on a local installation. Be sure to have access to the email account associated with the site administrator account to see the resulting email.

If you're using Local as your development server, you can go to the **TOOLS** tab of your site configuration and click on **OPEN MAILHOG** to capture emails. Once the **MailHog** page appears in your browser, click on **Connected** and use the **Refresh** button (button with two spinning arrows) to see captured emails after completing this recipe.

How to do it...

Follow these steps to add code that will prepare email data for new reviews and send it using the `wp_mail` function:

1. Navigate to the `ch7-book-review-user-submission` directory of the WordPress `plugins` folder of your development installation.
2. Open the `ch7-book-review-user-submission.php` file in a text editor.
3. Add this following code at the end of the plugin file to define a new filter function to set the sending email address when using the `wp_mail` function:

```
function ch7_brus_custom_wp_mail_from( $orig_email ) {
    return str_replace( 'wordpress@', 'bookreviews@',
```

```
                                $orig_email );
}
```

4. Add the following block of code at the end of the plugin file to define a second filter function to define the name of the sender when using the `wp_mail` function:

```
function ch7_brus_custom_wp_mail_from_name( $from ) {
    return get_bloginfo( 'name' ) . ' - ' .
        'Book Review System';
}
```

5. Insert the following line of code to register a function to be called back when new posts are submitted:

```
add_action( 'wp_insert_post', 'ch7_brus_send_email',
            10, 2 );
```

6. Insert the following block of code to implement the `ch7_brus_send_email` function:

```
function ch7_brus_send_email( $post_id, $post ) {
    if ( !isset(
            $_POST['ch7_brus_user_book_review'] ) ||
            'book_reviews' != $post->post_type ) {
        return;
    }

    $admin_mail = get_option( 'admin_email' );
    $headers = 'Content-type: text/html';
    $message = 'New user book review.<br />';
    $message .= 'Book Title: ';
    $message .= esc_html( $post->post_title );
    $message .= '<br /><a href="';
    $message .= add_query_arg( array(
                    'post_status' => 'draft',
                    'post_type' => 'book_reviews' ),
                    admin_url( 'edit.php' ) );
    $message .= '">Moderate new book reviews</a>';
    $email_title = "Book Reviews - ";
```

```
        $email_title .= esc_html( $_POST['book_title'] );

        add_filter( 'wp_mail_from',
                    'ch7_brus_custom_wp_mail_from' );
        add_filter( 'wp_mail_from_name',
                    'ch7_brus_custom_wp_mail_from_name' );

        wp_mail( $admin_mail, $email_title, $message,
                 $headers );

        remove_filter( 'wp_mail_from',
                       'ch7_brus_custom_wp_mail_from' );
        remove_filter( 'wp_mail_from_name',
            'ch7_brus_custom_wp_mail_from_name' );
    }
```

7. Save and close the plugin file.
8. Go back and visit the book review submission form and submit a book review.

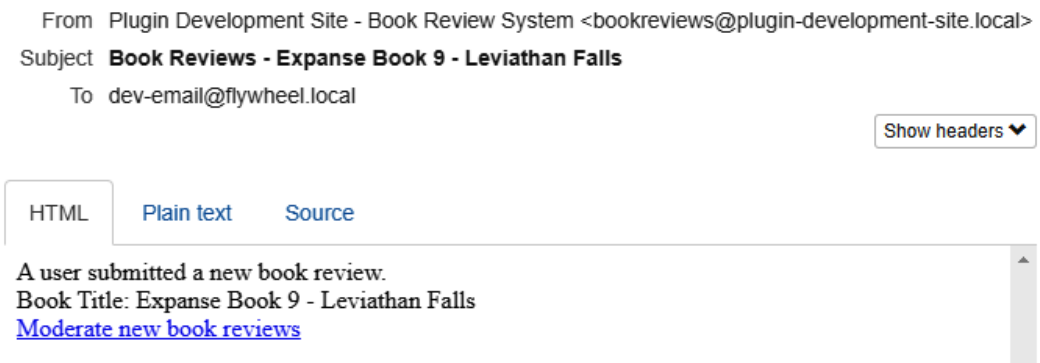


Figure 7.2 – Email generated by user submission. Captured with the Mailhog tool in the Local web server

An email will be sent to the address associated with the website administrator user account, containing some information from the new review and a link to view all draft book reviews.

How it works...

The `wp_mail` function can be used by any plugin to send out email messages. It takes five arguments to define all the elements of the outgoing message:

```
wp_mail( $destination, $subject, $message, [$headers],
         [$attachments] );
```

The first three arguments are required and define, respectively, the email address of the intended recipient, the title of the message, and its content. As we have seen in this recipe, the content is mainly specified using standard HTML syntax, while the target email address is retrieved from the site options table using the `get_option` function. As for the title, it is built from static text and the book review title to create the final result.

The next parameter is optional and provides header information for the email, with the most important piece of information in that section being the character set. The last parameter can optionally be used to specify the path of one or more files to be sent as email attachments.

To make it easier for website administrators to manage new entries, part of the message body contains a link to the book reviews management page of the WordPress website administration area to quickly display all the unapproved entries (set as `draft` items).

The recipe also makes use of the `wp_mail_from` and `wp_mail_from_name` filters to change the email address and sender name when calling the `wp_mail` function. These filters are registered just before calling `wp_mail` and removed right after the function call to avoid affecting other calls to `wp_mail` in WordPress or other plugins.

See also

- The *Creating a client-side content submission form* recipe

Implementing a CAPTCHA on user forms using an online service

A common security measure on website forms is to use **CAPTCHA** codes. These codes use distorted letters or some other form of test to check that the person submitting data is not a spam robot. The form that we have been building to accept visitor-submitted book reviews could benefit from this type of technology to avoid weeding through unwanted entries.

This recipe shows how to integrate Google's reCAPTCHA service in our book review submission form. If you prefer using a local CAPTCHA script to avoid being dependent on an online service or to be sure that your form can be used around the globe, jump to the next recipe, *Using a local library to implement a CAPTCHA on user forms*.

Getting ready

You should be running the final version of the Book Reviews plugin created in *Chapter 4*, *The Power of Custom Post Types*, and should have already followed the *Sending email notifications upon new submissions* recipe. Alternatively, you can get the resulting files from the book's GitHub page (`ch4/ch4-book-reviews/ch4-book-reviews-v12.php` and `ch7/ch7-book-review-user-submission/ch7-book-review-user-submission-v3.php`) and rename `ch7-book-review-user-submission-v3.php` to `ch7-book-review-user-submission.php` before starting the recipe. This recipe also requires having a Google account to be able to register for the reCAPTCHA service.

How to do it...

Follow these steps to integrate the reCAPTCHA service in your book review submission form:

1. Visit the Google reCAPTCHA administrator website (`https://www.google.com/recaptcha/admin`) and log in if you are not already connected to Google services.
2. Register a new website by specifying a **Label** (for example, `Development website`) and setting the type of reCAPTCHA to **reCAPTCHA V2** using the radio selector.
3. In the additional list of options that appear, select the **I'm not a robot** checkbox.
4. Enter the domain name of the web server you will be using to test your code in the **Domains** section. If you are running Local as your development server, this may be `plugin-development-site.local`.
5. Check the box to accept the terms of service and then click on **Submit** to complete the new website creation process.
6. On the resulting page, take note of the **site key** and **secret key** fields that the service assigned to your test website. They will be inserted into the recipe code in further steps.

7. Navigate to the `ch7-book-review-user-submission` directory of your `plugins` folder and open the file named `ch7-book-review-user-submission.php` in a code editor.
8. Insert the following line of code at the end of the file to register a function that will be called when WordPress prepares the list of scripts to be loaded on the website:

```
add_action( 'wp_enqueue_scripts',
            'ch7_brus_recaptcha_script' );
```

9. Add the following block of code to provide an implementation for the `ch7_brus_recaptcha_script` function:

```
function ch7_brus_recaptcha_script() {
    wp_enqueue_script( 'google_recaptcha',
        'https://www.google.com/recaptcha/api.js',
        array(), false, true );
}
```

10. Locate the `ch7_brus_book_review_form` function in your code and add an extra row just before the table closing code (`</tr></table>`) to display the CAPTCHA. Replace `[my-site-key]`, including the square brackets, with the site key obtained earlier from the reCAPTCHA service, leaving the double quotes in place:

```
</tr><tr>
    <td colspan="2">
    <div class="g-recaptcha"
    data-sitekey="[my-site-key]"></div>
    </td>
```

11. Save your plugin and visit your book review submission page. You should now see the Google reCAPTCHA box appear in the form.

Figure 7.3 – Google reCAPTCHA verification box

12. Visit the Google reCAPTCHA GitHub page (`https://github.com/google/recaptcha`) and download the latest version of the repository to your computer.
13. Extract the repository contents and copy the resulting `recaptcha-master\src` folder to the `ch7-book-review-user-submission` directory, renaming the `src` folder to `recaptcha`.
14. Locate the `ch7_brus_process_user_book_reviews` function in your code and add the following lines of code at the very top of the function implementation. Replace `[my-secret-key]`, including the square brackets, with the secret key obtained earlier from the reCAPTCHA service, leaving the single quotes in place:

```
require_once plugin_dir_path( __FILE__ ) .
                '/recaptcha/autoload.php';

$recaptcha =
    new \ReCaptcha\ReCaptcha( '[my-secret-key]' );
$resp = $recaptcha->verify(
            $_POST['g-recaptcha-response'],
            $_SERVER['REMOTE_ADDR'] );
if ( !$resp->isSuccess() ) {
    $abort_message = 'Missing or incorrect captcha. ';
    $abort_message .= 'Please go back and try again.';
    wp_die( $abort_message );
    exit;
} else {
```

15. Add an extra closing bracket (`}`) just before the end of the `ch7_brus_process_user_book_reviews` function to close out the `else` section of the code inserted in the previous step.
16. Save and close the code file.
17. Submit a new book review without checking the **reCAPTCHA** checkbox to see that it displays an error message indicating that the captcha is missing or incorrect.

How it works...

Google's reCAPTCHA service is a free service that can generate and display a CAPTCHA box in your form. It has evolved over time from asking users to decipher a scrambled message in an image to displaying addresses and other street signs. Other versions asked users to enter what they saw. In its latest incarnation, reCAPTCHA simply asks the user to check a box to indicate that they are not a robot. If it suspects the visitor might be an automated system, then it asks more advanced questions. When visitors check the box, a hidden field is populated with a validation code that gets sent to our data processing function along with the rest of the form data.

Adding a reCAPTCHA to our form is actually quite easy. It only requires us to load a JavaScript file and add a line of code to our form that gets transformed into the service's trademark checkbox. Once the form data is posted, validating it is a bit more complex than the nonces we have used before. More specifically, our plugin needs to communicate with Google's servers to check the validation code. Thankfully, Google offers an easy-to-use library to hide away much of this operation's complexity. If the reCAPTCHA code received is valid, the previously created data processing and storage code is executed as before. Otherwise, an error message is displayed to users. In addition to being easy to integrate, a benefit of using a third-party service is that most code updates are done by the service provider. You would still need to check for occasional updates to the PHP validation library, but that is only a small part of this service's functionality.

If you are planning to distribute a plugin that makes use of the reCAPTCHA service, it would not make sense to leave your own website and secret keys in the final plugin code. You should instead create an administration panel, as we learned to do in *Chapter 3*, *User Settings and Administration Pages*. This way, users can enter their own keys and have them be used on the website.

> **Note**
>
> It should be noted that while Google reCAPTCHA is one of the most popular CAPTCHA generation services, it does not directly work in China at the time of writing. It has not worked there for some time and might also be blocked by other countries. If you are developing a plugin for a wide audience, you might want to consider supporting more than one CAPTCHA service or library. This way, your users will have options, no matter where they or their audience resides. Read the next recipe for an example of using a local library to generate a CAPTCHA.

See also

- The *Sending email notifications upon new submissions* recipe
- The *Creating an administration page menu item in the Settings menu* recipe from *Chapter 3*, *User Settings and Administration Pages*
- The *Rendering the admin page contents using HTML* recipe from *Chapter 3*, *User Settings and Administration Pages*
- The *Processing and storing plugin configuration data* recipe from *Chapter 3*, *User Settings and Administration Pages*

Using a local library to implement a CAPTCHA on user forms

As discussed in the previous recipe, *Implementing a CAPTCHA on user forms using an online service*, adding a CAPTCHA to visitor-facing forms helps reduce unwanted submissions to a website. After seeing how to integrate an online third-party service, this recipe shows how to download and integrate a local PHP script to generate and validate CAPTCHA images locally.

Getting ready

You should be running the final version of the Book Reviews plugin created in *Chapter 4*, *The Power of Custom Post Types*. You should also have already followed the *Sending email notifications upon new submissions* recipe. You should not have any Google reCAPTCHA code from the previous recipe.

Alternatively, you can get the resulting files from the book's GitHub page (`ch4/ch4-book-reviews/ch4-book-reviews-v12.php` and `ch7/ch7-book-review-user-submission/ch7-book-review-user-submission-v3.php`). The `ch7-book-review-user-submission-v3.php` file should be renamed to `ch7-book-review-user-submission.php` before starting the recipe.

The GD and FreeType PHP libraries need to be installed and activated in your development web server's PHP installation to be able to generate a CAPTCHA image. Most of the pre-packaged local web servers listed in the *Installing a web server on your computer* recipe from *Chapter 1*, *Preparing a Local Development Environment*, come with these libraries activated. They are also commonly enabled on most hosted web servers. That being said, you should still mention that they are required when distributing a plugin containing the following code to a larger audience.

How to do it...

Follow these steps to integrate a local library to provide CAPTCHA protection for your book review submission form:

1. Visit the **Securimage** website (`https://www.phpcaptcha.org/download/`) to download the latest version of their CAPTCHA library.
2. Open the resulting archive and extract the entire `securimage` directory to the `ch7-book-review-user-submission` folder of your plugin directory.
3. In the plugin directory, open the file named `ch7-book-review-user-submission.php` in a code editor.
4. Locate the `ch7_brus_book_review_form` function in your code and add an extra row just before the table closing code (`</tr></table>`) to display the CAPTCHA:

```
  </tr><tr>
     <td>Enter text from image</td>
     <td><img id="captcha"
              src="<?php echo plugins_url(
                  'securimage/securimage_show.php',
                  __FILE__ ); ?>"
              alt="CAPTCHA Image" />
          <input type="text" name="captcha_code"
                 size="10" maxlength="6" />
          <a href="#"
                 onclick="document.getElementById(
                 'captcha' ).src =
                 '<?php echo plugins_url(
                     '/securimage/securimage_show.php',
                     __FILE__ ); ?>?'
                 + Math.random(); return false">
                 [ New Image ]</a>
  </td>
```

5. Save your plugin and visit your book review submission page. You should now see the CAPTCHA image appear in the form.

Figure 7.4 – Captcha generated with a local library

6. Locate the `ch7_brus_process_user_book_reviews` function in your code and add the following lines of code at the top of the function implementation:

```
if ( PHP_SESSION_NONE == session_status() ) {
    session_start();
}
include_once plugin_dir_path( __FILE__ ) .
    '/securimage/securimage.php';
    $securimage = new Securimage();

if ( false == $securimage->check(
        $_POST['captcha_code'] ) ) {
    // Display message if required fields are missing
    $abort_message = 'Missing or incorrect captcha. ';
    $abort_message .= 'Please go back and try again.';
    wp_die( $abort_message );
    exit;
} else {
```

7. Add an extra closing bracket (}) just before the end of the `ch7_brus_process_user_book_reviews` function to close out the `else` section of the code inserted in the previous step.
8. Save and close the code file.
9. **Submit** a new book review without entering the CAPTCHA code to see that it displays an error message indicating that the captcha is missing or incorrect.

How it works...

The **Securimage** script is a simple tool that can generate and display a CAPTCHA image, as well as store the string that it generated using session data.

The recipe's code displays the CAPTCHA image by using a standard HTML `img` tag that uses one of the scripts in the `securimage` directory as the image path. On the data processing side, our code first starts by calling `session_start` to reconnect to the session that was initiated by the image generator to store the CAPTCHA code. It then proceeds to check whether the user CAPTCHA text matches the image that was displayed using the `check` method of the `securimage` class. Based on this result, we display an error message or continue verifying that the rest of the required data fields have been submitted correctly.

While there are many PHP scripts available for generating CAPTCHA images, this particular one was selected for this recipe since it uses the BSD license. This makes it compatible with WordPress licensing. You would therefore be able to distribute this script with your plugin on the official WordPress plugin repository.

> **Note**
> When using a third-party library in your plugin, you should regularly check whether that library has been updated by its author and incorporate new versions in your work to make sure that you don't expose your plugin to security vulnerabilities.

For more advanced content filtering methods, look up the **Akismet** API (`https://akismet.com/development/api/`).

See also

- The *Implementing a CAPTCHA on user forms using an online service* recipe

8
Customizing User Data

After learning how to customize the post editor and taxonomies with custom data fields, the next areas that we will be augmenting are the user creation and editing sections of WordPress. This type of modification can be used for many different purposes, including tracking when users have last visited a site or storing additional data on the types of services they are interested in. This chapter expands on the private content plugin that we started in the *Creating a new enclosing shortcode* recipe in *Chapter 2*, *Plugin Framework Basics*, and the *Storing stylesheet data in user settings* recipe in *Chapter 3*, *User Settings and Administration Pages*. This time around, we are adding a second level of content protection that will restrict specially identified post or page content to only be displayed for paid users.

This chapter shows us how to augment the WordPress user editor and use the additional data associated with user accounts through the following recipes:

- Adding custom fields to the user editor
- Processing and storing custom user data
- Displaying new user data on the user list page
- Using custom user data in an enclosing shortcode

Technical requirements

You can find complete code examples on GitHub in the following repository: `https://github.com/PacktPublishing/WordPress-Plugin-Development-Cookbook-Third-Edition`.

Adding custom fields to the user editor

Similar to the taxonomy editor, the user editor allows developers to add extra fields through action hooks. This task is facilitated by using the same HTML structure, whether a new user is being created or an existing one is modified.

Getting ready

To create a complete plugin with a stylesheet and administration panel, you should have already followed the *Storing stylesheet data in user settings* recipe in *Chapter 3*, *User Settings and Administration Pages,* to have a starting point for this recipe. Alternatively, you can get the resulting plugin code directory (`ch3/ch2-private-item-text/*`) from the book's GitHub page and rename `ch2-private-item-text-v2.php` to `ch2-private-item-text.php`.

It is also possible to start fresh and create a new empty plugin file and follow this chapter's recipes by making a folder called `ch2-private-item-text` in your plugin directory with a text file called `ch2-private-item-text.php`, and only adding a standard plugin header at the top of the file.

How to do it...

Follow these steps to add a new section to the user editor:

1. Navigate to the `ch2-private-item-text` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch2-private-item-text.php` file in a text editor.
3. Add the following code snippet at the end of the file to declare a global variable and initialize the variable's content:

```
global $user_levels;
$user_levels = array( 'regular' => 'Regular',
                      'paid' => 'Paid' );
```

4. Add the following lines of code to register a function to be called when new users are added to the site through the administration interface or when users are edited:

```
add_action( 'user_new_form',
            'ch2pit_show_user_profile' );
add_action( 'edit_user_profile',
            'ch2pit_show_user_profile' );
```

5. Add the following block of code to provide an implementation for the `ch2pit_show_user_profile` function:

```
function ch2pit_show_user_profile( $user ) {
    global $user_levels;
    $current_user_level = '';

    if ( !empty( $user ) &&
            isset( $user->data->ID ) ) {
        $user_id = $user->data->ID;
        $current_user_level =
            get_user_meta( $user_id, 'user_level',
                           true );
    } ?>

    <h3>Site membership level</h3>
    <table class="form-table">
        <tr><th>Level</th>
            <td><select name="user_level"
                        id="user_level">
            <?php foreach( $user_levels as
                 $user_level_idx => $user_level ) { ?>
                <option value="<?php
                    echo $user_level_idx; ?>"
                <?php selected( $current_user_level,
                                $user_level_idx ); ?>>
                <?php echo $user_level; ?></option>
            <?php } ?>
        </select></td></tr>
```

```
        </table>
  <?php }
```

6. Save and close the plugin file.
7. Go to the **Plugins** section of the administration interface and make sure that the **Chapter 2 - Private Item Text** plugin is activated.
8. Select the **Add New** menu item under the **Users** section to view the user creation page. You should see the new **Site membership level** section at the bottom of the form:

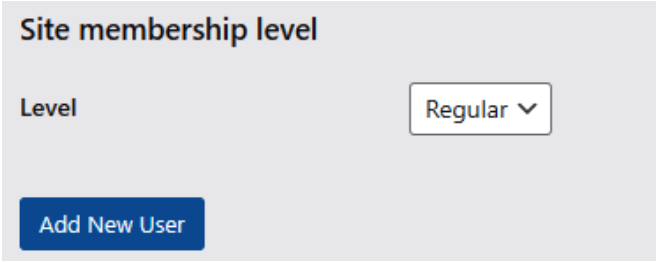


Figure 8.1 – Site membership level option

9. Edit one of the existing users (except your own administrator user account) to see that the additional section is also displayed. If you try to save the user in either situation, the newly added field will not be saved until you perform the steps in the next recipe.

How it works...

The `user_new_form` and `edit_user_profile` action hooks allow developers to add content to the user editor when a new user is added to the site and when users are edited, respectively.

> **Note**
>
> WordPress offers a third action hook related to editing users, called `show_user_profile`. It should be noted that functions associated with that hook get called when users view their own profile. You should carefully consider whether or not it makes sense to register a function with this action hook depending on the nature of your plugin and the additional information to be stored. If you do not register any function with this hook, as we are doing in this recipe, you will not see the additional fields created when viewing your own user profile on the site, even if you are logged in as an administrator. That being said, you will still be able to see any locked content since we will display it to users with administrator privileges.

Our callback function first checks to see whether it received a valid user through the `$user` parameter or whether a new user is being created. Based on the result, it retrieves the associated user-level data by calling the `get_user_meta` function or initializes a blank variable to store the user level. The syntax of `get_user_meta` follows the same structure as the `get_post_meta` and `get_term_meta` functions. More specifically, the first parameter is used to specify the user ID and the remaining fields indicate the field name to be retrieved and whether data should be returned as a single variable or an array:

```
get_user_meta( $user_id, $field_name, $single );
```

The rest of our callback function is responsible for rendering a basic HTML select list and selecting one of the items in the list based on the current user membership level.

The only additional element that we put in place in this recipe is the declaration of an array of user membership levels as a `global` variable. We reference this variable in multiple functions in this chapter, including when we're rendering the drop-down selection list in this recipe.

See also

- The *Storing stylesheet data in user settings* recipe in *Chapter 3*, *User Settings and Administration Pages*

Processing and storing custom user data

As user profiles are created or updated, WordPress offers an easy way to register a callback to save custom user data in the site's database.

Getting ready

You should have already followed the *Adding custom fields to the user editor* recipe to have a starting point for this recipe. The resulting plugin should still be active on your development site. Alternatively, you can get the resulting code directory (`ch8/ch2-private-item-text/*`) from the book's GitHub page and rename `ch2-private-item-text-v3.php` to `ch2-private-item-text.php`. All other `.php` files can be deleted to avoid confusion when activating the plugin.

How to do it...

Follow these steps to register an action hook to save custom user data:

1. Navigate to the `ch2-private-item-text` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch2-private-item-text.php` file in a text editor.
3. Add the following lines of code at the end of the file to register functions to be called when user data is stored upon initial creation or when a user is updated:

```
add_action( 'user_register',
            'ch2pit_save_user_data' );
add_action( 'profile_update',
            'ch2pit_save_user_data' );
```

4. Add the following block of code to provide an implementation for the `ch2pit_save_user_data` function:

```
function ch2pit_save_user_data( $user_id ) {
    global $user_levels;
    if ( isset( $_POST['user_level'] ) &&
            !empty( $_POST['user_level'] ) &&
            array_key_exists( $_POST['user_level'],
                $user_levels ) ) {
        update_user_meta( $user_id, 'user_level',
            sanitize_text_field(
                $_POST['user_level'] ) );
    } else {
        update_user_meta( $user_id, 'user_level',
                                'regular' );
    }
}
```

5. Save and close the plugin file.
6. Edit one of the existing user accounts on your site that is not your own or create a new one to see that the user-level value is saved.

How it works...

Similar to adding fields to the user editor, WordPress offers two different action hooks to register functions that are called when a user is initially registered (`user_register`) and when a user profile is updated (`profile_update`). As you can see, we are registering the same callback function in both cases. When called, our code checks to see whether we received a value indicating the desired user level and uses the `update_user_meta` function to save it. Similar to the post and term meta update functions, this function has three parameters. The first parameter indicates the user ID that the information should be associated with. The remaining parameters specify the name of the custom data field and the value to be stored:

```
update_user_meta( $user_id, $field_name, $single );
```

As an additional precaution, we check to see whether the value we received is present in our global array of user levels using the PHP `array_key_exists` function.

The `else` branch of the `if` statement is put in place to set a default user level if the field is absent, which can happen if the site allows visitors to register for accounts by themselves.

Of course, this current process of classifying users as regular users or paid users is very manual, requiring site administrators to log in to the site's dashboard and update their status individually. A more elaborate solution could be to set the value of this field based on receiving a payment to the site using an online payment service.

See also

- The *Adding custom fields to the user editor* recipe

Displaying new user data on the user list page

As new fields are added to user profiles, it can be very useful for site administrators to see this data in the site user list and be able to filter users to only see the ones that were assigned certain values. This recipe shows how to add extra columns to the user listing of the WordPress administration area and add a filter function to reduce the number of records shown.

Getting ready

You should have already followed the *Processing and storing custom user data* recipe to have a starting point for this recipe. The resulting plugin should still be active on your development site. Alternatively, you can get the resulting code directory (`ch8/ch2-private-item-text/*`) from the book's GitHub page and rename `ch2-private-item-text-v4.php` to `ch2-private-item-text.php`. All other `.php` files can be deleted to avoid confusion when activating the plugin.

How to do it...

Follow these steps to display custom user data when managing users:

1. Navigate to the `ch2-private-item-text` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch2-private-item-text.php` file in a text editor.
3. Add the following line of code at the end of the file to register a function to be called when WordPress is preparing the list of columns to be displayed on the user listing page:

```
add_filter( 'manage_users_columns',
            'ch2pit_add_user_columns' );
```

4. Add an implementation for the `ch2pit_add_user_columns` function with the following block of code:

```
function ch2pit_add_user_columns( $columns ) {
    $new_columns = array_slice( $columns, 0, 2, true )
        + array( 'level' => 'User Level' ) +
        array_slice( $columns, 2, NULL, true );
    return $new_columns;
}
```

5. Add the following line of code to assign a function to be called when WordPress prepares the content of each column as it displays a list of all the users:

```
add_filter( 'manage_users_custom_column',
    'ch2pit_display_user_columns_data', 10, 3 );
```

6. Provide an implementation for the `ch2pit_display_user_columns_data` function by inserting the following code section:

```
function ch2pit_display_user_columns_data( $val,
                                $column_name, $user_id ) {
    global $user_levels;
    $val = '';
    if ( 'level' == $column_name ) {
        $current_user_level =
            get_user_meta( $user_id, 'user_level',
                                true );
        if ( !empty( $current_user_level ) ) {
            $val = $user_levels[$current_user_level];
        }
    }
    return esc_html( $val );
}
```

7. Add the following code snippet to register a function to be called when WordPress prepares the list of quick actions that can be performed by administrators on the user listing page:

```
add_action( 'restrict_manage_users',
            'ch2pit_add_user_filter' );
```

8. Add this code section to provide a body for the `ch2pit_add_user_filter` function:

```
function ch2pit_add_user_filter() {
    global $user_levels;
    $filter_value = '';
    if ( isset( $_GET['user_level'] ) ) {
        $filter_value =
            sanitize_text_field(
                $_GET['user_level'] );
    } ?>
    <select name="user_level" class="user_level"
            style="float:none;">
    <option value="">No filter</option>
```

```
    <?php foreach( $user_levels as
            $user_level_idx => $user_level ) { ?>
        <option value="<?php echo $user_level_idx; ?>"
        <?php selected( $filter_value,
                        $user_level_idx ); ?>>
        <?php echo $user_level; ?></option>
    <?php } ?>
    <input type="submit" class="button"
           value="Filter">
<?php }
```

9. Add the following line of code to register a function to be called when WordPress is rendering the footer of the administration pages:

```
add_action( 'admin_footer', 'ch2pit_user_filter_js' );
```

10. Provide an implementation for the `ch2pit_user_filter_js` function by adding the following code section:

```
function ch2pit_user_filter_js() {
    global $current_screen;
    if ( 'users' != $current_screen->id ) {
        return;
    } ?>
    <script type="text/javascript">
        jQuery( document ).ready( function() {
            jQuery( '.user_level' ).first().change(
            function() {
                jQuery( '.user_level' ).
                  last().val( jQuery( this ).val() );
            });

            jQuery( '.user_level' ).last().change(
            function() {
                jQuery( '.user_level' ).
                  first().val( jQuery( this ).val() );
```

```
            });
        });
    </script>
<?php }
```

11. Add this line of code to register a function that will be executed when WordPress is preparing the query to retrieve the list of users from the database:

```
add_filter( 'pre_get_users', 'ch2pit_filter_users' );
```

12. Insert the following block of code to define the `ch2pit_filter_users` function:

```
function ch2pit_filter_users( $query ) {
    global $pagenow, $user_levels;

    if ( is_admin() && 'users.php' == $pagenow &&
            isset( $_GET['user_level'] ) ) {
        $filter_value =
            sanitize_text_field(
                $_GET['user_level'] );
        if ( !empty( $filter_value ) &&
                array_key_exists( $_GET['user_level'],
                                  $user_levels ) ) {
            $query->set( 'meta_key', 'user_level' );
            $query->set( 'meta_query', array(
                        array( 'key' => 'user_level',
                        'value' => $filter_value ) ) );
        }
    }
}
```

13. Save and close the plugin file.

14. Visit the **Users** section of the administration interface to see the new **User Level** column.

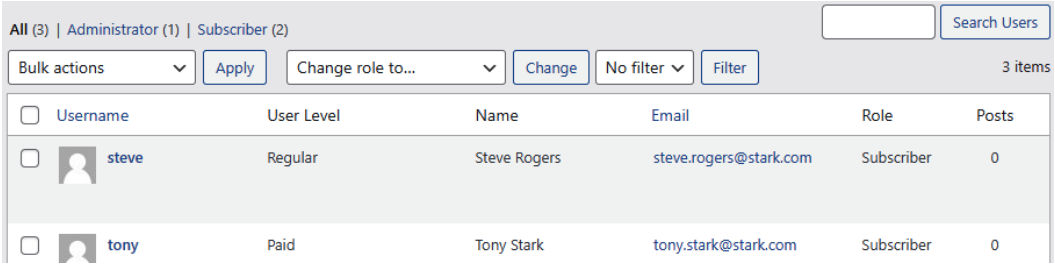


Figure 8.2 – Custom fields in the user management section

15. Use the filter drop-down list and click on **Filter** to limit the number of records displayed.

How it works...

The first code section in this recipe is a filter function that gets a list of all columns that the system intends on displaying in the **Users** section of the administration interface. Similar to what we did in the *Displaying additional columns on the custom post list page* recipe in *Chapter 4*, *The Power of Custom Post Types*, we use the PHP `array_slice` function to split the columns array into two sections. We then proceed to insert our new column name to display it as the second column in the table.

We continue this recipe by providing a function that displays the user level assigned to each user. This is done by simply retrieving the information based on the current user ID and returning the data to be displayed. It should be noted that the user management page takes an approach that is slightly different from the post and page custom column content. Instead of an action hook, it uses a filter function where we need to return data instead of directly outputting HTML code to the browser.

The next code section is responsible for rendering a drop-down list of user levels that can be used to filter the user list. This is done by using standard HTML and code to loop through the list of user levels and display them. That being said, the callback code we register to display the drop-down list is going to be called twice, before and after listing users. This results in two drop-down lists on the same page with no way to differentiate them. To avoid having issues with the two lists sending different values to our filtering code, the next function places a block of JavaScript code in the page footer of all user list pages. This code allows us to synchronize either drop-down list with the other's value when the user interacts with them.

Our last step consists of modifying the user query if we find a filtering argument in the page URL when displaying the user listing. If an argument is found and the page being displayed is the user list, we will check whether the value is part of our user levels array and then add it to the system's current query variables to filter the results.

See also

- The *Processing and storing custom user data* recipe

Using custom user data in an enclosing shortcode

Now that we have additional data stored in user profiles and have made it easy to view this information on the administration pages, we can finally complete our content subscription system. The only missing component to achieve our goal of restricting site content to paid users is to introduce a new enclosing shortcode that checks a user's membership level before displaying content.

Getting ready

You should have already followed the *Displaying new user data on the user list page* recipe to have a starting point for this recipe. The resulting plugin should still be active on your development site. Alternatively, you can get the resulting code directory (`ch8/ch2-private-item-text/*`) from the book's GitHub page and rename `ch2-private-item-text-v5.php` to `ch2-private-item-text.php`. All other `.php` files can be deleted to avoid confusion when activating the plugin.

How to do it...

Follow these steps to create a new enclosing shortcode that restricts content to paid users:

1. Navigate to the `ch2-private-item-text` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch2-private-item-text.php` file in a text editor.
3. Add the following line of code at the end of the file to define a new shortcode and define the function to be called when it is used:

```
add_shortcode( 'paid', 'ch2pit_paid_shortcode' );
```

4. Provide an implementation for `ch2pit_paid_shortcode` with the following code section:

```
function ch2pit_paid_shortcode( $atts,
                                $content = null ) {
    $blockoutput = '<div class="register">';
    $blockoutput .= 'You need to be a paid member to';
    $blockoutput .= ' access this content.</div>';

    if ( !is_user_logged_in() ) {
        return $blockoutput;
    }

    $current_user = wp_get_current_user();
    $current_user_level = get_user_meta(
        $current_user->ID, 'user_level', true );
    if ( 'paid' == $current_user_level ||
            current_user_can( 'activate_plugins' ) ) {
        return '<div class="paid">' . $content .
            '</div>';
    } else {
        return $blockoutput;
    }
}
```

5. Save and close the plugin file.
6. Create a new post and wrap some or all of the content with the `[paid]` and `[/paid]` tags:

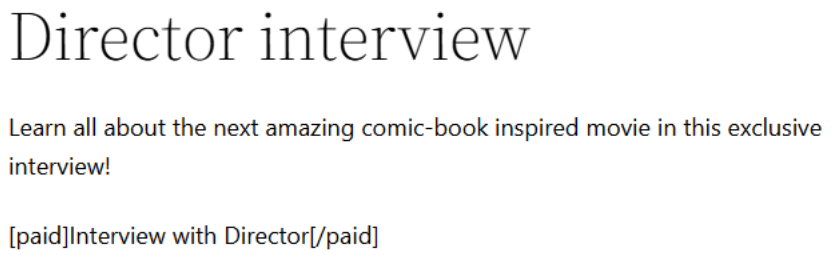


Figure 8.3 – Shortcode to restrict content to paid users

7. View the page as an administrator, a visitor who is not logged in, a registered regular user, and a registered paid user to validate that the content is only displayed in the first and last cases.

How it works...

Similar to the `[private]` shortcode that we created back in the *Creating a new enclosing shortcode* recipe in *Chapter 2*, *Plugin Framework Basics*, the new `[paid]` shortcode can be used by content creators to identify content that can only be viewed by paid members.

To do this, our shortcode implementation first checks whether the visitor is logged in. If that is the case, it further validates the user by asking WordPress to provide data on the current user with the `wp_get_current_user` function. Once it has the current user ID, our recipe code uses it to retrieve the `user_level` metadata that we stored in the previous recipes in this chapter. Alternatively, it checks whether the user is a site administrator by using the `current_user_can` WordPress function and specifying `'activate_plugins'` as the argument, since only administrators can perform this action. If either of these conditions is met, the enclosed post content will be displayed. Otherwise, a message will be shown to indicate that users must be paid members to see this content.

See also

- The *Displaying new user data on the user list page* recipe

9

Leveraging JavaScript, jQuery, and AJAX Scripts

JavaScript libraries, especially the very popular jQuery library along with its numerous plugins, can do wonders in bringing a website to life with slick animations, dynamic data queries, and advanced graphical features. Unfortunately, for all of their benefits, these scripts can also be difficult to work with. For example, loading more than one copy of jQuery can destroy everything that was set up by another instance, and errors in one script usually prevent other scripts from running correctly.

WordPress' answer to this convoluted architecture is twofold. As a first step, it comes pre-packaged with a copy of jQuery and many other popular JavaScript libraries that plugin developers can use without having to load their own versions. Then, to prevent multiple copies from being loaded on a page, it offers easy-to-use functions that queue up scripts and styles to identify duplicates before rendering pages.

This chapter shows how to safely load JavaScript and jQuery files that are provided with WordPress or that come from external sources. Using these libraries, developers can add powerful new functionalities to front-facing pages and plugin configuration panels. We'll also see how to securely run AJAX queries to refresh partial page sections.

This chapter focuses on incorporating JavaScript in plugins by exploring the following recipes:

- Safely loading jQuery onto WordPress web pages
- Displaying a pop-up dialog using the built-in ThickBox plugin
- Controlling pop-up dialog display using shortcodes
- Displaying a calendar day selector using the Datepicker plugin
- Adding tooltips to admin page form fields using the TipTip plugin
- Using AJAX to dynamically update partial page contents

Technical requirements

You can find complete code examples on GitHub in the following repository: `https://github.com/PacktPublishing/WordPress-Plugin-Development-Cookbook-Third-Edition`.

Safely loading jQuery onto WordPress web pages

When working on a project that would benefit from the jQuery library, it might be tempting to provide your own copy of jQuery or to access a copy from the Google API website. However, there is no need to do this as WordPress provides a copy of jQuery as part of its installation and makes it very easy to load.

By using the appropriate utility function to load jQuery, developers make a request to WordPress to load this library instead of doing it themselves. Once all the requests have been received, they are analyzed for duplicates. Then, a single instance of each script is loaded to reduce the chance of conflicts between multiple copies of the same library.

Getting ready

You should have access to a WordPress development environment.

How to do it...

Follow these steps to load the jQuery library for use on front-facing website pages:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch9-load-jquery`.
3. Navigate to the directory and create a text file called `ch9-load-jquery.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 9 - Load jQuery`.
5. Add the following line of code to register a function to be called when script loading requests are processed:

```
add_action( 'wp_enqueue_scripts',
            'ch9lj_front_facing_pages' );
```

6. Add the following code segment to provide an implementation for the `ch9lj_front_facing_pages` function:

```
function ch9lj_front_facing_pages() {
    wp_enqueue_script( 'jquery' );
}
```

7. Save and close the plugin file.
8. Go to the **Themes** menu section located under **Appearance** in the WordPress administration interface.
9. Activate the **Twenty Twenty-Two** theme if it is not already active or install it if you don't have a copy of it in your installation.
10. Visit your website and view the page source code, searching for instances of the `jquery.js` library. Your search should come up empty unless you have activated other plugins that are already requesting for jQuery to be loaded.
11. Navigate to the **Plugins** management page and **Activate** the **Chapter 9 - Load jQuery** plugin.
12. Go back to any page on your website and view the page source once again.
13. Search for the keyword `jquery` within the page source code:

```
<script src='http://plugin-development-site.local/
wp-includes/js/jquery/jquery.min.js?ver=3.6.0'
id='jquery-core-js'></script>
```

```
<script src='http://plugin-development-site.local/
wp-includes/js/jquery/jquery-migrate.min.js?ver=3.3.2'
id='jquery-migrate-js'></script>
```

As we can see, a copy of the `jquery.min.js` script is now loaded from the WordPress `wp-includes` folder, along with the `jquery-migrate.min.js` script for backward compatibility.

> **Note**
>
> Note that the website address (`plugin-development-site.local`) and the exact version numbers may differ based on your version of WordPress and development environment setup.

How it works...

The key component of this recipe is the `wp_enqueue_script` function. This function allows developers to load their own JavaScript files or to ask WordPress to load one of the scripts that it comes packaged with. While the function requires many arguments when loading your own scripts, which we'll cover in a later recipe in this chapter, it only needs a single argument to load built-in scripts. In this example, that argument is `jquery`. To get a full list of default scripts available with WordPress, check out the code reference page for the function (`https://developer.wordpress.org/reference/functions/wp_enqueue_script/`).

Once you know which script to load, the call to `wp_enqueue_script` should be made from one of three action hooks, depending on the target page(s) where the script should be loaded. These are `wp_enqueue_scripts` for front-facing pages, `admin_enqueue_scripts` for administration pages, and `login_enqueue_scripts` for the login page, with the first one fulfilling our requirement for this recipe.

> **Note**
>
> We made sure to activate a specific theme that does not load jQuery. Many modern themes already make a request for the popular library to be loaded to animate menus or provide other interactive functionalities. While users of your plugins may use a theme that already loads jQuery, the script enqueuing function will make sure that only a single copy is loaded.

There's more...

Veteran jQuery developers should be aware that the copy delivered with WordPress has a small caveat.

jQuery no-conflict mode

To avoid internal conflicts with other JavaScript and jQuery libraries, the version of jQuery that comes bundled with WordPress is configured in **no-conflict** mode. This means that the `$` shortcut that can normally be used to access jQuery will not be available. As such, all the examples found in this chapter spell out the `jQuery` keyword.

If you are in a situation where you need to regain access to this shortcut, you can use the following syntax in your code:

```
jQuery( document ).ready( function($) {
    // $ shortcut is now available for this function
} );
```

Displaying a pop-up dialog using the built-in ThickBox plugin

As annoying as they can be to visitors, pop-up dialogs are a feature that many website administrators use to help them advertise special offers or get readers to subscribe to their content. Since it uses pop-up dialogs in its own administrative pages, WordPress comes bundled with a jQuery script called ThickBox that can be used to display these types of dialogs.

Getting ready

You should have access to a WordPress development environment.

How to do it...

Follow these steps to load the ThickBox script and use it to render a pop-up dialog:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch9-pop-up-dialog`.
3. Navigate to the directory and create a text file called `ch9-pop-up-dialog.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 9 - Pop-Up Dialog`.

5. Add the following line of code to register a function to be called when script loading requests are made:

```
add_action( 'wp_enqueue_scripts',
            'ch9pud_load_scripts' );
```

6. Add the following code segment to provide an implementation for the `ch9pud_load_scripts` function:

```
function ch9pud_load_scripts() {
    wp_enqueue_script( 'jquery' );
    add_thickbox();
}
```

7. Insert the following line of code to register a function to display content in the page footer:

```
add_action( 'wp_footer', 'ch9pud_footer_code' );
```

8. Append the following block of code to provide an implementation for the `ch9pud_footer_code` function:

```
function ch9pud_footer_code() { ?>
    <script type="text/javascript">
    jQuery( document ).ready( function() {
        setTimeout( function() {
            tb_show( 'Pop-Up Message',
            '<?php echo plugins_url(
            'content.html?width=420&height=220',
            __FILE__ ); ?>', null );
        }, 2000 );
    } );
</script>
<?php
}
```

9. Save and close the plugin file.
10. Create a new HTML file in the `ch9-pop-up-dialog` directory named `content.html` and open it in a code editor.

11. Insert the following HTML code as the file's content:

```
<!DOCTYPE html>
<html>
    <body>
        <div>This is the pop-up content.</div>
    </body>
</html>
```

12. Save and close the HTML file.
13. Navigate to the **Plugins** management page and **Activate** the **Chapter 9 - Pop-Up Dialog** plugin.
14. Visit any page of the website:

Figure 9.1 – Custom JavaScript pop-up dialog

As seen in *Figure 9.1*, a new pop-up dialog appears 2 seconds after the whole page is displayed.

How it works...

Using a similar technique to the previous recipe, we start by assigning a function to the `wp_enqueue_scripts` action hook. When executed, the callback makes a call to `wp_enqueue_script` to request for jQuery to be loaded from the local copy of WordPress. It also calls the `add_thickbox` function, a utility function that makes multiple calls to `wp_enqueue_script` and `wp_enqueue_style` to load the appropriate JavaScript and style sheet in the page header.

Once all the required elements are loaded, the next section of the recipe outputs a block of JavaScript code to the page footer. This code uses the jQuery `ready` method to register a function that will be called when the entire page is loaded. When this happens, the `setTimeout` JavaScript function is used to register a function that will be called `2000` milliseconds later and takes care of calling `tb_show` to display the pop-up dialog. `tb_show` has three arguments, with the first indicating the dialog title, the second containing the address of the content to render within the box, and the third expecting a path to a group of images to be displayed. In our case, that last argument is left as null. Notice that the width and height (in pixels) of the dialog are indicated in the address of the page to be displayed and could be made into options with a plugin configuration page.

There's more...

While the recipe displays a valid dialog, developers might want a bit more control over how it can be closed and when it gets displayed.

Removing the dialog close button

By default, the ThickBox script offers a close button in the top-right corner of the pop-up dialog that can be used to close it at any time. This may not be desirable if you expect visitors to provide feedback or perform a specific action before dismissing the dialog. By adding the `modal` keyword to the content URL—set to the value of `true`—ThickBox will remove the dialog title bar, including the close button:

```
tb_show( 'Pop-Up Message', '<?php echo plugins_url(
    'content.html?width=420&height=220&modal=true',
    __FILE__ ); ?>', null );
```

Once the close button is gone, we can call the `tb_remove` JavaScript function to close the dialog. The following is an example of a simple button that could be added in `content.html` to close the dialog, after the current content `div` section:

```
<div>
    <button onclick="tb_remove();">Close button</button>
</div>
```

Displaying pop-up dialogs on select pages

While the recipe's original code displays a pop-up dialog on every single page of a website, it may be better to show it only on specific pages, such as the front page, to avoid over-exposure. To accomplish this, we can move the two `add_action` calls inside of an action hook callback and check whether the visitor is making a request to see the front page before loading our scripts:

```
add_action( 'template_redirect',
            'ch9pud_template_redirect' );

function ch9pud_template_redirect() {
    if ( is_front_page() ) {
        add_action( 'wp_enqueue_scripts',
                    'ch9pud_load_scripts' );
        add_action( 'wp_footer', 'ch9pud_footer_code' );
    }
}
```

A similar technique can be used by substituting the `is_front_page` function with the `is_page( 'id_title_or_slug' )` function, which checks whether the current page numeric ID, title, or post slug matches the value that it receives as an argument. In that situation, a plugin configuration page could allow users to easily select one or more pages on which the dialog should appear.

Controlling pop-up dialog display using shortcodes

As you may be aware, loading scripts and styles on a page where they won't be used unnecessarily slows down that page's loading time. This happens since the browser still needs to download, validate, and execute the content of these external files. The previous recipe's *There's more...* section offered one technique to select specific pages where scripts and styles should be loaded. However, a different approach is to analyze the page contents for the presence of a special code to make that decision.

This recipe shows how to add a filter to the previous recipe to search for a shortcode in posts and pages to decide when to display a pop-up dialog.

Getting ready

You should have already followed the *Displaying a pop-up dialog using the built-in ThickBox plugin* recipe to have a starting point for this recipe. Alternatively, you can get the resulting code (`ch9/ch9-pop-up-dialog/ch9-pop-up-dialog-v1.php`) from the book's GitHub page and rename the file as `ch9-pop-up-dialog.php`. If you implemented the `template_redirect` method shown in the *There's more…* section of that recipe, you should revert the code to the original recipe code. If you do not do this, you will not be able to see the popup on other pages even if they contain the `[popup]` shortcode.

How to do it…

Follow these steps to add logic to trigger the display of a pop-up dialog on pages containing a custom shortcode:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Navigate to the `ch9-pop-up-dialog` directory and then edit `ch9-pop-up-dialog.php`.
3. Add the following lines of code at the top of the file, just under the plugin header:

```
global $load_scripts;
$load_scripts = false;
```

4. Find the `ch9pud_load_scripts` function and add the following highlighted lines of code:

```
function ch9pud_load_scripts() {
    global $load_scripts;
    if ( $load_scripts ) {
        wp_enqueue_script( 'jquery' );
        add_thickbox();
    }
}
```

5. Locate the `ch9pud_footer_code` function and modify the code, adding the following highlighted lines of code to the function body between the existing function opening parenthesis ({) and the closing PHP statement (?>) and just before the function's closing parenthesis (}):

```
function ch9pud_footer_code() {
    global $load_scripts;
    if ( $load_scripts ) { ?>
<script type="text/javascript">
    jQuery( document ).ready( function() {
        setTimeout( function(){
            tb_show( 'Pop-Up Message',
            '<?php echo plugins_url(
            'content.html?width=420&height=220',
            __FILE__ ); ?>', null );
        }, 2000 );
    });
</script>
<?php }
}
```

6. Add the following lines of code to register a function that will filter post and page contents before any other parsing and formatting is performed:

```
add_filter( 'the_posts',
    'ch9pud_conditionally_add_scripts_and_styles' );
```

7. Append the following block of code to provide an implementation for the `ch9pud_conditionally_add_scripts_and_styles` function:

```
function ch9pud_conditionally_add_scripts_and_styles(
                                            $posts ) {
    if ( empty( $posts ) ) {
        return $posts;
    }

    global $load_scripts;
    foreach ( $posts as $post ) {
```

```
        if ( has_shortcode( $post->post_content,
                                'popup' ) ) {
            $load_scripts = true;
            return $posts;
        }
    }
    return $posts;
}
```

8. Insert the following function call to declare a new shortcode along with a function responsible for replacing it with content:

```
add_shortcode( 'popup', 'ch9pud_popup_shortcode' );
```

9. Add the following code block to provide a simple implementation for the `ch9pud_popup_shortcode` function:

```
function ch9pud_popup_shortcode() {
    return;
}
```

10. Save and close the plugin file.
11. Visit the website's front page and you will notice that the pop-up dialog is no longer displayed.
12. Create a new page and insert the `[popup]` shortcode in the page contents.
13. View the new page to see that the new pop-up dialog appears, while the `[popup]` shortcode is not shown as part of the page text.

How it works...

Our recipe code first creates a `global` variable that will be used to determine whether or not they should load scripts and output code to the page footer. It then moves on to modify our two existing action hook callbacks so they query this global variable to decide whether they should perform their actions. With this in place, the bulk of the work is actually done by the filter function associated with the `the_posts` action hook. This function receives an array of all posts and pages to be displayed and must determine whether a special keyword is present to set the `load_scripts` variable appropriately.

As you can see from the recipe's code, the text that we chose to look for, `[popup]`, is a shortcode. While we could have selected any text as the trigger, a shortcode is easy to hide by providing a simple rendering function that returns no content.

See also

- The *Displaying a pop-up dialog using the built-in ThickBox plugin* recipe

Displaying a calendar day selector using the Datepicker plugin

As you develop configuration interfaces for your plugins or meta boxes to associate data to custom post types, you may need users to enter dates in text fields. Instead of only relying on the user to correctly enter date information using simple text, a much more reliable data entry method is to use a pop-up calendar. This type of tool allows users to navigate through visual representations of each month and pick the desired date.

Getting ready

You should have access to a WordPress development environment.

How to do it...

Follow these steps to use the jQuery Datepicker script that is provided by default with WordPress to display a pop-up calendar to provide an easy way to select dates:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch9-calendar-picker` along with a subdirectory called `css`.
3. Visit `https://www.jqueryui.com/download` and download the latest **Stable** package from the **Quick Downloads** section, located just under the **Download Builder** header.
4. Open the resulting file with an archive management tool and extract the file `jquery-ui.min.css` to the `css` folder of the newly created plugin directory.
5. Create a text file called `ch9-calendar-picker.php` in the plugin directory.
6. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 9 - Calendar Picker`.
7. Add the following line of code to register a function to be called when script loading requests are made:

```
add_action( 'admin_enqueue_scripts',
            'ch9cp_admin_scripts' );
```

8. Add the following code segment to provide an implementation for the `ch9cp_admin_scripts` function:

```
function ch9cp_admin_scripts() {
    $screen = get_current_screen();
    if ( 'post' == $screen->base &&
            'post' == $screen->post_type ) {
        wp_enqueue_script( 'jquery' );
        wp_enqueue_script( 'jquery-ui-core' );
        wp_enqueue_script( 'jquery-ui-datepicker' );
        wp_enqueue_style( 'datepickercss',
            plugins_url( 'css/jquery-ui.min.css',
                __FILE__ ), array(), '1.12.1' );
    }
}
```

9. Insert the following line of code to register a function to be called when meta boxes are created:

```
add_action( 'add_meta_boxes',
            'ch9cp_register_meta_box' );
```

10. Append the following block of code to provide an implementation for the `ch9cp_register_meta_box` function:

```
function ch9cp_register_meta_box() {
    add_meta_box( 'ch9cp_datepicker_box',
                  'Assign Date',
                  'ch9cp_date_meta_box', 'post',
                  'normal');
}
```

11. Insert the following code block to implement the `ch9cp_date_meta_box` function that was declared in the call to `add_meta_box`:

```
function ch9cp_date_meta_box( $post ) { ?>
    <input type="text" id="ch9cp_date"
           name="ch9cp_date" />
    <script type='text/javascript'>
        jQuery( document ).ready( function() {
```

```
            jQuery( '#ch9cp_date' ).datepicker(
                { minDate: '+0',
                dateFormat: 'yy-mm-dd', showOn: 'both',
                constrainInput: true} );
            } );
        </script>
    <?php }
```

12. Save and close the plugin file.
13. Navigate to the **Plugins** management page and **Activate** the **Chapter 9 - Calendar Picker plugin**.
14. Select any item in the **Posts** management section and edit it to see the new **Assign Date** meta box.
15. Click on the **...** button or click in the **Assign Date** textbox:

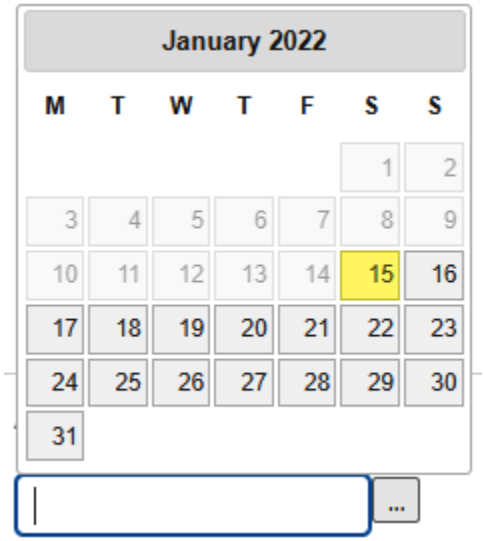


Figure 9.2 – Datepicker calendar date selector

As seen in *Figure 9.2*, a pop-up calendar appears to allow the user to select a date.

How it works...

Just like we saw with jQuery and ThickBox in the previous recipes, WordPress comes bundled with many jQuery libraries. Two of these libraries, jQuery UI and jQuery UI Datepicker, can be used to display a pop-up calendar and associate it with a text field on a form. That being said, the distribution of these scripts is missing the associated style sheet and images that are required to display a fully rendered calendar.

This recipe starts by visiting the jQuery UI website and downloading a copy of the complete jQuery UI library. Once the download is complete, we are only interested in getting a copy of the style sheet data. All other necessary scripts are provided by WordPress. After registering a function with `admin_enqueue_scripts`, we make three function calls to load the required JavaScript files in the admin page header. We also make a call to load the style sheet that we just downloaded. When extracting the style sheet from the downloaded archive, we extract the minified version of the CSS file to have the smallest version available.

The `wp_enqueue_style` function has many parameters. In this example, we are providing values for the first four of them to indicate the name of the style, the path to the style sheet file, an empty list of dependencies, and a version number. This function also has a fifth parameter, which we are not using here. That parameter indicates whether the script should be loaded in the header or footer, where the default is the header.

Once all of the required scripts are in place, the remainder of the code creates a meta box in the post editor and displays a text field in that box. It also outputs JavaScript code that will be called when the page is completely rendered to associate the pop-up calendar with the text field. We specify that the user will only be able to select future dates as part of the calendar's options with the `minDate` parameter, along with the desired date format. We are not saving any data entered in this field in this recipe at this time.

Adding tooltips to admin page form fields using the TipTip plugin

Documentation is a very important step of plugin development. It allows users to understand how to configure the plugins you create. That being said, users will not typically go very far to find the information they need, resulting in many unnecessary questions in discussion forums or emails.

As discussed in *Chapter 3*, *User Settings and Administration Pages*, one way to provide documentation is to create a **Help** tab that appears in the top-right corner of the plugin's configuration panel. While that approach is much easier for users than to find a README file or visit the official WordPress plugin repository, it still requires them to actively seek and click a link to open that section. That's where tooltips come into play. Using a jQuery plugin to render clean, good-looking tooltips, we can add documentation to a plugin that appears contextually based on the currently selected field.

Getting ready

You should have already followed the *Displaying a calendar day selector using the Datepicker plugin* recipe to have a starting point for this recipe. Alternatively, you can get the resulting code files (`ch9/ch9-calendar-picker/*`) from the book's GitHub page and rename `ch9-calendar-picker-v1.php` to `ch9-calendar-picker.php`.

How to do it...

Follow these steps to download and integrate the **TipTip jQuery** library to display tooltips when configuration fields are used:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Navigate to the `ch9-calendar-picker` directory.
3. Create a new subdirectory called `tiptip`.
4. Visit the **TipTip jQuery** home page, available at `https://drew.tenderapp.com/kb/tiptip-jquery-plugin/tiptip-downloads`.
5. Download version 1.3 of the plugin source code to your local computer.
6. Open the resulting file with an archive management tool and extract the `jquery.tipTip.minified.js` and `tipTip.css` files to the `tiptip` subdirectory.
7. Open the main plugin file, `ch9-calendar-picker.php`, in a code editor.
8. Find the `ch9cp_admin_scripts` function and add the following lines of code within the existing `if` condition and after the existing enqueuing calls, so that the new scripts are only loaded on the post editor, like the others:

```
wp_enqueue_script( 'tiptipjs',
    plugins_url( 'tiptip/jquery.tipTip.minified.js',
        __FILE__ ), array(), '1.3' );
wp_enqueue_style( 'tiptipcss',
    plugins_url( 'tiptip/tipTip.css', __FILE__ ),
        array(), '1.3' );
```

9. Locate the `ch9cp_date_meta_box` function and modify the line that renders the textbox, as shown in the following highlighted code, to add a class and title:

```
<input type="text" id="ch9cp_date"
       class="ch9cp_tooltip"
       title="Please enter a date"
       name="ch9cp_date" />
```

10. Again, in the `ch9cp_date_meta_box` function, add the following highlighted block of code to the existing block of JavaScript code:

```
<script type='text/javascript'>
    jQuery( document ).ready( function() {
        jQuery( '#ch9cp_date' ).datepicker(
            { minDate: '+0',
            dateFormat: 'yy-mm-dd', showOn: 'both',
                constrainInput: true } );
        jQuery( '.ch9cp_tooltip' ).each( function() {
            jQuery( this ).tipTip();
        } );
    });
</script>
```

11. Save and close the plugin file.
12. Select any item in the **Posts** management section and edit it.
13. Move the mouse over the **Assign Date** field.

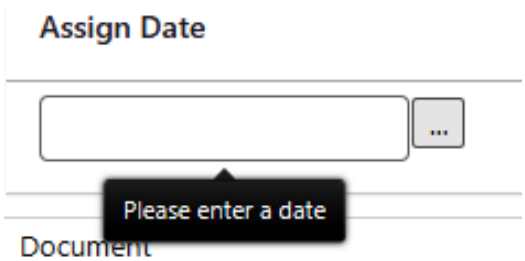


Figure 9.3 – Admin page tooltip

As seen in *Figure 9.3*, a tooltip now appears with the text assigned to the `title` tag of the `input` field.

How it works...

The **TipTip** library turns regular HTML `title` tags into nice-looking tooltips that appear when users position their mouse cursor over an item or interact with it.

This recipe starts by downloading the **TipTip** script from the plugin author's website. After downloading the library, we only extract two of the three files that the archive contains. The first is a minified version of the library, while the second is the library's style sheet.

Once we have the desired files in place, we load them in the admin page header by adding calls to the `wp_enqueue_script` and `wp_enqueue_style` functions in the callback that was already associated with the `admin_enqueue_script` action hook. Similar to `wp_enqueue_style`, the `wp_enqueue_script` function has five parameters. These indicate the name of the script, the location of the script file, a list of any dependencies for the script, a version number, and an option to indicate whether the script should be loaded in the site header or footer.

Once the library is loaded, activating the tooltips is quite simple. We first select a class name for our items (`ch9cp_tooltip`) and add it to all the items that are destined to have help text associated with them. We then add our help text in a `title` tag on each item. Note that the items in question could be anything from a `div` to a form `input` component or a table row. Finally, we make a call to the jQuery `each` function to find all the items that have the right class and execute the `tipTip` function on them. After execution, all the selected items will have their title text appear as tooltips.

See also

- The *Displaying a calendar day selector using the Datepicker plugin* recipe

Using AJAX to dynamically update partial page contents

When WordPress users create complex sites with lots of dynamic content, such as Twitter widgets or other third-party embedded components, navigation can become a grueling experience for visitors. This happens since the browser needs to refresh all page elements and fetch external data every time a user interacts with the website.

In such situations, using **Asynchronous JavaScript and XML** (**AJAX**) can greatly accelerate user navigation by displaying subsets of data and dynamically retrieving updates for isolated sections on request. More specifically, AJAX allows the browser to send requests to a web server and insert an HTML content response back in the web page, replacing or augmenting the original content.

Getting ready

You should have already followed the *Displaying custom post type data in shortcodes* recipe in *Chapter 4*, *The Power of Custom Post Types*, to have a starting point for this recipe. Alternatively, you can get the resulting code (`ch4/ch4-book-reviews/ch4-book-reviews-v5.php`) from the book's GitHub page and rename the file as `ch4-book-reviews.php`. Any subsequent version of the plugin's main code file will also work to perform this recipe.

How to do it...

Follow these steps to add AJAX support to the book review list that we created back in the *Displaying custom post type data in shortcodes* recipe in *Chapter 4*, *The Power of Custom Post Types*:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Navigate to the `ch4-book-reviews` directory and edit `ch4-book-reviews.php`.
3. Go to the bottom of the file and add the following line of code to register a function to be called when WordPress resolves the scripts to be loaded on visitor-facing pages:

```
add_action( 'wp_enqueue_scripts',
            'ch4_br_load_jquery' );
```

4. Add the following block of code to provide an implementation for the `ch4_br_load_jquery` function:

```
function ch4_br_load_jquery() {
    wp_enqueue_script( 'jquery' );
}
```

5. Locate the `add_shortcode` function call and insert the next two lines of code on the following lines:

```
add_action( 'wp_ajax_ch4_br_ajax',
            'ch4_br_book_review_list' );
add_action( 'wp_ajax_nopriv_ch4_br_ajax',
            'ch4_br_book_review_list' );
```

6. Find the `ch4_br_book_review_list` function implementation in the plugin's code.
7. Replace the `ch4_br_book_review_list` function with the following code to redefine it:

```
function ch4_br_book_review_list() {
    global $paged;
    $output = '<style>';
    $output .= '.nav-previous, .nav-next ';
    $output .= '{ cursor: pointer; ';
    $output .= 'text-decoration: none; }';
    $output .= '</style>';

    if ( isset( $_POST['action'] ) &&
                'ch4_br_ajax' == $_POST['action'] ) {
        check_ajax_referer( 'ch4_br_ajax' );
    }

    $query_params = array( 'post_type' =>
                               'book_reviews',
                           'post_status' => 'publish',
                           'posts_per_page' => 5 );

    $page_num = 1;
    if ( isset( $_POST['page_number'] ) &&
                !empty( $_POST['page_number'] ) ) {
        $page_num = intval( $_POST['page_number'] );
        $paged = $page_num;
    } elseif ( !empty( get_query_var( 'paged' ) ) ) {
        $page_num = intval(
```

```
                            get_query_var( 'paged' ) );
        } else {
            $output .= '<script type="text/javascript">';
            $output .= 'var page_num = ' . $page_num;
            $output .= ';</script>';
        }

        $query_params['paged'] = $page_num;

    }
```

8. Continue the implementation of the `ch4_br_book_review_list` function by inserting the following block of code within the function body, just before the closing bracket (}):

```
    $book_review_query = new WP_Query( $query_params );
    if ( $book_review_query->have_posts() ) {

    }
    wp_reset_postdata();

    if ( isset( $_POST['action'] ) &&
                'ch4_br_ajax' == $_POST['action'] ) {
        echo $output;
        die();
    } else {
        return $output;
    }
```

9. Insert the following block of code in the empty body of the `if ( $book_review_query->have_posts() ) {` statement to complete the implementation of the `ch4_br_book_review_list` function:

```
    $output .= '<div id="book_review_table">';
    $output .= '<table>';
    $output .= '<tr><th style="text-align:left;">';
    $output .= '<strong>Title</strong></th>';
    $output .= '<th style="text-align:left;">';
    $output .= '<strong>Author</strong></th></tr>';
```

```
while ( $book_review_query->have_posts() ) {
    $book_review_query->the_post();
    $output .= '<tr><td style="padding-right: 20px">';
    $output .= '<a href="' . get_permalink();
    $output .= '">' . get_the_title( get_the_ID() );
    $output .= '</a></td><td>';
    $output .= esc_html( get_post_meta( get_the_ID(),
                                'book_author', true ) );
    $output .= '</td></tr>';
}
$output .= '</table>';

if ( $book_review_query->max_num_pages > 1 ) {
    $output .= '<nav id="nav-below">';
    $output .= '<div class="nav-previous">';
    $output .= get_next_posts_link (
        '<span class="meta-nav">&larr;</span>' .
        ' Older reviews',
        $book_review_query->max_num_pages );
    $output .= '</div><div class="nav-next">';
    $output .= get_previous_posts_link(
        'Newer reviews ' .
        '<span class="meta-nav">&rarr;</span>',
        $book_review_query->max_num_pages );
    $output .= '</div></nav>';
}

$output .= '</div>';
```

10. Add the following line of code at the end of the plugin file to register a callback function to be executed when rendering the page footer:

```
add_action( 'wp_footer', 'ch4_br_footer_scripts' );
```

11. Insert the following block of code to provide an implementation for the `ch4_br_footer_scripts` function:

```
function ch4_br_footer_scripts() {
    $nonce = wp_create_nonce( 'ch4_br_ajax' ); ?>
<script type="text/javascript">
    function remove_links_register_callbacks() {
        jQuery( "#book_review_table .nav-previous > a"
            ).removeAttr( "href" );
        jQuery( "#book_review_table .nav-next > a"
            ).removeAttr( "href" );
        jQuery( ".nav-previous" ).click( function() {
            page_num++;
            replacecontent( page_num );
        } );
        jQuery( ".nav-next" ).click( function() {
            page_num--;
            replacecontent( page_num );
        } );
    }

    </script>
<?php }
```

12. Insert the following block of code in the `ch4_br_footer_scripts` function body, before the closing `</script>` bracket:

```
function replacecontent( page_num ) { jQuery.ajax( {
    type: "POST",
    url:
      "<?php echo admin_url( 'admin-ajax.php' ); ?>",
    data: { action: "ch4_br_ajax",
            _ajax_nonce: «<?php echo $nonce; ?>»,
            page_number: page_num },
    success: function( data ) {
         jQuery( "#book_review_table" ).html( data );
         remove_links_register_callbacks();
    }
```

```
})}

jQuery( document ).ready( function() {
    remove_links_register_callbacks();
});
```

13. Visit the page you previously created that contains the `[book-review-list]` shortcode or create a new one to see the book review list:

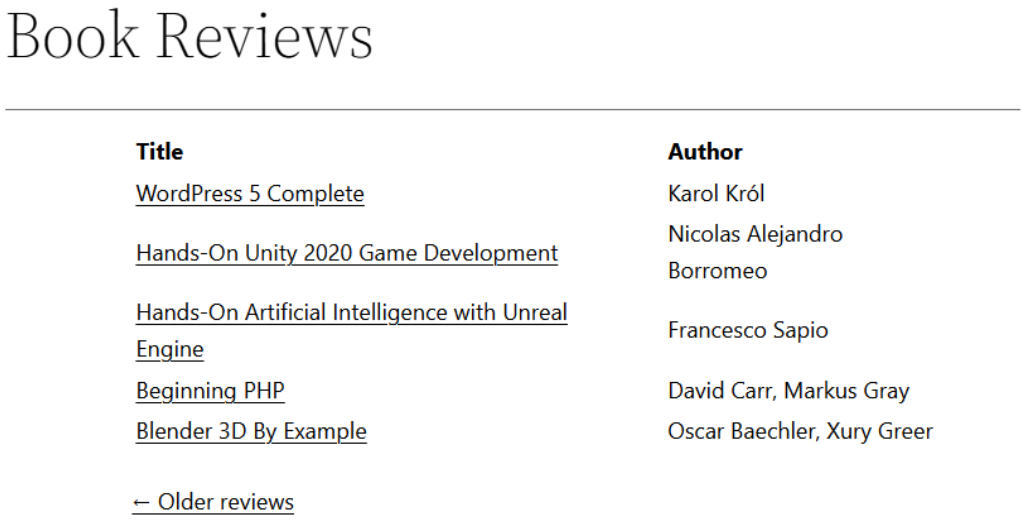


Figure 9.4 – Book review list

14. Click on the **Older reviews** link to see older items. Notice that the page updates without causing the browser to refresh the entire page. If you hover your mouse over the link, it does not show any URL assigned to it in the bottom-left corner of your browser.

How it works...

AJAX page interactions are powered by JavaScript code and allow users to create pages with content that gets dynamically updated. To add this functionality to our book review list, the very first thing we do is to ask WordPress to queue the jQuery library as part of visitor-facing page output. We've already learned how to do this in the *Safely loading jQuery onto WordPress web pages* recipe. This basic task will allow us to call jQuery functions in the rest of our recipe.

Once this is done, we move on to replace the implementation of the function that is responsible for displaying our book review list, `ch4_br_book_review_list`. While large parts of the function are identical to the way they were in the *Displaying custom post type data in shortcodes* recipe in *Chapter 4*, *The Power of Custom Post Types*, you'll see that we have made several changes throughout its code. The first of these changes is that we are now gaining access to a global variable called `$paged`. We also initialize the value of the shortcode output with an empty string at the very beginning of the function. We've then added a call to the `check_ajax_referer` function, which is essentially an AJAX version of `check_admin_referer`. Its purpose is to check for the presence of a security `nonce` value in the AJAX requests that are received. You will notice that this call is wrapped in a conditional statement that checks if the `$_POST['action']` variable exists and if its value is set to `ch4_br_ajax`. This allows us to only check for the presence of a nonce when we are performing a dynamic content update. The check won't be done the first time that the shortcode content is displayed.

We made other modifications to the `ch4_br_book_review_list` function. The first is to check for the presence of a current page number using one of two methods depending on how the content is rendered. The other change is to output a small chunk of JavaScript code containing the initial page value in the case of a shortcode content display. We also wrapped the table of book review titles in an HTML `div` tag. This allows us to select this section with a jQuery selector to attach callbacks or to replace its content when doing a dynamic page update.

The next large addition is a block of JavaScript code that is inserted in the page footer. It contains the definition of two JavaScript functions, `replacecontent` and `remove_links_register_callback`. It also makes a call to the `remove_links_register_callback` function when the page has been completely loaded. This JavaScript function has two main purposes. It first removes the `href` tags from the **Older reviews** and **Newer reviews** links that are displayed along with the list of book reviews. This is done in order to avoid going to different pages when these items are clicked. The function also registers callbacks to be called when the user clicks on either of the navigation links (`.nav-previous` and `.nav-next`). In either case, our code increments or decrements the page number to be displayed. It then calls our second custom JavaScript function, `replacecontent`, with the calculated page number.

The `replacecontent` function contains a call to a single function, the jQuery `ajax` function, to perform the dynamic content update. This function takes a number of arguments, starting with the type of operation, which is set to `POST`. This indicates that all of the variables sent in the request URL will be stored in a standard `$_POST` variable array. The second parameter is the URL to which the request should be sent, which we set to the WordPress admin URL for AJAX requests. Next up is a `data` array that contains a number of data elements, such as the name of the `action`, a `nonce` field to secure the request, and the `page_number` that should be displayed. All members of this array will populate the `$_POST` array that our rendering function receives. Finally, the `success` parameter indicates the code that should be executed when data is successfully received back. In our case, the data that is received back from the AJAX request is used to replace the HTML content of the `#book_review_table` div section. We also make a call to the `remove_links_register_callbacks` function to remove the new navigation links' `href` tags and re-attach callbacks to be called when they are next clicked. This must be done every time we update our content since the `jQuery(document).ready` code does not get fired when content is dynamically updated.

The final bit of code that we have added in our plugin is to assign the `ch4_br_book_review_list` function to be called when one of two variable name actions are matched: `wp_ajax_<actionname>` or `wp_ajax_nopriv_<actionname>`. The first of these action hooks is triggered when the user making the AJAX request is logged in on the site. The other one is for regular non-connected visitors. This allows a plugin to register different callbacks for different types of users. In our case, we register the same function for both, where `<actionname>` is the string that was sent as part of the `data` parameters in the AJAX request. Upon receiving the request, the callback generates an updated book review table and echoes the resulting HTML code so it can be caught by our AJAX function. It also makes a call to the standard PHP `die()` function before the function ends. While this last step might seem strange, it is needed to avoid having a trailing `1` at the end of the new HTML, indicating that the AJAX processing was successfully performed by WordPress.

See also

- The *Displaying custom post type data in shortcodes* recipe in *Chapter 4*, *The Power of Custom Post Types*

10
Adding New Widgets to the WordPress Library

Widgets have been present in WordPress from the early days of the platform. They allow users to easily populate sidebars or other widget areas of their website theme with content that is provided by WordPress itself (post or page data) or by any plugins that have been installed (for example, book review system information). Starting with the recent version 5.8, WordPress introduced the ability to use blocks from the block editor in the widget area. That being said, the platform continued supporting legacy widgets, coded using a combination of PHP and HTML. In the subsequent version 5.9, WordPress introduced the concept of block themes, with the first example theme not supporting widgets. We can therefore say that the future of widgets is a bit uncertain, but they are still relevant to the large majority of site themes out there today.

This chapter shows how to use a widget class to create a custom content widget. It also covers the second type of widget, dashboard widgets. These can be used to display plugin-specific information on the front page of the administrative area of single-site and network WordPress installations.

In this chapter, you will learn how to create your own widget through the following topics:

- Creating a new widget in WordPress
- Displaying widget configuration options
- Validating widget configuration options
- Adding a custom dashboard widget
- Adding a custom widget to the network dashboard

Technical requirements

You can find complete code examples on GitHub in the following repository: `https://github.com/PacktPublishing/WordPress-Plugin-Development-Cookbook-Third-Edition`.

Creating a new widget in WordPress

The first step in creating a custom widget is to define its name and indicate the class that contains all of its implementation functions. We also need to specify a function to render the widget's contents. Once the new element has been registered with the system, it will immediately appear in the widget list, where users will be able to drag and drop it to their widget areas.

Getting ready

You should have already followed the *Updating the page title to include custom post data using plugin filters* recipe in *Chapter 4*, *The Power of Custom Post Types*, to have a starting point for this recipe. Alternatively, you can get the resulting code (`ch4/ch4-book-reviews/ch4-book-reviews-v12.php`) from the book's GitHub page and make sure that the **Chapter 4 - Book Reviews** plugin is activated.

How to do it...

Follow these steps to define a new widget that displays recent book reviews from the custom post type created in *Chapter 4*, *The Power of Custom Post Types*:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch10-book-review-widget`.

3. Navigate to the directory and create a text file called `ch10-book-review-widget.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 10 - Book Review Widget`.
5. Add the following line of code to register a function to be called when widgets are initialized:

```
add_action( 'widgets_init',
            'ch10brw_create_widgets' );
```

6. Add the following code segment to provide an implementation for the `ch10brw_create_widgets` function:

```
function ch10brw_create_widgets() {
    register_widget( 'Book_Reviews' );
}
```

7. Insert the following block of code at the end of the plugin file to declare the `Book_Reviews` class along with its constructor method:

```
class Book_Reviews extends WP_Widget {
    function __construct () {
        parent::__construct( 'book_reviews',
            'Book Reviews',
            array( 'description' =>
                   'Displays recent book reviews' ) );
    }

}
```

8. Insert the following block of code within the `Book_Reviews` class, after the `__construct` method's closing parenthesis (}) and just before the class' final closing parenthesis (}), to define the class' `widget` rendering method:

```
function widget( $args, $instance ) {
    extract( $args );
    $nb_reviews =
        ( isset( $instance['nb_reviews'] ) &&
        !empty( $instance['nb_reviews'] ) ?
        intval( $instance['nb_reviews'] ) : 5 );
```

```
        $widget_title =
            ( isset( $instance['widget_title'] ) &&
            !empty( $instance['widget_title'] ) ?
            esc_html( $instance['widget_title'] ) :
                        'Book Reviews' );
        $query_array =
            array( 'post_type' => 'book_reviews',
                   'post_status' => 'publish',
                   'posts_per_page' => $nb_reviews );

        $book_review_query = new WP_Query();
        $book_review_query->query( $query_array );
        echo $before_widget . $before_title;
        echo apply_filters( 'widget_title',
                            $widget_title );
        echo $after_title;

    }
```

9. Continue the implementation of the `widget` method of the `Book_Reviews` class by inserting the following code at the end of the function body, right after the call to `echo $after_title` and before the method's closing parenthesis:

```
    if ( $book_review_query->have_posts() ) {
        echo '<ul>';
        while ( $book_review_query->have_posts() ) {
            $book_review_query->the_post();
            echo '<li><a href="' . get_permalink() . '">';
            echo get_the_title( get_the_ID() );
            if ( isset( $instance['show_author'] ) &&
                    'true' == $instance['show_author'] ) {
                echo ' - ' . get_post_meta(
                    get_the_ID(), 'book_author', true );
            }
            echo '</a></li>';
        }
```

```
        echo '</ul>';
    }
    wp_reset_query();
    echo $after_widget;
```

10. Save and close the plugin file.
11. Go to the **Appearance** section of your development site's **Dashboard** and **Activate** a theme other than **Twenty Twenty-Two** (such as **Twenty Twenty-One** or **Twenty Twenty**). You may need to install one of these themes if they are not present in your development environment.

> **Note**
>
> As mentioned previously, WordPress theme creation has started down a new path in version 5.9 with the introduction of block themes. The **Twenty Twenty-Two** theme is an example of this new theme development method and does not offer a **Widgets** editor, nor does it allow widgets to be added when editing the site layout. It only offers the ability to display blocks just like the ones we created in *Chapter 6*, *Extending the Block Editor*. We therefore switched to a different theme to be able to use our new widget. That being said, the majority of themes available online today do have a **Widgets** editor and are compatible with this recipe.

12. Navigate to the **Plugins** management page and **Activate** the **Chapter 10 - Book Review Widget** plugin.
13. Visit the **Widgets** menu, found in the **Appearance** administration menu section.
14. Click on the + sign next to the editor title in the top-left corner and scroll down to the **WIDGETS** section to see the newly created **Book Reviews** widget. You can also search for it using the **Search** box.

15. Click on the new widget to add it to the first widget section available and see it indicate that it does not have any options available. You can also click the + sign under or above existing widgets to add the **Book Reviews** widget to a specific location:

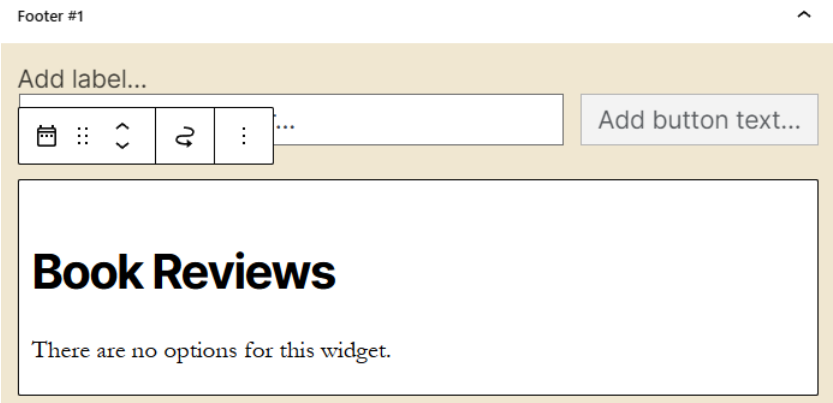


Figure 10.1 – Book Reviews widget empty options panel in the Twenty Twenty theme

16. Click in an empty area outside of the widget.

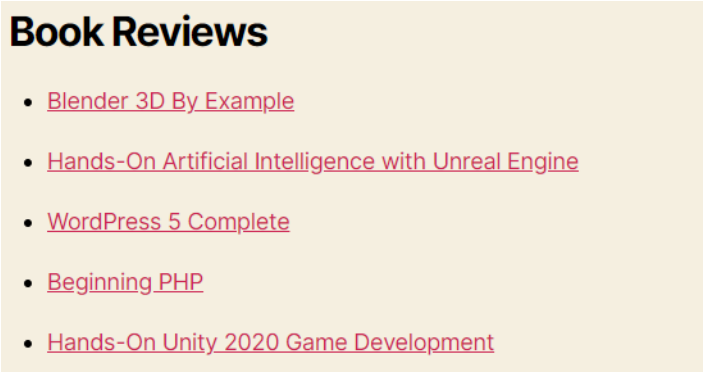


Figure 10.2 – List of book reviews in the Widgets editor using the Twenty Twenty theme

As seen in *Figure 10.2*, the widget now displays a list of up to five book reviews from your site's database. This is a direct preview of what your widget will look like on your website.

17. Click on **Update** in the top-right corner of the **Widgets** editor to save your new widget instance.
18. Visit your development site to see your new widget appear in the footer or another widget area location where you instantiated it.

How it works...

The `widgets_init` action hook is used to register a function to be executed when widgets are being created by WordPress. When the callback occurs, we create a new widget by calling the simple `register_widget` function. As can be seen in the recipe, this function requires a single argument that indicates the name of a widget implementation class.

The rest of the recipe declares this widget implementation class, which extends the WordPress `WP_Widget` class. This initial implementation provides two essential methods to create and display a widget: the class constructor and its rendering method, the `widget` method. The constructor initializes the object instance by specifying a unique identifier, a title, and a description embedded within an array of optional parameters. The `widget` method is responsible for rendering the content of the widget. As with any other functions declared in plugins, it is important to give a unique name to the widget class and widget identifier in order to avoid conflicts with other plugins.

Similar to action hooks, which we have seen in earlier chapters, the `widget` method is meant to directly output HTML code. This code is displayed in the browser when an instance of the new widget has been created in a widget area.

The `widget` method starts by calling the standard PHP `extract` function on the first parameter received, an array named `$args`. This array contains a list of styling tidbits to render the widget. Calling the `extract` function parses the array and creates individual variables for each element found. This makes it easier for the following code to access the elements that should be placed before and after the widget title and widget content.

After this initial statement, the recipe continues by retrieving the number of items to display and the widget title from the `$instance` array. This array is received as the second parameter of the `widget` method. To check the values contained within the `$instance` array, we use a code structure that we have not seen before called a **PHP ternary conditional operator** (`?:`). This operator expects three expressions ordered as follows: `(expr1)?(expr2):(expr3)`. Based on the values received, it returns `expr2` if `expr1` is true and `expr3` if it's false.

Now, as we saw when we instantiated the widget, our new widget does not currently have any options. Therefore, the `$instance` array will actually be empty when rendering our widget for the moment. We are laying down some groundwork to be ready for the widget options that will be created in the following recipes in this chapter. This means that the default widget title (`Book Reviews`) and default number of reviews to display (`5`) will be used to display the widget.

The rest of the code is very similar to the book review shortcode created in the *Displaying custom post type data in shortcodes* recipe in *Chapter 4*, *The Power of Custom Post Types*. More specifically, we assemble a query parameter array that indicates the type, post status, and maximum quantity of items that we want to retrieve from the database. This query array is used to create a new instance of the WordPress `WP_Query` object. If results are found, the following recipe code cycles through all the entries and outputs HTML code to render an unordered list of all the items found. As part of this HTML output, the recipe formats the widget content by inserting the values of the `$before_widget`, `$after_widget`, `$before_title`, and `$after_title` variables in the right spots.

See also

- The *Updating the page title to include custom post data using plugin filters* recipe in *Chapter 4*, *The Power of Custom Post Types*

Displaying widget configuration options

Similar to plugin configuration pages, widgets can have one or more options to allow users to specify how some aspects of the component will behave. These options can be configured individually for each instance of a widget that is added to a website layout. To handle all of the logistics around multiple possible widget instances, WordPress actually takes care of most of the data handling and storage tasks.

Getting ready

You should have already followed the *Creating a new widget in WordPress* recipe to have a starting point for this recipe. Alternatively, you can get the resulting code (`ch10/ch10-book-review-widget/ch10-book-review-widget-v1.php`) from the book's GitHub page and rename the file to `ch10-book-review-widget.php`. You should also be running the **Chapter 4 - Book Reviews** plugin. If needed, you can get that plugin's code (`ch4/ch4-book-reviews/ch4-book-reviews-v12.php`) from the book's GitHub page and **Activate** it.

How to do it...

Follow these steps to add a new method to the book review widget class to display configuration options in the **Widgets** editor:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Navigate to the `ch10-book-review-widget` directory and edit `ch10-book-review-widget.php`.

3. Find the `Book_Reviews` class and add the following block of code within the body of the class after the `widget` method's closing parenthesis (}) and before the closing class parenthesis (}) to define the `form` method:

```
function form( $instance ) {
    $show_author =
        ( isset( $instance['show_author'] ) &&
        !empty( $instance['show_author'] ) ?
        $instance['show_author'] : 'true' );
    $nb_reviews =
        ( isset( $instance['nb_reviews'] ) &&
        !empty( $instance['nb_reviews'] ) ?
        $instance['nb_reviews'] : 5 );
    $widget_title =
        ( isset( $instance['widget_title'] ) &&
        !empty( $instance['widget_title'] ) ?
        esc_attr( $instance['widget_title'] ) :
                    'Book Reviews' ); ?>

    <p><label for="<?php echo
        $this->get_field_id( 'show_author' ); ?>">
        Show author names
    <select id="<?php echo
        $this->get_field_id( 'show_author' ); ?>"
        name="<?php echo
        $this->get_field_name( 'show_author' ); ?>">
        <option value="true"
          <?php selected( $show_author, 'true' ); ?>>
        Yes</option>
        <option value="false"
          <?php selected( $show_author, 'false' ); ?>>
        No</option>
    </select>
    </label></p>

<?php }
```

4. Continue the implementation of the `form` method of the `Book_Reviews` class by inserting the following code in the body after the `</label></p>` line of code and before the method's closing `<?php }` tag:

```
<p><label for="<?php echo
    $this->get_field_id( 'widget_title' ); ?>">
    Widget Title
    <input type="text" id="<?php echo
        $this->get_field_id( 'widget_title' );?>"
           name="<?php
        echo $this->get_field_name(
                                 'widget_title' ); ?>"
           value="<?php echo $widget_title; ?>" />
    </label></p>
<p><label for="<?php echo
     $this->get_field_id( 'nb_reviews' ); ?>">
     Number of reviews to display
     <input type="text"
          id="<?php echo
          $this->get_field_id( 'nb_reviews' ); ?>"
          name="<?php echo
          $this->get_field_name(
                                 'nb_reviews' ); ?>"
          value="<?php echo $nb_reviews; ?>" />
</label></p>
```

5. Save and close the plugin file.

6. Go back to the **Appearance | Widgets** administration page and click on the **Book Reviews** widget instance to see its newly created options:

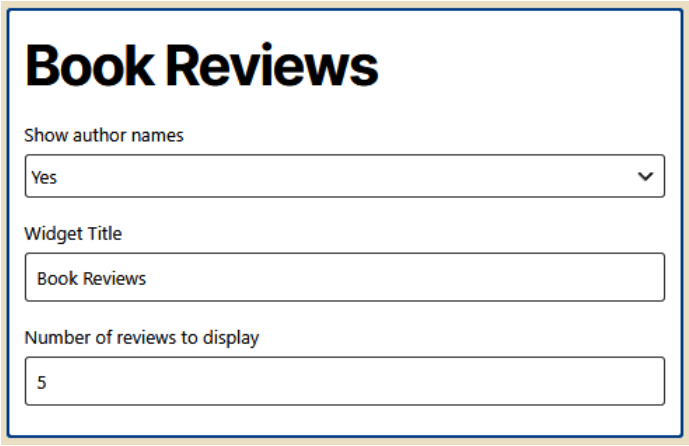


Figure 10.3 – Widget configuration options

7. Change one or more of the widget options and click on **Save** to update its configuration.

How it works...

When users create a new widget instance, WordPress automatically manages configuration options for that element using an array variable. It also calls the widget class' `form` method to render the widget instance's options in a configuration panel.

The first few lines of code in the `form` method verify that the `instance` array contains proper values. These values should specify whether author names are to be displayed, the title that should be displayed at the top of the widget, and the number of book reviews to be shown. All of these lines use the same PHP ternary conditional operator that we just saw in the *Creating a new widget in WordPress* recipe.

With these variables in place, the rest of the `form` method's code uses a mix of HTML and PHP code to render the configuration fields that are shown in the widget editor. The `get_field_id` and `get_field_name` methods, seen throughout this code, are used to generate unique identifiers that will help WordPress to store data separately for all widget instances.

As can be seen in this recipe, the widget class is able to automatically process and save widget configuration parameters. However, it should be noted that allowing WordPress to handle this task by itself means that no validation will be performed on the data entered. This could cause problems if a user enters text instead of the number of reviews to be displayed. The next recipe shows how to handle data validation.

See also

- The *Creating a new widget in WordPress* recipe

Validating widget configuration options

The widget configuration panel that was put in place in the previous recipe is functional and allows users to change options and save updated values in the website database. That being said, all WordPress does by default when the user saves a widget is store values directly in the site database. Accepting user data blindly is never a good idea and can lead to functionality problems and security risks if wrong or malicious values are entered. Therefore, it is preferable to add data validation rules through the creation of an `update` method that will be able to verify configuration data before it is saved.

Getting ready

You should have already followed the *Displaying widget configuration options* recipe to have a starting point for this recipe. Alternatively, you can get the resulting code (`ch10/ch10-book-review-widget/ch10-book-review-widget-v2.php`) from the book's GitHub page and rename the file to `ch10-book-review-widget.php`. You should also be running the **Chapter 4 - Book Reviews** plugin. If needed, you can get that plugin's code (`ch4/ch4-book-reviews/ch4-book-reviews-v12.php`) from the book's GitHub page and **Activate** it.

How to do it...

Follow these steps to implement a widget's `update` method:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Navigate to the `ch10-book-review-widget` directory and edit `ch10-book-review-widget.php`.

3. Find the `Book_Reviews` class and add the following block of code within the class after the `form` method's closing parenthesis (}) and before the closing class parenthesis (}) to define the `update` method:

```
function update( $new_instance, $instance ) {
    if ( is_numeric(
            $new_instance['nb_reviews'] ) ) {
        $instance['nb_reviews'] =
            intval( $new_instance['nb_reviews'] );
    }

    $instance['widget_title'] =
        sanitize_text_field(
            $new_instance['widget_title'] );
    $instance['show_author'] =
        sanitize_text_field(
            $new_instance['show_author'] );

    return $instance;
}
```

4. Save and close the plugin file.
5. Visit the **Widgets** section of the **Appearance** administration menu and click on the **Book Reviews** widget instance to see its configuration fields.
6. Enter a textual value (such as `Test`) in the **Number of reviews to display** field and click outside of the widget to see it again. You will see that the widget display is unchanged.
7. Click on **Update** in the top right of the **Widgets** editor, then navigate to another page of the WordPress administration section and come back. You will see that the field's value reverts to the last valid number saved for this field.

How it works...

The `update` method receives two arrays of data and must return a single array to be saved in the website database. The two incoming arrays contain the new option values entered by the user and the values that were previously stored for the widget, respectively.

The method's implementation starts by using the PHP `is_numeric` function to check whether the value entered for the number of entries to display is a number. If it is, it calls the `intval` function on the incoming value to be sure it's an integer number and stores it in the array of new values for the widget. If the value received is anything but a number, it does not change the previous widget value for the number of entries. Unfortunately, it is not possible to display an error message when this type of validation of widget options is performed.

Once the numeric value has been validated, our code calls the `sanitize_text_field` function to remove potentially harmful HTML or PHP tags from text fields. It saves the return value in the `$instance` array. Once all fields have been validated, the validated `$instance` array is returned.

See also

- The *Displaying widget configuration options* recipe

Adding a custom dashboard widget

Content widgets are used by website administrators to easily add content to their front-facing websites. WordPress contains another type of widget that plugin developers can use to enhance the administrator experience. Dashboard widgets are sections that appear on the front page of a website's administration area. These sections can offer any kind of functionality, from simple information displays indicating how much data is stored in a plugin to forms that allow website administrators to quickly perform configuration tasks.

Getting ready

You should have already followed the *Updating the page title to include custom post data using plugin filters* recipe in *Chapter 4*, *The Power of Custom Post Types*, to have a starting point for this recipe. Alternatively, you can get the resulting code (`ch4/ch4-book-reviews/ch4-book-reviews-v12.php`) from the code bundle and activate the **Chapter 4 - Book Reviews** plugin.

How to do it...

Follow these steps to add a new dashboard widget that indicates how many book reviews are stored in the system, along with links to quickly access them:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch10-book-review-dashboard-widget`.

3. Navigate to the directory and create a text file called `ch10-book-review-dashboard-widget.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 10 - Book Review Dashboard Widget`.
5. Add the following line of code to register a function to be called when the dashboard contents are being prepared:

```
add_action( 'wp_dashboard_setup',
            'ch10brdw_add_dashboard_widget' );
```

6. Add the following code segment to provide an implementation for the `ch10brdw_add_dashboard_widget` function:

```
function ch10brdw_add_dashboard_widget() {
    wp_add_dashboard_widget(
        'book_reviews_dashboard_widget',
        'Book Reviews',
        'ch10brdw_dashboard_widget' );
}
```

7. Insert the following block of code to implement the `ch10brdw_dashboard_widget` function declared in the previous step:

```
function ch10brdw_dashboard_widget() {
    $book_review_count =
        wp_count_posts( 'book_reviews' );
    if ( !empty( (array) $book_review_count ) ) { ?>
        <div>
        <a href="<?php echo add_query_arg( array(
            'post_status' => 'publish',
            'post_type' => 'book_reviews' ),
            admin_url( 'edit.php' ) ); ?>">
        <strong>
            <?php echo $book_review_count->publish; ?>
        </strong> Published</a><br />
        <a href="<?php echo add_query_arg( array(
            'post_status' => 'draft',
            'post_type' => 'book_reviews' ),
```

```
                admin_url( 'edit.php' ) ); ?>">
            <strong>
                <?php echo $book_review_count->draft; ?>
            </strong> Draft</a>
            </div><br />

            <style>
                #book_reviews_dashboard_widget a {
                    text-decoration: none;
                }
            </style>
        <?php }
    }
```

8. Save and close the plugin file.
9. Navigate to the **Plugins** management page and **Activate** the **Chapter 10 - Book Review Dashboard Widget** plugin.
10. Navigate to the website's **Dashboard**.

Figure 10.4 – Book Reviews dashboard widget

As seen in *Figure 10.4*, the new **Book Reviews** widget appears at the bottom of the page and displays information on the number of book reviews found on the site. It can be repositioned anywhere on the **Dashboard**.

How it works...

Any plugin can register its own dashboard widget when WordPress is putting together content for the administrative landing page. After registering a function to be called during the dashboard setup phase, our recipe makes a call to `wp_add_dashboard_widget`. This function allows us to register our own element for the website dashboard and provide a callback function to display its contents.

The `wp_add_dashboard_widget` function requires three parameters that provide a unique identifier for the new item, a title to be displayed at the top of the widget, and a function that will be responsible for generating the widget's contents. The `wp_add_dashboard_widget` function also has an optional fourth parameter that can be used when the widget needs to process form data as part of the dashboard widget contents.

As can be seen in the previous screenshot, dashboard widgets are displayed using WordPress meta boxes. Any HTML code echoed by the content display function directly appears in the box.

While the display function is mostly composed of HTML code, we also make a call to the `wp_count_posts` utility function, which easily returns the number of posts for a given post type.

The new widget can be hidden and moved to a new location on the **Dashboard**, like any other built-in widget. Just like the front-facing widget plugin created earlier in this chapter, it should be noted that all of the code in this plugin is in a separate file from the original book review plugin. This helps us organize our code separately from the original plugin file created in *Chapter 4*, *The Power of Custom Post Types*, and shows how developers can extend plugins they don't develop themselves.

See also

- The *Updating the page title to include custom post data using plugin filters* recipe in *Chapter 4*, *The Power of Custom Post Types*

Adding a custom widget to the network dashboard

As discussed in the *Creating a network-level plugin with admin pages* recipe in *Chapter 3*, *User Settings and Administration Pages*, WordPress offers a very powerful mode called network mode that allows multiple websites to be served from a single installation of the platform. When creating a plugin, developers need to think about whether it would make sense for their plugin to offer a dashboard widget that would only be seen on the network administrator's dashboard instead of being seen on individual website dashboards, or whether their plugin's scope is really more relevant at each website's level.

Getting ready

You should have already followed the *Adding a custom dashboard widget* recipe to have a starting point for this recipe. Alternatively, you can get the resulting code (`ch10/ch10-book-review-dashboard-widget/ch10-book-review-dashboard-widget-v1.php`) from the book's GitHub page and rename the file `ch10-book-review-dashboard-widget.php`. You should also have access to a network site configured as a network to test all of this plugin's functionality. Finally, the final version of the **Chapter 4 - Book Reviews** plugin should be installed and active at the network level. The Local development web server makes it easy to create a network WordPress setup.

How to do it...

Follow these steps to modify the dashboard widget defined in the previous recipe. After these modifications, it will appear in the network administration panel on network installations, while still appearing on individual site administration dashboards:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Navigate to the `ch10-book-review-dashboard-widget` directory and edit `ch10-book-review-dashboard-widget.php`.
3. Locate the `add_action` call that was first added at the top of the plugin and add the following line of code right after the existing call:

```
add_action( 'wp_network_dashboard_setup',
            'ch10brdw_add_dashboard_widget' );
```

4. Locate the `ch10brdw_dashboard_widget` function and add the following code to the function around the existing implementation of the function. The new lines of code are shown in bold:

```
function ch10brdw_dashboard_widget() {
    if ( is_network_admin() ) {
        $sites_list = get_sites();
    } else {
        $sites_list = array( 'blog_id' =>
                                get_current_blog_id() );
    }

    foreach( $sites_list as $site ) {
        if ( is_network_admin() ) {
            switch_to_blog( $site->blog_id );
```

```
            }
            $site_name = get_bloginfo( 'name' );
            echo '<div>' . $site_name . '</div>';
            $book_review_count =
                wp_count_posts( 'book_reviews' );
            ?>
            // REST OF PREVIOUS EXISTING CODE
        <?php }
        }
        if ( is_network_admin() ) {
            restore_current_blog();
        }
    }
```

5. Save and close the plugin file.
6. Navigate to the website **Dashboard** (on a single site installation) or the network-level **Dashboard** (in a network installation).

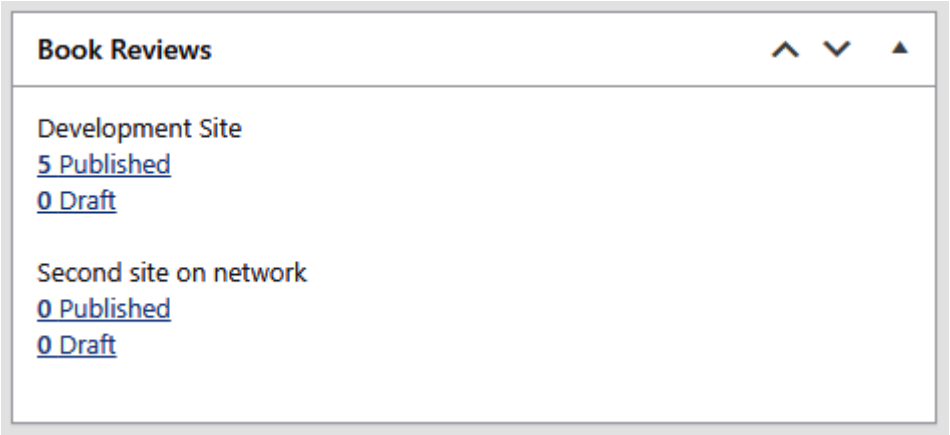


Figure 10.5 – Network-level Book Reviews widget showing all sites in network

As seen in *Figure 10.5*, we are now able to see a new network-level widget showing all sites when we visit the network **Dashboard**. When viewed on a single site installation, the widget looks as it did before, with the addition of the site name.

How it works...

The initial addition made in this recipe is to associate our book review count widget code with the `wp_network_dashboard_setup` action hook. This makes it appear on network-level dashboard pages in addition to individual site dashboards.

Once this is done, we modify the `ch10brdw_dashboard_widget` function. Using the `is_network_admin` function, the callback determines whether it is currently running on a network-level or site-level administration page. In the case of a network-level display, our rendering code first gets a list of all sites using the `get_sites` function, then cycles through the list of sites. Our code then makes use of the `switch_to_blog` function to access data from each site's database tables. Once all the blogs have been processed, we use the `restore_current_blog` function to go back to the original site that is configured as the top-level site in the network.

> **Note**
>
> It is important to use `restore_current_blog` after using `switch_to_blog` to avoid leaving some internal variables pointing to the wrong site.

When shown on a site-level administration page or a single-site installation of WordPress, we create a dummy list of sites instead of using the `get_sites` function. This dummy list contains a single entry for the current site to allow us to use the same `foreach` loop control structure that is used at the network level. We then avoid calling the functions that are related to switching between sites when the widget is not being displayed on the network-level dashboard. Otherwise, the actual code that queries how many book reviews are present and displays them is identical between both modes of the widget output.

See also

- The *Adding a custom dashboard widget* recipe

11
Fetching, Caching, and Regularly Updating External Site Data

After spending most of our time learning how to extend WordPress to allow site administrators and users to create local content, this chapter focuses on interacting with external data sources. While we did some work with displaying Twitter feed content in the *Creating a new shortcode with parameters* recipe in *Chapter 2*, *Plugin Framework Basics*, we had very little control over how this data was displayed since the content was shown in an **IFrame**. We also had no control over how this data was fetched and no way to cache it. The reason for this is that every user loading a page containing the Twitter feed shortcode fetched their own copy of the information to be displayed.

There are many different types of data out there that we could be interested in fetching, caching, and displaying. In this chapter, we will be focusing on two different types of information: **RSS** feeds and generic **XML** data.

More specifically, we will learn how to work with external data through the following recipes:

- Displaying RSS items from an external feed
- Optimizing plugin performance by storing external data using transients
- Scheduling plugin data updates using WP-Cron

Technical requirements

You can find complete code examples on GitHub in the following repository: `https://github.com/PacktPublishing/WordPress-Plugin-Development-Cookbook-Third-Edition`.

Displaying RSS items from an external feed

RSS is a technology that has been around since the early days of the internet. Its main purpose is to allow users to be easily notified when new content is published on their favorite sites. The underlying mechanism behind RSS feeds is a simple XML file that contains the site's headlines along with updated timestamps and part or all of the content of new items that are posted. Applications or services called **RSS aggregators** (for example, Feedly and NewsBlur) can monitor the content of RSS feeds and display new items as they are published or notify users of this new content. It is interesting to note that podcasts use the same RSS feed technology to publish their episodes online.

While RSS is not as popular as it used to be, most news-based websites still publish RSS feeds. WordPress includes a copy of the **SimplePie** library to allow developers to easily fetch and cache RSS feed data and easily display it on a site.

Getting ready

You should have access to a WordPress development environment.

How to do it...

Follow these steps to create a simple interface to manage a list of RSS feeds to be queried along with a shortcode to display each feed's information on a page:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch11-rss-feeder`.

3. Navigate to the directory and create a text file called `ch11-rss-feeder.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 11 - RSS Feeder`.
5. Add the following line of code to register a function to be called when WordPress is building the administration menu:

```
add_action( 'admin_menu', 'ch11rf_settings_menu' );
```

6. Add the following code segment to provide an implementation for the `ch11rf_settings_menu` function:

```
function ch11rf_settings_menu() {
    $options_page = add_options_page( 'RSS Feeder',
        'RSS Feeder', 'manage_options',
        'ch11rf-rss-feeder', 'ch11rf_config_page' );
}
```

7. Insert the following block of code to provide an implementation for the `ch11rf_config_page` function, which was declared when we created our menu item:

```
function ch11rf_config_page() {
    $rss_feed_list =
        get_option( 'ch11rf_rss_feed_list' ); ?>

    <div id="ch11rf-general" class="wrap">
    <h2>RSS Feeder</h2><br />

    <?php if ( isset( $_GET['message'] ) &&
                $_GET['message'] == '1' ) { ?>
    <div id='message' class='updated fade'>
        <p>
            <strong>Modifications Saved</strong>
        </p></div>
    <?php } ?>

    <form method="post" id="rss_feed_form"
          action="<?php echo
              admin_url( 'admin-post.php' ); ?>">
        <input type="hidden" name="action"
```

```
                value="save_ch11rf_options" />
    <?php wp_nonce_field( 'ch11rf' ); ?>

<?php }
```

8. Add this second block of code to continue the implementation of the `ch11rf_config_page` function, on the line following the call to `wp_nonce_field` and just before the closing PHP statement and final parenthesis (`<?php }`):

```
<table>
<?php if ( !empty( $rss_feed_list ) ) {
    foreach ( $rss_feed_list as $rss_feed ) { ?>
        <tr><td><input type="text" size="80"
                name="rss_feed[]"
                value="<?php echo $rss_feed; ?>"></td>
            <td><span style="cursor: pointer"
                class="dashicons dashicons-trash">
                </span></td>
    </tr>
    <?php } } ?>
    <tr><td>
        <input type="text" size="80" name="rss_feed[]"
                value=""
                placeholder="RSS Feed URL"></td>
        <td><input type="submit" value="Save"
                    class="button-primary"/></td></tr>
</table>
</form>
</div>
```

9. Add this third block of code to complete the implementation of the `ch11tr_config_page` function, after the closing `</div>` tag and before the final parenthesis (`<?php }`):

```
<script type="text/javascript">
    jQuery( document ).ready( function() {
        jQuery('.dashicons-trash').click( function() {
            jQuery( this ).closest( 'tr' ).remove();
            jQuery( '#rss_feed_form' ).submit();
```

```
        });
    });
</script>
```

10. Add the following lines of code at the end of the file to assign a callback function to be called when WordPress prepares to display administration pages and provide an implementation for this function:

```
add_action( 'admin_init', 'ch11rf_admin_init' );

function ch11rf_admin_init() {
    add_action( 'admin_post_save_ch11rf_options',
                'process_ch11rf_options' );
}
```

11. Add the following block of code to provide an implementation for the `process_ch11rf_options` function that was specified in the previous step:

```
function process_ch11rf_options() {
    if ( !current_user_can( 'manage_options' ) ) {
        wp_die( 'Not allowed' );
    }
    check_admin_referer( 'ch11rf' );

    if ( isset( $_POST['rss_feed'] ) &&
             !empty( $_POST['rss_feed'] ) ) {
        $rss_feed_array = array();
        foreach ( $_POST['rss_feed'] as $rss_feed ) {
            if ( !empty( $rss_feed ) ) {
                $rss_feed_array[] =
                    esc_url_raw( $rss_feed );
            }
        }
        update_option( 'ch11rf_rss_feed_list',
                       $rss_feed_array );
    }
    wp_redirect( add_query_arg( array(
        'page' => 'ch11rf-rss-feeder',
```

```
            'message' => '1' ),
            admin_url( 'options-general.php' ) ) );
        exit;
    }
```

12. Insert the following line of code at the end of the file to register a new shortcode:

```
add_shortcode( 'rss-feeder',
               'ch11rf_show_feed_list' );
```

13. Insert this final block of code at the end of the file to provide an implementation for the `ch11rf_show_feed_list` function:

```
function ch11rf_show_feed_list() {
    $rss_feed_list =
        get_option( 'ch11rf_rss_feed_list' );
    $output = '';
    if ( empty( $rss_feed_list ) ) { return; }

    $output .= '<div class="rss_feeder">';
    foreach ( $rss_feed_list as $rss_feed ) {
        $rss = fetch_feed( $rss_feed );
        if ( is_wp_error( $rss ) ) { return; }
        $feed_items = $rss->get_items( 0, 5 );
        if ( $feed_items ) {
            foreach ( $feed_items
                      as $index => $item ) {
                if ( 0 == $index ) {
                    $output .= '<h3>' .
                    $item->get_feed()->get_title() .
                    '</h3>';
                }
                $output .= '<div class="item">';
                $output .= '<h4><a href="' .
                    $item->get_permalink() . '">' .
                    $item->get_title() . '</a></h4>'
                    . '<div class="item_content">' .
                    wp_trim_words(
```

```
                                $item->get_description(), 40)
                            . '</div></div>';
                    }
                }
            }
            $output .= '</div>';
            return $output;
        }
```

14. Save and close the plugin file.
15. Navigate to the **Plugins** management page and **Activate** the **Chapter 11 – RSS Feeder** plugin.
16. Click on the **RSS Feeder** menu item, found under the **Settings** section of the WordPress administration pages, to view the plugin's configuration page.
17. Add a few RSS feed addresses by entering a URL and clicking on **Save** each time to save the item and add a new line.

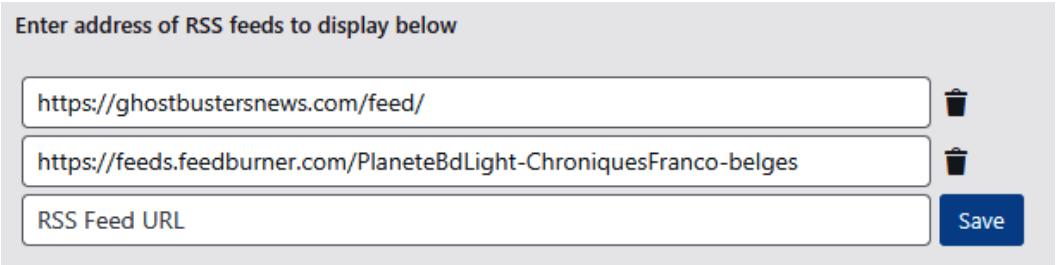


Figure 11.1 – RSS Feeder configuration page

18. Go to the **Pages** section of the administration area and click on **Add New** to create a new page.
19. Insert the `[rss-feeder]` shortcode in the page editor, **Publish** the page, and click on **View Page** to see the resulting news feed displayed on your development site.
20. Refresh the page to see that it comes up faster the second time around, since `fetch_feed` caches feed data locally.

How it works...

This recipe uses a lot of the tools that we covered in previous chapters to add a new menu item in the **Settings** menu, define a configuration page for our new plugin, and save the information that the user provides. That being said, it does introduce one new concept in the configuration page with input fields that all share the same name including square brackets (`rss_feed[]`). This standard nomenclature is used to indicate that all form fields with this name are to be collected and sent to the processing code as an array. Looking at the `process_ch11rf_options` function, we can see that our code checks to see whether we have received any data in the `$_POST['rss_feed']` variable. If data is found, our code cycles through all entries with a `foreach` loop. Each URL found is escaped and stored in the site option table using a single array.

Our recipe also creates a new shortcode called `[rss-feeder]` to display our RSS feed data. The main new element in this code is the `fetch_feed` function. This function takes the URL of an RSS feed as its only argument and returns a **SimplePie** object. Once we've executed the `fetch_feed` function, we check that the return value is not an error using the `is_wp_error` function. We then use the `get_items` method of the object to retrieve the `5` most recent items of the feed and iterate through all items with a `foreach` loop. The feed items themselves also offer multiple methods. We are calling these throughout our shortcode display function to get access to the information that they contain (`get_title`, `get_permalink`, `get_description`, and `get_feed`).

There's more...

In addition to facilitating the retrieval of RSS items from a remote site, the `fetch_feed` function also has built-in caching capabilities.

Changing the fetch_feed caching period

By default, any RSS data retrieved with the `fetch_feed` function will be saved in the WordPress site database using a caching mechanism called **transients** for a period of 12 hours. This means that the first time you try to access the page, it will take more time to appear as the server fetches all data and processes it before displaying it onscreen. Any subsequent visit during the next 12 hours will come up much faster. That being said, it will also show the exact same news items as the first visit. As with most aspects of WordPress, this caching period can be modified with a filter callback function. For example, adding the following code to the plugin file changes the caching period to 1 hour (3,600 seconds):

```
function ch11rf_feed_cache( $seconds ) {
    return 3600;
}
```

```
add_filter( 'wp_feed_cache_transient_lifetime',
            'ch11rf_feed_cache' );
```

The `wp_feed_cache_transient_lifetime` filter always sends and expects to receive a value in seconds.

See also

- The *Creating a new simple shortcode* recipe in *Chapter 2*, *Plugin Framework Basics*

Optimizing plugin performance by storing external data using transients

Some WordPress functions, such as the `fetch_feed` function that we saw in the *Displaying RSS items from an external feed* recipe, offer a built-in caching mechanism. This is not the case when building custom code to retrieve information from other online data sources. The following recipe takes a look at using the public API of a transportation agency, **Bay Area Rapid Transit**, in the San Francisco area. It shows how to retrieve and cache remote XML data in our local site database using transients to speed up site performance while also alleviating the load on the remote data source.

Getting ready

You should have access to a WordPress development environment.

How to do it...

Follow these steps to query, cache, and display transit data from an external source:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch11-transit-feed`.
3. Navigate to the directory and create a text file called `ch11-transit-feed.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 11 – Transit Feed`.

5. Insert the following block of code in your code file to define a utility function, `is_multi_array_or_convert`, which will check whether the variable it receives is a multidimensional array and make it one if it isn't:

```
function is_multi_array_or_convert( $array ) {
    if ( isset( $array[0] ) &&
            is_array( $array[0] ) ) {
        return $array;
    } else {
        return array( $array );
    }
}
```

6. Add the following block of code to the end of the plugin file to create a second utility function, `get_transit_data`, which will be responsible for fetching data from the **Bay Area Rapid Transit** website:

```
function get_transit_data() {
    $stations = array( '12th', '16th',
                       '19th', '24th' );
    $transit_data = array();

    foreach ( $stations as $station ) {
        if ( false === ( $transit_data[$station] =
            get_transient( 'transit_feed_'
                           . $station ) ) ) {
            $next_departures_xml =
                simplexml_load_file(
         'http://api.bart.gov/api/etd.aspx?cmd=etd&orig='
                 . $station . '&key=MW9S-E7SL-26DU-VV8V' );
            $next_departures_json =
                json_encode( $next_departures_xml );
            $transit_data[$station] =
                json_decode(
                    $next_departures_json, TRUE );
            set_transient( 'transit_feed_' . $station,
                $transit_data[$station], 300 );
        }
```

```
    }
    return $transit_data;
}
```

7. Insert the following line of code at the end of the plugin file to define a new shortcode:

```
add_shortcode( 'transit-feed',
               'ch11tf_transit_feed' );
```

8. Insert the following block of code to define the `ch11tf_transit_feed` function:

```
function ch11tf_transit_feed() {
    $output = '';
    $transit_data = get_transit_data();

    if ( empty( $transit_data ) ) { return; }
    $output .= '<div class="transit-feed">';
    foreach ( $transit_data as $station_data ) {
        $output .= '<h3>';
        $output .= $station_data['station']['name'];
        $output .= '</h3>';

        if ( empty(
                $station_data['station']['etd'] ) ) {
            continue;
        }
        $etd_array = is_multi_array_or_convert(
            $station_data['station']['etd'] );

        $output .= '<table><tr><th>Destination</th>';
        $output .= '<th>Direction</th>';
        $output .= '<th>Number of minutes</th></tr>';
        foreach ( $etd_array as $etd ) {
            if ( empty( $etd['estimate'] ) ) {
                continue;
            }
            $etd_estimate_array =
```

```
                    is_multi_array_or_convert(
                        $etd['estimate'] );

                foreach ( $etd_estimate_array as $eta ) {
                    $output .= '<tr><td class="dest">'
                        . $etd['destination']
                        . '</td><td>' . $eta['direction']
                        . '</td><td>' . $eta['minutes']
                        . '</td></tr>';
                }
            }
            $output .= '</table>';
        }

        $output .= '<style>';
        $output .= '.transit-feed th {text-align: left}';
        $output .= '.transit-feed table { width: 100% }';
        $output .= '.transit-feed .dest { width: 40% }';
        $output .= '</style>';

        $output .= '</div>';
        return $output;
    }
```

9. Save and close the plugin file.
10. Navigate to the **Plugins** management page and **Activate** the **Chapter 11 – Transit Feed** plugin.
11. Go to the **Pages** section of the administration area and click on **Add New** to create a new page.
12. Insert the `[transit-feed]` shortcode in the page editor, **Publish** the page, and click on **View Page**.

12th St. Oakland City Center

Destination	Direction	Number of minutes
Antioch	North	49
Antioch	North	79
Berryessa	South	53
Richmond	North	48

Figure 11.2 – Transit schedules for Bay Area Rapid Transit

As seen in *Figure 11.2*, our shortcode is converted into scheduling information for upcoming trains. Our sample code queries four stations of the Bay Area Rapid Transit system and could be expanded to cover all of them.

> **Note**
>
> When working with transients, a plugin such as **Transients Manager** (`https://wordpress.org/plugins/transients-manager/`) is very useful to view or delete transient data.
>
> It should also be noted that the `key` used as part of the URL to retrieve transit information in the `get_transit_data` function is a sample key that was found on the BART website. If the key does not work when you try to execute this recipe's code, sign up for your own free key (`https://api.bart.gov/api/register.aspx`) and replace it in the code.

How it works...

The bulk of the work in this recipe is done by the `get_transit_data` utility function. When called by our shortcode, this function first defines an array of abbreviated station names and an empty array of transit data. It then proceeds to loop through the station names array and uses an `if` statement that contains both a condition and an assignment statement:

```
if ( false === ( $transit_data[$station] =
            get_transient( 'transit_feed_'
                           . $station ) ) ) {
```

If we first look at the part of this code in the innermost parentheses, it calls the `get_transient` function and inserts the result as a new member in the `$transit_data` array. It uses the abbreviated station name as the array key. The transient name passed to the `get_transient` function also uses this abbreviated station name, along with a static string. Now, the actual conditional `if` statement is that we check if the value returned by `get_transient` and assigned to the array is the value `false`. If it is, no transient data was found under that name or the data has expired.

In the case where no data is returned by `get_transient`, we use the standard PHP `simplexml_load_file` function to get data from the public transit website and store it in a local variable. We then use a coding trick with the `json_encode` and `json_decode` functions to easily convert the XML object received from the `simplexml_load_file` function to a simpler PHP multidimensional array. We store the result in the `$transit_data` array, using the station code as the array key.

Last but not least, our utility function calls `set_transient` to store each station's data in the WordPress transients system with an expiration time set to `300` seconds. While this might not seem like much, it will still prevent a lot of data retrieval from the public transit site as well as speeding up page loads on a high-traffic website. It can also greatly help stay within limits when working with data services that have a maximum number of times you can access their services on an hourly or daily basis. While we could have gone for a longer period, we don't want the scheduling data to go completely stale either. Once all station data has been retrieved from the transient tables or refreshed, the resulting transit data array is returned.

> **Note**
> We could have saved the entirety of the transit data array in a single transient. We made the choice to store it in separate transients for each station to show that transient names don't have to be fixed strings. This approach could also be beneficial if we wanted to display individual station data on separate pages.

The remainder of the recipe implements a shortcode that requests the transit data from our utility function and renders it nicely to appear on our website. In addition to looping through all data and generating HTML, the shortcode implementation uses our second utility function, `is_multi_array_or_convert`. This function helps us make the data received from the public transit website more uniform. This is required since the site sends back a multidimensional array when multiple departures are found or a simple array of individual fields for single departures. Our utility function returns a multidimensional array either way. We can then easily loop through the results without needing to check how the data is structured.

See also

- The *Displaying RSS items from an external feed* recipe

Scheduling plugin data updates using WP-Cron

When working with external data sources, it's always important to consider the frequency and mechanism used to update this data, as well as caching data locally. While our code in the *Optimizing plugin performance by storing external data using transients* recipe is functional, it only checks whether the transient data is expired when users visit the shortcode. This slows down the page display if new data has to be fetched and processed. A better approach is to schedule our data update function to be periodically executed when any part of the site is visited, or even on a very specific schedule using **WP-Cron**.

Getting ready

You should have already followed the *Optimizing plugin performance by storing external data using transients* recipe to have a starting point for this recipe. Alternatively, you can get the resulting code (`ch11/ch11-transit-feed/ch11-transit-feed-v1.php`) from the book's GitHub page and rename the file as `ch11-transit-feed.php`.

How to do it...

Follow these steps to schedule our data update function so that it retrieves new transit data without requiring visits to the page containing the `[transit-feed]` shortcode:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Navigate to the `ch11-transit-feed` directory and edit `ch11-transit-feed.php`.
3. Add the following block of code at the end of the plugin file to register a function to be called when the plugin is initialized:

```
add_action( 'init', 'ch11tf_init' );
```

4. Insert the following code to provide an implementation for the `ch11tf_init` function:

```
function ch11tf_init() {
    add_action( 'ch11tf_transit_data_update',
                'get_transit_data' );
    register_deactivation_hook( __FILE__,
```

```
            'ch11tf_deactivation' );

        if ( !wp_next_scheduled(
                'ch11tf_transit_data_update' ) ) {
            wp_schedule_event( time(), 'five_minutes',
                'ch11tf_transit_data_update' );
        }
    };
```

5. Add the following block of code to the end of the plugin file to provide an implementation for the `ch11tf_deactivation` function:

```
    function ch11tf_deactivation() {
        wp_clear_scheduled_hook(
            'ch11tf_transit_data_update' );
    }
```

6. Go to the end of the plugin file and add the following block of code to register a function with the `cron_schedules` filter:

```
    add_filter( 'cron_schedules',
                'ch11tf_add_cron_period' );
```

7. Insert the following block of code to provide an implementation for the `ch11tf_add_cron_period` function:

```
    function ch11tf_add_cron_period( $schedules ) {
        $schedules['five_minutes'] = array(
            'interval' => 300,
            'display' => 'Five Minutes'
        );
        return $schedules;
    };
```

8. Save and close the plugin file.
9. Click on the **Add New** menu item found in the **Plugins** section of your site administration page.
10. Search for the **Advanced Cron Manager** plugin, then click **Install Now** and **Activate**.
11. Go to the **Tools** menu and select **Cron Manager**.

Event	Schedule	Arguments	Next execution ▲
ch11tf_transit_data_update	Five Minutes		In 2 mins January 30, 2022 7:38 pm

Figure 11.3 – Custom scheduled event

As seen in *Figure 11.3*, our custom event **ch11tf_transit_data_update** is shown in the list of scheduled tasks.

How it works...

WP-Cron provides a set of functions that allow developers to schedule the execution of custom code in the background as website visits take place.

The first of these functions is `wp_schedule_event`, which registers an action to be executed regularly on the website where it is activated. It should be noted that this is the first time that we are assigning an action instead of a function to execute our code. This means that we need to use the `add_action` function to associate our `get_transit_data` utility function to a new action hook, `ch11tf_transit_data_update`, before we can schedule it.

Once this is done, we use a WP-Cron utility function called `wp_next_scheduled` to check whether our custom action has already been scheduled before scheduling it. This must be done since `wp_schedule_event` schedules the given action blindly even if that exact hook was scheduled before.

When finally calling `wp_schedule_event`, its three parameters indicate the timestamp when the action hook should be executed next, the frequency for subsequent executions, and the name of the action hook to run, respectively. The timestamp that is most often sent for the first parameter is the return value of the standard PHP `time()` function. The result is that the action immediately runs before following its assigned schedule. When it comes to the execution frequency parameter, the four default periods in WordPress are `hourly`, `twicedaily`, `daily`, and `weekly`. The `five_minutes` frequency that we specify in our recipe is a custom period that is created through the `cron_schedules` filter hook, where the actual duration is specified in seconds.

The third WP-Cron function that we use in our recipe is `wp_clear_scheduled_hook`, which is placed in the plugin's deactivation function callback. It is very important to call this function to remove the action hook from the table of actions to be executed. Otherwise, that task will keep running, even if the plugin is deleted.

Once our action hook is registered with WP-Cron, it will check if it needs to be executed based on the specified schedule every time a site page is visited. On a highly visited site, this will help regularly update transit data.

There's more...

WP-Cron can be improved beyond its initial capabilities with a few additional manipulations.

Improving WP-Cron execution

WP-Cron helps check for out-of-date external data more frequently than only when the page displaying this data is viewed. That being said, site visits might not be frequent enough on a low-traffic site to keep the data updated. The built-in WP-Cron system also has an execution cost when any web page is visited since it needs to check if a scheduled task needs to run every single time.

To improve the WP-Cron system, a site owner can use a more reliable mechanism to schedule regular execution of data update code. This can be done using their own web server's **cron** functions or an external service through the following steps:

1. Edit the site's `wp-config.php` file and add the following line of code above the line that says `stop editing`:

   ```
   define( 'DISABLE_WP_CRON', true );
   ```

2. Create a cron job on your web server to call the following command at the desired frequency, where `YOUR_SITE_URL` would be replaced with your site address:

   ```
   wget --delete-after http://YOUR_SITE_URL/wp-cron.php
   ```

> **Note**
>
> Note that we are not providing exact steps to create a cron job as this will vary depending on your web server configuration and available management tools.
>
> We should also mention that free and paid third-party services (for example, `https://www.easycron.com/` and `https://cron-job.org/en/`) can also be used to access this special URL on a regular schedule.

See also

- The *Optimizing plugin performance by storing external data using transients* recipe

12
Enabling Plugin Internationalization

WordPress is a worldwide phenomenon, with users embracing the platform all around the globe. To create a more immersive experience for users in different locales, WordPress offers the ability to translate all of its administrator- and visitor-facing content. Like most other functionalities on the platform, internationalization is also available to plugin developers. By calling a set of easy-to-use functions, developers can include translation files with their creations.

To prepare a plugin to be localized, developers must use special internationalization functions when dealing with any textual element. Once this structure is in place, any site administrator can create localizations by themselves for languages that they are familiar with. They can submit these translations back to the plugin author for inclusion in a future update to the extension. WordPress.org also offers an online interface for users to be able to contribute translations without requiring the use of offline tools (`https://translate.wordpress.org/`).

This chapter explains how to prepare a plugin to be translated. It also shows how to use the Poedit tool to create a new language file for a simple plugin through the following topics:

- Changing the WordPress site language configuration
- Adapting default user settings for translation
- Making admin page code ready for translation
- Modifying shortcode output for translation
- Translating text strings using Poedit
- Loading a language file in the plugin initialization

Technical requirements

You can find complete code examples on GitHub in the following repository: `https://github.com/PacktPublishing/WordPress-Plugin-Development-Cookbook-Third-Edition`.

Changing the WordPress site language configuration

The first step to translate a plugin is to configure WordPress to a language setting other than English. This will automatically trigger mechanisms in the platform to look for alternate language content for any internationalized string.

Getting ready

You should have access to a WordPress development environment.

How to do it...

Follow these steps to configure the site to display in French:

1. Open your development site's **Dashboard** and navigate to the **General** section of the **Settings** menu.
2. Set **Site Language** to **Français du Canada** and click on **Save Changes**.

How it works...

The WordPress **Site Language** configuration option allows administrators to select the language that will be seen by visitors when they go to your website. It also determines the language that will be assigned to administrators by default when they visit the management section of the site.

As can be seen in the drop-down list of languages, WordPress has a single language installed by default, shown at the top of the list. The remainder of the list indicates languages that can be added to your website. When you set a new language, translation files are automatically downloaded for the selected language and the new selection becomes the default for visitors. If you look inside the `wp-content/languages` directory of your WordPress installation, you will see that a number of files with `.po`, `.mo`, and `.json` extensions were downloaded for the selected language. **Portable object** (`.po`) files are ASCII text files that contain a list of translations between the original English text from the WordPress source code and a target language. **Machine object** (`.mo`) files are binary versions of portable object files that can be efficiently loaded by PHP as a website is rendered. Finally, **JavaScript Object Notation** files (`.json`) contain translation text for some of the newer components of the WordPress administration interface.

> **Note**
>
> If you are not presented with a list of languages to install automatically in the **General** section of your development site's **Dashboard**, visit `https://wordpress.org/support/article/installing-wordpress-in-your-language/` to learn how to manually download and install translation files for WordPress.

Adapting default user settings for translation

Before we can create translation files for our own plugins, their code needs to be specifically written to allow text items to be translated. This work starts in the plugin's activation routine, where default plugin option values are set. By using translation functions, our code can search for alternate values when a language other than English is specified in the website's configuration file.

Getting ready

You should have already followed the *Changing the WordPress site language configuration* recipe to have configured a language other than English for your development site.

How to do it...

Follow these steps to assign a translated string to a plugin's default options array on initialization:

1. Navigate to the WordPress `plugins` directory of your development installation.
2. Create a new directory called `ch12-hello-world`.
3. Navigate to the directory and create a text file called `ch12-hello-world.php`.
4. Open the new file in a code editor and add an appropriate header at the top of the plugin file, naming the plugin `Chapter 12 - Hello World`.
5. Add the following line of code to register a function to be called when the plugin is activated:

```
register_activation_hook( __FILE__,
    'ch12hw_set_default_options_array' );
```

6. Insert the following block of code to provide an implementation for the `ch12hw_set_default_options_array` function:

```
function ch12hw_set_default_options_array() {
    if ( false === get_option( 'ch12hw_options' ) ) {
        $new_options = array();
        $new_options['message'] =
            __( 'Hello World',
                'ch12hw_hello_world' );
        add_option( 'ch12hw_options', $new_options );
    }
}
```

7. Save and close the plugin file.
8. Navigate to the **Plugins** management page (**Extensions** in French) and click on the **Activate** link (**Activer** in French) for the **Chapter 12 - Hello World** plugin.
9. Use your development server's database administration tool (**phpMyAdmin**, **Adminer**, or another) to run the following query against your site database:

```
SELECT * FROM wp_options where option_name =
'ch12hw_options'
```

option_id	option_name	option_value	autoload
1234	ch12hw_options	a:1:{s:7:"message";s:11:"Hello World";}	yes

Figure 12.1 – Hello World plugin options shown in Adminer

As seen in *Figure 12.1*, a new options array was created in the database, containing a default value for a message.

How it works...

The __ function (that's two underscores) is a WordPress internationalization utility function. When called, it tries to locate a translation for the text that it receives in its first argument within the text domain specified in the second argument. A text domain is essentially a subsection of the global translation table that is managed by WordPress. In this example, the text to be translated is the string `Hello World`, for which the system tries to find a translation in the `ch12hw_hello_world` domain. Since this domain is not available at this time, the function returns the original string that it received as its first parameter. The plugin code assigns the value it receives to the default configuration array.

It should be noted that the __ function is an alias for the `translate` function. While both functions have the same functionality, using __ makes our code shorter when it contains a lot of text elements to be translated.

> **Note**
> Developers should not use a variable or constant in the second parameter of the __ function. Doing so will cause problems with third-party plugins for site translation.

See also

- The *Changing the WordPress site language configuration* recipe

Making admin page code ready for translation

The previous recipe showed how to look up the translation of a text item and return its value for further processing in the plugin code. There are many instances where it is more practical to display the translated content immediately, as we'll see in this recipe.

Getting ready

You should have already followed the *Adapting default user settings for translation* recipe to have a starting point for this recipe. Alternatively, you can get the resulting code (`ch12/ch12-hello-world/ch12-hello-world-v1.php`) for that recipe from the book's GitHub page and rename the file to `ch12-hello-world.php`.

How to do it...

Follow these steps to translate the contents of a simple administration page for immediate display:

1. Navigate to the `ch12-hello-world` folder of the WordPress `plugins` directory of your development installation.
2. Open the `ch12-hello-world.php` file in a text editor.
3. Add the following line of code at the end of the file to register a function to be called when WordPress is building the administration page's menu:

```
add_action( 'admin_menu', 'ch12hw_settings_menu' );
```

4. Add the following code section to provide an implementation for the `ch12hw_settings_menu` function:

```
function ch12hw_settings_menu() {
    add_options_page(
        __( 'Hello World', 'ch12hw_hello_world' ),
        __( 'Hello World', 'ch12hw_hello_world' ),
        'manage_options',
        'ch12hw-hello-world', 'ch12hw_config_page' );
}
```

5. Insert the following block of code to create the `ch12hw_config_page` function declared in the call to `add_options_page`:

```
function ch12hw_config_page() {
    $options = get_option( 'ch12hw_options' ); ?>
    <div id="ch12hw-general" class="wrap">
    <h2><?php
        _e( 'Hello World Configuration',
            'ch12hw_hello_world' ); ?></h2>

```

```
        <form method="post" action="admin-post.php">
        <input type="hidden" name="action"
               value="save_ch12hw_options" />
        <?php wp_nonce_field( 'ch12hw' ); ?>
        <?php _e( 'Shortcode message',
                  'ch12hw_hello_world' ); ?>:
        <input type="text" name="message"
               value="<?php echo
                  esc_html( $options['message'] ); ?>"/>
        <br />
        <input type="submit" value="<?php _e( 'Submit',
                             'ch12hw_hello_world' ); ?>"
               class="button-primary"/>
        </form>
        </div>
    <?php }
```

6. Add the following line of code to register a function to be executed when the administration panel is being prepared to be displayed:

```
    add_action( 'admin_init', 'ch12hw_admin_init' );
```

7. Append the following code segment to provide an implementation for the `ch12hw_admin_init` function:

```
    function ch12hw_admin_init() {
        add_action( 'admin_post_save_ch12hw_options',
                    'process_ch12hw_options' );
    }
```

8. Provide code for the `process_ch12hw_options` function, declared in the previous step, by inserting the following code:

```
    function process_ch12hw_options() {
        if ( !current_user_can( 'manage_options' ) ) {
            wp_die( 'Not allowed' );
        }

        check_admin_referer( 'ch12hw' );
```

```
        $options = get_option( 'ch12hw_options' );
        $options['message'] = sanitize_text_field(
                                    $_POST['message'] );

        update_option( 'ch12hw_options', $options );
        wp_redirect( add_query_arg( 'page',
            'ch12hw-hello-world',
            admin_url( 'options-general.php' ) ) );
        exit;
    }
```

9. Save and close the plugin file.
10. Navigate to the administration page of your development WordPress installation.
11. Click on the **Settings** (**Réglages** in French) section on the left-hand navigation menu to expand it. Click on the new menu item labeled **Hello World**.

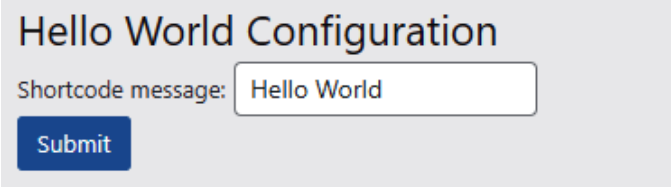


Figure 12.2 – Simple plugin administration page ready for translation

As seen in *Figure 12.2*, selecting the new entry displays the plugin's simple configuration form.

How it works...

This recipe makes use of the `__` function, covered in the previous recipe, along with the `_e` function. This second function's purpose is similar to `__`, except that it immediately echoes the outcome of the translation lookup to the browser. It should be used for any text elements that would previously have just been simple text in HTML code or can be used to replace calls to the `echo` function. Of course, making a call to this function requires the presence of standard opening and closing PHP tags (`<?` and `?>`) to be executed when placed among HTML code.

The rest of this plugin's code takes care of storing user updates in the website database, as covered previously in *Chapter 3*, *User Settings and Administration Pages*.

See also

- The *Adapting default user settings for translation* recipe

Modifying shortcode output for translation

As we have seen in numerous recipes, shortcodes are powerful tools that provide an easy way for users to add content to their website posts and pages. Since this content is presented to users, it can benefit from a translation just as much as the website's administration pages.

Getting ready

You should have already followed the *Making admin page code ready for translation* recipe to have a starting point for this recipe. Alternatively, you can get the resulting code (`ch12/ch12-hello-world/ch12-hello-world-v2.php`) for that recipe from the book's GitHub page and rename the file to `ch12-hello-world.php`.

How to do it...

Follow these steps to translate shortcode output before it is displayed and learn how to deal with variable data elements that can be positioned differently between languages:

1. Navigate to the `ch12-hello-world` folder of the WordPress plugin directory of your development installation.
2. Open the `ch12-hello-world.php` file in a text editor.
3. Add the following line of code at the end of the file to declare a new shortcode that will be available to content authors:

```
add_shortcode( 'hello-world',
               'ch12hw_hello_world_shortcode' );
```

4. Add the following code section to provide an implementation for the `ch12hw_hello_world_shortcode` function:

```
function ch12hw_hello_world_shortcode() {
    $options = get_option( 'ch12hw_options' );

    $output = '<p>';
    $output .=
        sprintf( __( 'The current message is: %s.',
```

```
                'ch12hw_hello_world' ),
                $options['message'] );
        $output .= '</p>';
        return $output;
    }
```

5. Save and close the plugin file.
6. Create a new page (using the **Ajouter** item of the **Pages** menu), insert the new shortcode `[hello-world]` in the content, and then publish the page (**Publier**).
7. View the page (**Voir la page**) to see the output of the shortcode:

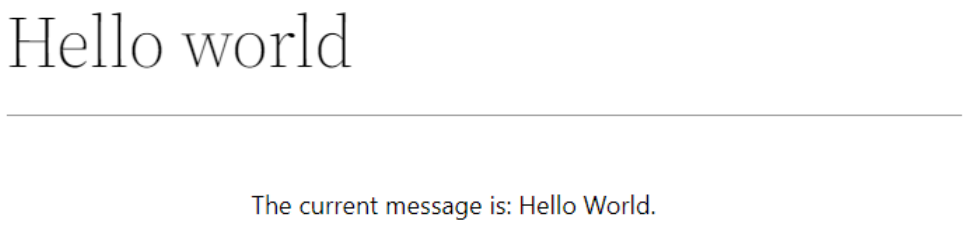


Figure 12.3 – Simple shortcode output ready for translation

How it works...

This recipe shows something that's a bit more complex than the *Adapting default user settings for translation* and *Making admin page code ready for translation* recipes. In this case, we want the shortcode output to be a combination of static text and a dynamic element. We also want that dynamic element to appear in different places based on the grammatical structure of the target language. The way to achieve this functionality is to combine the __ internationalization function with the `sprintf` standard PHP function.

The purpose of the `sprintf` function is to insert a variable in a string. It performs this task by looking for a placeholder in the target string sent in the first argument and replaces it with the variable that it receives as its second argument. Some examples of placeholders are `%s` for a string and `%d` for an integer. With this functionality in mind, we use a placeholder as part of the string to be translated so that users who create localization files can choose where the value will be placed as part of the sentence structure. Once the translation has been obtained by the __ function, we can send the alternate language string to `sprintf` along with the variable data to be inserted in the final string.

See also

- The *Adapting default user settings for translation* recipe

Translating text strings using Poedit

After inserting all the necessary code to look up translations for text elements in our plugin code, the next step is to create the actual translation files. While there are multiple tools available to perform this task, we will focus our efforts on the most popular one, the free multi-platform Poedit.

Getting ready

You should have already followed the *Modifying shortcode output for translation* recipe to have a starting point for this recipe. Alternatively, you can get the resulting code (`ch12/ch12-hello-world/ch12-hello-world-v3.php`) for that recipe from the book's GitHub page and rename the file to `ch12-hello-world.php`.

How to do it...

Follow these steps to extract all the strings to be translated from the plugin's code using Poedit, translate them, and save the resulting language files under the plugin directory:

1. Navigate to the `ch12-hello-world` folder of the WordPress plugin directory of your development installation.
2. Create a new subdirectory named `languages`.
3. Navigate to the Poedit download page and download the appropriate version of the tool for your computer (`https://www.poedit.net/download`).
4. Install and start the Poedit application.
5. Click on the **New** item under the application's **File** menu.

> **Note**
>
> It may be tempting to click on the **Create new…** item on the application's home page. However, this option can only be used when a translation template file (`.POT`) already exists, which is not our case here.
>
> It should also be noted that Poedit offers an automated way to create translations for WordPress themes or plugins as a paid service. We will focus on the free version of Poedit and highlight all steps required to create a translation in this mode.

6. Select **French (Canada)** for **Language of the translation**.
7. Click on the **Save** button from the toolbar and navigate to the `languages` subfolder created earlier in this recipe. Set the filename to `ch12hw_hello_world-fr_CA.po` and click on **Save** to save the new translation file.
8. Click on the **Translation | Properties…** menu item to open the **Translation Properties** dialog and select the **Sources paths** tab.
9. Create a new entry in the **Paths** list by clicking the **+** button under that section of the interface and selecting the **Add folders…** menu item.
10. Navigate one directory up from the `languages` directory and click on **Select Folder** to select the `ch12-hello-world` directory. This will change the **Base path** field in the **Sources paths** tab to the selected directory and create an entry in the **Paths** list that simply contains the period character (`.`).
11. Switch to the **Sources keywords** tab and add two new items to the **Additional keywords** list and set their values to `__` and `_e`.
12. Switch to the **Translation properties** tab and set the **Project name and version** field to `Hello World`.
13. Click on the **OK** button to close the **Translation properties** dialog, then click on **Save**.
14. Click on the **Update from code** toolbar button to extract all translatable items from our plugin's source code.
15. Select the items one by one in order to display them in the lower section of the window.

16. Enter a translation for each text element in the lower dialog box. The following screenshot shows the translations of each item to French:

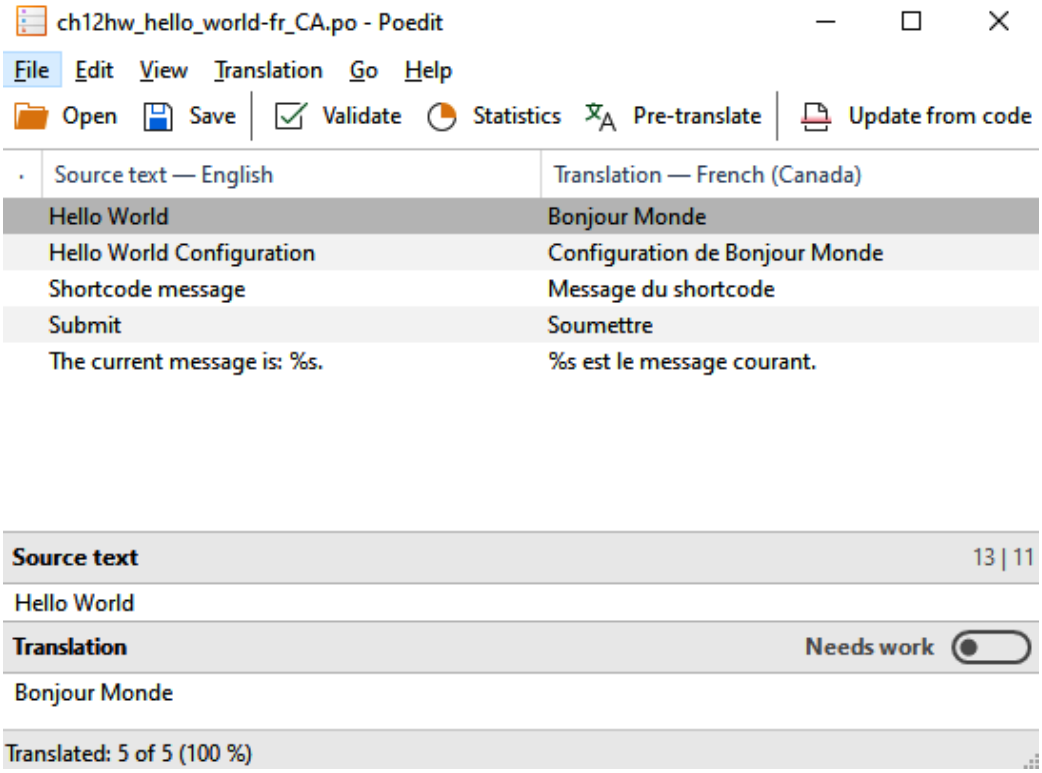


Figure 12.4 – French translation of all plugin strings using Poedit

17. **Save** the translation file once all strings have been translated.

How it works...

The Poedit tool searches through PHP files, looking for functions that have specific names, as specified in the **Sources keywords** configuration section. Based on the configuration that we specified, Poedit is able to find all the instances of the `__` and `_e` functions in the plugin code and retrieve the text strings that are set as the first argument to these functions. Once all the strings have been found, Poedit provides a simple interface to provide translations for each string and saves the resulting translation file. Upon saving a translation project, Poedit actually creates two files, the portable object file (`.po`) and the machine object file (`.mo`). The `.po` file is the one we open and modify in Poedit and the `.mo` file is the file that WordPress uses when executing our plugin code.

The name of the language files is made up of two parts: the name of the text domain, `ch12hw_hello_world`, and the target language code, `fr_CA`. The text domain was used in all of our calls to the `__` and `_e` functions, while `fr_CA` matches the language configuration that we set for our test site earlier in this chapter. To support the two other variants of the French language that WordPress supports (France French and Belgian French), we can make copies of the translation files. We can then set the proper language code in the new filenames (`ch12hw_hello_world-fr_FR.po` and `ch12hw_hello_world-fr_BE.po`) and edit them with Poedit to update each translation file's configuration.

There's more...

If you are only comfortable with English or don't want to distribute translation files with your plugins, you can, at a minimum, create a template file that users will be able to import to start their translation.

Translation template file

When you are only familiar with English, you can create a translation template that only contains the text to be translated by saving the catalog as a `.pot` file instead of a `.po`/`.mo` combination. In addition to the special extension, the filename should not contain a language tag (for example, `ch12hw_hello_world.pot`).

See also

- The *Adapting default user settings for translation* recipe

Loading a language file in the plugin initialization

The final step to plugin translation is to put the code in place to load a translation file. This is done by registering an action hook callback and calling a single function when it gets executed.

Getting ready

You should have already followed the *Making admin page code ready for translation* and *Translating text strings using Poedit* recipes to have the proper files required for this recipe. Alternatively, you can get the resulting code (the `ch12/ch12-hello-world/ch12-hello-world-v3.php` file and the `ch12/ch12-hello-world/languages` folder) for these recipes from the book's GitHub page. You should rename the file `ch12-hello-world-v3.php` to `ch12-hello-world.php` and copy the `languages` folder next to the renamed file before starting the recipe.

How to do it...

Follow these steps to load the translation file created in the previous recipe:

1. Navigate to the `ch12-hello-world` folder of the WordPress plugin directory of your development installation.
2. Open the `ch12-hello-world.php` file in a text editor.
3. Add the following line of code at the end of the file to register a function to be called when the plugin is initialized:

```
add_action( 'init', 'ch12hw_plugin_init' );
```

4. Add the following code section to provide an implementation for the `ch12hw_plugin_init` function:

```
function ch12hw_plugin_init() {
    load_plugin_textdomain( 'ch12hw_hello_world',
        false,
        dirname( plugin_basename( __FILE__ ) )
            . '/languages' );
}
```

5. Locate the `ch12hw_set_default_options_array` function in the plugin's code. Add the same call to `load_plugin_textdomain` at the top of the function, before the existing `if` statement:

```
load_plugin_textdomain( 'ch12hw_hello_world', false,
    dirname( plugin_basename( __FILE__ ) )
        . '/languages' );
```

6. Save and close the plugin file.

7. Navigate to the **Réglages** menu to see that the plugin's menu item has changed to show its translated name (**Bonjour Monde**).
8. Select the **Bonjour Monde** submenu item to see the translated configuration page.

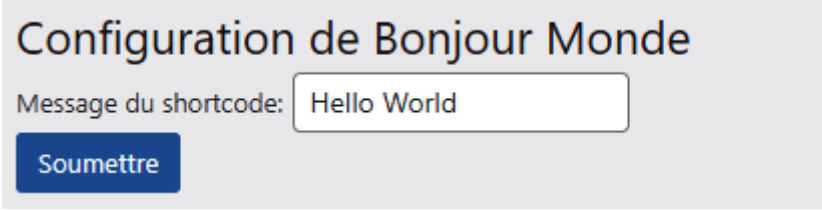


Figure 12.5 – Translated plugin configuration page

9. Use your development server's database management tool to run the following query and then **Delete** the resulting item:

```
SELECT * FROM wp_options where option_name =
'ch12hw_options'
```

10. Go to the **Extensions** menu and click on the **Désactiver** link under the **Chapter 12 – Hello World** plugin to deactivate it and reactivate it by clicking on **Activer**.
11. Use your development server's database management tool to run the following query. See that the message option was recreated containing a French string:

```
SELECT * FROM wp_options where option_name =
'ch12hw_options'
```

option_id	option_name	option_value	autoload
1256	ch12hw_options	a:1:{s:7:"message";s:13:"Bonjour Monde";}	yes

Figure 12.6 – Translated default plugin option

12. Visit the page containing the `[hello-world]` shortcode to see that the message appears in French and that the sentence is structured differently than in English.

Bonjour Monde est le message courant.

Figure 12.7 – Output of shortcode on French site

13. Open your development site's **Tableau de bord** and navigate to the **Général** section of the **Réglages** menu.
14. Set **Langue du site** to **English (United States)** and click on **Enregistrer les modifications** to bring your site configuration back to English.

How it works...

The `load_plugin_textdomain` function has three arguments. When called, it looks in the folder specified in the last parameter for a `.mo` file with a name starting with the text domain specified in the first parameter, followed by the language currently set in the WordPress configuration file. If found, the translation file is loaded in memory and is used to search for translations every time the `__` or `_e` functions are encountered during execution. The middle argument, set to a `false` value, is obsolete but is still needed for backward compatibility.

Once all hooks are in place in the plugin code and a first translation file (or template) is published with the plugin, users can easily modify text elements to other languages. They can immediately start using their new translations and can also provide these new translations back to the plugin author for inclusion in future updates. We added a second call to `load_plugin_textdomain` in the activation hook, since the `init` action hook is executed later than the plugin initialization function.

There's more...

As a plugin evolves over time, new text items may need to be translated. There may also be a need to use more advanced translation functions as well as to translate JavaScript code and custom block code.

Updating a translation file

When new calls to the `__` or `_e` functions are made in a plugin, the translation file needs to be updated to take new text elements into account. To do this, start the Poedit tool and open the existing catalog, then click on **Update from code** from the toolbar. This will extract all the text items and identify new entries. Once this is done, new items can be translated and saved back to the catalog file.

Advanced translation functions

While we used the most common internationalization functions in this chapter, there are a few more advanced functions that may be useful in your efforts:

- `_n( $singular, $plural, $number, $domain )`: This function will look up one of the first two strings received depending on whether the `$number` argument is one or more.
- `_x( $text, $context, $domain )`: Adds a parameter to the localization lookup to add a context parameter. This is useful when dealing with words that have the same spelling but different meanings.

- `_ex( $text, $context, $domain )`: The same as `_x` but echoes the result of the lookup.
- `_nx( $singular, $plural, $number, $context, $domain)`: The same as `_n` with the additional context parameter from `_x`.

There are also a number of functions that will perform a localization lookup immediately, followed by the escaping of the resulting string. These functions include `esc_attr__()`, `esc_attr_e()`, `esc_html__()`, `esc_html_x()`, and many more. For a full list of internationalization functions, visit `https://codex.wordpress.org/L10n`.

Localizing JavaScript files

JavaScript files are a bit trickier to translate as they are often read from an external file that cannot contain any PHP code. The solution to this is the `wp_localize_script` function. When called, this function inserts a new variable that is populated with one or more localized strings in scripts that have been queued up. Upon execution, the JavaScript code will be able to access these translations and display the proper text onscreen. The following code snippet is an example showing how to use the function:

```
wp_enqueue_script( 'script_handle' );
$translation_vars = array( 'string_val' =>
                          __( 'Text to be translated' ) );
wp_localize_script( 'script_handle', 'javascript_object',
                    $translation_vars );
```

After this code runs, a new variable array called `javascript_object` will be created inside the `script_handle` script. This object has a data member called `string_val` that contains a translation of the target text in the current WordPress language, if available.

Localizing custom blocks

Block editor extensions, such as the ones we created in *Chapter 6*, *Extending the Block Editor*, also need to be translated. The process to perform these translations shares some similarities with PHP-based plugin translation, but also differs in several ways that add complexity to the process.

A first step to translating blocks is to import the __ function at the top of our block source code files:

```
import { __ } from '@wordpress/i18n';
```

Once this is done, we can use this function around all text elements in our block code, similar to what we did earlier in this chapter. The following examples of the __ function come from the *Creating a server-side rendering block that leverages an existing shortcode* recipe in *Chapter 6*, *Extending the Block Editor*:

```
title: __( 'Twitter Embed', 'twitter-embed' ),
<TextControl label =
    { __( "Twitter Account User Name", 'twitter-embed' ) }
```

Once they have been wrapped with the __ function, all translatable strings can be extracted from the block source code and translated using Poedit, just like we did in the *Translating text strings using Poedit* recipe. The resulting portable object file cannot be used directly by our block and needs to be converted to a JavaScript Object Notation (`.json`) file. This conversion can be performed using a tool called **WP-CLI**, which stands for **WordPress Command-Line Interface** (`https://wp-cli.org/`), with the following command:

```
wp i18n make-json twitterembed-fr_CA.po --no-purge
```

Once the JSON version of our translations has been created, the resulting file can be renamed to `twitterembed-fr_CA-ch2twe-twitter-embed.json`. In this filename, the first part is the script translation domain, the second is the target translation language, and the third is the identifier of the script used when we called the `wp_register_script` function.

Last, but not least, you need to load the script translation and attach it to the block script code using the `wp_set_script_translations` function in a callback associated with the `init` action within your block plugin's main PHP file. The three arguments of this function are the script handle, the translation domain, and the path to the translation files:

```
wp_set_script_translations( 'ch2twe-twitter-embed',
    'twitterembed',
    plugin_dir_path( __FILE__ ) . 'languages' );
```

Once everything is in place, your custom block will appear translated when used in a page or post, as can be seen in the following figure:

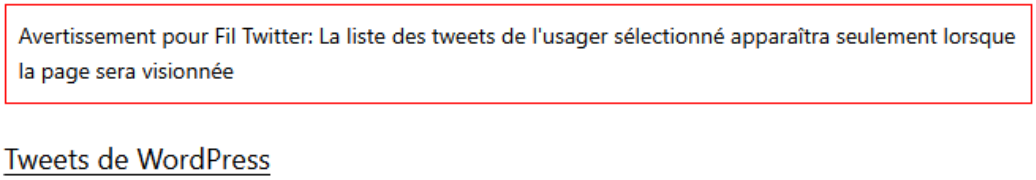


Figure 12.8 – Translated custom block on the French WordPress site

For more information on block translation, you can visit the Block Editor Handbook: `https://developer.wordpress.org/block-editor/how-to-guides/internationalization/`.

See also

- The *Translating text strings using Poedit* recipe

13
Distributing Your Plugin on WordPress.org

Once you have a version of your new plugin that is ready to be distributed to the masses, you need to decide whether you will join the official WordPress repository or self-publish it.

In most cases, the preferred option is to add your new extension to the official WordPress plugin repository. The official repository includes free hosting, built-in notification of new updates, and a powerful search engine that users can access on `WordPress.org` or from the **Plugins** section of their website's administration pages. Other benefits of hosting on the official repository include download statistics and the creation of a free forum to facilitate user support.

To qualify for this hosting, your work must be free and open source. It must also comply with the GNU General Public License, Version 2 (also known as GPL v2), a common open source software license that WordPress itself uses. This includes any code you wrote, along with any third-party PHP or JavaScript libraries that you have included in your plugin. To learn more about the GPL v2 license, visit `https://www.gnu.org/licenses/gpl-2.0.html`. It is also possible to publish a plugin with premium features that require a one-time payment or recurring subscription on the official repository. However, the plugin must offer a reasonable amount of free features and you cannot transform these features into paid functions over time.

In comparison, self-hosting gives you full control over pricing, distribution license, and the general presentation of your work. That being said, it also makes it harder for people to find your plugin and requires you to implement a custom update notification mechanism. Alternatively, users can manually download updates to your work as they become available. There are several popular plugin digital marketplaces, such as CodeCanyon (`https://codecanyon.net/`), that can help alleviate some of these drawbacks. However, your work will still have less visibility than it would on the official repository.

Before making your plugin publicly available, you should also be sure that you are ready to deal with user feedback and questions. Once your creation is available for download, WordPress website administrators will quickly find it and install it. As they use it, they may find that your work covers most, but not all, of their needs. When this happens, you will start getting requests to add functionality. This interaction with users is usually a great experience that can bring new ideas to enhance your work. However, you must also be ready to accept criticism and invest time in fixing issues and implementing new features. You also need to consider the time that will be involved in testing your plugin against new versions of WordPress, which typically come out two to three times a year.

This chapter explains how to prepare your work to be uploaded to the official plugin repository. It covers applying for an account, the actual code submission using Subversion, and how to customize your plugin page to give it a unique look.

More specifically, we will discuss how to distribute your creations through the following topics:

- Creating a README file for your plugin
- Applying for your plugin to be hosted on WordPress.org
- Uploading your plugin using Subversion
- Providing plugin banner and thumbnail images

Technical requirements

You can find complete code examples on GitHub in the following repository: `https://github.com/PacktPublishing/WordPress-Plugin-Development-Cookbook-Third-Edition`.

Creating a README file for your plugin

If you look at any plugins on the official WordPress repository, you will see that their page contains a lot of information. This includes a description of the extension, a list of frequently asked questions, and installation notes. As you may have noticed from the work that we have done so far, this data does not reside in the main plugin's code file. Instead, the official WordPress repository looks for this information in a specially formatted `readme.txt` file that needs to be included with the plugin.

Getting ready

You should have already followed the *Updating page title to include custom post data using plugin filters* recipe in *Chapter 4*, *The Power of Custom Post Types*, to have a starting point for this recipe. Alternatively, you can get the resulting code (`ch4/ch4-book-reviews/ch4-book-reviews-v12.php`) for that recipe from the book's GitHub page.

How to do it...

Follow these steps to create a `readme.txt` file for the Book Reviews plugin that we created in *Chapter 4*, *The Power of Custom Post Types*:

1. Navigate to the `ch4-book-reviews` folder of your WordPress plugins directory.
2. Create a new text file named `readme.txt` and open it in a code editor.
3. Insert the following text in the file:

```
=== Book Reviews ===
Contributors: ylefebvre
Donate link: http://ylefebvre.ca/wordpress-plugins/book-reviews
Tags: book, reviews
Requires at least: 5.0
Tested up to: 6.0
Stable tag: 1.0
License: GPLv2 or later
```

```
License URI: https://www.gnu.org/licenses/gpl-2.0.html

Create your own book review web site!

== Description ==

This plugin lets you add a book review system to your
WordPress site. Using custom post types, administrators
will be able to create and edit book reviews to be
published on your site.

== Installation ==

1. Download the plugin.
1. Upload the book-reviews folder to your site's
wp-content/plugins directory.
1. Activate the plugin in the WordPress Admin Panel.
1. Start creating new book reviews!
1. Use the [book-review-list] shortcode to list reviews
on a page.

== Changelog ==

= 1.0 =
* First version of the plugin.

== Frequently Asked Questions ==

There are currently no FAQs at this time.

== Screenshots ==

1. The review edition page
```

4. Save and close the text file.
5. Navigate to the **Book Reviews** section of the WordPress administration section of your development site and edit one of your entries.

6. Take a screenshot of the interface using a third-party screen capture tool or your operating system's built-in function.
7. Save the resulting image as `screenshot-1.jpg` in the plugin directory.

How it works...

The `readme.txt` file uses a wiki-like syntax, with the number of equal signs (`=`) indicating the level of each section header. The first and most important section is the header, which contains important information. This data includes the plugin's name, the author's `WordPress.org` username, donation link, search tags, supported versions, and stable Subversion code tag, along with a one-line description of its functionality. This last item will always be visible as users navigate through your plugin's pages.

The initial header is followed by multiple sections, which correspond to the various sections that appear within a plugin's display pages. More specifically, these sections contain a complete description of the extension's capability and a step-by-step guide to installing and using your work. They also contain a change log containing a list of all the versions with a summary of changes for each of them, frequently asked questions, and screenshots. It is also possible for plugin authors to create their own arbitrary sections using the same syntax.

As with the standard wiki syntax, the repeating `1.` in front of each installation step will be converted to incrementing values when they are displayed on the plugin's page. If screenshots are listed in the `readme.txt` file, the `WordPress.org` website will search for files whose names start with the keyword `screenshot-`, followed by a number. These images will be displayed in the screenshot section using their associated text as a legend. When taking screenshots of your plugin in action, make sure that they are clear and meaningful, as visitors will often decide whether or not they will download your creations based on these images.

You can find more details about the `readme.txt` file format in the `WordPress.org` plugin handbook (`https://developer.wordpress.org/plugins/plugin-basics/header-requirements/`).

There's more...

Let's take a closer look at the Subversion `Stable tag` property that is specified as part of the header information.

Releasing plugin versions using tags

Tags are a Subversion concept that allows developers to identify a group of file revisions at a specific point in time and label them with a name. This tag is used to specify the version of your plugin that `WordPress.org` visitors will be able to download. We specified `1.0` in this recipe, for the first version of our plugin. As you release new versions of your plugin, you will always be creating new tags for your code and updating the value of the `Stable tag` field to match the new tag name.

In addition to keeping your work organized, working with tags allows you to commit partially implemented new plugin features to your repository without users getting access to them. When your new features are ready, create a new tag and update the `Stable tag` to release your new work. It's also interesting to know that every version identified with a tag can be downloaded from the **Advanced View** section of your plugin page on the official repository. This makes it easy for users to roll back to a previous version if issues come up after an update. It also allows them to download an unreleased version of the plugin so they can test updates before you make them available publicly.

Applying for your plugin to be hosted on WordPress.org

After creating proper documentation for your creation, the next step toward its publication on the official plugin repository is to apply for hosting. This is simply done by submitting a request form in the **Developer Center** section on `WordPress.org`.

How to do it...

Follow these steps to apply for plugin hosting on the official WordPress plugin repository:

1. Create a zip archive containing your entire plugin directory, including the `readme.txt` file created in the previous recipe.
2. Visit the `https://wordpress.org/plugins/developers/add/` website and log in using the link in the top-right corner of the page or create a new account if you don't have one yet.
3. Select your ZIP file and upload it to the site.

How it works...

Plugin submission is a fairly simple process, giving developers access to a Subversion repository that they can use to upload their work and share it with the community.

Before submitting your plugin, you should search through existing plugins to be sure that you have not selected a name that already exists in the repository. If your selected name is already taken, your request will be turned down immediately. You can search using the plugin repository's search engine, as well as trying to access addresses based on your plugin name. For example, following our **Book Reviews** plugin example, you could check to see if the address `https://wordpress.org/plugins/book-reviews` exists.

It should be noted that your plugin needs to be functional when you apply to be listed on the repository. Once your plugin has been approved, you will get access to a hosted Subversion version control repository. This tool will allow you to easily keep backups of all the versions of your work during development and easily publish it. If you work on a plugin with one or more people, this infrastructure will make it very easy to exchange code between all contributors.

See also

- The *Creating a README file for your plugin* recipe

Uploading your plugin using Subversion

Using Subversion in the recipes of *Chapter 1*, *Preparing a Local Development Environment* might have seemed like overkill when you were working on a plugin locally. However, you will see that this knowledge will be very handy once your hosting request has been approved by the WordPress plugins team, as the system's backend relies on that version control system.

Getting ready

You should have already followed the *Applying for your plugin to be hosted on WordPress.org* recipe to have an approved repository on the official website. You should have also installed a Subversion client as shown in the *Creating a local Subversion repository* recipe in *Chapter 1*, *Preparing a Local Development Environment*. Finally, you should have plugin files ready for upload.

How to do it...

Follow these steps to submit your creation to the `WordPress.org` website once a repository has been created for you:

1. Open Command Prompt or Terminal and navigate to a work directory on your computer.
2. Check out your plugin's repository with the following command. You should use the address of your new repository, as indicated in your hosting approval email, for example:

```
svn checkout http://plugins.svn.wordpress.org/book-
reviews
```

> **Note**
>
> Calling the `checkout` command creates a directory whose name matches the last section of the repository URL (`book-reviews` in this case). This new directory contains three new empty folders: `branches`, `tags`, and `trunk`, reflecting your new plugin repository structure.

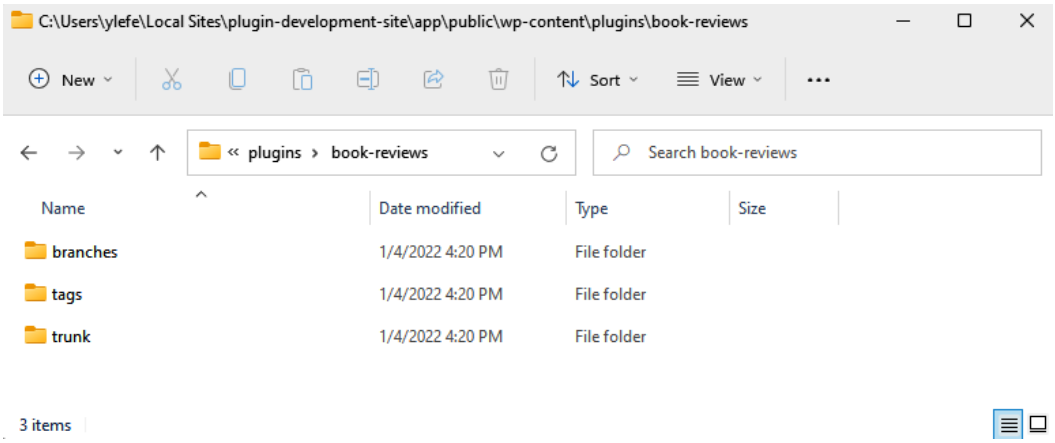


Figure 13.1 – Newly checked out empty plugin repository

3. Copy your plugin's files to the `trunk` folder of the resulting directory structure if it is empty. If the directory contains the initial files you submitted with your plugin application, update them if you made any changes since you applied.
4. If you added new files, add them using the `svn add` command.

5. If you added new files or updated them, commit your plugin files with the `svn commit` command:

```
svn commit -m "1.0 - First public version"
```

6. When prompted for authentication, use your `WordPress.org` username and password.

> **Note**
> After committing your files, you will receive an email from the WordPress repository confirming that they have been uploaded. The email also contains information on all changes that were made to each file.

7. Back in Command Prompt or Terminal, use the following command to tag the files that are currently in your `trunk` directory. The `1.0` number should match the stable tag number specified in the `readme.txt` file, for example:

```
svn copy https://plugins.svn.wordpress.org/book-reviews/
trunk/ https://plugins.svn.wordpress.org/book-reviews/
tags/1.0 -m "1.0 Release"
```

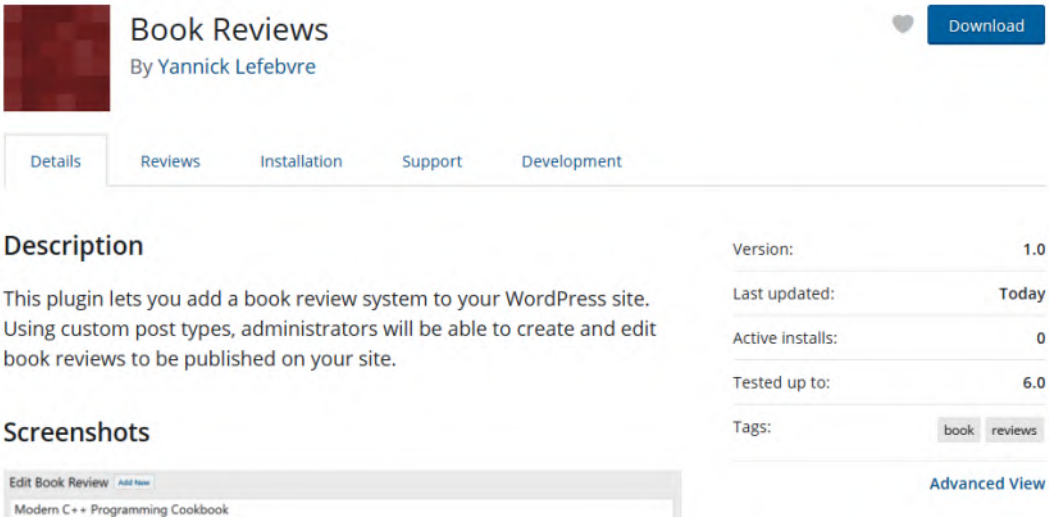


Figure 13.2 – Plugin repository page after commit

Once the commit and tag creation process are completed, you will be able to visit your plugin's page and download it. For our example, the **Book Reviews** plugin, the address of the page would be `https://wordpress.org/plugins/book-reviews`.

How it works...

The official WordPress plugin repository uses Subversion to manage all code files, provide version control services to developers, and find information to populate the extension's page. When your new repository is created, it contains three main directories: `trunk`, `tags`, and `branches`.

The `trunk` directory is usually the main location where you place the latest version of your plugin files. Following the steps in the recipe, we copy our files to this location and commit them to the server. Once uploaded, the WordPress.org servers take care of creating a zipped copy of your work for users to install on their site.

The `tags` directory is designed to hold pointers to various versions of your creation over time, as discussed in the *Creating a README file for your plugin* recipe. Used in conjunction with the **Stable tag** field of your plugin's `readme.txt` file, tags allow you to offer users a working plugin version while you commit and test new work to the `trunk`. New tags are created using the `svn copy` command. The `branches` directory has a similar function to `tags` but is more focused on the creation of alternate versions of plugins or in-development revisions that include specific functionality.

There's more...

If you want to execute your plugin's code in a local WordPress development installation as you are writing it, the following section shows you how to manage your code.

Checking out plugins to your development installation

When checking out the complete plugin repository folder, you end up with a structure that cannot be executed directly in WordPress for testing and development purposes. Instead of checking out the entire directory structure, you can limit your selection to the `trunk` directory. This will only copy the contents of that specific folder to your development environment.

See also

- The *Creating a README file for your plugin* recipe
- The *Creating a local Subversion repository* recipe in *Chapter 1*, *Preparing a Local Development Environment*.

Providing plugin banner and thumbnail images

While the plugin listing that we put in place by creating a `readme.txt` file is perfectly functional, it does not really stand out among the sea of extensions that are available on the website. Thankfully, `WordPress.org` introduced a mechanism allowing plugin developers to add a banner image to their listing. This image can be anything from a simple picture to a complex graphic to advertise your creation.

Getting ready

You should have already followed the *Applying for your plugin to be hosted on wordpress.org* and *Uploading your plugin using Subversion* recipes. You should have an approved repository on the official repository and plugin files uploaded to the server.

How to do it...

Follow these steps to prepare images for your plugin and learn how to upload them to your repository:

1. Navigate to the plugin repository folder created in the previous recipe.
2. Create a new directory called `assets`, next to the `branches`, `tags`, and `trunk` folders.
3. Create a new image that is exactly 772 x 250 pixels.
4. Save the image as a PNG file with the name `banner-772x250.png` and save it to the newly created `assets` directory.
5. Create smaller images that are exactly 256 x 256 pixels and 128 x 128 pixels in size and name the files `icon-256x256.png` and `icon-128x128.png`, respectively. The larger icon should be a higher-resolution version of the exact content of the smaller one. It will be used to replace the smaller one on high DPI displays.
6. Open Command Prompt or Terminal and navigate to the work directory created in the previous recipe.
7. Use the `svn add` command to identify the new `assets` folder and all of the files it contains to be added to the repository:

```
svn add assets
```

8. Commit your new files to your repository with the `svn commit` command:

```
svn commit -m "Graphical assets for plugin page"
```

9. Visit your plugin's page on `WordPress.org` to see the images in place:

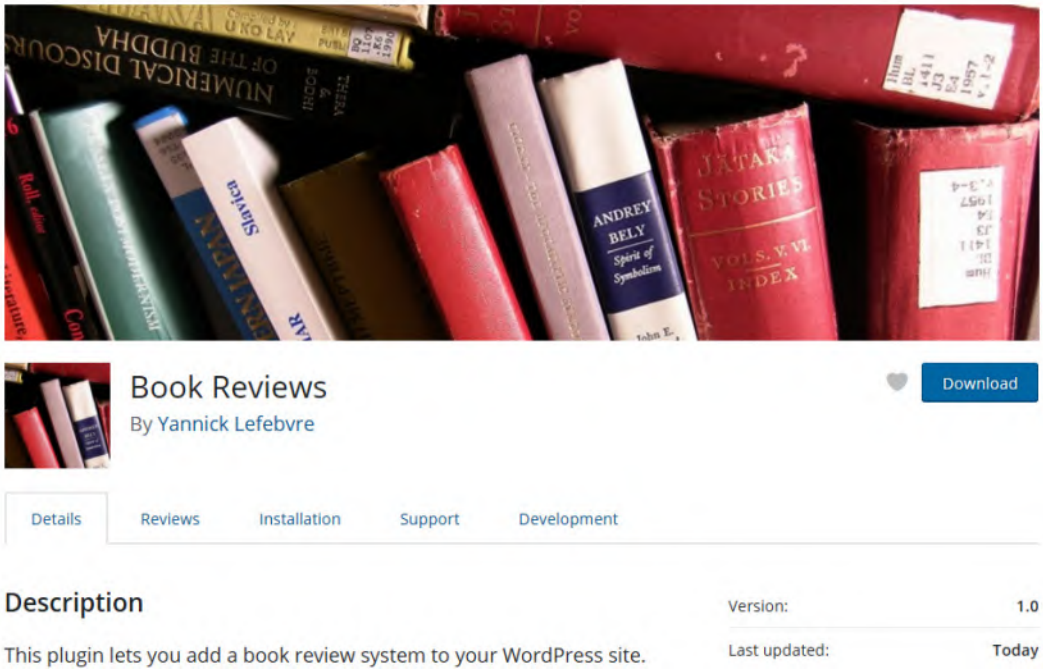


Figure 13.3 – Graphically-improved plugin repository page

10. Search for your plugin through the `https://wordpress.org/plugins` page or the **Add New** section of the **Plugins** management interface of WordPress.

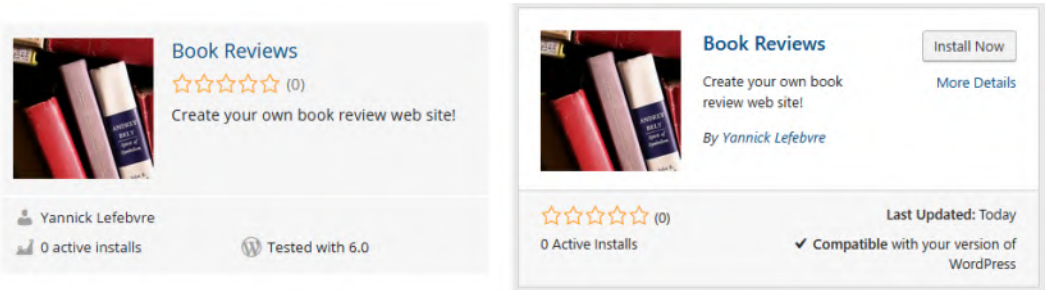


Figure 13.4 – Book review plugin appearing in search results

As seen in *Figure 13.4*, the icon and short plugin description that we provided appear in these results.

How it works...

When files are uploaded to the plugin repositories, the `WordPress.org` website checks for the presence of specific image files with specific names in a specific directory for the plugin banner and icon. If these files are present, it changes the layout of the plugin page to incorporate the images. It is important to respect the image format and specified dimensions when creating a plugin banner and icons to make sure that they are displayed properly on the website.

See also

- The *Uploading your plugin using Subversion* recipe

Index

Symbols

$paged 308

A

action hook
  about 19, 26
  online listings 27
  user settings, accessing from 102, 103
add_action function 26
admin code
  splitting, from main plugin file to optimize site performance 111, 112
administration page menu item
  creating, in settings menu 71-74
admin page code
  preparing, for translation 353-356
admin page contents
  rendering, with HTML 79-82
  rendering, with Settings API 92-99
admin page form fields
  tooltips, adding with TipTip plugin 298-301
admin pages
  formatting, with meta boxes 104-110
  used, for creating network-level plugin 131-135
admin_url() function 31
Akismet API
  reference link 265
apply_filters function 34
arrays
  used, for storing user settings 66-68
Asynchronous JavaScript and XML (AJAX)
  about 302
  using, to dynamically update partial page contents 301-309

B

BART
  reference link 343
block
  creating 219-225
  development environment, setting up 219
Block Editor 144
Block Editor Handbook
  reference link 368
built-in debugging features 47, 48

built-in ThickBox plugin
  used, for displaying pop-up
    dialog 287-290

C

calendar day selector
  displaying, with Datepicker
    plugin 295-298
CAPTCHA
  implementing, on user forms 257-260
  implementing on user forms,
    with local library 262-265
  working 261
category editor
  hiding, from custom post
    type editor 172-174
  working 175
check_ajax_referer function 308
checkout operation 12
Classic Editor
  reference link 201
client-side content submission form
  creating 244-248
code editing application
  features, prerequisites 15
code editor 1, 16, 121
coding errors
  troubleshooting 42-46
commit operation 12
computer
  local web server, installing on 2-5
confirmation message
  displaying, when options saved 87, 88
content_url() function 31
Cornerstone
  URL 13
cron functions 348
custom blocks
  localizing 366
  version control 225
custom Cascading Style Sheet (CSS) code
  inserting, in page output 56, 57
custom categories
  Quick Edit fields, adding 184-191
  working 183
custom dashboard widget
  adding 324-326
custom fields
  adding, to categories 168-171
  adding, to user editor 268-270
  working 171
Custom Fields editor 145
Custom Fields section
  hiding, in post editor 206-209
custom files
  attaching with media uploader, by
    extending post editor 210-214
custom help pages
  adding 89-92
custom layout
  single custom post type items,
    displaying with 151-153
  working 154
custom meta boxes
  adding, to post types 202, 203
  information, capturing with 198-201
  information, displaying with 198-201
  working 201, 202
custom post data
  displaying, with filter functions 203-205
custom post editor
  section, adding 146-149
  working 150

custom post list page
  additional columns, displaying 175-179
  filters, adding for custom
    categories 181, 182
  working 180
custom post type
  creating 139-144
  custom categories, adding 165-167
  data, displaying in shortcodes 160-164
  permalinks slug, modifying 146
  search output, tailoring for 155-159
  user-submitted content, saving 249-252
  WordPress Block Editor, using 144
  working 143
custom user data
  processing 271
  storing 272, 273
  using, in enclosing shortcode 279-281
custom widget
  adding, to network dashboard 327-330
  creating, in WordPress 312-318

D

Dashicons
  reference link 78
Datepicker plugin
  used, for displaying calendar
    day selector 295-298
deactivation function 65
Debug Bar plugin
  download link 48
dedicated code editor/text editor
  installing 14-16
default user settings
  adding, for translation 351-353
development installation
  plugins, checking out 378
drop-down list settings field
  rendering 100

E

EasyPHP
  URL 6
editable blockquote block
  creating 226-231
email notifications
  sending 254-257
enclosing shortcode
  creating 54-56
  custom user data, using 279-281
external files
  loading, with WordPress path
    utility functions 28-31
external images
  loading, with WordPress path
    utility functions 28-31

F

favicons
  URL 28
file history
  viewing 14
files
  updating, to latest repository version 14
filter functions
  custom post data, displaying
    with 203-205
filter hook
  about 19, 33
  online listings 34
  user settings, accessing from 102, 103

G

get_permalink() function 39
get_template_directory_uri() function 31
get_theme_root() function 31
get_the_title () function 39
Git
  URL 13
GitHub
  URL 13
Global PHP variables 109
Google Images
  URL 141
Google reCAPTCHA administrator
  URL 258
Google reCAPTCHA GitHub page
  URL 260
Google reCAPTCHA service
  working 261
Gutenberg 144

H

Homebrew
  URL 7
home_url() function 31
hook listings
  URL 27
hook priority
  setting, to determine menu order 75
hooks
  about 19, 26
  searching, in WordPress source code 28
HTML
  used, for rendering admin
    page contents 79-82

I

IconArchive
  URL 153
import Subversion command 12
includes_url() function 31
internationalization functions
  reference link 366
Internet Assigned Numbers Authority
    (IANA) media types
  reference link 215

J

JavaScript files
  about 366
  localizing 366
JavaScript Object Notation
    files (.json) 351
jQuery
  loading, onto WordPress
    web pages 284-286
  no-conflict mode 287

L

language file
  loading, in plugin initialization 362-364
link tracking code
  inserting, in page body 39-42
load_plugin_textdomain function 365
Local application
  components 5
local library
  using, for CAPTCHA implementation
    on user forms 262-265
local Subversion version control repository
  creating 7-11

local version control repository
  creating 7
local web server
  about 1
  benefits 2
  installing, on computer 2-5
Local website
  URL 3

M

machine object (.mo) files 351
MAMP
  URL 6
media uploader
  custom files, attaching with 210-214
  working 215
menu icons
  sources 78, 79
Mercurial
  URL 13
meta boxes
  used, for formatting admin pages 104-110
modified files
  differences, viewing 14
multi-level administration menu
  creating 75-77
multiple sets, of user settings
  managing, from single admin page 122-130
MySQL 4

N

network dashboard
  custom widget, adding to 328-330
network-level plugin
  creating, with admin pages 131-135
Node.js 219
Notepad++
  URL 15

O

object-oriented PHP
  used, for writing plugins 58-60
output content
  adding, to page headers 24-27
OWASP Top Ten
  reference link 82

P

page body
  link tracking code, inserting in 39-42
page headers
  output content, adding to 24-27
page title
  updating, with plugin filters 193-195
parameters
  shortcode, creating 51-54
partial page contents
  updating, dynamically with AJAX 301-309
path utility functions
  using, to load external files 28-31
  using, to load external images 28-31
PHP 4
PHP Extension Pack 17
PHP ternary conditional operator 317
plugin actions
  output content, adding to page headers 24-27

plugin banner
  providing 379-381
plugin configuration data
  processing 83-86
  storing 83-86
plugin data
  removing, on deletion 69-71
plugin data updates
  scheduling, with WP-Cron 345-347
plugin distribution
  files requirement 226
plugin file
  creating 20-23
plugin filters
  link tracking code, inserting
    in page body 39-42
  used, for adding text after each
    item's content 35-38
  used, for modifying site
    generator meta tag 32-34
plugin handbook
  reference link 373
plugin header
  creating 21, 22
plugin hosting
  applying, on WordPress plugin
    repository 374, 375
plugin initialization
  language file, loading in 362-364
  user settings, creating 63-65
plugin output
  formatting, by loading style sheet 56-58
plugin performance
  optimizing by storing external data,
    with transients 339-344
plugins
  checking out, to development
    installation 378
  README file, creating for 371-373
  uploading, with Subversion 375-378
  writing, with object-oriented PHP 58-60
plugins_url function 30, 31
plugin versions
  releasing, with tags 374
Poedit
  download link 359
  text strings, translating with 359-362
  working 361, 362
pop-up dialog
  close button, removing 290
  display, controlling with
    shortcodes 291-294
  displaying, on select page 291
  displaying, with built-in
    ThickBox plugin 287-290
portable object (.po) files 351
post editor
  Custom Fields section, hiding 206-209
  extending, to attach custom files
    with media uploader 210-214
preg_replace function 34

Q

Quick Edit fields
  adding, for custom categories 184-192
  working 192

R

README file
  creating, for plugin 371-373
register_taxonomy function
  reference link 168
  working 168

remote web development environment
  creating 7
replacecontent function 309
REST API 144, 231
roles and capabilities
  reference link 74
RSS 332
RSS aggregators 332
RSS feeds
  displaying, from external feed 332-337
  fetch_feed caching period,
    modifying 338
  working 338

S

Scalable Vector Graphic (SVG) 78
Screem
  URL 15
search output
  tailoring, for custom post
    type items 155-159
Securimage
  reference link 263
server-side rendering block
  creating, that leverages existing
    shortcode 234-241
Settings API
  about 79
  used, for rendering admin
    page contents 92-99
settings menu
  administration page menu
    item, creating 71-74
shortcodes
  about 49
  creating 49, 50
  creating, with parameters 51-54
  custom post type data,
    displaying 160-163
  output, modifying for
    translation 357, 358
  used, for controlling pop-up
    dialog display 291-294
  working 164
show_user_profile 270
SimplePie library 332
single admin page
  multiple sets of user settings,
    managing from 122-130
single custom post type items
  displaying, with custom layout 151-153
site generator meta tag
  modifying, with plugin filters 32-34
site_url() function 31
SmartSVN
  URL 13
style sheet
  loading, to format plugin output 56-58
style sheet data
  storing, in user settings 113-121
Sublime Text
  URL 15
Subversion
  about 11
  used, for uploading plugin 375-378
SVN 17

T

tags
  used, for releasing plugin versions 374
taxonomy 165
text area settings field
  rendering 101

TextMate
  URL 15
text strings
  translating, with Poedit 359-362
thumbnail images
  providing 379-381
TipTip plugin
  used, for adding tooltips to admin page form fields 298-301
tooltips
  adding, to admin page form fields with TipTip plugin 298-301
TortoiseSVN
  integrating, in Windows File Explorer 12
  URL 7, 12
transients
  about 338
  using, for plugin performance optimization 339-344
Transients Manager
  reference link 343
translation
  admin page code, preparing for 353-356
  advanced translation functions 365, 366
  custom blocks, localizing 366-368
  default user settings, adapting 351-353
  JavaScript files, localizing 366
  shortcode output, modifying 357, 358
  template file 362
  translation file, updating 365
Twenty Twenty-Two theme 31, 47

U

uncommitted file changes
  reverting 14
user capability 74
user editor
  adding, to custom fields 268-270
user list page
  user data, displaying 273-279
user settings
  accessing, from action and filter hooks 102, 103
  creating, on plugin initialization 63-65
  storing, with arrays 66-68
  style sheet data, storing 113-121
user-submitted content
  moderating 253
  saving, in custom post types 249-252

V

variable content
  printing 43-47
Varying Vagrant Vagrants (VVV)
  about 6
  URL 6
version control 7
version control system 1, 13
Versions
  URL 13
Visual Studio Code
  about 17
  installing 15, 16
  URL 15

W

WampServer
  URL 6
Wandisco
  URL 7
web server packages
  reference link 6

widget configuration options
  displaying 318-322
  validating 322, 323
Windows File Explorer
  TortoiseSVN integration 12
WordPress
  built-in debugging features 47, 48
  custom widget, creating 312-318
WordPress Code Reference
  reference link 143, 144, 252
  URL 27
WordPress Codex
  URL 27
WordPress Hooks Intellisense 17
WordPress.org
  applying, for plugin hosting on 374, 375
WordPress plugin boilerplate generator
  URL 60
WordPress Plugins
  reference link 380
WordPress REST endpoint
  creating 231-233
WordPress rewrite module 146
WordPress site language configuration
  modifying 350, 351
  reference link 351
WordPress source code
  hooks, searching 28
WordPress web pages
  jQuery, loading onto 284-286
wp_add_dashboard_widget function 326
WP-CLI
  URL 367
WP-Cron
  execution, improving 348
  plugin data updates, scheduling
    with 345-347
  references, for third-party services 348
wp_enqueue_script function 286
wp_enqueue_style function 298
wp_nonce_field function 82
wp_upload_dir() function 31

X

Xdebug
  using 48
XnViewMP tool
  URL 153

Hi!

I, Yannick Lefebvre, author of *WordPress Plugin Development Cookbook*, really hope you enjoyed reading this book and found it useful for increasing your productivity and efficiency in developing custom plugins for the WordPress platform.

It would really help me (and other potential readers!) if you could leave a review on Amazon sharing your thoughts on this cookbook.

Go to the link below to leave your review:

`https://packt.link/r/180181077X`

Your review will help me to understand what's worked well in this book, and what could be improved upon for future editions, so it really is appreciated.

Best wishes,

Yannick Lefebvre

Packt.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

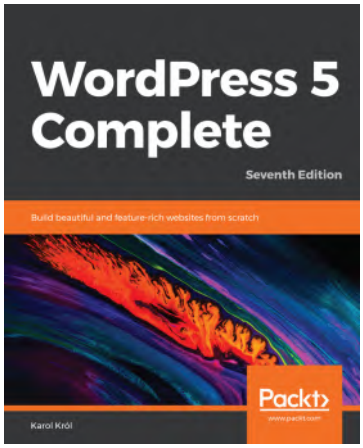
- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at packt.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at customercare@packtpub.com for more details.

At www.packt.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:

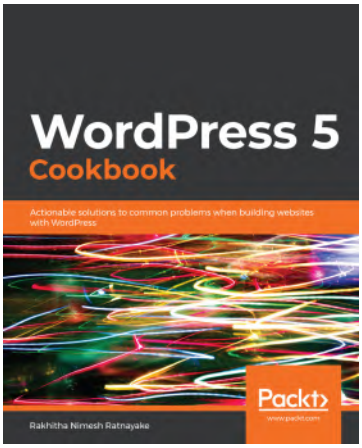


WordPress 5 Complete - Seventh Edition

Karol Król

ISBN: 978-1-78953-201-2

- Learn to adapt your plugin with the Gutenberg editor
- Create content that is optimized for publication on the web
- Craft great looking pages and posts with the use of block editor
- Structure your web pages in an accessible and clear way
- Install and work with plugins and themes



WordPress 5 Cookbook

Rakhitha Nimesh Ratnayake

ISBN: 978-1-83898-650-6

- Install and customize WordPress themes and plugins for building websites
- Develop modern web designs without the need to write any code
- Explore the new Gutenberg content editor introduced in WordPress 5 (Bebo)
- Use the existing WordPress plugins to add custom features and monetize your website
- Improve user interaction and accessibility for your website with simple tricks

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit `authors.packtpub.com` and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.